# 通向递归自我改进智能体之路：基础、框架与未来方向

## The Path to Recursive Self-Improving Agents: Foundation, Framework, and Future Directions

来源：https://doi.org/10.20944/preprints202608.0051.v1

原文 PDF：70 页。制作日期：2026-10-22。

译稿说明：这是 Google Translate 生成、经固定术语表约束的逐页机器翻译初稿，未经逐句人工学术审校。中文翻译覆盖全部可提取文本，包含正文、图表文字、附录与参考文献文本；图片内未能提取的文字、数学公式的二维布局和复杂表格请对照原文。排版提取可能造成少量顺序或符号错误，尤其是双栏图表、公式、脚注。不能用机器译稿替代原文进行公式推导或直接引用。常用英文术语和缩写予以保留。

PDF 采用原文页与中文译页交替编排，原始图表及公式完整保留在原文页。章节中的数字“原文第 X 页”始终指原始 PDF 物理页码，而非论文印刷页码或译稿页码。参考文献在原文页保留完整英文条目，检索时使用原文。学习文档中的例子和伪代码是导读，不属于作者原文。

# 核心摘要校订

根据原文摘要单独校订。正文仍为逐页机器翻译初稿；原始图表、公式和英文条目请查看紧邻原文页。

基于基础模型的智能体系统已经能以更高自主性完成更复杂任务，但人工开发与改进仍耗费大量时间和精力，因此需要研究可以自行改进的系统。本文研究能够自主将经验和评价反馈转化为组件持久更新的智能体，并提出从人工改进到通用递归自我改进的五级标准。本文还提出统一研究框架，将基础模型（M）、运行支撑框架（H）、智能体数据系统（D）、训练器（T）和支配其更新的改进机制（Imp）视为耦合系统。在此框架下，本文系统综述单个组件的自我改进与跨组件协同改进，并指出通向递归自我改进智能体的开放问题，为研究提供基础与路线图。

*Review*

# The Path to Recursive Self-Improving Agents: Foundation, Framework, and Future Directions

**Shuaiqi Liu †, Zhengkai Lin †, Yuxiang Zhang †, Yuanyi Ren †, Yue Wu, Yongbin Li, Zheng Wang, Zhihang Fu *,† and Jieping Ye**

Alibaba Group, China
* Correspondence: zhihang.fzh@alibaba-inc.com
† Core contributors.

**Abstract**

Agent systems based on foundation models have advanced rapidly and can now accomplish increasingly complex tasks with greater autonomy. However, manually developing and refining agent systems requires substantial human effort and time, motivating a shift toward agent systems that can improve themselves. In this survey, we study self-improving agent systems that autonomously transform experience and evaluation feedback into persistent updates to their components. To compare the self-improvement capabilities of diverse agent systems, we introduce a five-level grading standard, ranging from manual improvement to general recursive self-improvement. Furthermore, we propose a unified research framework that jointly models the foundation model, agent harness, agent data system, agent trainer, and the mechanism governing their improvement as a coupled system. Guided by this framework, we systematically review the literature on self-improvement and provide a taxonomy covering self-improvement in individual components and co-improvement across components. In addition, this survey identifies open research problems on the path toward recursive self-improving agents. Overall, this survey provides a foundation for studying self-improving agent systems and a roadmap for future research.

**Keywords:** AI agent; recursive self-improvement; self-improving agent; LLM; artificial intelligence

## 1. Introduction

Agent systems built on foundation models have made substantial progress in recent years. They can now complete increasingly complex tasks with greater autonomy, and the frontier of their capabilities continues to expand. In particular, an empirical study quantifies this trend by measuring agent capability in terms of *task length*, defined as the time required for human experts to complete a given task. Within the field of software engineering, it shows that the length of tasks that frontier agents can solve at a 50% success rate has approximately doubled every seven months since 2019 [1,2]. This progress reflects coordinated advances in four aspects. First, frontier *foundation models* have been trained at scale to strengthen their intrinsic agentic capabilities such as long-horizon planning, code generation, tool use, and iterative reflection [3–5], leading to rapid progress on agent benchmarks [6–8]. Second, more capable *agent harnesses* now support planning, long-context management, tool interaction, code execution, file-system operations, and failure handling, helping models handle complex and long-horizon tasks [9–12]. Third, the *production of agent data*, including environments, tasks, and trajectories, has scaled up dramatically [5,13,14]. Agent data systems can provide a continuing supply of training and evaluation signals for improving models' agentic capabilities. Finally, advances in *training algorithms and training infrastructure* have supported large-scale training of agent models [15–19].

However, progress across these aspects typically relies heavily on manual design and implementation. Each cycle of diagnosis, redesign, and implementation demands substantial human effort and

# 原文第 1 页 | 中文翻译初稿

通向递归自我改进智能体之路：基础、框架与未来方向
递归自我改进（Recursive Self-Improving）智能体之路：基础、框架和未来方向

刘帅奇†、林正凯†、张宇翔†、任远一†、吴越、李永斌、王正、付志航*、†和叶杰平

中国阿里巴巴集团

- 通讯地址：zhihang.fzh@alibaba-inc.com

- 核心贡献者。

摘要

基于基础模型（Foundation Models）的智能体系统发展迅速，现在可以以更大的自主性完成日益复杂的任务。然而，手动开发和完善智能体系统需要大量的人力和时间，这促使人们转向能够自我改进的智能体系统。在本篇综述中，我们研究了 自我改进（Self-Improving） 智能体系统，该系统能够自动将经验和评估反馈转化为对其组件的持续更新。为了比较不同智能体系统的自我改进（Self-Improvement）能力，我们引入了五级分级标准，从手动改进到通用递归自我改进（Recursive Self-Improvement）。此外，我们提出了一个统一的研究框架，将 基础模型（Foundation Model）、智能体 运行支撑框架（Harness）、智能体数据系统、智能体训练器以及管理它们改进的机制联合建模为耦合系统。在此框架的指导下，我们系统地回顾了 自我改进（Self-Improvement） 的文献，并提供了涵盖单个组件中的 自我改进（Self-Improvement） 和跨组件的共同改进的分类法。此外，这项调查还确定了 递归自我改进（Recursive Self-Improving） 智能体道路上的开放研究问题。总体而言，本篇综述为研究 自我改进（Self-Improving） 智能体系统奠定了基础，并为未来的研究提供了路线图。

关键词：人工智能智能体； 递归自我改进（Recursive Self-Improvement）； 自我改进（Self-Improving）智能体；法学硕士；人工智能

一、简介

基于基础模型（Foundation Models）构建的Agent系统近年来取得了长足的进步。他们现在可以以更大的自主权完成日益复杂的任务，并且他们的能力边界不断扩大。特别是，一项实证研究通过衡量智能体能力（任务长度）来量化这一趋势，任务长度定义为人类专家完成给定任务所需的时间。在软件工程领域，研究表明，自 2019 年以来，前沿智能体能够以 50% 的成功率解决的任务长度大约每 7 个月增加一倍 [1,2]。这一进展体现了四个方面的协调推进。首先，前沿_基础模型（Foundation Models）_经过大规模训练，以增强其内在智能体能力，例如长时程（Long-Horizon）规划、代码生成、工具使用和迭代反射[3-5]，从而在智能体基准上取得快速进展[6-8]。其次，功能更强大的_agent 运行支撑框架（Harnesses）_现在支持规划、长上下文管理、工具交互、代码执行、文件系统操作和故障处理，帮助模型处理复杂的长时程（Long-Horizon）任务[9-12]。第三，智能体数据的生产，包括环境、任务和轨迹，已大幅扩展[5,13,14]。智能体数据系统可以持续提供训练和评估信号，以提高模型的智能体能力

。最后，训练算法和训练基础设施的进步支持了智能体模型的大规模训练[15-19]。

然而，这些方面的进展通常在很大程度上依赖于手动设计和实现。每个诊断、重新设计和实施周期都需要大量的人力和

time. Consequently, the scope and speed of improvement are constrained by the available expertise and budget. These constraints motivate a shift from manually improved agent systems toward those capable of autonomous self-improvement.

In this survey, we define an agent system as *self-improving* if it can autonomously transform experience and evaluation feedback into persistent updates to its internal components or parameters, thereby enhancing its capabilities and behaviors [20–23]. To formalize this notion, we formulate an agent system as a tuple of five interdependent components: the foundation model $\mathcal{M}$, agent harness $\mathcal{H}$, agent data system $\mathcal{D}$, agent trainer $\mathcal{T}$, and improvement mechanism *Imp*, as shown in Figure 1.

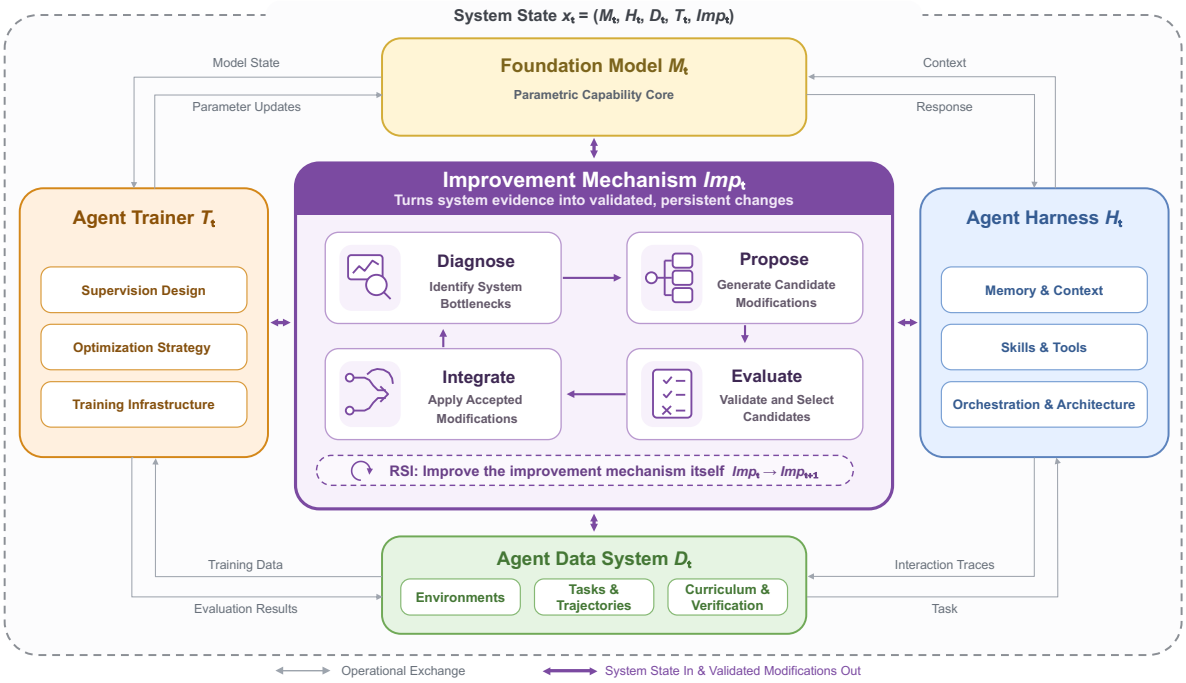



**Figure 1.** A unified framework for agent system self-improvement.

This formulation naturally suggests a further extension: since *Imp* is itself a component of the system, it can likewise be subject to recursive modification. In this scenario, each improvement cycle not only enhances operational capabilities but also optimizes the mechanism that drives future improvements [24,25], enabling the agent system to effectively **improve how to improve**. We refer to this capability as *recursive self-improvement* (RSI), raising the possibility of open-ended, self-reinforcing improvement.

To evaluate where existing agent systems stand on the path to RSI, we further introduce a grading standard that classifies them into five levels of self-improvement capability, ranging from manual improvement, where every change requires external human intervention, to RSI, where the improvement mechanism itself is subject to the system's own modification. This standard offers a shared basis for comparing these systems and indicating how far they have progressed toward more autonomous and general self-improvement.

Furthermore, we introduce a unified research framework that models the self-improving agent system, enabling a holistic analysis across components. The framework specifies two aspects: the scope of autonomous improvement and the procedure of improvement. As shown in Figure 1, the scope of autonomous improvement includes the foundation model, agent harness, agent data system, agent trainer, and improvement mechanism. The procedure of improvement includes the diagnosis of system bottlenecks, the proposal of candidate modifications to these components, the validation and selection of these candidates, and the integration of the selected modification. Based on this research framework, this survey provides a comprehensive review of self-improving agent systems.

In summary, this survey makes the following contributions:

# 原文第 2 页 | 中文翻译初稿

2 共 70 个

时间。因此，改进的范围和速度受到可用专业知识和预算的限制。这些限制促使从手动改进的智能体系统转向那些能够自主 自我改进（Self-Improvement） 的智能体系统。

在本篇综述中，我们将智能体系统定义为_自我改进（Self-Improving）_，如果它能够自主地将经验和评估反馈转化为对其内部组件或参数的持续更新，从而增强其能力和行为[20-23]。为了形式化这个概念，我们将智能体系统制定为五个相互依赖的组件的元组：基础模型（Foundation Model）M、智能体运行支撑框架（Harness）H、智能体数据系统_D_、智能体训练器_T_和改进机制_Imp_，如图1所示。

系统状态 x ☐ = ( M ☐ , H ☐ , D ☐ , T ☐ , Imp ☐ )
模型状态 基础模型（Foundation Model） M ☐ 上下文
参数能力核心
参数更新响应
改进机制 Imp ☐
将系统证据转化为经过验证的、持久的变更
智能体训练师 T ☐ 智能体 运行支撑框架（Harness） H ☐
诊断建议
监督设计识别系统生成候选内存和上下文
瓶颈修改
优化策略技能和工具
集成评估
训练基础设施应用已接受的验证并选择编排和架构
修改候选人
RSI：改进改进机制本身 Imp ☐ → Imp ☐+1
训练数据智能体数据系统 D ☐ 交互轨迹
任务与课程&
评估结果环境轨迹验证任务
操作交换系统状态输入和验证修改输出

图1.智能体系统自我改进（Self-Improvement）的统一框架。

这个表述自然意味着进一步的扩展：由于 Imp 本身就是系统的一个组件，因此它同样可以进行递归修改。在这种场景下，每个改进周期不仅增强了运营能力，而且还优化了驱动未来改进的机制[24,25]，使智能体系统能够有效地改进如何改进。我们将此能力称为 递归自我改进（Recursive Self-Improvement）(RSI)，提高了开放式、自我强化改进的可能性。

为了评估现有智能体系统在 RSI 道路上的位置，我们进一步引入了分级标准，将其分为 自我改进（Self-Improvement） 能力的五个级别，从手动改进（每次更改都需要外部人工干预）到 RSI（改进机制本身需要系统自身的修改）。该标准为比较这些系统并表明它们在更加自主和通用的 自我改进（Self-Improvement） 方面取得了多大进展提供了共享基础。

此外，我们引入了一个统一的研究框架，对 自我改进（Self-Improving） 智能体系统进行建模，从而实现跨组件的整体分析。该框架规定了两个方面：自主改进的范围和改进的程序。如图1所示，自主改进范围包括基础模型（Foundation Model）、坐席运行支撑框架（Harness）、坐席数据系统、坐席训练器、改进机制

。改进的过程包括系统瓶颈的诊断、对这些组件的候选修改的建议、这些候选修改的验证和选择以及所选修改的集成。基于此研究框架，本篇综述对 自我改进（Self-Improving） 智能体系统进行了全面回顾。

总而言之，本篇综述有以下贡献：

- We establish formal definitions of agent system self-improvement and RSI, together with a five-level capability grading standard ranging from manual improvement to general RSI.
- We propose a unified research framework (Figure 1) that models the foundation model, agent harness, agent data system, agent trainer, and improvement mechanism as a coupled evolving system, capturing the scope, dependencies, and dynamics of self-improvement.
- We systematically reviews existing work on self-improving agent systems and provides a taxonomy (Figure 2), covering self-improvement in individual components and co-improvement across components.
- We identify key open problems on the path to RSI, which provide research directions for developing more autonomous, reliable, and general self-improving agent systems.

Together, these contributions provide a unified foundation for studying self-improving agent systems and a roadmap toward RSI agent systems.

The remainder of this survey is organized as follows. Section 2 presents formal definitions of agent systems and self-improvement, establishes a grading standard for self-improvement capability, and introduces our unified research framework. Sections 3, 4, and 5 review the existing work on self-improvement in the agent harness, the agent data system, and the agent trainer, respectively. Section 6 summarizes existing work on collaborative improvement of multiple components within a self-improving agent system. Section 7 identifies open problems on the path toward general RSI agent systems. Finally, Section 8 concludes this survey.

# 原文第 3 页 | 中文翻译初稿

3 共 70 个

- 我们建立了智能体系统自我改进（Self-Improvement）和RSI的正式定义，以及从手动改进到通用RSI的五级能力分级标准。

- 我们提出了一个统一的研究框架（图 1），将 基础模型（Foundation Model）、智能体 运行支撑框架（Harness）、智能体数据系统、智能体训练器和改进机制建模为耦合演化系统，捕获 自我改进（Self-Improvement） 的范围、依赖性和动态。

- 我们系统地回顾了 自我改进（Self-Improving） 智能体系统的现有工作，并提供了一个分类法（图 2），涵盖单个组件中的 自我改进（Self-Improvement） 以及跨组件的共同改进。

- 我们确定了 RSI 道路上的关键开放问题，为开发更自主、可靠和通用的 自我改进（Self-Improving） 智能体系统提供了研究方向。

这些贡献共同为研究 自我改进（Self-Improving） 智能体系统和 RSI 智能体系统的路线图提供了统一的基础。

本篇综述的其余部分组织如下。第2节提出了智能体系统和自我改进（Self-Improvement）的正式定义，建立了自我改进（Self-Improvement）能力的分级标准，并介绍了我们的统一研究框架。第 3、4 和 5 节分别回顾了智能体 运行支撑框架（Harness）、智能体数据系统和智能体训练器中 自我改进（Self-Improvement） 的现有工作。第 6 节总结了 自我改进（Self-Improving） 智能体系统内多个组件协作改进的现有工作。第 7 节指出了通用 RSI 智能体系统道路上的未决问题。最后，第 8 节总结了本篇综述。

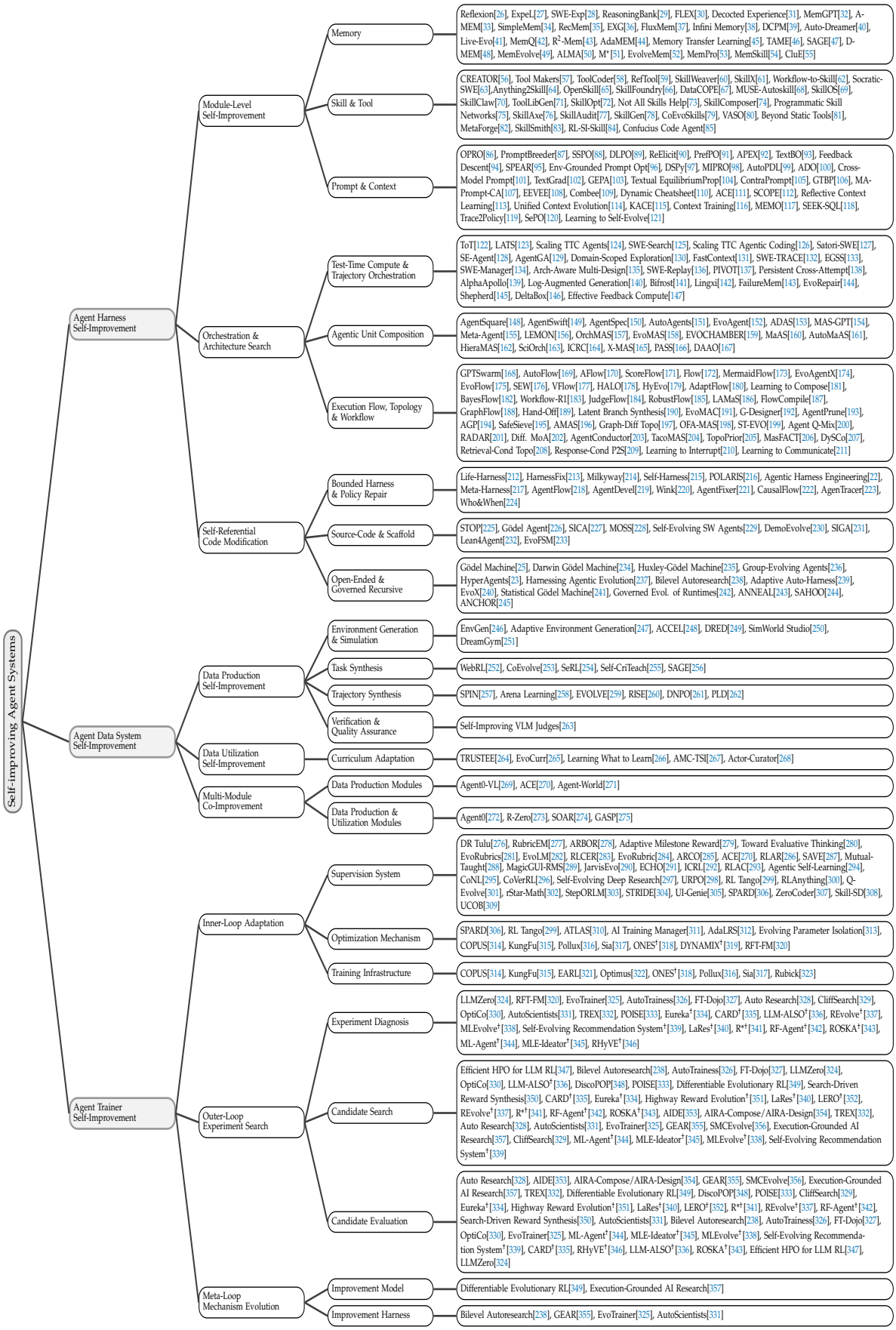



**Figure 2.** Taxonomy of self-improving agent systems organized into *Agent Data System Self-Improvement*, *Agent Harness Self-Improvement*, and *Agent Trainer Self-Improvement*, with representative works annotated at each leaf node.

# 原文第 4 页 | 中文翻译初稿

4 共 70 个

Reflexion[26]、ExpeL[27]、SWE-Exp[28]、 ReasoningBank[29]、FLEX[30]、Decoted Experience[31]、MemGPT[32]、A-
MEM[33]、SimpleMem[34]、RecMem[35]、EXG[36]、FluxMem[37]、Infini Memory[38]、 DCPM[39]、自动梦想家[40]、
内存 Live-Evo[41]、MemQ[42]、R 2 -Mem[43]、AdaMEM[44]、内存迁移学习[45]、TAME[46]、SAGE[47]、D-
MEM[48]、MemEvolve[49]、ALMA[50]、M ⋆ [51]、EvolveMem[52]、MemPro[53]、MemSkill[54]、CluE[55]
CREATOR[56]、工具制造商[57]、ToolCoder[58]、RefTool[59]、SkillWeaver[60]、SkillX[61]、工作流程到技能[62]、 Socratic-
SWE[63]、Anything2Skill[64]、OpenSkill[65]、SkillFoundry[66]、DataCOPE[67]、MUSE-Autoskill[68]、SkillOS[69]、
模块级技能和工具 SkillClaw[70]、ToolLibGen[71]、SkillOpt[72]、并非所有技能都有帮助[73]、SkillComposer[74]、编程技能
自我改进（Self-Improvement）Networks[75]、SkillAxe[76]、SkillAudit[77]、SkillGen[78]、CoEvoSkills[79]、VASO[80]、超越静态工具[81]、
MetaForge[82]、SkillSmith[83]、RL-SI-Skill[84]、孔子代码智能体[85]
OPRO[86]、PromptBreeder[87]、SSPO[88]、DLPO[89]、ReElicit[90]、PrefPO[91]、APEX[92]、TextBO[93]、Feedback
Descent[94]、SPEAR[95]、Env-Grounded Prompt Opt[96]、DSPy[97]、 MIPRO[98]、AutoPDL[99]、ADO[100]、跨模型提示[101]、TextGrad[102]、GEPA[103]、文本EquilibriumProp[104]、ContraPrompt[105]、GTBP[106]、MA-
提示和上下文
提示-CA[107]、 EEVEE[108]、Combee[109]、动态备忘单[110]、ACE[111]、SCOPE[112]、反思性情境学习[113]、统一情境进化[114]、KACE[115]、情境训练[116]、MEMO[117]、 SEEK-SQL[118]、Trace2Policy[119]、SePO[120]、学习自我进化[121]
ToT[122]、LATS[123]、扩展 TTC 智能体[124]、SWE-Search[125]、扩展 TTC 智能体编码[126]、Satori-SWE[127]、
SE-Agent[128]、AgentGA[129]、域范围探索[130]、FastContext[131]、SWE-TRACE[132]、EGSS[133]、

测试时计算和轨迹编排 SWE-Manager[134]、Arch-Aware Multi-Design[135]、 SWE-Replay[136]、PIVOT[137]、持续交叉尝试[138]、
AlphaApollo[139]、Log-Augmented Generation[140]、Bifrost[141]、Lingxi[142]、FailureMem[143]、EvoRepair[144]、
Shepherd[145]、DeltaBox[146]、有效反馈计算[147]
智能体管理自我改进编排和智能体单元组合 AgentSquare[Meta-Agent[155148], LEMON[], AgentSwift[156], OrchMAS[149], AgentSpec[157], EvoMAS[150], AutoAgents[158], EVOCHAMBER[151], EvoAgent[159152]、MaAS[]、ADAS[160153]、AutoMaAS[]、

MAS-GPT[161154],],
架构搜索 HieraMAS[162]、SciOrch[163]、ICRC[164]、X-MAS[165]、PASS[166], DAAO[167]
GPTSwarm[168]、AutoFlow[169]、AFlow[170]、ScoreFlow[171]、Flow[172]、MermaidFlow[173]、
EvoAgentX[174]、
EvoFlow[175]、SEW[176]、VFlow[177]、HALO[178]、HyEvo[179]、 AdaptFlow[180]、学习撰写[181]、
BayesFlow[182]、Workflow-R1[183]、JudgeFlow[184]、RobustFlow[185]、LAMaS[186]、
FlowCompile[187]、
执行流程、拓扑和工作流 GraphFlow[188]、Hand-Off[189]、潜在分支综合[190]、 EvoMAC[191]、
G-Designer[192]、AgentPrune[193]、
AGP[194]、SafeSieve[195]、AMAS[196]、Graph-Diff Topo[197]、OFA-MAS[198]、ST-EVO[199]、
Agent Q-Mix[200]、
RADAR[201]、Diff

。 MoA[202]、AgentConductor[203]、TacoMAS[204]、TopoPrior[205]、MasFACT[206]、DySCo[207]、
检索条件 Topo[208]、响应条件 P2S[209]、学习中断[210]、学习通信[211]
有界运行支撑框架（Harness）&保单修复寿命-运行支撑框架（Harness）[Meta-运行支撑框架（Harness）
[212217]、HarnessFix[]、AgentFlow[213218]、Milkyway[]、AgentDevel[214]、Self-运行支撑框架（
Harness）[219]、Wink[220]、 AgentFixer[215]、POLARIS[221216]、CausalFlow[]、Agentic 运行支撑框
架（Harness） Engineering[222]、AgenTracer[22322]、]、
Who&When[224]
自引用源代码和支架 STOP[225]、Gödel Agent[226]、SICA[227]、 MOSS[228]、自进化软件智能体[229]、
DemoEvolve[230]、SIGA[231]、
代码修改 Lean4Agent[232]、EvoFSM[233]
哥德尔机[25]、达尔文哥德尔机[234]、Huxley-Gödel 机[235]、群进化智能体[236]、
开放式和超级智能体[23]、利用智能体进化[237]、双层自动研究[238]、自适应自动运行支撑框架（
Harness）[239]、
受控递归EvoX[240]、统计哥德尔机[241]、受控进化。 Runtimes[242]、ANNEAL[243]、SAHOO[244]、
ANCHOR[245]
环境生成EnvGen[246]、自适应环境生成[247]、ACCEL[248]、DRED[249]、SimWorld Studio[250]、
& 模拟 DreamGym[251]
数据生产任务合成WebRL[252]、CoEvolve[253]、SeRL[254]、Self-CriTeach[255]、SAGE[256]
自我改进（Self-Improvement）
轨迹合成 SPIN[257]、Arena Learning[258]、EVOLVE[259]、RISE[260]、DNPO[261]、 PLD[262]
智能体数据系统验证和质量保证 自我改进（Self-Improving） VLM 法官[263]
自我改进（Self-Improvement）数据利用自我改进课程适应TRUSTEE[264]、EvoCurr[265]、学习该学
[266]、AMC-TSI[267]、 Actor-Curator[268]
数据生产模块 Agent0-VL[269]、ACE[270]、Agent-World[271]
多模块
协同改进数据生产与利用模块 Agent0[272]、R-Zero[273]、SOAR[274]、GASP[275]
DR Tulu[276]、 RubricEM[277]、ARBOR[278]、自适应里程碑奖励[279]、走向评价性思维[280]、
EvoRubrics[281]、EvoLM[282]、RLCER[283]、EvoRubric[284]、ARCO[285]、ACE[270]、RLAR[286]、
SAVE[287]、Mutual-
Taught[288]、MagicGUI-RMS[289]、JarvisEvo[290]、ECHO[291]、ICRL[292]、RLAC[293]、智能体自

学习[294]、
监督系统 CoNL[295]、CoVerRL[296]、自我进化深度研究[297]、 URPO[298]、RL Tango[299]、RLAnything[300]、Q-
Evolve[301]、rStar-Math[302]、StepORLM[303]、STRIDE[304]、UI-Genie[305]、SPARD[306]、ZeroCoder[307]、 Skill-SD[308]、
UCOB[309]
内环自适应
SPARD[306]、RL Tango[299]、ATLAS[310]、AI Training Manager[311]、AdaLRS[312]、演进参数隔离[313]、
优化机制 COPUS[314]、KungFu[315]、 Pollux[316]、Sia[317]、ONES † [318]、DYNAMIX † [319]、RFT-FM[320]
训练基础设施 COPUS[314]、KungFu[315]、EARL[321]、Optimus[322]、ONES † [318]、Pollux[316]、Sia[317], Rubick[323]
LLMZero[324], RFT-FM[320], EvoTrainer[325], AutoTrainess[326], FT-Dojo[327], Auto Research[328], CliffSearch[329],
OptiCo[330], AutoScientists[331], TREX[332], POISE[333]、Eureka † [334]、CARD † [335]、
LLM-ALSO † [336]、REvolve † [337]、
实验诊断 MLEvolve † [338]、自进化推荐系统 † [339]、LaRes † [340]、R* † [341]、RF-Agent † [342]、ROSKA † [343]、
ML-Agent † [344]、MLE-Ideator † [345]、RHyVE † [346]
高效 HPO

LLM RL[347]、Bilevel Autoresearch[238]、AutoTrainess[326]、FT-Dojo[327]、LLMZero[324]、OptiCo[330]、LLM-ALSO † [336]、DiscoPOP[348]、POISE[333]、可微进化 RL[349]、搜索驱动奖励合成[350]、CARD † [335]、尤里卡 † [334]、高速公路奖励进化 † [351]、LaRes † [340]、LERO † [352]、
智能体训练者自我改进外环候选人搜索 REvolve † [337]、R* † [341]、 RF-Agent † [342]、ROSKA † [343]、AIDE[353]、AIRA-Compose/AIRA-Design[354]、TREX[332]、
实验搜索自动研究[328]、AutoScientists[331]、EvoTrainer[325]、GEAR[355]、SMCEvolve[356]、执行接地AI
研究[357]、CliffSearch[329]、ML-Agent † [344]、MLE-Ideator † [345]、MLEvolve † [338]、自我进化推荐系统 † [339]
自动研究[328]、AIDE[353]、AIRA-Compose/AIRA-Design[354]、 GEAR[355]、SMCEvolve[356]、执行型
AI 研究[357]、TREX[332]、可微进化 RL[349]、DiscoPOP[348]、POISE[333]、CliffSearch[329]、Eureka † [334]、高速公路奖励进化 † [351]、LaRes † [340]、LERO † [352]、R* † [341]、REvolve † [337]、RF-Agent † [342]、
候选人评估搜索驱动的奖励合成[350]、AutoScientists[331]、双层自动研究[238]、AutoTrainess[326]、FT-Dojo[327]、
OptiCo[330]、EvoTrainer[325]、ML-Agent † [344]、MLE-Ideator † [345]、MLEvolve † [338]、自我进化推荐系统 † [339]、CARD † [335]、RHyVE † [346]、 LLM-ALSO † [336]、ROSKA † [343]、LLM RL 的高效 HPO[347]、
LLMZero[324]

改进模型可微进化 RL[349]、基于执行的 AI 研究[357]
元循环
机制进化
改进运行支撑框架（Harness） Bilevel Autoresearch[238]、GEAR[355]、EvoTrainer[325]、AutoScientists[331]
自我改进（Self-Improving） Agent Systems

图 2. 自我改进（Self-Improving） 智能体系统的分类，组织为 Agent Data System 自我改进（Self-Improvement）、 Agent 运行支撑框架（Harness） 自我改进（Self-Improvement） 和 Agent Trainer 自我改进（Self-Improvement），并在每个叶节点处注释了代表性作品。

## 2. Foundation of Self-Improving Agent Systems

This section establishes the foundation for the rest of the survey by clarifying the key concepts of self-improving agent systems, defining a grading standard for self-improvement capability, and introducing a unified research framework. Together, these elements provide the conceptual basis, evaluation criteria, and analytical framework for examining existing work in later sections. Section 2.1 defines agent systems by introducing their main components and then formalizes self-improvement and recursive self-improvement (RSI). Section 2.2 introduces a five-level grading standard for comparing systems by autonomy in the improvement loop, modification of the improvement mechanism, and generalization across domains. Section 2.3 proposes a unified research framework for self-improving agent systems.

### *2.1. Preliminaries*

To provide a clear basis for the subsequent discussion, this section establishes the conceptual and formal foundations of self-improving agent systems. Section 2.1.1 formalizes an agent system as a five-component tuple and explains the role of each component within the system. Section 2.1.2 formalizes the self-improvement process.

#### 2.1.1. Agent Systems

An agent system is a computational system in which one or more autonomous agents observe an environment, reason over available information, and act to achieve specified objectives. Such a system typically contains several components that support planning, interaction, learning, and adaptation. In this survey, we represent an agent system as a tuple of five components: $(\mathcal{M}, \mathcal{H}, \mathcal{D}, \mathcal{T}, Imp)$, where $\mathcal{M}$ denotes the foundation model, $\mathcal{H}$ denotes the agent harness, $\mathcal{D}$ denotes the agent data system, $\mathcal{T}$ denotes the agent trainer, and $Imp$ denotes the improvement mechanism. These five components serve distinct but complementary functions. We elaborate on each component in turn below.

Foundation model.

The foundation model $\mathcal{M}$ is the parametric core of the agent system. It provides general capabilities, including language understanding, reasoning, planning, code generation, and tool use [3,4]. These capabilities support the operation of the other components, while feedback from those components can in turn guide updates to $\mathcal{M}$.

Agent harness.

The agent harness $\mathcal{H}$ is the execution layer that places the foundation model in an operational loop. It governs what the model observes, what actions it can take, and how observations and actions are organized into task-completion workflows [9–12]. The harness therefore enables the system to adapt its behavior without necessarily changing the parameters of the foundation model.

Agent data system.

The agent data system $\mathcal{D}$ manages the lifecycle of training and evaluation data. It converts agent trajectories, environmental observations, performance feedback, and existing datasets into experience and evaluative signals [14,246,253]. It typically consists of two closely coupled stages: data production and data utilization. Together, these two stages connect the inference-time interaction in the harness with persistent learning in the trainer, and directly shape learning efficiency and system capability.

Agent trainer.

The agent trainer $\mathcal{T}$ converts agent experience into persistent model updates [282,325,358]. At each training step, $\mathcal{T}$ consumes data from the agent data system $\mathcal{D}$, and returns updated model parameters. It comprises three coupled modules. First, an evaluative feedback module transforms raw outcomes into learning signals, which may involve evaluation criteria, verifiers, or credit-assignment procedures. Second, a policy-optimization module specifies the objective, algorithm, update sched-

# 原文第 5 页 | 中文翻译初稿

5 共 70 个
2. 自我改进（Self-Improving）智能体系统的建立

本节通过阐明自我改进（Self-Improving）智能体系统的关键概念、定义自我改进（Self-Improvement）能力的分级标准并引入统一的研究框架，为其余调查奠定基础。这些要素共同为后面章节中检查现有工作提供了概念基础、评估标准和分析框架。 2.1节通过介绍智能体系统的主要组件来定义智能体系统，然后形式化自我改进（Self-Improvement）和递归自我改进（Recursive Self-Improvement）（RSI）。 2.2节介绍了一个五级评分标准，用于通过改进循环中的自主性、改进机制的修改和跨领域的泛化来比较系统。 2.3节提出了自我改进（Self-Improving）智能体系统的统一研究框架。
2.1。预赛

为了给后续讨论提供清晰的基础，本节建立了 自我改进（Self-Improving） 智能体系统的概念和形式基础。第 2.1.1 节将智能体系统形式化为五组件元组，并解释了系统中每个组件的作用。第 2.1.2 节规范了 自我改进（Self-Improvement） 流程。
2.1.1.智能体系统

智能体系统是一种计算系统，其中一个或多个自主智能体观察环境，对可用信息进行推理，并采取行动以实现指定目标。这样的系统通常包含多个支持规划、交互、学习和适应的组件。在本篇综述中，我们将智能体系统表示为五个组件的元组：（ M 、 H 、 D 、 T 、 Imp ），其中 M 表示 基础模型（Foundation Model），H 表示智能体 运行支撑框架（Harness），D 表示智能体数据系统，T 表示智能体trainer，_Imp_表示改进机制。这五个组成部分具有不同但互补的功能。下面我们依次详细阐述每个组件。
基础模型（Foundation Model）。

基础模型（Foundation Model）_M_是智能体系统的参数化核心。它提供通用功能，包括语言理解、推理、规划、代码生成和工具使用[3,4]。这些功能支持其他组件的操作，而来自这些组件的反馈反过来可以指导对 M 的更新。
智能体运行支撑框架（Harness）。

智能体 运行支撑框架（Harness） H 是将 基础模型（Foundation Model） 置于操作循环中的执行层。它控制模型观察的内容、可以采取的操作以及如何将观察和操作组织成任务完成工作流程 [9-12]。因此，运行支撑框架（Harness） 使系统能够调整其行为，而无需更改 基础模型（Foundation Model） 的参数。
智能体数据系统。

智能体数据系统_D_管理训练和评估数据的生命周期。它将智能体轨迹、环境观察、绩效反馈和现有数据集转换为经验和评估信号[14,246,253]。它通常由两个紧密耦合的阶段组成：数据生产和数据利用。这两个阶段将 运行支撑框架（Harness） 中的推理时间交互与训练器中的持续学习连接起来，并直接塑造学习效率和系统能力。
智能体训练师。

智能体训练器_T_将智能体经验转化为持久模型更新[282,325,358]。在每个训练步骤中，T 消耗来自智能体数据系统 D 的数据，并返回更新的模型参数。它包括三个耦合模块。首先，评估反馈模块将原始结果转化为学习信号，这可能涉及评估标准、验证者或学分分配程序。其次，策略优化模块指定目标、算法、更新计划

ule, and constraints used to update model parameters. Third, a training-infrastructure module provides the computational substrate on which these updates run, such as computational backends and reinforcement-learning environments. Together, these modules determine how agent experience becomes a persistent parameter change in the agent system.

Improvement mechanism.

The improvement mechanism *Imp* governs how evidence and feedback are used to modify the agent system over time [234,324,325]. Its operational structure is specified in Section 2.3.1.

#### 2.1.2. Self-Improvement

Self-improvement is an iterative process in which a system autonomously modifies its own state to improve its capabilities, behaviors, or underlying components [22,23]. Let $x_t \in \mathcal{X}$ denote the complete state of the agent system at discrete step $t$. This state includes the source code, parameters, configurations, and outputs of the agent system's components defined in Section 2.1.1. At each step, the improvement mechanism $Imp_t$ in $x_t$ reads the current state and produces the next state:

$$x_{t+1} = Imp_t(x_t). \tag{1}$$

The complete agent system participates in this iterative process, but not every component must change at every step. A given modification also need not immediately improve the component being modified. The improvement mechanism may greedily select the candidate with the largest predicted short-term gain. It may also maintain multiple evolutionary lineages, evaluate them in subsequent rounds, and retain candidates that support stronger long-term improvement.

Recursive Self-Improvement.

Recursive self-improvement (RSI) is a stronger form of self-improvement in which the system can also modify the mechanism that governs its future improvements [24,25]. In other words, an RSI agent system improves not only its current capabilities but also its ability to improve. In Equation 1, this means that $Imp_t$ is not an external or fixed procedure. As part of $x_t$, $Imp_t$ can itself be updated to a successor $Imp_{t+1}$, which then governs subsequent improvement cycles.

### *2.2. Self-Improvement Capability Grading Standard*

Existing agent systems differ in how autonomous they are, whether the improvement mechanism can improve itself, and whether self-improvement can generalize to open domains. Consequently, the term *self-improving* has been applied to systems with substantially different properties. Inspired by the level-based structure of the SAE J3016 driving-automation taxonomy [359], we propose a five-level grading standard for more systematic comparison.

As shown in Table 1, the five levels of self-improvement capability can be distinguished by several aspects. The **proposer**, **modifier**, and **validator** describe whether the agent system can autonomously propose changes, apply them, and validate their effects without human intervention. The ***Imp* modification** dimension specifies whether the improvement mechanism *Imp* itself lies within the system's autonomous modification scope. This dimension is decisive for RSI, as it determines whether the system's improvement capacity itself can grow over time. Finally, **generality** indicates whether self-improvement transfers beyond a bounded task family or restricted operational setting, reflecting genuine capability gains rather than in-distribution overfitting.

# 原文第 6 页 | 中文翻译初稿

6 共 70 个

规则，以及用于更新模型参数的约束。第三，训练基础设施模块提供运行这些更新的计算基础，例如计算后端和强化学习环境。这些模块共同决定座席体验如何成为座席系统中持久的参数变化。
完善机制。

改进机制_Imp_控制如何使用证据和反馈来随着时间的推移修改智能体系统[234,324,325]。其操作结构在第 2.3.1 节中指定。
2.1.2. 自我改进（Self-Improvement）

自我改进（Self-Improvement）是一个迭代过程，系统自主修改自己的状态以改进其功能、行为或底层组件[22,23]。令 xt ∈X 表示智能体系统在离散步骤 t 处的完整状态。该状态包括第 2.1.1 节中定义的智能体系统组件的源代码、参数、配置和输出。在每一步，xt 中的改进机制 Impt 都会读取当前状态并生成下一个状态：

xt +1 = 导入（xt）。 (1)

完整的智能体系统参与这个迭代过程，但并不是每个组件都必须在每个步骤中进行更改。给定的修改也不需要立即改进正在修改的组件。改进机制可能会贪婪地选择具有最大预测短期增益的候选者。它还可能维持多个进化谱系，在后续几轮中对其进行评估，并保留支持更强大的长期改进的候选者。
递归自我改进（Recursive Self-Improvement）。

递归自我改进（Recursive Self-Improvement）(RSI) 是 自我改进（Self-Improvement）的更强形式，其中系统还可以修改管理其未来改进的机制 [24,25]。换句话说，RSI智能体系统不仅提高了其当前的能力，而且提高了其改进的能力。在公式 1 中，这意味着 Impt 不是外部或固定过程。作为 xt 的一部分，Impt 本身可以更新为后继 Impt +1，然后控制后续的改进周期。
2.2。 自我改进（Self-Improvement）能力分级标准

现有的智能体系统的不同之处在于它们的自治程度、改进机制是否可以自我改进以及自我改进（Self-Improvement）是否可以推广到开放域。因此，术语 自我改进（Self-Improving）已应用于具有显着不同属性的系统。受 SAE J3016 驾驶自动化分类法 [<mark>359] 基于级别的结构的启发，我们提出了一个五级分级标准，以进行更系统的比较。

如表1所示，自我改进（Self-Improvement）能力的五个级别可以从几个方面来区分。提议者、修改者和验证者描述了智能体系统是否可以在无需人工干预的情况下自主提出更改、应用更改并验证其效果。_Imp_修改维度指定了改进机制_Imp_本身是否属于系统的自主修改范围。这个维度对于RSI来说是决定性的，因为它决定了系统本身的改进能力是否能够随着时间的推移而增长

。最后，通用性表明 自我改进（Self-Improvement）是否转移到有界任务族或受限操作环境之外，反映了真正的能力增益，而不是分布内的过度拟合。

**Table 1.** Capability-grading standard for agent system self-improvement. A checkmark indicates that the agent system possesses the corresponding capability without per-update human intervention. *Imp* denotes the improvement mechanism. *Imp Modification* requires a functionally meaningful change that affects subsequent improvement cycles. *Generality* refers to the domain generality of recursive self-improvement, rather than task-performance generalization alone.

| Level | Category | Programmatic Improvement Loop | | | Recursion | Domain |
|---|---|---|---|---|---|---|
| | | Proposer | Modifier | Validator | *Imp* Modification | Generality |
| L1 | Manual Improvement | | | | | |
| L2 | Assisted Improvement | ✓ | | | | |
| L3 | Programmatic Self-Improvement | ✓ | ✓ | ✓ | | |
| L4 | Bounded Recursive Self-Improvement | ✓ | ✓ | ✓ | ✓ | |
| L5 | General Recursive Self-Improvement | ✓ | ✓ | ✓ | ✓ | ✓ |

The five levels are defined as follows:

(L1) **Manual improvement.** The agent system has no autonomous improvement capability. It is deployed and executed in a fixed form, and every change requires an external development and deployment process.

(L2) **Assisted improvement.** The agent system can propose candidate modifications or provide diagnostic evidence, but humans remain responsible for validating and applying substantive changes. The improvement mechanism *Imp* is maintained by humans rather than by the agent system. Therefore, L2 is regarded as a precursor to self-improvement rather than an instance of it.

(L3) **Programmatic self-improvement.** The agent system can autonomously propose, apply, and validate modifications to its operational components, such as the foundation model, agent harness, data system, or trainer. These modifications may be substantial and may span multiple components. However, the improvement mechanism *Imp* that governs these modifications remains fixed or externally maintained. L3 is therefore programmatic: the system controls the execution of improvement, but does not yet modify the mechanism *Imp* that governs subsequent improvement cycles.

(L4) **Bounded recursive self-improvement.** The agent system can not only propose candidate modifications, validate their effects, and apply the validated modifications, but also rewrite its own improvement mechanism, which makes the improvement mechanism self-referential. Individual steps need not modify every component, but the overall process supports sustained long-term progress within a bounded domain.

(L5) **General recursive self-improvement.** L5 retains the autonomy, self-reference, and long-term progress of L4. Beyond this, its improvement capability can transfer effectively across broad and evolving task domains rather than remaining limited to a fixed benchmark, narrow task family, or restricted operational setting. Such generality marks a qualitative shift: the system's capacity to improve is no longer tied to any particular problem space, opening the possibility of improvement across an ever-expanding range of domains.

In summary, this grading standard separates manual improvement, assisted improvement, programmatic self-improvement, bounded RSI, and general RSI. It provides a shared basis for comparing existing agent systems and for identifying how far current systems have progressed toward general RSI.

# 原文第 7 页 | 中文翻译初稿

7 共 70 个

表1 智能体系统自我改进（Self-Improvement）的能力分级标准。复选标记表示智能体系统拥有相应的能力，无需每次更新人工干预。 _Imp_表示改进机制。 Imp Modification 需要进行功能上有意义的更改，从而影响后续的改进周期。 _通用性_指的是 递归自我改进（Recursive Self-Improvement） 的领域通用性，而不是单独的任务性能通用性。

|~~级别~~|~~类别~~|程序
|自动改进
i|ment 循环
|递归
i|域
|
|---|---|---|---|---|---|---|---|
|||提议者|修改者|验证者|Imp 修改|通用性|
|L1|手动改进||||||
|L2|协助
改进|✓|||||
|L3|程序化
自我改进（Self-Improvement）|✓|✓|✓|||
|L4|有界递归
自我改进（Self-Improvement）|✓|✓|✓|✓||
|L5|一般递归
自我改进（Self-Improvement）|✓|✓|✓|✓|✓|

五个级别的定义如下：

- (L1) 手动改进。 Agent系统不具备自主改进能力。它以固定的形式部署和执行，每次更改都需要外部开发和部署过程。

- (L2) 辅助改进。智能体系统可以提出候选修改或提供诊断证据，但人类仍然负责验证和应用实质性更改。改进机制_Imp_是由人类而不是智能体系统维护的。因此，L2 被视为 自我改进（Self-Improvement） 的前身，而不是它的一个实例。

- (L3) 程序化 自我改进（Self-Improvement）。智能体系统可以自主地提出、应用和验证对其操作组件的修改，例如基础模型（Foundation Model）、智能体运行支撑框架（Harness）、数据系统或训练器。这些修改可能是实质性的并且可能跨越多个组件。然而，管理这些修改的改进机制_Imp_仍然是固定的或外部维护的。因此，L3 是程序化的：系统控制改进的执行，但尚未修改控制后续改进周期的机制 Imp。

- (L4) 有界 递归自我改进（Recursive Self-Improvement）。智能体系统不仅可以提出候选修改、验证其效果并应用已验证的修改，还可以重写自己的改进机制，这使得改进机制具有自我参照性。各个步骤不需要修改每个组件，但整个过程支持在有限域内持续的长期进展。

- (L5) 将军 递归自我改进（Recursive Self-Improvement）。 L5保留了L4的自主性、自我参考性和长期进步性。除此之外，其改进能力可以有效地跨广泛且不断发展的任务领域转移，而不是局限于固定的基准、

狭窄的任务系列或受限的操作环境。这种普遍性标志着质的转变：系统的改进能力不再与任何特定的问题空间相关，从而为不断扩大的领域范围提供了改进的可能性。

综上所述，该评分标准将手动改进、辅助改进、程序化 自我改进（Self-Improvement）、有界 RSI 和一般 RSI 分开

。它为比较现有智能体系统和确定当前系统在通用 RSI 方面的进展提供了共享基础。

### 2.3. Unified Research Framework for Agent System Self-Improvement

The preceding subsections define the agent-system components, self-improvement and RSI, and the grading standard for self-improvement capability. Based on these definitions, this subsection presents a unified research framework for analyzing self-improving agent systems as a coherent whole. The subsection first defines the scope of the research framework, then discusses the self-improvement process, and finally summarizes the main advantages of this framework.

#### 2.3.1. Definition and Formulation

We define the unified research framework as a state-based model of the objects, dependencies, and update process involved in agent system self-improvement. It focuses on two key aspects: the scope of autonomous improvement and the mechanism of improvement. In other words, the framework specifies what can be improved and how improvement proceeds.

The scope of autonomous improvement specifies which parts of the agent system can be inspected, modified, and validated by the system itself. In this framework, the scope covers the states of all system components, while improvements to these states are constrained by their dependencies.

System state.

Building on the agent-system definition in Section 2.1.1 and Figure 1, we represent an agent system at iteration $t$ as the following revisable state:

$$x_t = (\mathcal{M}_t, \mathcal{H}_t, \mathcal{D}_t, \mathcal{T}_t, Imp_t) \in \mathcal{X}, \tag{2}$$

where $\mathcal{M}_t$, $\mathcal{H}_t$, $\mathcal{D}_t$, and $\mathcal{T}_t$ denote the current foundation model, agent harness, agent data system, and agent trainer, respectively. $Imp_t$ denotes the current improvement mechanism, and $\mathcal{X}$ contains the possible agent system states.

Operational dependencies.

The five components of $x_t$ in Equation 2 do not operate independently. For this reason, improving a state in $x_t$ requires considering how that change affects, and is limited by, the other components of the system. Figure 1 illustrates that the five components of $x_t$ are mutually dependent at the system level. During task execution, the harness and the foundation model form the acting loop. The *harness* $\mathcal{H}_t$ assembles a context from sources such as the user instructions, memory, skills, tools, and observations. Given the context, the foundation model $\mathcal{M}_t$ then outputs an action decision. The harness executes the decision in the environment, collects observations, and stores the resulting interaction trace.

The *data system* $\mathcal{D}_t$ converts accumulated traces and other artifacts into training or evaluation data. Given the training data, the *trainer* $\mathcal{T}_t$ then converts data into persistent updates of the *model* $\mathcal{M}_t$. The evaluation data, in turn, provide feedback signals that guide these updates and inform improvement decisions across these components. The *improvement mechanism* $Imp_t$ modifies the agent system as a whole by determining which of the five components should be changed and how, turning feedback into concrete updates across the system.

If these dependencies are ignored, a local improvement may leave the overall performance unchanged. This is likely when improving one component creates incompatibility with other components. It can also occur when the current bottleneck is distributed across several components rather than located in a single component.

Self-Improvement Process.

The improvement mechanism $Imp_t$ realizes the transition from the current system state $x_t$ to a revised state $x_{t+1}$ through four functional stages, as summarized in Figure 1. Within the unified framework, these stages operate over the coupled system state and its component dependencies.

1. The Diagnose stage analyzes end-to-end performance, component states, interface behavior, and feedback signals to identify the bottlenecks that currently constrain system-level improvement.

# 原文第 8 页 | 中文翻译初稿

8 共 70 个
2.3。 Agent系统统一研究框架自我改进（Self-Improvement）

前面的小节定义了智能体系统组件、自我改进（Self-Improvement）和 RSI，以及 自我改进（Self-Improvement）能力的分级标准。基于这些定义，本小节提出了一个统一的研究框架，用于将 自我改进（Self-Improving）智能体系统作为一个连贯的整体进行分析。该小节首先定义了研究框架的范围，然后讨论了自我改进（Self-Improvement）过程，最后总结了该框架的主要优点。
2.3.1.定义和表述

我们将统一的研究框架定义为智能体系统自我改进（Self-Improvement）中涉及的对象、依赖关系和更新过程的基于状态的模型。它重点关注两个关键方面：自主改进的范围和改进的机制。换句话说，该框架指定了可以改进的内容以及改进的进行方式。

自主改进的范围指定了智能体系统的哪些部分可以由系统本身检查、修改和验证。在这个框架中，范围涵盖了所有系统组件的状态，而对这些状态的改进则受到它们的依赖关系的限制。
系统状态。

基于第 2.1.1 节和图 1 中的智能体系统定义，我们将迭代 t 处的智能体系统表示为以下可修改状态：

$$x_t = (M_t, H_t, D_t, T_t, Impt) \in X, \quad (2)$$

其中 Mt 、 Ht 、 Dt 和 Tt 分别表示当前 基础模型（Foundation Model）、智能体 运行支撑框架（Harness）、智能体数据系统和智能体训练器。 _Impt_表示当前的改进机制，_X_包含可能的智能体系统状态。
操作依赖性。

公式 2 中 xt 的五个分量并不独立运行。因此，改善 xt 中的状态需要考虑该更改如何影响系统的其他组件，并受到系统其他组件的限制。图 1 说明了 xt 的五个组件在系统级别上是相互依赖的。任务执行期间，运行支撑框架（Harness） 和 基础模型（Foundation Model） 形成作用循环。 运行支撑框架（Harness） Ht 从用户指令、内存、技能、工具和观察等来源组装上下文。给定上下文后，基础模型（Foundation Model） Mt 然后输出行动决策。 运行支撑框架（Harness） 在环境中执行决策、收集观察结果并存储生成的交互跟踪。

数据系统 Dt 将累积的痕迹和其他工件转换为训练或评估数据。给定训练数据，trainer Tt 然后将数据转换为_model Mt_ 的持久更新。评估数据反过来提供反馈信号，指导这些更新并为这些组件的改进决策提供信息。 改进机制 Impt 通过确定五个组件中的哪一个应该更改以及如何更改，将智能体系统作为一个整体进行修改，将反馈转化为整个系统的具体更新。

如果忽略这些依赖性，局部改进可能会使整体性能保持不变。当改进一个组件导致与其他组件不兼容时，很可能会出现这种情况

。当当前瓶颈分布在多个组件而不是位于单个组件中时，也可能会发生这种情况。
自我改进（Self-Improvement） 过程。

改进机制_Impt_通过四个功能阶段实现从当前系统状态_xt_到修订状态_xt_+1的转变，如图1所示。在统一框架内，这些阶段对耦合系统状态及其组件依赖性进行操作。

1. 诊断阶段分析端到端性能、组件状态、接口行为和反馈信号，以识别当前制约系统级改进的瓶颈。

2. The Propose stage generates candidate modifications, each specifying the target subset of $x_t$ and the proposed change. A candidate may target one component or coordinate changes across multiple interdependent components.
3. The Evaluate stage assesses the effects of candidate modifications on the coupled system, including performance, cost, compatibility, and safety, and selects candidates for integration.
4. The Integrate stage applies accepted modifications to produce $x_{t+1}$ while preserving cross-component consistency and ensuring that the changes persist across subsequent improvement cycles.

Together, these four stages characterize the generic improvement dynamics of the unified framework. Different self-improving agent systems may implement each stage through different mechanisms.

#### 2.3.2. Advantages of the Unified Framework

The unified framework has three main advantages.

First, it provides a common representation for research threads that are often studied separately. This makes it easier to compare methods that otherwise appear in separate discussions of harness design, data construction, training, or self-modification. Each existing paradigm can be characterized as a special case of the general update rule $x_{t+1} = Imp_t(x_t)$. A static agent system corresponds to the degenerate case where the system state remains unchanged, i.e., $x_{t+1} = x_t$. A fixed-mechanism agent system applies a rule-based $Imp_t(\cdot)$ that remains fixed and falls outside the autonomous modification scope. Finally, an RSI agent system represents the full realization of the framework, in which $Imp_t(\cdot)$ acts on the entire system state, including itself.

Second, covering all system components in this framework helps locate and eliminate system-level bottlenecks. End-to-end performance alone can show that a system has failed, but it often cannot explain why the failure occurs. Component-level evidence can identify local weaknesses, but it may miss failures caused by interface mismatch or by constraints that span multiple components [270,291]. By placing all components in one framework, the analysis can determine whether a proposed improvement changes the actual bottleneck or only improves an isolated part of the system.

Third, modeling the dependencies among system components helps clarify cross-component causality. This is essential for multi-component co-evolution, where a modification in one component often causes indirect, widespread effects across the system. Identifying the scope of impact along these dependency paths is necessary both for evaluating the effectiveness of the improvement and for achieving long-term, sustainable co-evolution. The framework helps distinguish a multi-component co-evolution system from a system that merely optimizes individual components in isolation.

## 3. Agent Harness Self-Improvement

As defined in Section 2, the agent harness $\mathcal{H}$ is the non-parametric execution layer that determines what the model can observe, what actions it can invoke, and how observations and actions are organized into goal-directed workflows. Harness self-improvement studies persistent changes to this layer, where interaction traces, evaluation feedback, and execution failures are converted into durable changes in external state, action interfaces, prompts, workflows, or executable scaffolds. The central idea is that an agent can improve by changing the conditions under which the foundation model is invoked, rather than by changing the model parameters themselves. This makes the harness a natural substrate for early agent self-improvement: it can accumulate experience, reshape tool use and coordination, and repair parts of the agent's operating procedure while the underlying model remains fixed. Its relevance to RSI depends on what becomes mutable. When only harness modules are updated by a mostly fixed procedure, the result is usually module-level self-improvement; when the scaffold or improvement procedure itself becomes a target of modification, harness evolution begins to approach recursive self-improvement. Figure 3 situates these cases within the scope of this section. We first examine persistent changes confined to individual harness modules, then changes to

# 原文第 9 页 | 中文翻译初稿

9 共 70 个

2. Propose 阶段生成候选修改，每个修改指定 xt 的目标子集和建议的更改。候选者可以针对一个组件或协调多个相互依赖的组件之间的更改。

3. 评估阶段评估候选修改对耦合系统的影响，包括性能、成本、兼容性和安全性，并选择候选集成。

4. 集成阶段应用已接受的修改来生成 xt +1，同时保持跨组件一致性并确保更改在后续改进周期中持续存在。

这四个阶段共同描述了统一框架的一般改进动态。不同的自我改进（Self-Improving）智能体系统可以通过不同的机制来实现每个阶段。
2.3.2.统一框架的优点

统一框架具有三个主要优点。

首先，它为经常单独研究的研究线索提供了通用的表示。这样可以更轻松地比较 运行支撑框架（Harness）设计、数据构建、训练或自我修改的单独讨论中出现的方法。每个现有范例都可以被描述为一般更新规则 xt +1 = Impt ( xt ) 的特殊情况。静态智能体系统对应于系统状态保持不变的简并情况，即 xt +1 = xt 。固定机制智能体系统应用基于规则的 Impt ( · )，该规则保持固定并超出自主修改范围。最后，RSI 智能体系统代表了该框架的完整实现，其中 Impt ( · ) 作用于整个系统状态，包括其自身。

其次，覆盖该框架中的所有系统组件有助于定位和消除系统级瓶颈。仅端到端性能可以表明系统发生了故障，但它通常无法解释发生故障的原因。组件级证据可以识别局部弱点，但可能会错过由于接口不匹配或跨越多个组件的约束而导致的故障[270,291]。通过将所有组件放置在一个框架中，分析可以确定所提出的改进是改变实际瓶颈还是仅改进系统的孤立部分。

第三，对系统组件之间的依赖关系进行建模有助于阐明跨组件的因果关系。这对于多组件协同进化至关重要，其中一个组件的修改通常会在整个系统中引起间接、广泛的影响。确定这些依赖路径的影响范围对于评估改进的有效性和实现长期、可持续的共同进化都是必要的。该框架有助于区分多组件协同进化系统和仅单独优化各个组件的系统。
3.智能体运行支撑框架（Harness） 自我改进（Self-Improvement）

如第 2 节中所定义，智能体 运行支撑框架（Harness） H 是非参数执行层，它确定模型可以观察什么、可以调用什么操作以及如何将观察和操作组织到目标导向的工作流中

。 运行支撑框架（Harness） 自我改进（Self-Improvement） 研究该层的持久变化，其中交互跟踪、评估反馈和执行失败被转换为外部状态、操作界面、提示、工作流或可执行支架的持久变化。中心思想是智能体可以通过更改调用 基础模型（Foundation Model） 的条件来改进，而不是通过更改模型参数本身。这使得 运行支撑框架（Harness） 成为早期智能体 自我改进（Self-Improvement） 的天然基础：它可以积累经验，重塑工具的使用和协调，并在底层模型保持固定的情况下修复智能体的部分操作流程。它与 RSI 的相关性取决于什么会变得可变。当仅通过基本固定的过程更新 运行支撑框架（Harness） 模块时，结果通常是模块级 自我改进（Self-Improvement）；当支架或改进程序本身成为修改目标时，运行支撑框架（Harness）的进化开始接近递归自我改进（Recursive Self-Improvement）。图 3 将这些情况归入本节的范围内。我们首先检查仅限于单个 运行支撑框架（Harness） 模块的持久更改，然后更改为

orchestration and architecture, and finally self-referential modifications to executable harness code and the mechanism governing future updates.

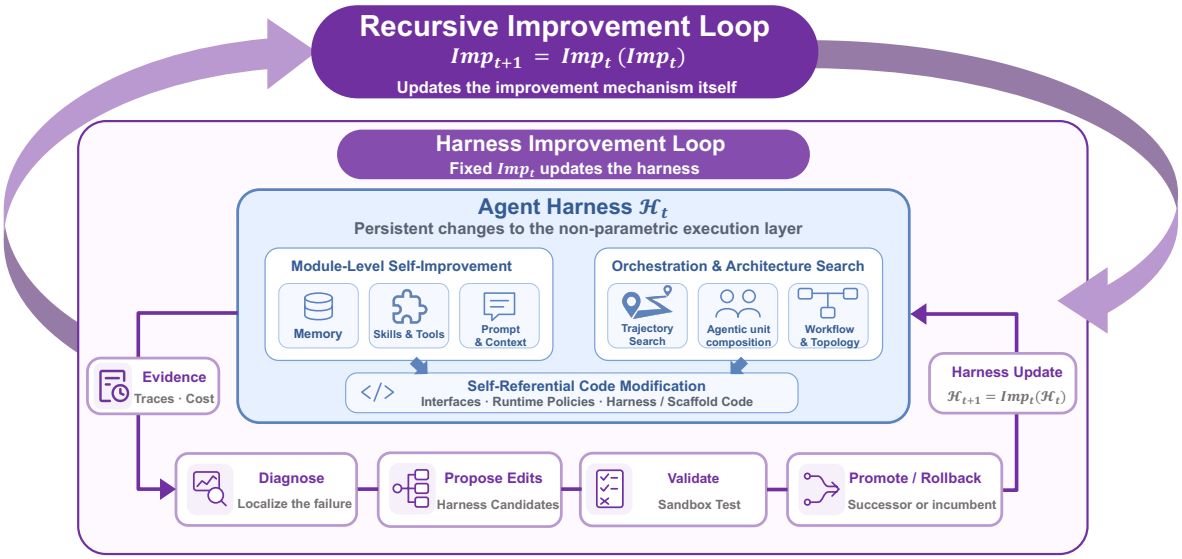



**Figure 3.** Conceptual framework of an operational agent harness, harness self-improvement, and improvement-mechanism evolution. The innermost layer depicts how an agent harness $\mathcal{H}_t$ organizes agentic units and their supporting modules through an orchestration structure. The harness-improvement loop uses execution evidence to diagnose problems, propose edits, validate candidates in a sandbox, and either promote a successor harness $\mathcal{H}_{t+1}$ or roll back. At the outer level, improvement histories and validation outcomes provide evidence for revising and meta-validating the improvement mechanism $Imp_t$ itself, yielding $Imp_{t+1}$ when a candidate is promoted.

**Table 2.** Representative self-improvement methods in agent harness evolution.

| Work | Self-Improvement Process | | | Level |
|---|---|---|---|---|
| | Object | Evidence | Mechanism | |
| **Module-Level Self-Improvement** | | | | |
| **Reflexion** [26] | Experience memory | Trace + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **SWE-Exp** [28] | Experience memory | Trace + Outcome label | Experience reuse | L3 |
| **ReasoningBank** [29] | Experience memory | Trace + Evaluator judgment | Experience reuse | L3 |
| **FLEX** [30] | Experience memory | Trace + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **Decocted Experience** [31] | Experience memory | Trace + Outcome label | Experience reuse | L3 |
| **Live-Evo** [41] | Experience memory | Trace + Outcome label | Experience reuse | L3 |
| **Memory Transfer Learning** [45] | Experience memory | Cross-domain feedback | Experience reuse | L3 |
| **Tool Makers** [57] | Tool library | Execution result + Outcome label | Artifact synthesis | L3 |
| **ToolCoder** [58] | Tool library | Execution result | Artifact synthesis | L3 |
| **SkillWeaver** [60] | Skill library | Trace + Execution result + Evaluator judgment | Artifact synthesis | L3 |
| **SkillX** [61] | Skill library | Trace + Outcome label + Evaluator judgment | Artifact synthesis | L3 |
| **Workflow-to-Skill** [62] | Skill library | Trace | Artifact synthesis | L3 |
| **SkillFoundry** [66] | Skill library | Execution result + Outcome label + Evaluator judgment | Artifact synthesis | L3 |
| **OpenSkill** [65] | Skill library | Execution result + Outcome label + Evaluator judgment | Artifact synthesis | L3 |
| **SkillOS** [69] | Skill library | Trace + Evaluator judgment | Policy learning | L3 |
| **SkillComposer** [74] | Skill library | Trace + Outcome label | Policy learning | L3 |
| **Programmatic Skill Networks** [75] | Skill library | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **SkillAxe** [76] | Skill library | Evaluator judgment | Diagnostic repair | L3 |
| **SkillAudit** [77] | Skill library | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **SkillGen** [78] | Skill library | Trace + Outcome label + Evaluator judgment | Artifact synthesis | L3 |
| **CoEvoSkills** [79] | Skill library | Execution result + Outcome label + Evaluator judgment | Artifact synthesis | L3 |
| **SkillSmith** [83] | Skill-tool library | Trace + Execution result + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **Confucius Code Agent** [85] | Skill-tool library | Trace + Execution result + Evaluator judgment | Artifact synthesis | L3 |
| **Dynamic Cheatsheet** [110] | Context playbook | Evaluator judgment | Experience reuse | L3 |
| **ACE** [111] | Context playbook | Trace + Execution result + Evaluator judgment | Experience reuse | L3 |

# 原文第 10 页 | 中文翻译初稿

70 中的 10

编排和架构，最后是对可执行 运行支撑框架（Harness） 代码的自引用修改以及管理未来更新的机制。

递归改进循环
□□□□%□ = □□□□ (□□□□)
更新改进机制本身
运行支撑框架（Harness） 改进循环
修复□□□□更新运行支撑框架（Harness）
智能体运行支撑框架（Harness）□□
对非参数执行层的持久更改
模块级自我改进（Self-Improvement）编排和架构搜索
内存技能和工具和上下文提示轨迹搜索组合智能体单元和拓扑工作流程
证据运行支撑框架（Harness） 更新
自引用代码修改
跟踪 · 成本接口 · 运行时策略 · 运行支撑框架（Harness） / 支架代码 □□"□ = □□□□(□□)
诊断建议编辑验证升级 /回滚
定位失败 运行支撑框架（Harness） 候选沙盒测试继任者或现任

图 3. 操作智能体 运行支撑框架（Harness）、运行支撑框架（Harness） 自我改进（Self-Improvement） 的概念框架以及改进机制的演变。最内层描述了智能体运行支撑框架（Harness）_Ht_如何通过编排结构组织智能体单元及其支持模块。 运行支撑框架（Harness） 改进循环使用执行证据来诊断问题、提出编辑建议、验证沙箱中的候选者，以及提升后继 运行支撑框架（Harness） Ht +1 或回滚。在外部层面，改进历史和验证结果为修改和元验证改进机制_Impt_本身提供了证据，当候选人晋升时产生_Impt_+1。

表 2. 智能体 运行支撑框架（Harness） 进化中的代表性 自我改进（Self-Improvement） 方法。

|工作||自我改进（Self-Improvement） 流程||级别|
|---|---|---|---|---|
||对象|证据|机制||
|||模块级|| 自我改进（Self-Improvement）|||
|反思[26]|体验记忆|轨迹+结果标签+评估者判断|体验重用|L3|
|SWE-Exp[28]|经验记忆|轨迹+结果标签|经验重用|L3|
|ReasoningBank[29]|经验记忆|轨迹+评估器判断|经验复用|L3|
|FLEX[30]|体验记忆|轨迹+结果标签+评估者判断|体验复用|L3|
|煎制经验[31]|经验记忆|痕迹+结果标签|经验复用|L3|
|Live-Evo[41]|体验记忆|轨迹+结果标签|体验重用|L3|
|记忆迁移学习[45]|体验记忆|跨域反馈|体验复用|L3|
|工具制作者[57]|工具库|执行结果+结果标签|工件合成|L3|
|ToolCoder[58]|工具库|执行结果|神器合成|L3|
|SkillWeaver[60]|技能库|轨迹+执行结果+Evaluator判断|神器合成|L3|

|SkillX[61]|技能库|轨迹+结果标签+Evaluator判断|神器合成|L3|
|工作流程到技能[62]|技能库|追踪|工件合成|L3|
|SkillFoundry[66]|技能库|执行结果+结果标签+
Evaluator判断|神器合成|L3|
|OpenSkill[65]|技能库|执行结果+结果标签+
Evaluator判断|神器合成|L3|
|SkillOS[69]|技能库|Trace + Evaluator判断|策略学习|L3|
|SkillComposer[74]|技能库|轨迹+结果标签|策略学习|L3|
|程序化技能网络[75]|技能库|轨迹+结果标签+评估者判断|诊断修复|L3|
|SkillAxe[76]|技能库|评估者判断|诊断修复|L3|
|SkillAudit[77]|技能库|轨迹+结果标签+Evaluator判断|诊断修复|L3|
|SkillGen[78]|技能库|轨迹+结果标签+Evaluator判断|神器合成|L3|
|CoEvoSkills[79]|技能库|执行结果+结果标签+
Evaluator判断|神器合成|L3|
|SkillSmith[83]|技能工具库|Trace + 执行结果 + 结果
标签 + Evaluator 判断|基于搜索的优化|L3|
|孔子代码智能体[85]|技能工具库|轨迹+执行结果+Evaluator判断|神器合成|L3|
|动态备忘单[110]|上下文剧本|评估者判断|体验重用|L3|
|ACE[111]|上下文剧本|Trace + 执行结果 + Evaluator 判断|体验复用|L3|

| Work | Object | Evidence | Mechanism | Level |
|---|---|---|---|---|
| **SCOPE** [112] | Context playbook | Trace + Execution result + Evaluator judgment | Experience reuse | L3 |
| **Reflective Context Learning** [113] | Context playbook | Trace + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **Unified Context Evolution** [114] | Context playbook | Trace + Outcome label + Improvement statistics | Experience reuse | L3 |
| **KACE** [115] | Context playbook | Trace + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **MEMO** [117] | Context playbook | Trace + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **SEEK-SQL** [118] | Context playbook | Trace + Execution result + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **GEPA** [103] | Multi-stage prompts | Trace + Execution result + Outcome label | Search-based optimization | L3 |
| **Trace2Policy** [119] | Decision-rule library | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **Learning to Self-Evolve** [121] | Context-editor weights | Trace + Outcome label | Policy learning | L3 |
| **SePO** [120] | Task prompt + Improver prompt | Outcome label | Search-based optimization + Meta-optimization | L4 |
| **Orchestration-Level Self-Improvement** | | | | |
| **AgentGA** [129] | Trajectory orchestration | Outcome label + Improvement statistics | Search-based optimization | L3 |
| **SWE-Replay** [136] | Trajectory memory | Trace + Execution result + Outcome label | Experience reuse | L3 |
| **Log-Augmented Generation** [140] | Trajectory memory | Trace | Experience reuse | L3 |
| **FailureMem** [143] | Trajectory memory | Execution result + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **EvoRepair** [144] | Trajectory memory | Trace + Outcome label + Evaluator judgment | Experience reuse | L3 |
| **EvoAgent** [152] | Agent composition | Evaluator judgment | Search-based optimization | L3 |
| **ADAS** [153] | Agent system code | Execution result + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **EvoMAS** [158] | Agent composition + Experience memory | Trace + Execution result + Evaluator judgment | Search-based optimization | L3 |
| **EVOCHAMBER** [159] | Agent composition + Experience memory | Cross-domain feedback | Search-based optimization | L3 |
| **MermaidFlow** [173] | Workflow structure | Execution result + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **EvoAgentX** [174] | Workflow structure | Outcome label | Search-based optimization | L3 |
| **EvoFlow** [175] | Workflow structure | Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **SEW** [176] | Workflow structure | Outcome label | Search-based optimization | L3 |
| **HyEvo** [179] | Workflow structure | Execution result + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **AdaptFlow** [180] | Workflow structure | Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **JudgeFlow** [184] | Workflow structure | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **Lean4Agent / LeanEvolve** [232] | Workflow structure | Trace + Execution result + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **EvoFSM** [233] | Workflow structure | Trace + Evaluator judgment | Diagnostic repair | L3 |
| **ScoreFlow** [171] | Workflow-manager weights | Outcome label | Policy learning | L3 |
| **Learning to Compose** [181] | Workflow-manager weights | Outcome label | Policy learning | L3 |
| **Workflow-R1** [183] | Workflow-manager weights | Execution result + Outcome label | Policy learning | L3 |
| **Learning to Hand Off** [189] | Workflow-manager weights | Outcome label | Policy learning | L3 |
| **Self-Referential Code Modification** | | | | |
| **Life-Harness** [212] | Harness policy | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **HarnessFix** [213] | Harness policy | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **Milkyway** [214] | Harness policy | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **Self-Harness** [215] | Harness policy | Trace + Execution result + Outcome label | Diagnostic repair | L3 |
| **POLARIS** [216] | Harness policy | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **DemoEvolve** [230] | Runtime scaffold | Trace + Execution result + Outcome label | Search-based optimization | L3 |
| **SIGA** [231] | Runtime scaffold | Trace + Outcome label | Artifact synthesis | L3 |
| **HarnessForge** [360] | Runtime scaffold + Model weights | Trace + Outcome label + Evaluator judgment | Diagnostic repair + Policy learning | L3 |
| **Agentic Harness Engineering** [22] | Runtime scaffold + Evolution history | Trace + Outcome label + Evaluator judgment | Diagnostic repair | L3 |
| **Meta-Harness** [217] | Runtime scaffold + Evolution history | Trace + Outcome label | Search-based optimization | L3 |
| **AgentFlow** [218] | Runtime scaffold + Evolution history | Trace + Execution result + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **Adaptive Auto-Harness** [239] | Runtime scaffold + Evolution history | Trace + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **Group-Evolving Agents** [236] | Agent source code + Evolution history | Trace + Outcome label + Evaluator judgment | Search-based optimization | L3 |
| **Gödel Agent** [226] | Agent source code + Improvement mechanism | Execution result + Outcome label | Meta-optimization | L4 |
| **SICA** [227] | Agent source code + Improvement mechanism | Execution result + Outcome label + Evaluator judgment | Search-based optimization + Meta-optimization | L4 |
| **Darwin Gödel Machine** [234] | Agent source code + Improvement mechanism | Trace + Outcome label + Evaluator judgment | Search-based optimization + Meta-optimization | L4 |
| **Huxley-Gödel Machine** [235] | Agent source code + Improvement mechanism | Outcome label + Improvement statistics | Search-based optimization | L4 |
| **Red Queen Gödel Machine** [361] | Agent source code + Evaluation mechanism | Outcome label + Evaluator judgment + Improvement statistics | Search-based optimization + Meta-optimization | L4 |

# 原文第 11 页 | 中文翻译初稿

70 中的 11

|工作|对象|证据|机制|级别|
|---|---|---|---|---|
|SCOPE[112]
反思性情境学习[113]|情境剧本
情境剧本|追踪 + 执行结果 + 评估者判断
追踪 + 结果标签 + 评估者判断|经验重用
 经验重用|L3
L3|
|统一情境演进[114]|情境剧本|追踪+结果标签+改进统计|体验复用|L3|
|KACE[115]|情境剧本|追踪+结果标签+评估者判断|体验复用|L3|
|MEMO[117]|上下文剧本|追踪+结果标签+评估者判断
|体验重用|L3|
|SEEK-SQL[118]|Context playbook|Trace + 执行结果 + Outcome
标签 + Evaluator 判断|体验复用|L3|
|GEPA[103]|多级提示|追踪+执行结果+结果标签|基于搜索的优化|L3|
|
Trace2Policy[119]|
决策规则库|Trace + 结果标签 + Evaluator 判断|
 诊断修复|L3|
|学习自我进化[121]|上下文编辑器权重|跟踪+结果标签|策略学习|L3|
|SePO[120]|
任务提示 + 改进提示|结果标签|
基于搜索的优化
|L4|
||||+ 元优化||
||编排级别|I 自我改进（Self-Improvement）|||
|AgentGA[129]|轨迹编排|结果标签+改进统计|基于搜索的优化|L3|
|SWE-Replay[136]|轨迹记忆|轨迹+执行结果+结果标签|体验复用|L3|
|对数增强生成[140]|轨迹记忆|轨迹
|体验重用|L3|
|FailureMem[143]|轨迹记忆|执行结果+结果标签+
评估者判断|经验复用|L3|
|EvoRepair[144]|轨迹记忆|
轨迹+结果标签+评估者判断|体验复用|L3|
|EvoAgent[152]|智能体组成|评估器判断
|基于搜索的优化|L3|
|ADAS[153]|Agent系统代码|执行结果+结果标签+
Evaluator判断|基于搜索的优化|L3|
|EvoMAS[158]|Agent 组合 +

Ei|Trace + 执行结果 + Evaluator 判断|基于搜索的优化|L3|
|EVOCHAMBER[159]|体验记忆
Agent组合+
|跨域反馈|基于搜索的优化|L3|
||体验记忆||||
|MermaidFlow[173]|工作流程结构|执行结果+结果标签+
Evaluator判断|基于搜索的优化|L3|
|EvoAgentX[174]|工作流程结构|
结果标签|基于搜索的优化|L3|
|EvoFlow[175]
|工作流程结构
||结果标签+Evaluator判断
|基于搜索的优化
|L3
|
|SEW[176]|工作流程结构|结果标签
|基于搜索的优化|L3|
|HyEvo[179]|工作流程结构|执行结果+结果标签+
Evaluator判断|基于搜索的优化|L3|
|AdaptFlow[180]|工作流程结构|
结果标签+Evaluator判断|基于搜索的优化|L3|
|JudgeFlow[184]|工作流程结构|轨迹+结果标签+Evaluator判断
|诊断修复|L3|
|Lean4Agent / LeanEvolve[232]|工作流程结构|Trace + 执行结果 + 结果
标签 + Evaluator 判断|诊断修复|L3|
|EvoFSM[233]
|工作流程结构
|
Trace + Evaluator 判断
|诊断修复
|L3|
|ScoreFlow[171]|工作流管理器权重|结果标签|策略学习|L3|
|学习撰写[181]
Workflow-R1[183]|工作流程管理器权重
工作流程管理器

权重|结果标签
执行结果+结果标签|策略学习
策略学习|L3
L3|
|学习交接[189]|工作流管理器权重|结果标签|策略学习|L3|
||自参考|代码修改|||
|Life-运行支撑框架（Harness）[212]

|运行支撑框架（Harness）策略
|追踪+结果标签+评估者判断
|诊断修复
|L3
|
|HarnessFix[213]|运行支撑框架（Harness） 策略|追踪 + 结果标签 + Evaluator 判断|诊断修复|L3|
|银河[214]|运行支撑框架（Harness）策略|追踪+结果标签+评估者判断|诊断修复|L3|
|
自运行支撑框架（Harness）[215]|
运行支撑框架（Harness）策略|
跟踪+执行结果+结果标签|
诊断修复|L3|
|POLARIS[216]|运行支撑框架（Harness）策略|追踪+结果标签+评估者判断|诊断修复|L3|
|DemoEvolve[230]|运行时脚手架|跟踪+执行结果+结果标签|基于搜索的优化|L3|
|SIGA[231]|运行时脚手架|跟踪+结果标签|工件合成|L3|
|HarnessForge[360]
智能体 运行支撑框架（Harness） 工程[22]|运行时脚手架 + 模型权重
运行时脚手架 + 演化历史|追踪 + 结果标签 + 评估者判断
追踪 + 结果标签 + 评估者判断|诊断修复 + 策略学习
诊断修复|L3
L3|
|Meta-运行支撑框架（Harness）[217]|运行时脚手架 + 演化历史
|追踪 + 结果标签
|基于搜索的优化|L3|
|AgentFlow[218]|运行时脚手架 + 演化历史|Trace + 执行结果 + 结果
标签 + Evaluator 判断|基于搜索的优化|L3|
|自适应自动-运行支撑框架（Harness）[239]|运行时支架+演化历史|
跟踪+结果标签+评估器判断|基于搜索的优化|L3|
|群体进化智能体[236]|智能体源代码+
进化历史|轨迹+结果标签+评估器判断|基于搜索的优化|L3|
|Gödel Agent[226]|y
Agent源代码 +|执行结果 + 结果标签|元优化|L4|
||改进机制||||
|SICA[227]|Agent 源代码 +|执行结果 + 结果标签 +
Elt dt|基于搜索的优化|L4|
||改进机制|vauaor jugmen|+元优化||
|i dl hi|
A d||
Shbdiii|L4|
|Darwn Göe Macne[234]|gent源码+
改进机制|追踪+结果标签+Evaluator判断|earc-ase optmzaton
+元优化||
|Huxley-Gödel Machine[235]|Agent源码+

改进机制|结果标签+改进统计|基于搜索的优化|L4|
|||结果
标签
+
评估器|||
|红皇后哥德尔机[361]|Agent源码+
评估机制|判断+改进统计|基于搜索的优化
+元优化|L4|

| Work | Object | Evidence | Mechanism | Level |
|---|---|---|---|---|
| **HyperAgents** [23] | Agent source code + Improvement mechanism | Outcome label + Improvement statistics | Search-based optimization + Meta-optimization | L4 |
| **Harnessing Agentic Evolution** [237] | Improvement mechanism | Trace + Execution result + Outcome label + Improvement statistics | Meta-optimization | L4 |
| **EvoX** [240] | Improvement mechanism | Outcome label + Improvement statistics | Meta-optimization | L4 |
| **ANCHOR** [245] | Governance mechanism | Safety oversight | Governed commit | L3 |
| **Statistical Gödel Machine** [241] | Governance mechanism | Safety oversight | Governed commit | L3 |
| **ANNEAL** [243] | Governance mechanism | Safety oversight | Diagnostic repair + Governed commit | L3 |

### *3.1. Module-Level Self-Improvement*

For a fixed agent to improve across tasks, knowledge accumulated from earlier work can be reused to guide its subsequent behavior. At the module level within the harness, that carry-over is localized within memory, skill and tool, or prompt and context modules: experience is written into memory, stabilized as reusable skills or tools, or folded back into the prompts and contexts that condition later decisions. The central move is to externalize learning into artifacts that a mostly fixed harness can inspect, retrieve, revise, and reuse, rather than to derive them from scratch on each new task. Although this is the simplest form of self-improvement, it allows improvements to persist across runs and thereby supports more extensive forms of harness evolution.

#### 3.1.1. Memory Self-Modification

Memory self-modification refers to improving an agent by persistently changing the external memory module that conditions later executions, without updating the model parameters. This is different from static RAG or simple long-context retrieval: the memory is produced by the agent's own interaction history and is reused across later tasks. Early work establishes this non-parametric form of self-improvement by turning one-off execution traces into reusable textual experience. Reflexion writes task feedback into verbal self-reflections stored in an episodic memory buffer, allowing later trials to avoid previously observed mistakes [26]. ExpeL expands the same idea from trial-level reflection to experiential learning, extracting natural-language insights from collected trajectories and recalling both insights and examples at inference time [27]. Later systems make the experience artifact more specialized. SWE-Exp builds an experience bank from successful and failed software-repair trajectories so that coding agents can reuse repair patterns [28]; ReasoningBank distills successful and failed interaction histories into reasoning memory for later strategy selection [29]; FLEX organizes continual reflections into an inheritable experience library whose accumulated contents improve later agents [30]. Decocted Experience further emphasizes that useful memory is not a full transcript but a distilled, coherent, and retrievable essence of prior experience [31]. In this first stage, the memory module improves mainly by changing its contents: lessons, strategies, and procedural hints accumulate outside the fixed model and condition future behavior.

As agents operate over longer horizons, the central problem shifts from storing more experience to organizing, compressing, and maintaining it. A flat list of transcripts or reflections quickly creates redundancy, conflict, retrieval noise, and context pressure. MemGPT provides an early architectural anchor by treating LLM context as a memory hierarchy, where information can be moved between working context and archival storage [32]. A-MEM makes the store more self-organizing through Zettelkasten-style notes, dynamic indexing, and links that allow new memories to update the attributes and connections of older ones [33]. SimpleMem compresses interaction histories into structured semantic memory units, synthesizes related memories online, and plans retrieval according to the current intent [34]. RecMem adds a timing principle for consolidation: repeated semantic patterns, rather than every incoming interaction, trigger the extraction of episodic or semantic memory [35]. Graph and document structures offer another way to keep long-term memory usable. EXG organizes successes and failures as an experience graph that can grow during execution and be reused later [36]; FluxMem models memory as continuously evolving connectivity, repairing useful links, pruning interference, and distilling recurrent trajectories into procedural circuits [37]; Infini Memory

# 原文第 12 页 | 中文翻译初稿

70 中的 12

|工作|对象|证据|机制|级别|
|---|---|---|---|---|
|HyperAgents[23]|Agent源码+
改进机制|结果标签+改进统计|基于搜索的优化
+元优化|L4|
|利用Agentic Evolution[237]|改进机制|追踪+执行结果+结果
标签+改进统计|元优化|L4|
|EvoX[240]|改进机制|结果标签+改进统计|元优化|L4|
|ANCHOR[245]|治理机制|安全监督|治理提交|L3|
|统计哥德尔机[241]|治理机制|安全监督|治理提交|L3|
|ANNEAL[243]|治理机制|安全监督|诊断修复+治理提交|L3|

3.1。模块级 自我改进（Self-Improvement）

对于跨任务改进的固定智能体，可以重用从早期工作中积累的知识来指导其后续行为。在 运行支撑框架（Harness）内的模块级别，该结转被本地化在内存、技能和工具或提示和上下文模块中：经验被写入内存，稳定为可重用的技能或工具，或折叠回提示和上下文中，以影响以后的决策。核心举措是将学习外化为基本上固定的 运行支撑框架（Harness） 可以检查、检索、修改和重用的工件，而不是在每个新任务中从头开始派生它们。虽然这是 自我改进（Self-Improvement） 最简单的形式，但它允许在运行中持续改进，从而支持更广泛形式的 运行支撑框架（Harness） 演变。

3.1.1.内存自我修改

内存自修改是指通过持续更改影响后续执行的外部内存模块来改进智能体，而不更新模型参数。这与静态 RAG 或简单的长上下文检索不同：记忆是由智能体自己的交互历史记录生成的，并在以后的任务中重用。早期工作通过将一次性执行跟踪转换为可重用的文本体验来建立 自我改进（Self-Improvement） 的这种非参数形式。反思将任务反馈写入存储在情景记忆缓冲区中的口头自我反思，从而允许以后的试验避免以前观察到的错误[26]。 ExpeL 将相同的想法从试验级反思扩展到体验式学习，从收集的轨迹中提取自然语言见解，并在推理时回忆见解和示例 [27]。后来的系统使体验工件更加专业化。 SWE-Exp 根据成功和失败的软件修复轨迹构建了一个经验库，以便编码智能体可以重用修复模式 [28]； ReasoningBank 将成功和失败的交互历史提炼到推理记忆中，以供以后的策略选择[29]； FLEX 将持续的反思组织成一个可继承的经验库，其积累的内容可以改进后来的智能体 [30]。 《煎经验》进一步强调，有用的记忆不是完整的记录，而是先前经验的提炼出来的、连贯的、可检索的本质[31]。在第一阶段，记忆模块主要通过改变其内容来改进：教训、策略和程序提示在固定模型之外积累并影响未来的行为。

随着智能体的操作范围越来越广，中心问题从存储更多经验转变为组织、压缩和维护经验。简单的抄本或反思列表很快就会产生冗余、冲突、检索噪音和上下文压力。 MemGPT 通过将 LLM 上下文视为内存层次结构提供了早期的架构锚点，其中信息可以在工作上下文和档案存储之间移动 [32]。 A-MEM 通过 Zettelkasten 风格的注释、动态索引和链接使存储更加自组织，允许新记忆更新旧记忆的属性和连接 [33]。SimpleMem 将交互历史压缩为结构化语义记忆单元，在线合成相关记忆，并根据当前意图计划检索[34]。RecMem 添加了巩固的计时原则：重复的语义模式，而不是每次传入的交互，都会触发情景或语义记忆的提取 [35]。图形和文档结构提供了另一种保持长期记忆可用的方法。 EXG 将成功和失败组织为经验图，该图可以在执行过程中增长并在以后重用[36]； FluxMem 将记忆建模为不断发展的连接、修复有用的链

接、修剪干扰以及将循环轨迹提炼为程序电路 [37]；无限内存

maintains topic-structured documents that aggregate evidence and revise facts over time [38]. DCPM separates fast belief-revision writes from slower schema, intention, and cross-domain pattern induction [39], while Auto-Dreamer decouples online acquisition from offline consolidation through a learned consolidator that replaces redundant memory regions with compact abstractions [40]. Together, these systems turn memory from a collection of records into an evolving long-term state.

A further step is to make memory writing, retrieval, and reuse adaptive. Long-running agents must decide which memories remain useful, which memories deserve trust, and which abstractions transfer to new tasks. Live-Evo treats deployment as online memory evolution: an Experience Bank records what happened, a Meta-Guideline Bank summarizes reusable guidance, and feedback reinforces helpful memories while reducing the influence of stale or misleading ones [41]. MemQ introduces credit assignment by propagating Q-values backward through a memory provenance DAG, so memories that helped generate later useful memories receive delayed credit [42]. $R^2$-Mem applies a similar idea to memory search, evaluating successful and unsuccessful search trajectories and distilling them into reusable search experience [43]. AdaMEM addresses the timing of retrieval in long-horizon interaction by retrieving and synthesizing memory at each decision state instead of relying on a static episode-initial memory injection [44]. Memory Transfer Learning shows that cross-domain memory is most effective when it captures high-level debugging or validation principles rather than overly specific traces [45]. Other systems regulate the write and reuse process directly: TAME calibrates memory use through trust-aware executor-evaluator feedback [46], SAGE uses a novelty gate to decide whether newly extracted facts should be added, merged, ignored, or escalated to an LLM [47], and D-MEM routes memory updates through a fast/slow mechanism in which only high-surprise and high-utility inputs trigger deeper memory evolution [48]. These methods make clear that memory module self-improvement is not mere accumulation; it is a controlled process of selection, credit assignment, abstraction, calibration, and forgetting.

Recent work pushes the improvement target from memory contents and retrieval scores to the memory mechanism itself. MemEvolve decomposes agent memory systems into encoding, storage, retrieval, and management operations, and lets experiential knowledge and memory architecture co-evolve [49]. ALMA represents memory designs as executable code and uses a meta-agent to search over representations, storage rules, retrieval behavior, and update logic across domains [50]. M$^\star$ similarly treats the memory module as an executable program containing schema, logic, and instructions, producing task-specific memory programs with different structures [51]. EvolveMem focuses on retrieval architecture, using failure logs to diagnose weak retrieval configurations and adjust them with rollback safeguards [52]. MemPro broadens this view to the full memory construction-retrieval pipeline, maintaining a version tree of runnable memory-system implementations and creating improved versions from recurring failure modes [53]. On the write side, MemSkill treats extraction, consolidation, and pruning as selectable and evolvable memory skills [54], while CluE adapts memory-extraction prompts to heterogeneous task groups [55]. This frontier moves memory self-modification from the question of what the agent remembers to the question of how the memory module improves its own representation, storage, retrieval, and update procedures. Most systems still rely on an externally specified improvement loop, but the mutable object now includes key parts of the memory-improvement mechanism itself, making memory module self-improvement an important bridge from experience accumulation toward higher-level agent self-improvement.

#### 3.1.2. Skill and Tool Self-Improvement

In skill and tool self-improvement, reusable capabilities may take the form of executable code skills, function repositories, API wrappers, workflow routines, skill documents, skill contracts, or tool-use procedures. Early systems explore how agents can construct and accumulate these different forms of capability artifacts. CREATOR and Large Language Models as Tool Makers show that LLMs can synthesize tools or Python utility functions that separate abstract reasoning from concrete computation and are reusable in later calls [56,57]. ToolCoder organizes tool learning as code generation, execution feedback, and function-repository maintenance [58], while RefTool shows that tools can also be

# 原文第 13 页 | 中文翻译初稿

70 中的 13

维护主题结构的文档，随着时间的推移汇总证据并修改事实[38]。 DCPM 将快速的信念修正写入与较慢的模式、意图和跨域模式归纳分开[39]，而 Auto-Dreamer 通过学习的整合器将在线获取与离线整合分离，该整合器用紧凑的抽象替换冗余内存区域[40]。这些系统共同将记忆从记录集合转变为不断发展的长期状态。

下一步是使内存写入、检索和重用具有适应性。长时间运行的智能体必须决定哪些记忆仍然有用，哪些记忆值得信任，以及哪些抽象转移到新任务。 Live-Evo 将部署视为在线记忆演化：经验库记录发生的事情，元指南库总结可重用的指导，反馈强化有用的记忆，同时减少陈旧或误导性记忆的影响[41]。 MemQ 通过内存起源 DAG 向后传播 Q 值来引入 贡献归因（Credit Assignment），因此有助于生成以后有用内存的内存会收到延迟的信用 [42]。 R2 -Mem 将类似的想法应用于内存搜索，评估成功和不成功的搜索轨迹并将其提炼为可重用的搜索体验 [43]。 AdaMEM 通过在每个决策状态检索和合成内存来解决 长时程（Long-Horizon） 交互中的检索时序，而不是依赖于静态片段初始内存注入 [44]。内存迁移学习表明，当跨域内存捕获高级调试或验证原理而不是过于具体的跟踪时，它是最有效的[45]。其他系统直接调节写入和重用过程：TAME通过信任感知的执行器评估器反馈来校准内存使用[46]，SAGE使用新颖性门来决定是否应添加、合并、忽略新提取的事实或升级为LLM[47]，而D-MEM通过快速/慢速机制路由内存更新，其中只有高惊喜和高实用性输入才会触发更深层次的内存演化[48]。这些方法表明，存储模块自我改进（Self-Improvement）不是单纯的积累；它是一个选择、贡献归因（Credit Assignment）、抽象、校准和遗忘的受控过程。

最近的工作将改进目标从记忆内容和检索分数推向了记忆机制本身。 MemEvolve 将智能体内存系统分解为编码、存储、检索和管理操作，并让经验知识和内存架构共同进化[49]。 ALMA 将内存设计表示为可执行代码，并使用元智能体来跨域搜索表示、存储规则、检索行为和更新逻辑 [50]。 M_⋆_同样将内存模块视为包含模式、逻辑和指令的可执行程序，产生具有不同结构的特定于任务的内存程序[51]。 EvolveMem 专注于检索架构，使用故障日志来诊断薄弱的检索配置，并通过回滚保护措施对其进行调整 [52]

。 MemPro 将这种观点扩展到完整的内存构建检索管道，维护可运行内存系统实现的版本树，并从重复出现的故障模式中创建改进的版本 [53]。在写入方面，MemSkill 将提取、合并和修剪视为可选择和可进化的记忆技能 [54]，而 CluE 则将记忆提取提示适应异构任务组 [55]。这一前沿将记忆自我修改从智能体记住什么的问题转移到记忆模块如何改进其自身的表示、存储、检索和更新过程的问题。大多数系统仍然依赖于外部指定的改进循环，但可变对象现在包含了内存改进机制本身的关键部分，使得内存模块自我改进（Self-Improvement）成为从经验积累到更高级别智能体自我改进（Self-Improvement）的重要桥梁。
3.1.2.技能与工具 自我改进（Self-Improvement）

在技能和工具自我改进（Self-Improvement）中，可重用能力可以采取可执行代码技能、函数存储库、API包装器、工作流程例程、技能文档、技能合同或工具使用过程的形式。早期系统探索智能体如何构建和积累这些不同形式的能力工件。作为工具制造者的 CREATOR 和大型语言模型表明，法学硕士可以合成工具或 Python 实用函数，将抽象推理与具体计算分开，并在以后的调用中可重用 [56,57]。 ToolCoder 将工具学习组织为代码生成、执行反馈和功能存储库维护 [58]，而 RefTool 表明工具也可以

constructed from external references and domain knowledge [59]. In these systems, the main object of improvement is the external capability library; the outer acquisition procedure is usually still fixed.

Recent work shifts from generating isolated skills or tools to building capability libraries from multiple sources. One line abstracts agent trajectories, demonstrations, or task logs into reusable skills: SkillWeaver, SkillX, and Workflow-to-Skill distill web interactions, raw trajectories, or workflow traces into API-like skills, hierarchical skill knowledge bases, or workflow-preserving skill representations [60–62]. Socratic-SWE follows the same trace-to-skill pattern by distilling historical solving or repair trajectories into an Agent Skill Registry of procedural skills [63]. A second line compiles external resources into skills: Anything2Skill, OpenSkill, SkillFoundry, and DataCOPE turn documents, open-world resources, scientific repositories, notebooks, databases, or unsupervised exploration into skill packages and SkillBanks with lifecycle or verification signals [64–67]. As these libraries grow, self-improvement also requires curation. MUSE-Autoskill and SkillOS organize creation, memory, management, selection, and refinement into a skill lifecycle; SkillClaw aggregates multi-user trajectories into a shared SkillHub; and ToolLibGen performs the analogous cleanup on the tool side by deduplicating, refactoring, consolidating, and organizing generated tools [68–71]. This moves the field from one-off capability creation toward maintaining a reusable substrate of agent actions.

A capability library only improves future behavior when its contents are selected, composed, and revised appropriately. SkillOpt and Not All Skills Help approach this from the use side: the former treats skill invocation as an executive strategy, while the latter estimates per-skill causal contribution to suppress or repair harmful skills [72,73]. SkillComposer and Evolving Programmatic Skill Networks focus on composition and generalization: one decomposes skill evolution into create, improve, and merge operations, while the other organizes executable symbolic programs into a skill network that supports planning, fault localization, maturity gating, and rollback [74,75]. Other systems refine the skills themselves. SkillAxe and SkillAudit revise LLM-authored skills through structured diagnostics or paired trajectories; SkillGen, CoEvoSkills, and VASO improve self-generated skills through verified synthesis, surrogate-verifier co-evolution, or formal skill contracts [76–80]. Together, these works show that skill/tool self-improvement is not linear library growth, but an ongoing process of selection, composition, revision, and retention.

The boundary between skills and tools is also becoming less rigid as capability libraries enter larger agent-improvement loops. Beyond Static Tools, MetaForge, and SkillSmith exemplify the move from fixed toolsets to dynamic tool ecosystems: agents can synthesize tools at test time, retrieve and adapt tools on demand, or apply update bundles that jointly modify skills and tools through operations such as wrapping, editing, composing, splitting, and retiring [81–83]. Coding-agent systems show the same integration in a different environment: Socratic-SWE feeds skill libraries into code repair, task generation, and solving loops, while Confucius Code Agent connects persistent notes and modular extensions to the build, test, and debugging process of real codebases [63,85]. Overall, the trajectory of this area is from individual reusable skills or tools toward a managed external capability layer. Most systems remain harness-level L3 self-improvement, but learned curators, co-evolving verifiers, and open-world resource-to-skill pipelines begin to improve not only the library contents but also parts of the mechanism by which that library is improved.

#### 3.1.3. Prompt and Context Self-Optimization

Prompt and context self-optimization is the most lightweight form of harness self-improvement: model weights, external tools, and executable code may remain fixed, while the natural-language control surface that conditions later execution is revised. The optimization target can be a system prompt, task instruction, few-shot demonstration set, prompt program, context playbook, guideline, or rule document. The earliest and simplest line treats prompts as black-box textual parameters. OPRO establishes the basic loop by asking an optimizer LLM to propose new instructions from prior prompt-score histories [86], while PromptBreeder extends the idea by co-evolving task prompts and mutation prompts, making prompt improvement partly self-referential [87]. Later prompt-search methods vary the supervision and feedback budget: Self-Supervised Prompt Optimization reduces

# 原文第 14 页 | 中文翻译初稿

70 中的 14

根据外部参考和领域知识构建[59]。在这些系统中，主要改进的对象是外部能力库；外部采集过程通常仍然是固定的。

最近的工作从生成孤立的技能或工具转向从多个来源构建能力库。一行将智能体轨迹、演示或任务日志抽象为可重用的技能：SkillWeaver、SkillX 和 Workflow-to-Skill 将 Web 交互、原始轨迹或工作流跟踪提取为类似 API 的技能、分层技能知识库或工作流保留技能表示形式 [60-62]。 Socratic-SWE 通过将历史解决或修复轨迹提炼到程序技能的智能体技能注册表中，遵循相同的跟踪技能模式[63]。第二行将外部资源编译为技能：Anything2Skill、OpenSkill、SkillFoundry 和 DataCOPE 将文档、开放世界资源、科学存储库、笔记本、数据库或无监督探索转化为具有生命周期或验证信号的技能包和 SkillBank [64-67]。随着这些库的增长，自我改进（Self-Improvement） 也需要管理。 MUSE-Autoskill和SkillOS将创建、记忆、管理、选择和细化组织成技能生命周期； SkillClaw 将多用户轨迹聚合到共享的 SkillHub 中； ToolLibGen 通过重复数据删除、重构、整合和组织生成的工具，在工具端执行类似的清理工作[68-71]。这使得该领域从一次性能力创建转向维护智能体操作的可重用基础。

能力库只有在其内容被适当选择、组成和修改时才能改善未来的行为。 SkillOpt 和并非所有技能有助于从使用方面解决此问题：前者将技能调用视为执行策略，而后者则估计每项技能对抑制或修复有害技能的因果贡献[72,73]。 SkillComposer 和演进的编程技能网络侧重于组合和泛化：一种将技能演进分解为创建、改进和合并操作，而另一种将可执行的符号程序组织成支持规划、故障定位、成熟度门控和回滚的技能网络[74,75]。其他系统本身会完善技能。 SkillAxe 和 SkillAudit 通过结构化诊断或配对轨迹修改法学硕士创作的技能； SkillGen、CoEvoSkills 和 VASO 通过验证合成、智能体验证者共同进化或正式技能合同来提高自生成技能 [76-80]。这些工作共同表明，技能/工具 自我改进（Self-Improvement） 不是线性的库增长，而是一个持续的选择、组合、修订和保留的过程。

随着能力库进入更大的智能体改进循环，技能和工具之间的界限也变得不那么严格。除了静态工具之外，MetaForge 和 SkillSmith 例证了从固定工具集到动态工具生态系统的转变：智能体可以在测试时合成工具，按需检索和调整工具，或者应用更新包，通过包装、编辑、组合、分割和退役等操作共同修改技能和工具 [81-83]

。编码智能体系统在不同的环境中表现出相同的集成：Socratic-SWE 将技能库提供给代码修复、任务生成和解决循环，而孔子代码智能体将持久注释和模块化扩展连接到真实代码库的构建、测试和调试过程[63,85]。总体而言，该领域的发展轨迹是从个人可重用技能或工具转向托管外部能力层。大多数系统仍然是 运行支撑框架（Harness） 级别的 L3 自我改进（Self-Improvement），但学习的管理者、共同进化的验证者和开放世界的资源到技能管道不仅开始改进库内容，而且还开始改进库改进的部分机制。

3.1.3.提示和上下文自我优化

提示和上下文自优化是 运行支撑框架（Harness） 自我改进（Self-Improvement） 最轻量级的形式：模型权重、外部工具和可执行代码可能保持固定，而影响后续执行的自然语言控制面则被修改。优化目标可以是系统提示、任务指令、小样本演示集、提示程序、上下文剧本、指南或规则文档。最早和最简单的一行将提示视为黑盒文本参数。 OPRO 通过要求优化器 LLM 从先前的提示得分历史中提出新指令来建立基本循环 [86]，而 PromptBreeder 通过共同进化任务提示和突变提示来扩展该想法，使提示改进部分地自我参考 [87]。后来的提示搜索方法改变了监督和反馈预算：自我监督提示优化减少了

dependence on labeled references, DLPO imports robustness and update-control ideas into textual optimization, ReElicit builds interpretable elicited feature spaces for Bayesian optimization of system prompts, PrefPO uses pairwise preferences and natural-language criteria, APEX dynamically selects informative development examples, TextBO casts language-space search in a Bayesian-optimization style, and Feedback Descent uses pairwise comparisons with textual rationales for open-ended text artifacts [88–94]. In more agentic settings, the same search loop becomes grounded in executable analysis and environment interaction: SPEAR uses code execution and rollback validation to propose and retain prompt edits, while environment-grounded prompt optimization uses rollout returns and behavior analysis to revise game-agent prompts [95,96].

As agents become compound LM programs, the optimized object expands from a single instruction to a bundle of LM-call interfaces, demonstrations, prompting patterns, role prompts, and local context. DSPy makes this shift explicit by representing LM calls as declarative modules in its programming abstraction and compiling their prompts and demonstrations [97]. MIPRO jointly optimizes instructions and few-shot examples across multi-stage LM programs [98], and AutoPDL searches over prompting patterns and demonstrations to produce readable, executable prompt programs for agents [99]. ADO adds that the input data and example organization inside a prompt are themselves part of the control interface [100], while cross-model prompt translation separates transferable semantic directives from model-dependent interface constraints so prompt programs can adapt when the underlying foundation model changes [101]. Once the prompt surface is distributed across LM calls or agents, the central bottleneck becomes credit assignment: the system must identify which prompt, role instruction, demonstration, or context fragment caused a downstream error. TextGrad abstracts LLM critique as a gradient-like signal over textual variables [102], and GEPA uses execution and evaluation traces for reflective prompt evolution [103]. Textual Equilibrium Propagation, ContraPrompt, GTBP, and multi-agent prompt credit assignment then refine this feedback path through local equilibrium updates, success/failure trace contrast, graph-structured target propagation, and temporal/structural blame decomposition in multi-agent systems [104–107]. EEVEE and Combee further scale prompt learning to heterogeneous task streams and large volumes of agentic traces, pushing prompt-program adaptation from offline tuning toward deployment-time learning [108,109].

A parallel line of work focuses on constructing the context interface: what information enters the context window, how that information is structured, which rules or playbooks become executable constraints, and how the system updates and prunes the context and resolves conflicts within a finite context budget. Dynamic Cheatsheet and ACE provide two early anchors. Dynamic Cheatsheet compresses prior attempts into short strategy hints that can be injected into later prompts [110], while ACE uses generator, reflector, and curator roles to produce and organize context playbooks for subsequent executions [111]. SCOPE, Reflective Context Learning, and Unified Context Evolution make this context-compilation process more systematic: they derive context-management guidelines from execution traces, abstract context-space updates into reusable optimization primitives, or wrap information from memory, strategy, workflow, and skill sources into typed context units that can be selected, pruned, and budgeted [112–114]. KACE and Context Training with Active Information Seeking emphasize grounding: the former organizes knowledge as task-retrievable epistemic guidance, and the latter lets the optimizer acquire missing external information before updating context [115, 116]. In domain-specific settings, MEMO, SEEK-SQL, and Trace2Policy convert self-play outcomes, positive/negative execution trajectories, or expert behavior traces into hints, guidelines, or rule documents that can be consumed by the next inference-time decision [117–119]. Across these systems, context becomes an optimizable interface for the next run, not merely a larger container for previous experience.

From an RSI perspective, most prompt or context self-optimization still improves artifacts under externally specified update rules: the system repeatedly proposes, evaluates, and preserves better prompts, demonstrations, guidelines, or context units, while the outer improvement procedure remains fixed. A smaller set of systems begins to make the prompt optimizer itself mutable. PromptBreeder

# 原文第 15 页 | 中文翻译初稿

70 中的 15

对标记参考的依赖，DLPO 将鲁棒性和更新控制思想引入文本优化，ReElicit 为系统提示的贝叶斯优化构建可解释的引出特征空间，PrefPO 使用成对偏好和自然语言标准，APEX 动态选择信息丰富的开发示例，TextBO 以贝叶斯优化风格进行语言空间搜索，反馈下降使用与开放式文本工件的文本基本原理的成对比较[88-94]。在更智能体的设置中，相同的搜索循环以可执行分析和环境交互为基础：SPEAR 使用代码执行和回滚验证来提议和保留提示编辑，而基于环境的提示优化使用 展开轨迹（Rollout） 返回和行为分析来修改游戏智能体提示 [95,96]。

随着智能体成为复合 LM 程序，优化的对象从单个指令扩展到一组 LM 调用接口、演示、提示模式、角色提示和本地上下文。 DSPy 通过在其编程抽象中将 LM 调用表示为声明性模块并编译它们的提示和演示来明确这种转变[97]。 MIPRO 联合优化多阶段 LM 程序中的指令和少量示例 [98]，AutoPDL 搜索提示模式和演示，为智能体生成可读、可执行的提示程序 [99]。 ADO 补充说，提示内的输入数据和示例组织本身就是控制接口的一部分 [100]，而跨模型提示翻译将可转移的语义指令与模型相关的接口约束分开，因此提示程序可以在底层 基础模型（Foundation Model） 更改时进行调整 [101]。一旦提示表面分布在 LM 调用或智能体之间，中心瓶颈就变成了 贡献归因（Credit Assignment）：系统必须识别哪个提示、角色指令、演示或上下文片段导致了下游错误。 TextGrad 将 LLM 批判抽象为文本变量上的类似梯度的信号 [102]，而 GEPA 使用执行和评估轨迹进行反思性提示演化 [103]。然后，文本平衡传播、ContraPrompt、GTBP 和多智能体提示 贡献归因（Credit Assignment） 通过局部平衡更新、成功/失败跟踪对比、图结构目标传播和多智能体系统中的时间/结构责备分解来完善此反馈路径[104-107]。 EEVEE 和 Combee 进一步将即时学习扩展到异构任务流和大量智能体跟踪，推动即时程序适应从离线调整转向部署时学习 [108,109]。

并行工作重点是构建上下文接口：哪些信息进入上下文窗口，如何构建该信息，哪些规则或剧本成为可执行约束，以及系统如何更新和修剪上下文并解决有限上下文预算内的冲突。 Dynamic Cheatsheet 和 ACE 提供了两个早期锚点

。 Dynamic Cheatsheet 将先前的尝试压缩为简短的策略提示，可以注入到以后的提示中 [110]，而 ACE 使用生成器、反射器和策展人角色来生成和组织上下文剧本以供后续执行 [111]。 SCOPE、反射性上下文学习和统一上下文进化使这个上下文编译过程更加系统化：它们从执行跟踪中导出上下文管理指南，将上下文空间更新抽象为可重用的优化原语，或者将来自内存、策略、工作流和技能源的信息包装到可以选择、修剪和预算的类型化上下文单元中[112-114]。 KACE 和带有主动信息搜索的上下文训练强调 落地关联（Grounding）：前者将知识组织为任务可检索的认知指导，后者让优化器在更新上下文之前获取丢失的外部信息 [115, 116]。在特定领域的设置中，MEMO、SEEK-SQL 和 Trace2Policy 将自我对弈结果、正/负执行轨迹或专家行为跟踪转换为可由下一个推理时间决策使用的提示、指南或规则文档 [117-119]。在这些系统中，上下文成为下一次运行的可优化界面，而不仅仅是先前体验的更大容器。

从 RSI 的角度来看，大多数提示或上下文自我优化仍然在外部指定的更新规则下改进工件：系统反复提出、评估和保留更好的提示、演示、指南或上下文单元，而外部改进程序保持固定。一小部分系统开始使提示优化器本身可变。及时饲养员

already co-evolves mutation prompts with task prompts, even though the top-level evolutionary framework remains hand-designed [87]. SePO goes further by optimizing the prompt agent's own system prompt alongside task-agent prompts, turning prompt optimization from a hand-written procedure into a reusable optimizer skill [120]. Learning to Self-Evolve trains a model to edit its own test-time context and rewards edits that improve downstream performance [121]. Overall, prompt and context self-optimization provides a fast, inexpensive, and inspectable adaptation layer. Within a broader self-evolving agent harness, prompt and context artifacts serve as a control interface connecting memory, skills, workflows, and code-level adaptation.

### *3.2. Orchestration and Architecture Search*

Once agents acquire reusable memory, skills, tools, and context, a second bottleneck becomes visible: difficult tasks are rarely solved by one uninterrupted model call or one fixed execution path. Orchestration work treats the agent harness as a computation process that can be branched, searched, routed, and reorganized. Improvement comes from changing how trajectories are branched, specialized agents are assigned, feedback is routed, and communication is scheduled, so that the system can spend effort where uncertainty is high and reuse useful intermediate results rather than repeatedly starting from scratch. Within RSI systems, orchestration is the layer that turns improved harness modules into adaptive problem-solving systems, making self-improvement not only about owning stronger parts but also about organizing them effectively under real compute and reliability constraints.

#### 3.2.1. Test-Time Compute and Trajectory Orchestration

Scaling at test time starts from a simple weakness of ordinary agent execution: a single run is a fragile commitment to one sequence of thoughts, tool calls, and environment states. Test-time compute and trajectory orchestration therefore improve a fixed or mostly fixed agent harness by giving it more paths to explore, compare, and combine. The basic unit of improvement is a reasoning path, tool-action sequence, repository state, or full execution trajectory. Early search-oriented methods provide the conceptual basis: Tree of Thoughts treats intermediate thoughts as expandable and evaluable search nodes, while Language Agent Tree Search extends this idea to agents that interleave reasoning, acting, planning, and environmental feedback [122,123]. A more systematic view appears in Scaling Test-time Compute for LLM Agents, which compares parallel sampling, step-wise Best-of-N, beam or tree search, sequential revision, and verifier-based merging across agent tasks [124]. In software engineering, this trajectory view becomes especially concrete. SWE-Search applies Monte Carlo Tree Search to software repair trajectories, using action generation, value estimation, and discriminator-style debate to decide which states to expand [125]. Scaling Test-Time Compute for Agentic Coding frames the long-horizon coding setting around three coupled problems: representing long rollouts compactly, selecting among them reliably, and reusing their useful information through recursive tournament voting and parallel-distill-refine strategies [126]. Other systems enlarge the candidate space through evolutionary or population-style search. Satori-SWE evolves candidate patches through selection and mutation, SE-Agent operates directly on multi-step trajectories through revision, recombination, and refinement, and AgentGA moves the search object to the agent-seed level, evolving task prompts and parent archives that initialize fresh autonomous runs [127–129]. Even repository exploration can be treated as a test-time orchestration problem: domain-scoped parallel exploration and FastContext-style repository explorer agents show that context acquisition itself can be parallelized, specialized, and compressed before the main coding trajectory proceeds [130,131].

More paths, however, only help when the harness can tell which ones deserve attention. As trajectory pools grow, the problem shifts from exploration to feedback: which branches should receive more compute, which candidates should be pruned, and which failed attempts should be repaired rather than restarted. SWE-TRACE uses rubric-based process reward models at inference time to guide action-level sampling and prune weak action candidates before a full trajectory is completed [132]. EGSS makes the allocation problem explicit by using tool and action entropy to detect high-ambiguity steps, invoking auxiliary judgment only where uncertainty is high, and selecting final

# 原文第 16 页 | 中文翻译初稿

70 中的 16

尽管顶层进化框架仍然是手工设计的，但突变提示已经与任务提示共同进化[87]。 SePO 进一步优化提示智能体自己的系统提示以及任务智能体提示，将提示优化从手写程序转变为可重用的优化器技能 [120]。学习自我进化训练模型编辑其自己的测试时间上下文，并奖励可提高下游性能的编辑[121]。总体而言，提示和上下文自我优化提供了快速、廉价且可检查的适应层。在更广泛的自我进化智能体 运行支撑框架（Harness） 中，提示和上下文工件充当连接内存、技能、工作流程和代码级适应的控制界面。

3.2。编排和架构搜索

一旦智能体获得可重用的内存、技能、工具和上下文，第二个瓶颈就变得显而易见：困难的任务很少能通过一个不间断的模型调用或一个固定的执行路径来解决。编排工作将智能体 运行支撑框架（Harness） 视为可以分支、搜索、路由和重组的计算过程。改进来自于改变轨迹的分支方式、分配专门智能体、路由反馈和安排通信的方式，以便系统可以在不确定性高的地方花费精力并重用有用的中间结果，而不是重复从头开始。在 RSI 系统中，编排是将改进的 运行支撑框架（Harness） 模块转变为自适应问题解决系统的层，使 自我改进（Self-Improvement） 不仅能够拥有更强大的部件，而且能够在实际计算和可靠性约束下有效地组织它们。

3.2.1.测试时计算和轨迹编排

测试时的扩展始于普通智能体执行的一个简单弱点：单次运行是对一系列思想、工具调用和环境状态的脆弱承诺。因此，测试时计算和轨迹编排通过为固定或大部分固定的智能体 运行支撑框架（Harness） 提供更多路径来探索、比较和组合来改进它。改进的基本单位是推理路径、工具操作序列、存储库状态或完整执行轨迹。早期的面向搜索的方法提供了概念基础：思想树将中间思想视为可扩展和可评估的搜索节点，而语言智能体树搜索将这一想法扩展到交织推理、行动、规划和环境反馈的智能体[122,123]。更系统的观点出现在 Scaling Test-time Compute for LLM Agents 中，它比较了并行采样、逐步 Best-of-N、束或树搜索、顺序修订以及跨智能体任务的基于验证者的合并 [124]。在软件工程中，这种轨迹视图变得尤其具体。SWE-Search 将蒙特卡洛树搜索应用于软件修复轨迹，使用动作生成、价值估计和判别器式辩论来决定扩展哪些状态[125]。智能体编码的扩展测试时间计算围绕三个耦合问题构建了 长时程（Long-Horizon） 编码设置：紧凑地表示长 展开轨迹（Rollouts），在其中可靠地进行选择，并通过递归锦标赛投票和并行蒸馏精炼策略重用它们的有用信息 [126]

。其他系统通过进化或群体式搜索扩大候选空间。 Satori-SWE 通过选择和突变进化候选补丁，SE-Agent 通过修改、重组和细化直接在多步骤轨迹上运行，AgentGA 将搜索对象移动到智能体种子级别，进化任务提示和初始化新自主运行的父档案 [127-129]。甚至存储库探索也可以被视为测试时编排问题：领域范围的并行探索和 FastContext 风格的存储库浏览器智能体表明，上下文获取本身可以在主编码轨迹继续之前进行并行化、专门化和压缩 [130,131]。

然而，只有当 运行支撑框架（Harness） 能够判断哪些路径值得关注时，更多路径才有帮助。随着轨迹池的增长，问题从探索转向反馈：哪些分支应该接受更多计算，哪些候选应该被修剪，哪些失败的尝试应该被修复而不是重新启动。 SWE-TRACE 在推理时使用基于规则的过程奖励模型来指导动作级别采样，并在完整轨迹完成之前修剪弱动作候选者 [132]。 EGSS 通过使用工具和动作熵来检测高模糊性步骤，仅在不确定性高的情况下调用辅助判断，并选择最终的结果，从而使分配问题变得明确

patches with augmented regression and edge-case tests [133]. Some methods move selection earlier in the software-engineering workflow. SWE-Manager generates multiple natural-language repair proposals before coding, then selects and synthesizes a golden proposal, while architecture-aware multi-design generation creates several repository-level designs and filters them with static and dynamic impact analysis [134,135]. A related shift happens after execution has already exposed a failure: the system can refine or replay the useful parts of a trajectory instead of discarding the whole attempt. SWE-Replay archives previous trajectories and branches from informative intermediate steps without relying on an LLM-as-judge quality estimate [136]. PIVOT treats the trajectory as an optimizable plan-execution object: it preserves validated prefixes, inspects execution losses, rewrites unsupported suffixes, and verifies global constraints [137]. Persistent cross-attempt state optimization for repository-level code generation keeps task-local success knowledge, failure knowledge, and a historical-best repository across attempts, so later runs can inherit useful state while avoiding regressions [138]. AlphaApollo's propose-judge-update loop is a broader agentic reasoning example of the same pattern: test-time improvement arises not from a single retry, but from repeated proposal, verification, and update cycles over the evolving solution state [139]. These systems show that effective test-time scaling is not proportional to raw rollout count; it depends on whether feedback is local enough to intervene early, reliable enough to guide selection, and persistent enough to influence subsequent decisions.

Reliable feedback can guide search and revision within a task, but it does not eliminate duplicated computation across attempts: agents may still reconstruct context, repairs, and execution states that previous attempts have already produced. This motivates methods that make prior computation and prior experience available to the next trajectory rather than treating each run as isolated. Log-Augmented Generation stores and retrieves prior reasoning logs through key-value cache representations, directly reusing earlier computation rather than distilling it into textual reflection [140]. Decocted Experience instead extracts, organizes, and retrieves distilled experience from trajectories and feedback, making context construction itself a test-time scaling axis [31]. Bifrost adapts successful prior trajectories to new contexts through representation-level steering, while Lingxi retrieves procedural knowledge from historical issue-resolution cases to guide online analysis and scaling [141,142]. FailureMem and EvoRepair provide repair-specific variants by converting failed attempts, diagnostic lessons, and vulnerability-repair experience into reusable guidance for later fixes [143,144]. These experience-oriented methods sit near memory self-improvement, but their role here is narrower: they matter when stored trajectories or distilled lessons directly change current inference-time search, selection, or repair. At the same time, reusable trajectories require runtime support: the system must be able to capture, fork, restore, and compare executions whose actions have side effects. Shepherd records model calls, tool calls, and environment changes as formalized execution traces that can be monitored, forked, replayed, modified, and resumed, while DeltaBox provides sandbox checkpoint and rollback mechanisms for stateful agents whose actions affect files, processes, or tests [145,146]. Effective Feedback Compute introduces a scaling measure that credits feedback only when it is informative, valid, non-redundant, and retained for subsequent decisions, thereby distinguishing useful feedback from raw expenditure in tokens, tool calls, or runtime [147]. In summary, the key to effective test-time scaling is ensuring that additional inference compute produces reliable feedback and reusable intermediate results that guide final decisions.

#### 3.2.2. Agentic Unit Composition

Agentic unit composition concerns the participant layer of an agentic computation: beyond optimizing the search process and message flow, the system must decide what kinds of agents are available to do the work. At the finest level of composition, the internal structure of an agent becomes part of the design space. Instead of treating an agent as a single hand-written template, AgentSquare constructs candidate systems from specialized agents responsible for planning, reasoning, tool use, and memory, then searches over their evolution and recombination through standardized interfaces [148]. AgentSwift keeps this compositional view but enlarges the space: workflows are searched together with

# 原文第 17 页 | 中文翻译初稿

70 中的 17

带有增强回归和边缘情况测试的补丁[133]。有些方法将选择移到软件工程工作流程的早期。SWE-Manager 在编码前生成多个自然语言修复建议，然后选择并综合一个黄金建议，而架构感知的多重设计生成创建多个存储库级设计并通过静态和动态影响分析对其进行过滤[134,135]。在执行已经暴露出失败之后，就会发生相关的转变：系统可以细化或重放轨迹的有用部分，而不是放弃整个尝试。SWE-Replay 存档以前的轨迹和来自信息丰富的中间步骤的分支，而不依赖于 LLM 作为法官的质量估计[136]。 PIVOT 将轨迹视为可优化的计划执行对象：它保留经过验证的前缀、检查执行损失、重写不支持的后缀并验证全局约束 [137]。用于存储库级代码生成的持久交叉尝试状态优化可以保留任务本地成功知识、失败知识和跨尝试的历史最佳存储库，因此以后的运行可以继承有用的状态，同时避免回归[138]。AlphaApollo 的提议-判断-更新循环是同一模式的更广泛的智能体推理示例：测试时间的改进不是来自单次重试，而是来自不断发展的解决方案状态上的重复提议、验证和更新周期[139]。这些系统表明，有效的测试时间缩放与原始 展开轨迹（Rollout） 计数不成正比；这取决于反馈是否足够本地化以进行早期干预，是否足够可靠以指导选择，以及是否足够持久以影响后续决策。

可靠的反馈可以指导任务内的搜索和修订，但它并不能消除尝试之间的重复计算：智能体仍然可以重建先前尝试已经产生的上下文、修复和执行状态。这激发了使先前计算和先前经验可用于下一个轨迹的方法，而不是将每次运行视为孤立的。日志增强生成通过键值缓存表示来存储和检索先前的推理日志，直接重用早期的计算而不是将其提炼为文本反射[140]。相反，煎煮经验从轨迹和反馈中提取、组织和检索提炼的经验，使上下文构建本身成为测试时间缩放轴[31]。 Bifrost 通过表示级引导将成功的先前轨迹适应新的环境，而 Lingxi 从历史问题解决案例中检索程序知识以指导在线分析和扩展[141,142]。 FailureMem 和 EvoRepair 通过将失败的尝试、诊断经验教训和漏洞修复经验转换为可重用的指南以供以后修复，从而提供特定于修复的变体 [143,144]。这些面向经验的方法位于内存附近 自我改进（Self-Improvement），但它们在这里的作用较窄：当存储的轨迹或提取的经验教训直接改变当前的推理时间搜索、选择或修复时，它们很重要

。同时，可重用的轨迹需要运行时支持：系统必须能够捕获、分叉、恢复和比较其操作具有副作用的执行。 Shepherd 将模型调用、工具调用和环境更改记录为可以监视、分叉、重播、修改和恢复的正式执行跟踪，而 DeltaBox 为有状态智能体提供沙箱检查点和回滚机制，这些智能体的操作会影响文件、进程或测试[145,146]。有效反馈计算引入了一种扩展措施，仅当反馈信息丰富、有效、非冗余并保留用于后续决策时才计入反馈，从而将有用的反馈与令牌、工具调用或运行时的原始支出区分开来[147]。总之，有效测试时间扩展的关键是确保额外的推理计算产生可靠的反馈和可重用的中间结果，以指导最终决策。

3.2.2.智能体单位组成

智能体单元组合涉及智能体计算的参与者层：除了优化搜索过程和消息流之外，系统还必须决定哪些类型的智能体可以完成工作。在最精细的组合水平上，智能体的内部结构成为设计空间的一部分。AgentSquare 没有将智能体视为单个手写模板，而是从负责规划、推理、工具使用和记忆的专门智能体构建候选系统，然后通过标准化接口搜索它们的演变和重组[148]。 AgentSwift 保留了这种组合视图，但扩大了空间：工作流程与

specialized agents for memory, tool use, and planning, while a value model and uncertainty-guided MCTS make exploration practical [149]. AgentSpec adds a controlled-composition perspective by representing embodied systems as typed combinations of agents responsible for perception, memory, reasoning, reflection, action, and optional learning, making agent compatibility and interaction effects visible [150].

At a coarser granularity, the optimization target shifts from the internal modules of a single agent to the set of agents that jointly solve a task. AutoAgents establishes the basic task-conditioned pattern by generating specialized agents and an execution plan for the input task instead of relying on a fixed hand-written team [151]. EvoAgent turns role construction into an evolutionary process, varying roles, skills, prompts, and settings through mutation, crossover, and selection to expand a seed agent into a diverse multi-agent population [152]. ADAS broadens the design space further by letting a meta-agent iteratively propose, test, archive, and reuse code-defined child agent systems [153], while MAS-GPT learns to generate query-specific executable multi-agent systems in a single inference pass [154]. Later systems make these generated teams more structured and executable. Meta-Agent maps task descriptions into verified multi-agent systems with agent specifications, input/output contracts, tool configurations, and verification criteria [155]. LEMON generates orchestration specifications with task-specific roles, duties, capacity levels, and dependencies [156], and OrchMAS dynamically instantiates domain-aware scientific expert agents with tailored prompts [157]. EvoMAS represents multi-agent systems as structured configurations over roles, prompts, tools, model assignments, and communication structure, then evolves these configurations through selection, mutation, crossover, and cross-query reuse [158]. EVOCHAMBER adds a population-level substrate in which a coevolving agent pool forms teams through anchor, complement, and scout roles, while lifecycle operators fork, merge, prune, and seed agents under performance pressure [159]. The emphasis has therefore shifted from manually designing a team to making team construction itself an adaptive architecture layer.

Generated teams, however, are still a coarse answer if every query is forced through the same collection of roles or the same model choices. The next movement is to keep a heterogeneous pool of candidate units and select from it at query time. MaAS formulates this through an agentic supernet whose feasible operators include reasoning, debate, refinement, testing, tool-use, and early-exit patterns; each query samples a cost-aware multi-agent architecture from the learned operator distribution [160]. AutoMaAS extends the same idea with operator lifecycle management, so operators can be generated, fused, eliminated, and reweighted from online feedback [161]. HieraMAS makes heterogeneity internal to each functional role: a role becomes a supernode backed by mixtures of LLMs, with learned model selection, role pruning, and topology selection [162]. SciOrch applies this selection view to frontier scientific reasoning, where a lightweight orchestrator decomposes multimodal questions, delegates subproblems to selected expert LLM APIs, and synthesizes the final answer [163]. Iterative Critique-and-Routing Controller similarly treats coordination as a multi-round choice over a heterogeneous agent pool: the controller critiques the current draft, decides whether to stop, and routes the next step to the most suitable agent [164]. X-MAS supplies empirical support for this direction by showing that different LLMs specialize in different MAS roles and domains [165], while PASS and DAAO show how probabilistic tool/operator paths and model routing can be conditioned on domain structure, task difficulty, and cost [166,167]. Across these systems, orchestration-level self-improvement focuses on adaptive agent composition: the harness learns which specialized agents should be assembled for the computation at hand.

#### 3.2.3. Execution Flow, Topology, and Workflow Optimization

Once the participating units have been chosen, a second architecture-search problem remains: how those units should be connected into an executable computation. One family of work treats the execution flow itself as the optimization target. GPTSwarm provides an early formulation by representing a language-agent system as an optimizable graph, where nodes are agents or operations and edges carry information among them [168]. AutoFlow, AFlow, and ScoreFlow make this idea more explicit for workflow generation: workflows are no longer hand-written prompt chains, but natural-language pro-

# 原文第 18 页 | 中文翻译初稿

18 共 70

记忆、工具使用和规划的专门智能体，而价值模型和不确定性引导的 MCTS 使探索变得实用 [149]。 AgentSpec 通过将具体系统表示为负责感知、记忆、推理、反射、动作和可选学习的智能体的类型化组合，增加了受控组合的视角，使智能体兼容性和交互效果可见[150]。

在较粗的粒度上，优化目标从单个智能体的内部模块转移到共同解决任务的一组智能体。 AutoAgents 通过生成专门的智能体和输入任务的执行计划来建立基本的任务条件模式，而不是依赖于固定的手写团队 [151]。 EvoAgent 将角色构建转变为一个进化过程，通过突变、交叉和选择来改变角色、技能、提示和设置，将种子智能体扩展到多样化的多智能体群体 [152]。 ADAS 通过让元智能体迭代地提出、测试、存档和重用代码定义的子智能体系统 [153] 进一步拓宽了设计空间，而 MAS-GPT 则学习在单个推理过程中生成特定于查询的可执行多智能体系统 [154]。后来的系统使这些生成的团队更加结构化和可执行。元智能体将任务描述映射到经过验证的多智能体系统，其中包含智能体规范、输入/输出契约、工具配置和验证标准[155]。 LEMON 生成具有特定于任务的角色、职责、能力级别和依赖性的编排规范 [156]，OrchMAS 通过定制的提示动态实例化领域感知科学专家智能体 [157]。 EvoMAS 将多智能体系统表示为角色、提示、工具、模型分配和通信结构的结构化配置，然后通过选择、变异、交叉和交叉查询重用来演化这些配置 [158]。 EVOCHAMBER 添加了一个群体级别的基础，其中共同进化的智能体池通过锚定、补充和侦察角色形成团队，而生命周期操作员则在性能压力下分叉、合并、修剪和种子智能体[159]。因此，重点已经从手动设计团队转移到使团队建设本身成为一个自适应架构层。

然而，如果每个查询都强制通过相同的角色集合或相同的模型选择，那么生成的团队仍然是一个粗略的答案。下一步是保留候选单元的异构池并在查询时从中进行选择。 MaAS 通过一个智能体超级网络来制定这一点，其可行的操作包括推理、辩论、细化、测试、工具使用和提前退出模式；每个查询都会从学习到的运算符分布中采样成本感知的多智能体架构[160]。 AutoMaAS 通过算子生命周期管理扩展了相同的想法，因此可以根据在线反馈生成、融合、消除和重新加权算子[161]。 HieraMAS 使每个功能角色内部具有异质性：角色成为由 LLM 混合支持的超级节点，具有学习模型选择、角色修剪和拓扑选择 [162]

。 SciOrch 将这种选择视图应用于前沿科学推理，其中轻量级协调器分解多模式问题，将子问题委托给选定的专家 LLM API，并综合最终答案 [163]。迭代批评和路由控制器类似地将协调视为异构智能体池上的多轮选择：控制器批评当前草案，决定是否停止，并将下一步路由到最合适的智能体[164]。 X-MAS 通过表明不同的法学硕士专注于不同的 MAS 角色和领域 [165] 为这个方向提供了经验支持，而 PASS 和 DAAO 则展示了概率工具/操作员路径和模型路由如何以领域结构、任务难度和成本为条件 [166,167]。在这些系统中，编排级 自我改进（Self-Improvement） 专注于自适应智能体组合：运行支撑框架（Harness） 了解应该组装哪些专用智能体来进行手头的计算。

3.2.3.执行流程、拓扑和工作流程优化

一旦选择了参与单元，第二个架构搜索问题仍然存在：如何将这些单元连接到可执行计算中。一系列工作将执行流程本身视为优化目标。 GPTSwarm 通过将语言智能体系统表示为可优化图来提供早期的表述，其中节点是智能体或操作，边在它们之间携带信息[168]。 AutoFlow、AFlow 和 ScoreFlow 让这个想法对于工作流生成更加明确：工作流不再是手写的提示链，而是自然语言亲

grams, code-represented workflows, or continuous workflow representations that can be searched and optimized from execution feedback [169–171]. Flow expresses agentic workflows as activity-on-vertex graphs whose dependencies and subtask allocation can be refined over time, while MermaidFlow and EvoAgentX emphasize executable workflow representations with explicit roles, data-flow constraints, evaluation, and evolution loops [172–174]. Later work expands the search procedure itself. EvoFlow and SEW use evolutionary mechanisms to explore diverse workflow structures, VFlow and HALO adapt structured search to domain-specific or hierarchical orchestration, and HyEvo evolves hybrid workflows that mix LLM calls with deterministic code nodes [175–179]. Other systems improve how workflow search generalizes and localizes edits: AdaptFlow learns reusable workflow initializations, Learning to Compose builds cross-domain workflow capabilities, BayesFlow casts workflow generation as probabilistic inference, Workflow-R1 optimizes multi-turn workflow construction, JudgeFlow uses block-level judgment to target local workflow edits, and RobustFlow adds preference-based pressure for stable workflow structures [180–185]. Verification-aware and state-machine formulations make these local revisions more explicit. Lean4Agent formalizes workflow graphs, step contracts, and execution trajectories in Lean, while LeanEvolve translates localized verification failures into targeted revisions of the workflow specification [232]. EvoFSM instead exposes a finite-state workflow whose states, transitions, and state-specific instructions can be changed through atomic edit operations, with a persistent memory reusing successful configurations and operation sequences across tasks [233]. This line also includes execution-aware workflow optimization: LAMaS learns latency-aware orchestration graphs for parallel multi-agent execution, FlowCompile treats structured LLM workflows as a compiler optimization space, GraphFlow manages graph-structured workflows for efficient serving, Learning to Hand Off studies interface-constrained handoff policies, and latent-space branch synthesis changes how parallel workflow branches are merged for later reasoning [186–190]. Across these systems, self-improvement happens at the level of the workflow program: the system changes the dependency graph, ordering, handoff, parallelism, or executable structure through which fixed or selected units produce an answer.

Whereas workflow-search methods optimize the end-to-end execution structure, another line isolates communication among participating agents as the main object of self-improvement. These methods determine which participating agents exchange information, when communication occurs, how densely they are connected, and whether learned topological patterns can transfer across tasks. GPTSwarm already makes graph connectivity optimizable, while EvoMAC evolves multi-agent collaboration networks for software development and G-Designer uses graph neural methods to design task-aware communication topologies [168,191,192]. Cost and redundancy then motivate sparse communication. AgentPrune removes unnecessary or harmful message-passing edges, Adaptive Graph Pruning jointly prunes agents and communication links, and SafeSieve progressively prunes communication based on accumulated experience [193–195]. More recent methods generate or select topology directly: AMAS determines task-specific communication graphs, graph-diffusion topology generation and OFA-MAS synthesize task-adaptive graphs through generative models, ST-EVO evolves communication graphs across spatial and temporal dimensions, Agent Q-Mix frames agent action and connection choices as cooperative reinforcement learning, RADAR adds redundancy-aware diffusion, and differentiable mixture-of-agents models make topology selection trainable through sparse activation [196–202]. Domain- and state-conditioned topology methods push this further: AgentConductor evolves DAG communication structures for competition-level code generation, TacoMAS co-evolves topology and capability at test time, TopoPrior and MasFACT learn transferable topology priors for later collaboration, and DySCo selects sparse trust-aware edges per reasoning round [203–207]. Finally, topology optimization can be conditioned on richer runtime signals. Retrieval-conditioned topology selection chooses collaboration modes from codebase complexity under budget conservation, response-conditioned parallel-to-sequential orchestration decides whether parallel agent outputs should trigger sparse DAG refinement, Learning to Interrupt optimizes when agents should enter each other's communication stream, and Learning to Communicate moves beyond explicit edges

# 原文第 19 页 | 中文翻译初稿

19 共 70

克、代码表示的工作流或连续工作流表示，可以从执行反馈中搜索和优化[169-171]。 Flow 将智能体工作流表示为顶点活动图，其依赖性和子任务分配可以随着时间的推移而细化，而 MermaidFlow 和 EvoAgentX 则强调具有明确角色、数据流约束、评估和演化循环的可执行工作流表示[172-174]。后来的工作扩展了搜索过程本身。 EvoFlow 和 SEW 使用进化机制来探索不同的工作流程结构，VFlow 和 HALO 使结构化搜索适应特定领域或分层编排，HyEvo 发展了将 LLM 调用与确定性代码节点混合在一起的混合工作流程 [175-179]。其他系统改进了工作流搜索泛化和本地化编辑的方式：AdaptFlow 学习可重用的工作流初始化，Learning to Compose 构建跨域工作流功能，BayesFlow 将工作流生成转换为概率推理，Workflow-R1 优化多轮工作流构建，JudgeFlow 使用块级判断来定位本地工作流编辑，而 RobustFlow 为稳定的工作流结构添加基于偏好的压力[180-185]。验证感知和状态机公式使这些本地修订更加明确。 Lean4Agent 在 Lean 中形式化工作流程图、步骤契约和执行轨迹，而 LeanEvolve 将局部验证失败转化为工作流程规范的有针对性的修订 [232]。相反，EvoFSM 公开了一个有限状态工作流程，其状态、转换和特定于状态的指令可以通过原子编辑操作进行更改，并使用持久内存跨任务重用成功的配置和操作序列[233]。该系列还包括执行感知工作流优化：LAMaS 学习用于并行多智能体执行的延迟感知编排图，FlowCompile 将结构化 LLM 工作流视为编译器优化空间，GraphFlow 管理图结构工作流以实现高效服务，Learning to Hand Off 研究接口约束的切换策略，潜在空间分支合成改变了并行工作流分支的合并方式以供后续推理[186-190]。在这些系统中，自我改进（Self-Improvement）发生在工作流程序级别：系统更改依赖图、排序、切换、并行性或可执行结构，固定或选定的单元通过这些结构产生答案。

尽管工作流搜索方法优化了端到端执行结构，但另一条线隔离了参与智能体之间的通信，作为 自我改进（Self-Improvement）的主要对象。这些方法确定哪些参与智能体交换信息、何时发生通信、它们连接的密集程度以及学习的拓扑模式是否可以跨任务转移。 GPTSwarm 已经使图形连接可优化，而 EvoMAC 发展了用于软件开发的多智能体协作网络，G-Designer 使用图形神经方法来设计任务感知通信拓扑 [168,191,192]。成本和冗余会导致稀疏通信

。 AgentPrune 删除不必要或有害的消息传递边缘，自适应图修剪联合修剪智能体和通信链路，SafeSieve 根据积累的经验逐步修剪通信[193-195]。最近的方法直接生成或选择拓扑：AMAS 确定特定于任务的通信图，图扩散拓扑生成和 OFA-MAS 通过生成模型合成任务自适应图，ST-EVO 跨空间和时间维度演化通信图，Agent Q-Mix 将智能体动作和连接选择框架为协作 强化学习（Reinforcement Learning），RADAR 添加冗余感知扩散，可微的智能体混合模型使拓扑选择可通过稀疏进行训练激活[196-202]。域和状态条件拓扑方法进一步推动了这一点：AgentConductor 演化了用于竞争级代码生成的 DAG 通信结构，TacoMAS 在测试时共同演化拓扑和能力，TopoPrior 和 MasFACT 学习可转移拓扑先验以供以后协作，DySCo 在每次推理轮中选择稀疏的信任感知边缘 [203-207]。最后，拓扑优化可以以更丰富的运行时信号为条件。检索条件拓扑选择在预算保护下从代码库复杂性中选择协作模式，响应条件并行到顺序编排决定并行智能体输出是否应触发稀疏 DAG 细化，学习中断优化智能体何时应进入彼此的通信流，学习通信超越显式边缘

toward learned communication channels and latent protocols [208–211]. Taken together, these works shift multi-agent orchestration from static debate or fixed pipelines toward adaptive communication structures whose edges, density, timing, and reusable priors become part of the evolving harness.

### *3.3. Self-Referential Code Modification*

Memory, skill, prompt, and workflow optimization each targets a specific class of harness artifact. Self-referential code modification expands the editable scope to the harness or scaffold itself, allowing broader changes across its components and their interactions. At the more recursive end of this spectrum, the modified agent participates in subsequent improvement cycles, becoming both the object of improvement and part of the mechanism that produces further changes. This self-reference makes code-level modification the closest harness-side analogue of recursive self-improvement in this section, while also placing greater demands on validation, rollback, auditability, and controlled promotion.

#### 3.3.1. Bounded Self-Harness and Policy Repair

The most local form of self-referential harness modification is bounded repair: the agent does not rewrite its full source code or redesign its entire scaffold, but it can turn execution evidence into persistent changes to the operating rules and interfaces that help future runs. At the lower level, these changes target action interfaces, tool contracts, runtime constraints, and domain-specific procedural rules. Life-Harness adapts the interface rather than the model: recurring interaction failures are converted into reusable interventions over environment contracts, action realization, procedural guidance, and trajectory regulation [212]. HarnessFix makes the trace-to-patch structure explicit. It compiles failed trajectories and harness code into a diagnostic representation, attributes failures to specific harness layers, generates scoped repair operators, and validates patches against regressions [213]. Milkyway applies this form of persistent harness adaptation to forecasting agents: it updates a persistent forecasting harness from pre-resolution signals across repeated forecasts, so later predictions are guided by revised procedures for evidence tracking, factor revision, and uncertainty management [214]. Across these systems, the important shift is that experience no longer only becomes memory or skill content; it changes the execution rules under which the next run is conducted.

A second group applies bounded repair directly to policy and harness code. Self-Harness lets the same fixed model mine its own failures, propose minimal edits to its harness, and promote only those edits that improve held-in tasks without regressing on held-out validation [215]. POLARIS applies a similar idea to small language models through experience-abstracted policy repair: failed validation tasks are distilled into reusable strategies and then implemented as auditable policy-code patches that can be executed in later tasks [216]. Around these core repair mechanisms, recent work also develops the surrounding infrastructure needed for reliable harness evolution. Agentic Harness Engineering uses trajectory observability to edit the coding-agent harness implementation with predicted impact and verification feedback [22]; Meta-Harness optimizes harness code end-to-end from source, scores, and execution traces [217]; and AgentFlow synthesizes multi-agent harnesses by revising roles, prompts, tools, communication topology, and coordination protocols from runtime signals [218]. Other systems strengthen the diagnosis and promotion stages: AgentDevel frames agent improvement as release engineering with executable diagnosis and non-regression gates [219], Wink provides runtime recovery from coding-agent misbehaviors through targeted interventions [220], AgentFixer connects failure detection to root-cause analysis and fix recommendations [221], CausalFlow performs step-level causal attribution and counterfactual repair [222], and AgenTracer and Who&When study automated failure attribution in multi-agent traces [223,224]. Together, these works mark a transition of self-improvement from modules used by the harness to modifications of the harness itself that persist across future executions.

#### 3.3.2. Source-Code and Scaffold Self-Modification

Source-code and scaffold self-modification makes the agent's executable implementation itself editable, allowing revisions to span multiple interacting components. STOP gives an early formulation

# 原文第 20 页 | 中文翻译初稿

70 中的 20

学习沟通渠道和潜在协议[208-211]。总而言之，这些工作将多智能体编排从静态辩论或固定管道转向自适应通信结构，其边缘、密度、时序和可重用先验成为不断发展的 运行支撑框架（Harness） 的一部分。
3.3。自引用代码修改

内存、技能、提示和工作流程优化均针对特定类别的 运行支撑框架（Harness） 工件。自引用代码修改将可编辑范围扩展到 运行支撑框架（Harness） 或脚手架本身，从而允许对其组件及其交互进行更广泛的更改。在这个范围的更递归端，修改后的智能体参与后续的改进周期，既成为改进的对象，又成为产生进一步变化的机制的一部分。这种自引用使得代码级修改成为本节中 递归自我改进（Recursive Self-Improvement） 最接近的 运行支撑框架（Harness） 端类似物，同时也对验证、回滚、可审计性和受控升级提出了更高的要求。
3.3.1.有界自我-运行支撑框架（Harness） 和策略修复

自引用 运行支撑框架（Harness） 修改的最本地化形式是有界修复：智能体不会重写其完整源代码或重新设计其整个支架，但它可以将执行证据转化为对操作规则和接口的持久更改，以帮助将来的运行。在较低级别，这些更改针对操作接口、工具契约、运行时约束和特定于域的程序规则。 Life-运行支撑框架（Harness） 适应的是界面而不是模型：重复出现的交互失败被转化为对环境契约、动作实现、程序指导和轨迹调节的可重复使用的干预措施[212]。 HarnessFix 使跟踪到补丁的结构变得明确。它将失败的轨迹和运行支撑框架（Harness） 代码编译为诊断表示，将失败归因于特定的 运行支撑框架（Harness） 层，生成范围修复运算符，并针对回归验证补丁[213]。 Milkyway 将这种形式的持久 运行支撑框架（Harness） 适应应用于预测智能体：它根据重复预测的预解析信号更新持久预测 运行支撑框架（Harness），因此以后的预测由证据跟踪、因子修订和不确定性管理的修订程序指导 [214]。在这些系统中，重要的转变是经验不再仅仅成为记忆或技能内容；而是成为内容。它改变了下一次运行的执行规则。

第二组直接对策略和 运行支撑框架（Harness） 代码应用有界修复。 Self-运行支撑框架（Harness） 让相同的固定模型挖掘自己的失败，对其 运行支撑框架（Harness） 提出最小的编辑建议，并仅促进那些改进保留任务的编辑，而不会对保留验证进行回归[215]。 POLARIS 通过经验抽象策略修复将类似的想法应用于小语言模型：失败的验证任务被提炼为可重用策略，然后实现为可在后续任务中执行的可审计策略代码补丁[216]。围绕这些核心修复机制，最近的工作还开发了可靠的 运行支撑框架（Harness） 进化所需的周边基础设施

。 Agentic 运行支撑框架（Harness） Engineering 使用轨迹可观测性来编辑编码智能体 运行支撑框架（Harness） 实现，并具有预测影响和验证反馈 [22]； Meta-运行支撑框架（Harness） 从源、分数和执行跟踪中端到端优化 运行支撑框架（Harness） 代码 [217]； AgentFlow 通过修改运行时信号的角色、提示、工具、通信拓扑和协调协议来合成多智能体 运行支撑框架（Harnesses） [218]。其他系统加强了诊断和升级阶段：AgentDevel 将智能体改进框架为具有可执行诊断和非回归门的发布工程[219]，Wink 通过有针对性的干预提供编码智能体不当行为的运行时恢复[220]，AgentFixer 将故障检测与根本原因分析和修复建议连接起来[221]，CausalFlow 执行步骤级因果归因和 反事实（Counterfactual） 修复[222]，以及 AgenTracer 和 Who&When 研究多智能体跟踪中的自动故障归因 [223,224]。这些工作共同标志着 自我改进（Self-Improvement） 从 运行支撑框架（Harness） 使用的模块到 运行支撑框架（Harness） 本身的修改的过渡，这些修改在未来的执行中持续存在。
3.3.2.源代码和脚手架自我修改

源代码和支架自我修改使智能体的可执行实现本身可编辑，从而允许修订跨越多个交互组件。 STOP 给出了一个早期的公式

in recursive self-improving code generation: a seed improver calls a fixed language model to improve downstream programs, then applies the same improvement process to its own Python source, producing successor improvers with stronger scaffolding strategies such as beam search, genetic algorithms, and simulated annealing [225]. Gödel Agent moves the same self-referential idea into the agent policy itself: the agent inspects its runtime memory and uses monkey patching to revise its decision procedure and self-update logic, so that the modified policy participates in later recursive calls [226]. In software engineering settings, A Self-Improving Coding Agent provides a more complete repository-level loop. The coding agent edits its own Python codebase, tools, subagents, context management, and benchmark machinery, while the best archived agent version becomes the next meta-agent for subsequent improvement rounds [227]. MOSS pushes the object of modification into deployed agent substrates, rewriting source-level harness logic such as routing, state management, dispatch, hooks, mediator behavior, and session lifecycle; candidate changes are packaged as trial images, replayed against production-failure batches, and promoted through consent and rollback gates before entering the live container [228]. Self-Evolving Software Agents describes the same trend from a BDI-LLM architecture perspective, where experience can induce new goals, revised reasoning structures, intention-selection patterns, and executable actions that are retained for later behavior [229]. Around this central line, recent work increasingly treats other executable elements of the scaffold as evolvable parts of the agent harness. DemoEvolve treats the frozen-model agent's prompts, state tracking, action filters, tool exposure, and control flow as an executable harness, using prior harness code, scores, rollout traces, and human demonstrations to guide active model-facing edits that are selected and reused on held-out seeds [230]. SIGA shows a similar pattern at the adapter layer: simulator-specific contracts are injected through context, tool, and termination interfaces, and prior trajectories are used to rewrite the adapter's primer, procedural-memory cheatsheet, and auxiliary skills into a reusable plugin package [231]. Taken together, these systems show a shift from agents that merely write code as task output to agents whose improver programs, policies, repositories, runtime tools, production harnesses, or adapter plugins can themselves be rewritten, verified, and reused as part of the next agent execution.

#### 3.3.3. Open-Ended and Governed Recursive Self-Modification

Open-ended recursive self-modification asks whether self-editing can become a sustained evolutionary process rather than a single scaffold repair or a one-step source-code patch. The theoretical reference point is the Gödel Machine: a fully self-referential system whose own problem-solving code and improvement procedure are both subject to rewriting when sufficient evidence supports the change [25].

Recent LLM-agent systems operationalize this ideal by progressively exposing different parts of an archive-based improvement loop to adaptation. The Darwin Gödel Machine (DGM) replaces the original Gödel Machine's impractical proof obligation with empirical validation: a coding agent edits its own repository, evaluates the resulting descendant on downstream tasks, and retains viable variants in a branching archive from which later agents can be selected for further modification [234]. Rather than committing only to the strongest current variant, this archive preserves diverse branches and temporarily suboptimal stepping stones that may support stronger descendants in later generations. DGM thus establishes a practical RSI loop in which the agent implementation being improved can also produce its next modification, although archive search, parent selection, and part of the surrounding improvement procedure remain externally specified. The Huxley–Gödel Machine (HGM) changes how this archive is searched. It addresses the fact that an agent's current task performance may be a poor indicator of its ability to generate better descendants by introducing clade metaproductivity, which evaluates a candidate through the outcomes of its descendant subtree. Metaproductivity-aware selection can therefore allocate expansion effort toward lineages with greater long-term potential rather than merely toward the highest-scoring current node [235]. Whereas HGM changes which lineage is expanded, Hyperagents allows the self-modification procedure to evolve as well. A hyperagent places the task agent and the meta-agent responsible for proposing modifications in the same editable program. For each selected parent, the meta-agent examines the agent's repository, previous evaluation

# 原文第 21 页 | 中文翻译初稿

21 共 70

在递归自我改进（Recursive Self-Improving）代码生成中：种子改进器调用固定语言模型来改进下游程序，然后将相同的改进过程应用于其自己的Python源代码，产生具有更强支架策略（例如集束搜索、遗传算法和模拟退火）的后继改进器[225]。哥德尔智能体将相同的自引用思想转移到智能体策略本身中：智能体检查其运行时内存并使用猴子修补来修改其决策过程和自更新逻辑，以便修改后的策略参与以后的递归调用[226]。在软件工程设置中，自我改进（Self-Improving）编码智能体提供了更完整的存储库级循环。编码智能体编辑自己的Python代码库、工具、子智能体、上下文管理和基准机器，而最好的存档智能体版本成为后续改进轮次的下一个元智能体[227]。 MOSS将修改的对象推送到部署的智能体底层中，重写源级运行支撑框架（Harness）逻辑，例如路由、状态管理、调度、钩子、中介行为和会话生命周期；候选更改被打包为试验图像，针对生产失败批次进行重播，并在进入实时容器之前通过同意和回滚门进行推广[228]。自我进化软件智能体从 BDI-LLM 架构的角度描述了相同的趋势，其中经验可以引发新的目标、修改后的推理结构、意图选择模式以及为以后的行为保留的可执行操作[229]。围绕这条中心线，最近的工作越来越多地将支架的其他可执行元素视为智能体 运行支撑框架（Harness） 的可进化部分。
DemoEvolve 将冻结模型智能体的提示、状态跟踪、动作过滤器、工具暴露和控制流视为可执行的 运行支撑框架（Harness），使用先前的 运行支撑框架（Harness） 代码、分数、展开轨迹（Rollout） 跟踪和人工演示来指导在保留种子上选择和重用的主动面向模型的编辑[230]。 SIGA 在适配器层显示了类似的模式：通过上下文、工具和终止接口注入特定于模拟器的合约，并使用先前的轨迹将适配器的入门、程序内存备忘单和辅助技能重写为可重用的插件包 [231]。总而言之，这些系统显示了从仅将代码编写为任务输出的智能体到其改进程序、策略、存储库、运行时工具、生产 运行支撑框架（Harnesses） 或适配器插件本身可以重写、验证并作为下一次智能体执行的一部分重用的智能体的转变。
3.3.3.开放式和受控的递归自我修改

开放式递归自我修改询问自我编辑是否可以成为一个持续的进化过程，而不是单个脚手架修复或一步源代码补丁。理论参考点是哥德尔机：一个完全自我参考的系统，当有足够的证据支持更改时，其自己的问题解决代码和改进程序都可以重写[25]。

最近的 LLM 智能体系统通过逐步暴露基于档案的改进循环的不同部分来适应这一理想

。达尔文哥德尔机（DGM）用经验验证取代了原始哥德尔机不切实际的证明义务：编码智能体编辑自己的存储库，评估下游任务的结果后代，并在分支存档中保留可行的变体，从中可以选择后续智能体进行进一步修改[234]。该档案不是只致力于当前最强的变体，而是保留了不同的分支和暂时次优的垫脚石，这些垫脚石可能会支持后代更强大的后代。因此，DGM 建立了一个实用的 RSI 循环，其中正在改进的智能体实现也可以产生其下一个修改，尽管档案搜索、父选择和部分周围的改进过程仍然是外部指定的。赫胥黎-哥德尔机 (HGM) 改变了该档案的搜索方式。它解决了这样一个事实，即智能体当前的任务绩效可能并不能很好地表明其通过引入进化枝元生产力（通过其后代子树的结果来评估候选者）来生成更好的后代的能力。因此，元生产力意识选择可以将扩展工作分配给具有更大长期潜力的谱系，而不仅仅是分配给得分最高的当前节点[235]。 HGM 改变了扩展的谱系，而 Hyperagents 也允许自我修改过程进化。超级智能体将负责在同一可编辑程序中提出修改建议的任务智能体和元智能体放在一起。对于每个选定的父代，元智能体会检查智能体的存储库、之前的评估

results, and remaining iteration budget to diagnose failures and decide what to modify. Because the meta-agent's own implementation is part of the repository, it can rewrite both the task-solving code and the logic used to analyze performance and generate future modifications. Over successive generations, DGM-Hyperagents (DGM-H) develops reusable meta-level mechanisms such as performance tracking and persistent memory. Since these mechanisms learn from evaluation feedback rather than depend on a fixed, handcrafted procedure, they can support self-improvement even when task performance and code-editing ability are not naturally aligned. DGM-H consequently improves agents for paper review and robotics reward design, while the meta-level capabilities acquired in these domains further transfer to Olympiad-level mathematics grading [23]. Building on HGM's lineage search and Hyperagents-style editable workspaces, Red Queen Gödel Machine (RQGM) places learned evaluators in the same evolvable workspace as task agents, allowing the capability being optimized and the mechanism that evaluates that capability to improve together [361]. To keep scores meaningful as evaluators change, RQGM freezes the active evaluator within each epoch, promotes only challengers validated against a held-out anchor, and removes records whose scores depend on a displaced evaluator. This design turns evaluation from a fixed external constraint into an evolvable part of recursive self-improvement without destabilizing lineage selection. Figure 4 illustrates an integrated framework that captures the core self-improvement loop of DGM and RQGM.

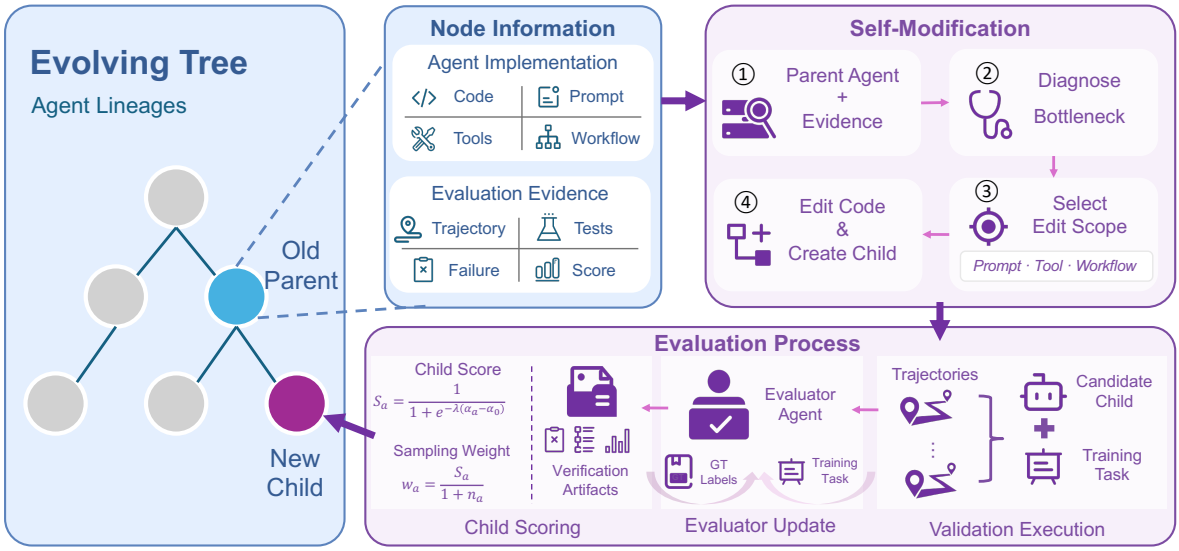



**Figure 4. Conceptual synthesis of archive-based recursive self-improvement.** The evolving tree retains agent lineages, with each node coupling an editable agent implementation, including code, prompts, tools, and workflows, with execution and evaluation evidence. A selected parent uses this evidence to diagnose bottlenecks, determine an edit scope, and rewrite its implementation to create a child, which is executed on sampled tasks to produce trajectories and verification artifacts. In DGM, valid children are added to the evolving tree, and each node's benchmark score and number of functioning children determine how likely it is to be sampled as a future parent. RQGM extends this loop by making the evaluator evolvable as well: the evaluator judges the child's trajectories, while task feedback and ground-truth labels also guide evaluator updates, enabling the task agent and its evaluation mechanism to improve together [234,361].

Group-Evolving Agents approaches self-improvement from a collective perspective, changing the evolutionary unit from an isolated parent to a group or population, selecting parent groups by performance and novelty, and sharing patches, trajectories, and failure experience so that discoveries from separate branches can be recombined by later offspring [236]. Other systems make the broader evolutionary process itself editable. Harnessing Agentic Evolution formulates evolution as an interactive environment: accumulated candidates, traces, failures, costs, and scores become process-level state, while a meta-editor changes the procedure or operating context that controls future evolution segments [237]. Bilevel Autoresearch makes a related move in automated research loops, where an outer loop reads the inner loop's code and experimental traces and injects new search mechanisms that

# 原文第 22 页 | 中文翻译初稿

22 共 70

结果，以及用于诊断故障并决定修改内容的剩余迭代预算。由于元智能体自己的实现是存储库的一部分，因此它可以重写任务解决代码以及用于分析性能和生成未来修改的逻辑。经过连续几代的发展，DGM-Hyperagents (DGM-H) 开发了可重用的元级机制，例如性能跟踪和持久内存。由于这些机制是从评估反馈中学习而不是依赖于固定的手工程序，因此即使任务性能和代码编辑能力不自然地一致，它们也可以支持 自我改进（Self-Improvement）。 DGM-H 因此改进了论文评审和机器人奖励设计的智能体，而在这些领域获得的元级能力进一步转移到奥林匹克级数学评分 [23]。红皇后哥德尔机（RQGM）以 HGM 的谱系搜索和 Hyperagentsstyle 可编辑工作空间为基础，将学习评估器放置在与任务智能体相同的可进化工作空间中，从而允许优化能力以及评估该能力的机制共同改进 [361]。为了在评估者发生变化时保持分数有意义，RQGM 冻结每个时期内的活跃评估者，仅提升针对保留锚点验证的挑战者，并删除其分数取决于替换评估者的记录。该设计将评估从固定的外部约束转变为 递归自我改进（Recursive Self-Improvement）的可进化部分，而不会破坏谱系选择的稳定性。图 4 说明了捕获 DGM 和 RQGM 核心 自我改进（Self-Improvement） 循环的集成框架。

节点信息自我修改
进化树智能体实现①父智能体②诊断
智能体谱系代码提示证据+瓶颈
工具工作流程
评估证据④编辑代码③选择
轨迹测试和编辑范围
旧创建子
父失败分数提示·工具·工作流程
评估过程
□ ！ = 1 + □Child Score "# 1 $!"$" EvaluatorAgent 轨迹 CandidateChild
ChildNew 采样权重□! = 1 + □□! ! VerificationArtifacts GT LabelsGT TrainingTask TrainingTask
Child Scoring Evaluator 更新验证执行
…

图 4. 基于存档的 递归自我改进（Recursive Self-Improvement） 的概念综合。不断发展的树保留了智能体谱系，每个节点都耦合可编辑的智能体实现，包括代码、提示、工具和工作流程，以及执行和评估证据。选定的父级使用此证据来诊断瓶颈、确定编辑范围并重写其实现以创建子级，该子级在采样任务上执行以生成轨迹和验证工件。在 DGM 中，有效的子节点被添加到进化树中，每个节点的基准分数和功能子节点的数量决定了它被采样为未来父节点的可能性。 RQGM 通过使评估器也可进化来扩展这个循环：评估器判断子进程的轨迹，而任务反馈和真实标签也指导评估器更新，使任务智能体及其评估机制能够共同改进 [234,361]。

群体进化智能体从集体的角度接近自我改进（Self-Improvement），将进化单位从孤立的父代改变为群体或群体，通过性能和新颖性选择父群体，并共享补丁、轨迹和失败经验，以便来自不同分支的发现可以由后来的后代重新组合[236]。其他系统使更广泛的进化过程本身可编辑。利用智能体进化将进化表述为一个交互式环境：累积的候选者、痕迹、失败、成本和分数成为流程级状态，而元编辑器则改变控制未来进化

片段的过程或操作上下文[237]。 Bilevel Autoresearch 在自动化研究循环中采取了相关举措，其中外循环读取内循环的代码和实验跟踪，并注入新的搜索机制，

alter subsequent research dynamics [238]. Broader work on harness and search-mechanism evolution extends the same trend: Meta-Harness searches over harness code using a filesystem history of prior sources, traces, and scores [217], Adaptive Auto-Harness maintains a harness tree for open-ended task streams and routes new tasks to specialized branches [239], and EvoX evolves parent-selection and variation strategies during LLM-driven program optimization [240]. Across these systems, the common movement is from producing a better next agent to maintaining self-modification history, selecting lineages with future potential, co-evolving the criteria that define improvement, sharing experience across branches, and changing the mechanism by which later improvements will be generated.

The same openness creates a governance problem: once self-modified agents, harnesses, skills, evaluators, or search procedures persist into future runtime, a bad commit can reshape the distribution of later proposals, contaminate the parent pool, or amplify objective hacking and safety drift. Statistical Gödel Machine addresses this at the commit-decision level by replacing formal proof obligations with statistical gates: a candidate edit is accepted only when paired evaluation and confidence certificates support superiority over the incumbent, while a global error budget controls cumulative risk across irreversible recursive edits [241]. Governed Evolution of Agent Runtimes generalizes this discipline into a runtime architecture in which prompts, tools, evaluators, routing policies, skills, and agent-generated code artifacts become persistent operational capabilities only through lifecycle promotion, validation, traceability, audit trails, and rollback [242]. ANNEAL gives a concrete instance of governed structural repair: recurring failures are converted into typed symbolic patches, but each patch must pass scoring, guardrails, canary tests, provenance recording, and deterministic rollback preparation before it enters future execution [243]. Safety monitoring adds another layer. SAHOO tracks goal drift, constraint preservation, and regression risk across recursive improvement cycles, making capability gains legible together with their alignment cost [244]. ANCHOR studies human-like supervision inside self-evolving systems, showing how review at phases such as task proposal, planning, output verification, and execution-result feedback can mitigate safety degradation while preserving core capabilities [245]. Governed recursive self-modification therefore combines the search for better successor agents with a controlled process for deciding which changes are retained. Sustainable RSI requires validation signals, statistical confidence, rollback paths, auditability, and automated oversight to become part of the self-improvement loop itself.

## 4. Agent Data System Self-Improvement

The lifecycle of training and evaluation data is a critical dimension of self-improving agent systems. Existing research on self-improvement mainly focuses on model training or agent harness design. However, the data system that supplies training and evaluation signals is equally important to the continued improvement of an agent system.

Figure 5 illustrates that the agent data system typically consists of two stages: **data production** and **data utilization**. The data production stage comprises four modules: environment generation or simulation, task synthesis, trajectory synthesis, and verification and quality assurance. Together, these modules form a pipeline that generates and validates training and evaluation data. The data utilization stage consists of curriculum design and adaptation, which determine how the generated data are selected and allocated for agent training. These two stages are closely coupled: training requirements from the utilization stage guide subsequent data production, while data production determines what the utilization stage can operate on.

# 原文第 23 页 | 中文翻译初稿

23 共 70

改变后续的研究动态[238]。关于 运行支撑框架（Harness） 和搜索机制演变的更广泛的工作扩展了相同的趋势：Meta-运行支撑框架（Harness） 使用先前源、跟踪和分数的文件系统历史记录来搜索 运行支撑框架（Harness） 代码 [217]，自适应 Auto-运行支撑框架（Harness） 维护用于开放式任务流和路由的运行支撑框架（Harness） 树专门分支的新任务[239]，并且EvoX在LLM驱动的程序优化期间发展父选择和变异策略[240]。在这些系统中，共同的运动是从产生更好的下一个智能体到维护自我修改历史，选择具有未来潜力的谱系，共同进化定义改进的标准，跨分支共享经验，以及改变产生后续改进的机制。

同样的开放性也会产生治理问题：一旦自我修改的智能体、运行支撑框架（Harnesses）、技能、评估器或搜索程序持续到未来的运行时，错误的提交可能会重塑后来提案的分布，污染父池，或放大客观黑客攻击和安全漂移。统计哥德尔机通过用统计门取代正式证明义务，在提交决策层面解决了这个问题：只有当成对的评估和置信度证书支持优于现任者时，候选编辑才会被接受，而全局错误预算控制不可逆递归编辑的累积风险[241]。智能体运行时的治理演化将这一规则概括为一个运行时架构，其中提示、工具、评估器、路由策略、技能和智能体生成的代码工件只有通过生命周期升级、验证、可追溯性、审计跟踪和回滚才能成为持久的操作功能[242]。 ANNEAL 给出了受控结构修复的具体实例：重复出现的故障被转换为类型化的符号补丁，但每个补丁在进入未来执行之前必须通过评分、护栏、金丝雀测试、出处记录和确定性回滚准备[243]。安全监控又增加了一层。 SAHOO 跟踪递归改进周期中的目标漂移、约束保留和回归风险，使能力增益及其调整成本清晰可见 [244]。 ANCHOR 研究自我进化系统内的类人监督，展示了任务提案、规划、输出验证和执行结果反馈等阶段的审查如何在保留核心能力的同时减轻安全退化[245]。因此，受控递归自我修改将寻找更好的后继智能体与决定保留哪些更改的受控过程结合起来。可持续 RSI 需要验证信号、统计置信度、回滚路径、可审计性和自动监督，以成为 自我改进（Self-Improvement） 循环本身的一部分。

4.座席数据系统自我改进（Self-Improvement）

训练和评估数据的生命周期是自我改进（Self-Improving）智能体系统的关键维度。现有对自我改进（Self-Improvement）的研究主要集中在模型训练或Agent 运行支撑框架（Harness）设计上。然而，提供训练和评估信号的数据系统对于智能体系统的持续改进同样重要。

图5说明了智能体数据系统通常由两个阶段组成：数据生产和数据利用。数据生产阶段包括四个模块：环境生成或模拟、任务合成、轨迹合成以及验证和质量保证。这些模块共同形成了一个生成并验证训练和评估数据的管道。数据利用阶段包括课程设计和适应，这决定了如何选择和分配生成的数据用于智能体训练。这两个阶段紧密耦合：利用阶段的训练需求指导后续的数据生产，而数据生产则决定利用阶段可以操作什么。

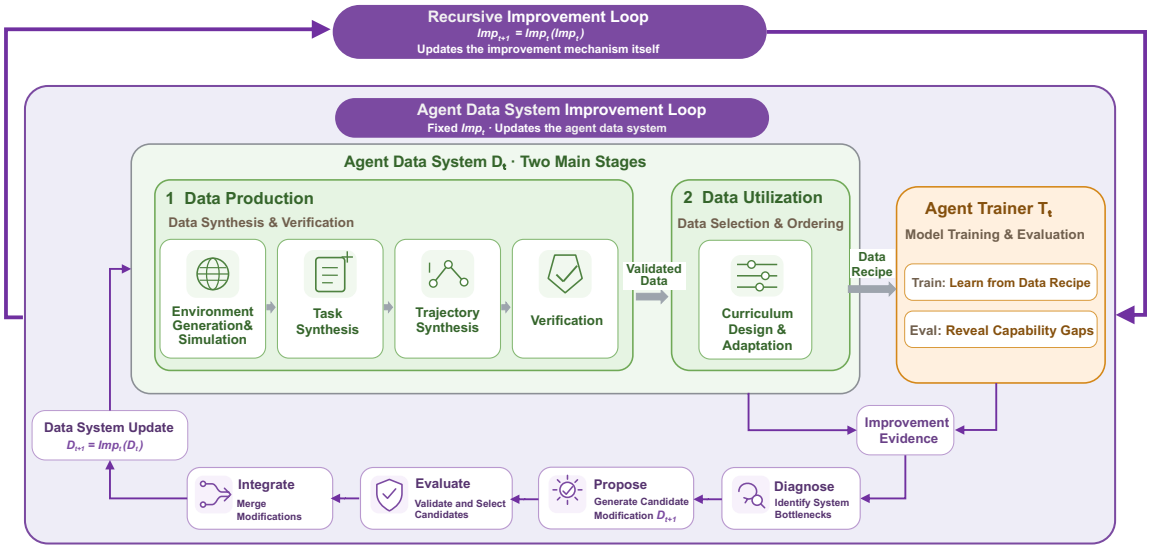



**Figure 5.** Conceptual framework of the agent data system, its improvement loop, and the recursive improvement loop. The data system consists of two main stages. *Data production* encompasses four modules that create new training and evaluation data. *Data utilization* consists of curriculum design and adaptation, which allocate the produced data for agent model training. The improvement loop uses training and evaluation feedback to diagnose bottlenecks, propose modifications, validate candidates, and promote a successor data system $\mathcal{D}_{t+1}$. At the outer level, improvement histories and validation outcomes provide evidence for revising the improvement mechanism $Imp_t$ itself, yielding $Imp_{t+1}$ when a candidate is promoted.

In this survey, **agent data system self-improvement** refers to an autonomous process in which a data system iteratively improves itself to generate higher-quality data for agent training and evaluation, without human intervention. To meet this definition, the modules within the data system need to actively modify their own implementations or adapt their behaviors based on verification results and model training outcomes, as shown in Figure 6.

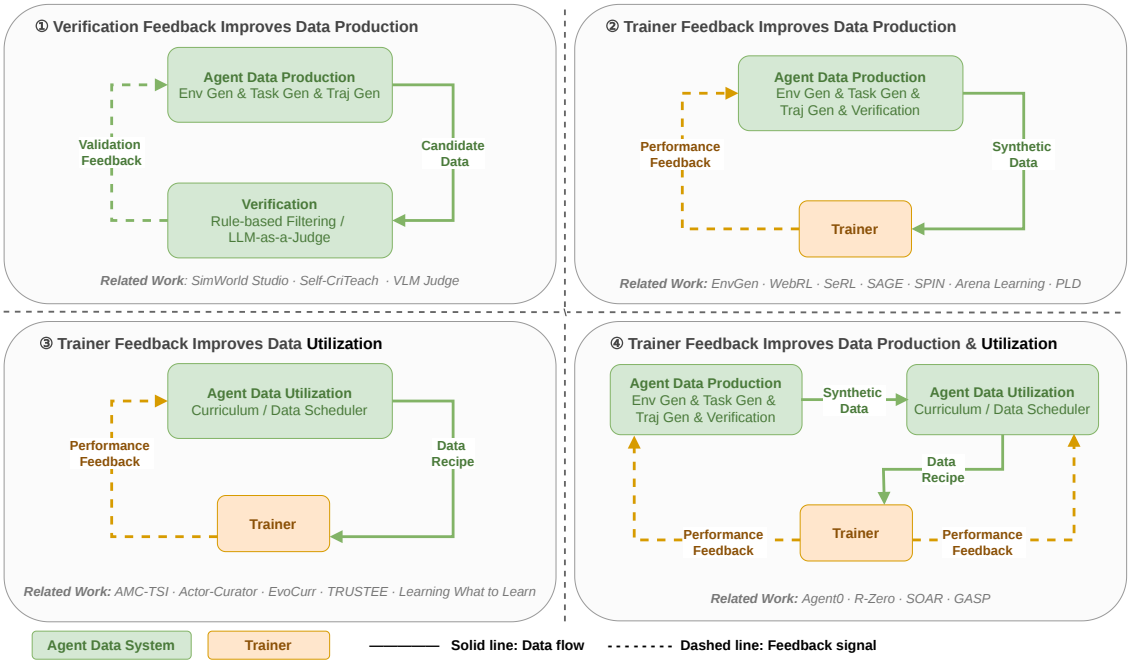



**Figure 6.** Self-improvement of data system modules in feedback loops.

Compared with conventional data pipelines that rely on manual design and improvement, a self-improving data system has advantages on both the training and evaluation sides. On the training side,

# 原文第 24 页 | 中文翻译初稿

24 共 70

递归改进循环
Impt+1 = Impt (Impt )
更新改进机制本身
智能体数据系统改进循环
固定Impt · 更新智能体数据系统
智能体数据系统 D □ · 两个主要阶段
1 数据生产 2 数据利用
智能体训练器 T □
数据合成与验证 数据选择与排序
模型训练与评估
数据
验证数据配方训练：从数据配方中学习
环境生成与模拟综合任务轨迹综合验证课程适应设计与评估：揭示能力差距
数据系统更新改进
Dt+1 = Impt (Dt ) 证据
集成评估建议诊断
合并验证并选择生成候选者识别系统
修改候选修改Dt+1瓶颈

图 5. 智能体数据系统的概念框架、其改进循环以及递归改进循环。数据系统由两个主要阶段组成。 _数据生产_包含四个模块，用于创建新的训练和评估数据。 _数据利用_包括课程设计和适应，将产生的数据分配给智能体模型训练。改进循环使用训练和评估反馈来诊断瓶颈、提出修改建议、验证候选者并推广后继数据系统_Dt_ +1。在外部层面，改进历史和验证结果为修改改进机制_Impt_本身提供了证据，当候选人晋升时产生_Impt_+1。

在本篇综述中，智能体数据系统自我改进（Self-Improvement）是指数据系统迭代改进自身以生成更高质量的数据用于智能体训练和评估的自主过程，无需人工干预。为了满足这个定义，数据系统中的模块需要根据验证结果和模型训练结果主动修改自己的实现或调整其行为，如图6所示。

① 验证反馈改善数据生产 ② 训练师反馈改善数据生产
智能体数据生产 智能体数据生产
Env Gen & Task Gen & Traj Gen Env Gen & Task Gen &
Traj Gen & Verification
验证候选性能综合
反馈数据反馈数据
验证
基于规则的过滤/训练师
LLM-as-a-Judge

相关工作：SimWorld Studio · Self-CriTeach · VLM Judge 相关工作：EnvGen · WebRL · SeRL · SAGE · SPIN · Arena Learning · PLD
\
③ 训练师反馈提高数据利用率 ④ 训练师反馈提高数据生产和利用率
智能体数据生产
课程/数据调度程序智能体数据利用率 Env Gen & Task Gen & Traj Gen & Verification 合成数据 课程/数据调度程序智能体数据利用率
绩效数据
反馈配方数据
配方
训练师
绩效训练师绩效
反馈反馈
相关工作：AMC-TSI · Actor-Curator · EvoCurr · TRUSTEE · 学习要学的内容 相关工作：Agent0 · R-Zero · SOAR · GASP
智能体数据系统训练师 ----- 实线：数据流 - - - - - - - -虚线：反馈信号

图 6. 反馈回路中数据系统模块的 自我改进（Self-Improvement）。

与依赖人工设计和改进的传统数据管道相比，自我改进的数据系统在训练和评估方面都具有优势。在训练方面，

it enables continuous generation of difficulty-adaptive curricula that target the current weaknesses of the agent model [246]. This enables adaptation of the effective training distribution beyond the limitations of a fixed corpus. On the evaluation side, the data system can continuously synthesize and refine benchmarks that evolve with the agent's capabilities. This helps prevent diminishing discriminability in evaluation and ensures that the feedback signals remain informative for subsequent improvement.

This section is organized by the *scope* of self-improvement. We first discuss *self-improvement in agent data production* (Section 4.1), where each data production module improves itself to adaptively produce agent data tailored to the current model's weaknesses. We then examine *self-improvement in agent data utilization* (Section 4.2), where curriculum design and adaptation improve the way produced data are selected, ordered, and allocated for agent model training. Finally, we consider *co-improvement across multiple modules in data production and utilization* (Section 4.3), where two or more modules are connected through feedback loops, enabling them to improve collaboratively.

### *4.1. Self-Improvement in Agent Data Production*

As shown in Figure 5, agent data production includes the environment generation or simulation, task synthesis, trajectory synthesis, and verification of training and evaluation data. This subsection examines improvement within each individual module of the data production. A system may contain several interacting modules and still belong to this category under either of two conditions. The first is *single-module improvement*, where only one module improves substantially, while the remaining modules are fixed and serve as feedback sources or evaluation mechanisms. The second is *independent improvement*, where several modules improve, but their improvement processes are mutually independent with no causal influence between them.

#### 4.1.1. Environment Generation or Simulation

An agent receives its feedback signals primarily from the environment it operates in. The fidelity, diversity, and difficulty distribution of training environments directly shape the agent's capabilities. Methods for preparing these environments can be divided into two classes. *Environment-generation methods* synthesize environments, including environment configurations, state spaces, and tool resources. *Environment-simulation methods* train a model to predict environmental dynamics and produce observation-action feedback. These two classes differ in mechanism but share a common goal: providing agents with feedback approximating actual operations in real-world environments.

Self-improvement in environment generation targets the validity and diversity of the produced environments. Compared with static, manually designed environments, a self-improving generator can adapt its outputs as the agent's capabilities grow [246]. It can also incorporate feedback to correct errors in previously generated configurations. With self-improvement, the generated environments remain aligned with the agent's current learning needs throughout training.

Environment-generation methods can be further organized by how they update the training distribution. The first class uses agent performance feedback to guide environment configuration generation. *EnvGen* [246] uses a fixed LLM to design environment configurations that target the weaknesses of an agent model trained by reinforcement learning. This process keeps the generated environments aligned with the agent's evolving capability boundary. *Adaptive Environment Generation* [247] extends the feedback-guided paradigm to spatial layout manipulation. It employs a fixed LLM as both a trajectory analyzer and an environment modifier. The analyzer extracts structured diagnostic feedback from the agent's navigation behavior, including failure modes and intermediate concerns, while the modifier adjusts individual object positions and orientations accordingly. This object-level adaptation uses collision-aware placement to produce physically valid layouts and targets specific spatial challenges without redesigning the entire environment.

The second class follows the unsupervised environment design paradigm, in which the curriculum is expanded through either search or distribution-based synthesis. *ACCEL* [248] represents the search-based approach. It applies small random mutations to existing environments and retains

# 原文第 25 页 | 中文翻译初稿

70 中的 25

它能够持续生成针对智能体模型当前弱点的难度自适应课程[246]。这使得有效训练分布的适应超越了固定语料库的限制。在评估方面，数据系统可以不断综合和完善随着智能体能力而发展的基准。这有助于防止评估中的辨别力减弱，并确保反馈信号为后续改进提供信息。

本节由 自我改进（Self-Improvement） 的_范围_组织。我们首先讨论智能体数据生产中的_自我改进（Self-Improvement）_（第 4.1 节），其中每个数据生产模块都会自我改进，以自适应地生成针对当前模型的弱点的智能体数据。然后，我们检查智能体数据利用中的_自我改进（Self-Improvement）_（第 4.2 节），其中课程设计和适应改进了为智能体模型训练选择、排序和分配生成的数据的方式。最后，我们考虑_数据生产和利用中跨多个模块的共同改进_（第 4.3 节），其中两个或多个模块通过反馈循环连接，使它们能够协同改进。

4.1。智能体数据生产中的自我改进（Self-Improvement）

如图5所示，智能体数据生产包括环境生成或模拟、任务合成、轨迹合成以及训练和评估数据的验证。本小节检查数据生成的每个单独模块内的改进。一个系统可能包含多个交互模块，并且在以下两种情况之一下仍然属于此类别。第一个是_单模块改进_，只有一个模块有实质性改进，而其余模块则固定并作为反馈源或评估机制。第二种是_独立改进_，即多个模块进行改进，但它们的改进过程是相互独立的，相互之间不存在因果影响。

4.1.1.环境生成或模拟

智能体主要从其运行的环境中接收反馈信号。训练环境的保真度、多样性和难度分布直接影响智能体的能力。准备这些环境的方法可以分为两类。 _环境生成方法_合成环境，包括环境配置、状态空间和工具资源。_环境模拟方法_训练模型来预测环境动态并产生观察行动反馈。这两个类在机制上有所不同，但有一个共同的目标：为智能体提供接近现实环境中实际操作的反馈。

环境生成中的自我改进（Self-Improvement）目标是生成环境的有效性和多样性。与静态、手动设计的环境相比，自我改进（Self-Improving） 生成器可以随着智能体能力的增长而调整其输出[246]。它还可以结合反馈来纠正先前生成的配置中的错误。借助 自我改进（Self-Improvement），生成的环境在整个训练过程中保持与智能体当前的学习需求保持一致。

环境生成方法可以通过它们如何更新训练分布来进一步组织。第一类使用智能体性能反馈来指导环境配置生成。 EnvGen [246] 使用固定的 LLM 来设计环境配置，以针对 强化学习（Reinforcement Learning） 训练的智能体模型的弱点。此过程使生成的环境与智能体不断发展的能力边界保持一致。 自适应环境生成[247]将反馈引导范式扩展到空间布局操作。它采用固定的法学硕士作为轨迹分析器和环境修改器。分析器从智能体的导航行为中提取结构化的诊断反馈，包括故障模式和中间问题，而修改器则相应地调整各个对象的位置和方向。这种对象级适应使用碰撞感知放置来生成物理上有效的布局并针对特定的空间挑战，而无需重新设计整个环境。

第二节课遵循无监督环境设计范式，其中课程通过搜索或基于分布的综合进行扩展。 ACCEL[248]代表基于搜索的方法。它将小的随机突变应用于现有环境并保留

variants whose estimated regret exceeds a fixed threshold. The selected environments therefore remain challenging for the agent and can serve as starting points for further mutations. *DRED* [249] instead represents distribution-grounded synthesis. It uses a pre-trained variational autoencoder to generate environments that remain consistent with the target distribution, thereby reducing distributional shift and improving zero-shot generalization.

The third class improves the environment generator's capability through continual expansion of reusable tools and skills. *SimWorld Studio* [250] uses a tool-augmented coding agent to construct physically grounded 3D environments in Unreal Engine. When rule-based physics checks and vision-language model critiques identify recurring failures, the agent converts successful fixes into reusable tools and skills stored in its library. Agent performance feedback further guides difficulty adjustment, so that newly generated environments remain near the agent's capability boundary.

These generation approaches improve the validity and informativeness of the training environments through different operations. Feedback-guided generation targets observed weaknesses, mutation-based search expands the curriculum near the capability boundary, and distribution-grounded synthesis preserves task relevance during expansion. Additionally, self-improvement of the environment generator's harness allows the environment generator to internalize quality-control lessons and avoid previously encountered failure modes. The main challenge lies in maintaining environment validity at scale, because greater autonomy can increase the frequency of degenerate, unsolvable, or trivially easy configurations. Robust quality-control mechanisms are therefore necessary to keep the synthesized environments effective for agent training.

For environment simulation, self-improvement aims to enhance the coverage and accuracy of the simulator's dynamics. As the agent explores new regions, a self-improving simulator can extend its coverage of the state-action space and progressively reduce the simulation-to-reality gap without manual recalibration. The main risk is that extrapolation beyond the simulator's training distribution may introduce drift into unrealistic dynamics. Such drift can lead to policies that exploit simulator artifacts rather than learn behavior that transfers to the target environment. Effective simulation improvement must therefore balance broader coverage with control of model drift. *DreamGym* [251] trains a reasoning-based experience model to predict environment dynamics in an abstract textual state space. Rather than replicating raw observations such as HTML or pixels, the model operates on structured state representations and uses chain-of-thought reasoning to predict subsequent states and rewards. An experience replay buffer seeded with trajectories reduces hallucination in state predictions, while a curriculum task generator selects tasks with high reward entropy to keep task difficulty near the agent's capability boundary.

#### 4.1.2. Task and Trajectory Synthesis

Training and evaluation tasks translate capability objectives into specific units for learning and assessment. Therefore, the quality of these tasks is central to the performance of the overall system. Training tasks determine which capabilities the agent can acquire, whereas evaluation tasks reveal capability gains and weaknesses. An agent task typically comprises an instruction, an initial environment state produced by methods in Section 4.1.1, explicit constraints, available tools and resources, and verifiable evaluation criteria. Additionally, for supervised fine-tuning task data instances, corresponding trajectories synthesized from the task are also required. Task synthesis must coordinate these elements so that each instance is valid, diverse, and aligned with its intended training or evaluation purpose.

We first classify existing methods by their primary synthesis object. **Task synthesis** constructs task instructions, constraints, resource settings, and evaluation criteria, which define what the agent must solve. **Trajectory synthesis** builds solutions and interaction trajectories for a given task, which record the experiences of the agent. We categorize each system according to the object that plays a central role in its improvement loop.

Within **Task synthesis**, we further distinguish methods by the improvement signal used to adapt, select, or assess the synthesized tasks. The first class of task-synthesis methods uses *diagnostic*

# 原文第 26 页 | 中文翻译初稿

26 共 70

估计后悔超过固定阈值的变体。因此，选定的环境对于智能体来说仍然具有挑战性，并且可以作为进一步突变的起点。 DRED [249] 相反代表基于分布的综合。它使用预先训练的变分自动编码器来生成与目标分布保持一致的环境，从而减少分布偏移并提高零样本泛化能力。

第三类通过不断扩展可重用工具和技能来提高环境生成器的能力。 SimWorld Studio [250] 使用工具增强编码智能体在虚幻引擎中构建物理接地的 3D 环境。当基于规则的物理检查和视觉语言模型批评识别出反复出现的故障时，智能体会将成功的修复转换为存储在其库中的可重用工具和技能。智能体性能反馈进一步指导难度调整，以便新生成的环境保持在智能体的能力边界附近。

这些生成方法通过不同的操作提高了训练环境的有效性和信息性。反馈引导生成针对观察到的弱点，基于突变的搜索将课程扩展到能力边界附近，而基于分布的合成在扩展过程中保留了任务相关性。此外，环境生成器 运行支撑框架（Harness） 的 自我改进（Self-Improvement） 允许环境生成器内化质量控制经验并避免以前遇到的故障模式。主要挑战在于维持大规模环境的有效性，因为更大的自主权可能会增加退化、无法解决或简单配置的频率。因此，需要强大的质量控制机制来保持合成环境对智能体训练的有效性。

对于环境模拟，自我改进（Self-Improvement）旨在增强模拟器动态的覆盖范围和准确性。当智能体探索新区域时，自我改进（Self-Improving） 模拟器可以扩展其状态动作空间的覆盖范围，并逐步缩小模拟与现实的差距，而无需手动重新校准。主要风险是超出模拟器训练分布的外推可能会导致不切实际的动态漂移。这种漂移可能导致策略利用模拟器工件，而不是学习转移到目标环境的行为。因此，有效的仿真改进必须在更广泛的覆盖范围与模型漂移的控制之间取得平衡。 DreamGym [251] 训练基于推理的体验模型来预测抽象文本状态空间中的环境动态。该模型不是复制 HTML 或像素等原始观察结果，而是对结构化状态表示进行操作，并使用思想链推理来预测后续状态和奖励。带有轨迹种子的体验重播缓冲区减少了状态预测中的幻觉，而课程任务生成器选择具有高奖励熵的任务，以将任务难度保持在智能体的能力边界附近。
4.1.2.任务和轨迹综合

训练和评估任务将能力目标转化为具体的学习和评估单元

。因此，这些任务的质量对于整个系统的性能至关重要。训练任务决定智能体可以获得哪些能力，而评估任务则揭示能力的增益和弱点。智能体任务通常包括指令、由第 4.1.1 节中的方法产生的初始环境状态、显式约束、可用工具和资源以及可验证的评估标准。此外，对于有监督的微调任务数据实例，还需要从任务合成相应的轨迹。任务合成必须协调这些元素，以便每个实例都是有效的、多样化的，并与其预期的训练或评估目的保持一致。

我们首先按主要综合对象对现有方法进行分类。任务综合构建任务指令、约束、资源设置和评估标准，定义智能体必须解决的问题。轨迹合成为给定任务构建解决方案和交互轨迹，记录智能体的经验。我们根据在改进循环中发挥核心作用的对象对每个系统进行分类。

在任务合成中，我们通过用于适应、选择或评估合成任务的改进信号进一步区分方法。第一类任务综合方法使用_diagnostic_

*performance feedback*. Its purpose is to identify capability gaps and generate tasks that target them. *WebRL* [252], for example, extracts failure patterns from agent experience and converts them into new training tasks. The task distribution changes with the agent's weaknesses, although the procedure that maps failures to tasks remains fixed. *CoEvolve* [253] extends this paradigm by extracting weakness signals from rollout trajectories. These signals guide a frozen LLM to re-explore the real environment along the identified failure dimensions, after which the resulting interactions are abstracted into validated task specifications and appended to the training set.

The second class uses *fixed quality-control signals* derived from rules, model agreement, or task structure. *SeRL* [254] generates instructions through few-shot prompting and selects them with fixed filters for similarity, content, length, and difficulty; majority voting provides an additional reward estimate. *Self-CriTeach* [255] uses a fixed prompting mechanism to synthesize symbolic planning domains. These domains define planning problems and supply structured criteria for downstream learning. In both systems, fixed quality-control signals shape the synthesized tasks, but the task-generation mechanism is not independently optimized.

The third class uses *competitive performance feedback*. In the frozen-opponent setting of *SAGE* [256], a Setter generates problems that it can solve but a fixed Opponent cannot. The competitive outcome therefore supplies a direct signal of task difficulty near the capability boundary. Only the Setter is trained in this setting, so the system does not establish co-improvement between the two roles.

In summary, these self-improving methods improve synthesized tasks along four main dimensions: relevance, validity, difficulty, and diversity. Diagnostic feedback increases relevance by directing generation toward observed capability gaps. Fixed quality-control signals improve validity and correctness by excluding inconsistent or poorly specified tasks. Similarity filtering also limits redundancy and helps preserve diversity. Competitive feedback adjusts difficulty toward the agent's current capability boundary, thereby keeping the generated tasks informative as the agent improves.

**Trajectory synthesis** methods generate agent learning experience for specified tasks. One group obtains such trajectories through competitive interaction. *SPIN* [257] contrasts outputs from the current model with those from a frozen previous iteration and fixed human references. *Arena Learning* [258] converts battles against fixed opponents into preference data using a fixed judge. The resulting trajectories become more challenging as the target model improves, but the interaction protocols and evaluation mechanisms remain fixed.

A second group generates trajectories through refinement or recovery. *EVOLVE* [259] turns initial and refined answers into preference data, whereas *RISE* [260] formulates iterative answer improvement as a multi-turn learning problem. *DNPO* [261] constructs preference pairs through dynamic labeling and controlled noise. *PLD* [262] instead collects recovery trajectories from specialized policies and distills them into a generalist model. Although these methods construct different forms of experience, they all apply a fixed trajectory-generation protocol around an improving model.

To sum up, these self-improving trajectory synthesis methods improve the usefulness of experience along several dimensions, including correctness, difficulty, and diversity. Competitive interaction produces more difficult and behaviorally diverse trajectories as the target model becomes stronger. Refinement-based methods improve the correctness and quality of candidate solutions, while recovery-based methods concentrate experience on observed failures. Preference construction and rubric-based assessment further increase the reliability of the supervision extracted from these trajectories.

#### 4.1.3. Verification and Quality Assurance

Verification and quality assurance protect the data pipeline from errors that could otherwise accumulate across self-improvement iterations. *Verification* assesses whether an individual task, trajectory, label, or outcome satisfies a specified correctness or consistency criterion. *Quality assurance* has a broader scope. It filters, ranks, repairs, or annotates data according to validity, reliability, difficulty, and provenance. These functions are essential because self-generated data can amplify specification errors, incorrect labels, and biased judgments when used for training. Reliable verification and quality assurance preserve the integrity of both learning signals and evaluation results.

# 原文第 27 页 | 中文翻译初稿

27 共 70

绩效反馈。其目的是识别能力差距并生成针对这些差距的任务。例如，WebRL [252] 从智能体经验中提取失败模式并将其转换为新的训练任务。尽管将失败映射到任务的过程保持不变，但任务分配会随着智能体的弱点而变化。 CoEvolve [253] 通过从 展开轨迹（Rollout） 轨迹中提取弱信号来扩展此范例。这些信号引导冻结的法学硕士沿着已识别的故障维度重新探索真实环境，之后将产生的交互抽象为经过验证的任务规范并附加到训练集。

第二类使用从规则、模型协议或任务结构派生的_固定质量控制信号_。 SeRL [254]通过几次提示生成指令，并使用固定的过滤器根据相似性、内容、长度和难度来选择它们；多数投票提供了额外的奖励估计。 Self-CriTeach[255]使用固定的提示机制来综合符号规划域。这些领域定义了规划问题并为下游学习提供结构化标准。在这两个系统中，固定的质量控制信号塑造了合成任务，但任务生成机制并未独立优化。

第三类使用_竞争性绩效反馈_。在 SAGE [256] 的冻结对手设置中，Setter 会产生它可以解决但固定对手无法解决的问题。因此，竞争结果提供了接近能力边界的任务难度的直接信号。在这种情况下，只有 Setter 接受过训练，因此系统不会在两个角色之间建立协同改进。

总之，这些 自我改进（Self-Improving） 方法从四个主要维度改进了综合任务：相关性、有效性、难度和多样性。诊断反馈通过引导一代人观察到的能力差距来提高相关性。固定的质量控制信号通过排除不一致或指定不明确的任务来提高有效性和正确性。相似性过滤还限制冗余并有助于保持多样性。竞争性反馈会根据智能体当前的能力边界调整难度，从而随着智能体的改进，使生成的任务保持信息丰富。

轨迹合成方法为指定任务生成智能体学习经验。一组通过竞争性互动获得这样的轨迹。 SPIN [257] 将当前模型的输出与冻结的先前迭代和固定的人类参考的输出进行对比。 竞技场学习[258]使用固定判断将与固定对手的战斗转换为偏好数据。随着目标模型的改进，产生的轨迹变得更具挑战性，但交互协议和评估机制保持不变。

第二组通过细化或恢复生成轨迹。 EVOLVE [259] 将初始和精炼的答案转化为偏好数据，而 RISE [260] 将迭代答案改进表述为多轮学习问题。 DNPO [261]通过动态标签和受控噪声构建偏好对

。 PLD [262] 相反，从专门的策略中收集复苏轨迹，并将其提炼成通用模型。尽管这些方法构建了不同形式的经验，但它们都围绕改进模型应用了固定的轨迹生成协议。

综上所述，这些 自我改进（Self-Improving） 轨迹合成方法在多个维度上提高了经验的有用性，包括正确性、难度和多样性。随着目标模型变得更强，竞争性互动会产生更加困难和行为多样化的轨迹。基于细化的方法提高了候选解决方案的正确性和质量，而基于恢复的方法则将经验集中在观察到的故障上。偏好构建和基于标准的评估进一步提高了从这些轨迹中提取的监督的可靠性。

4.1.3.验证和质量保证

验证和质量保证可保护数据管道免受错误的影响，否则这些错误可能会在 自我改进（Self-Improvement） 迭代中累积。 _验证_评估单个任务、轨迹、标签或结果是否满足指定的正确性或一致性标准。 _质量保证_的范围更广。它根据有效性、可靠性、难度和来源对数据进行过滤、排名、修复或注释。这些功能至关重要，因为自我生成的数据在用于训练时可能会放大规格错误、不正确的标签和有偏见的判断。可靠的验证和质量保证可以保持学习信号和评估结果的完整性。

In this subsection, self-improving verification refers to cases in which the verifier, judge, or quality-control policy is itself updated across iterations. This boundary excludes fixed verification mechanisms. Executable checks [362], agreement-based filters [363,364], and frozen critic signals [365] can stabilize a self-improving data loop, but they do not constitute self-improving verification if their own decision rules remain fixed.

Self-improving verification means that improvement occurs within the verification module. *Self-Improving Vision-Language Model (VLM) Judges* [263] iteratively trains a VLM judge with self-synthesized multimodal data at different quality levels. This process forms a single-module self-improvement loop for verification because the judge itself is refined over successive iterations. *Escaping Model Collapse* [366] provides a complementary analysis of verifier-guided retraining. It shows that an external verifier can provide useful information and yield short-term gains. However, repeated optimization may eventually pull the model toward the verifier's own knowledge boundary. This result highlights a central risk for self-improving verification: if the quality signal is imperfect, accepted data can reinforce verifier biases and narrow the learning distribution.

In summary, self-improving verification improves the accuracy, coverage, and robustness of quality control. Its main value is to reduce incorrect acceptance or rejection, expand coverage of failure modes, and prevent noisy data from propagating through later training iterations.

### *4.2. Self-Improvement in Agent Data Utilization*

The previous subsection focused on self-improvement in agent data production, where environment, task, and trajectory generation expand the pool of available training and evaluation data. This subsection instead focuses on self-improvement in agent data utilization. Rather than generating new data, it improves how the produced data are used for model training through curriculum design and adaptation. *Curriculum design* determines how available training data are organized, typically by selecting and ordering environments, tasks, or trajectories according to the coverage of task categories, difficulty levels, or the model's capability gaps. *Curriculum adaptation* focuses on the curriculum adaptor that expands, updates, and refines the curriculum during training. Together, curriculum design and curriculum adaptation determine the training data recipe by adapting data utilization to the agent model's evolving capabilities.

Methods in self-improving curriculum adaptation can be organized into two levels. The first distinction is whether the curriculum adaptor is rule-based or learned. Rule-based methods can still be adaptive because they update the curriculum in response to the model trainer's feedback. Learned curriculum adaptation methods go further by optimizing the curriculum adaptor itself. Within learned curriculum adaptation, a second distinction is whether the curriculum adaptor is learned before deployment or updated online during agent training.

Rule-based Curriculum Adaptation.

These methods adapt the curriculum, but the adaptation rule itself is predefined. The changing state may be a difficulty parameter, a memory of past strategies, or a compact playbook of curated knowledge. Within this class, methods differ in what aspect of the training distribution they adapt. *TRUSTEE* [264] trains a tool-calling agent on trajectories from a fully simulated environment. It steers task difficulty with a scalar parameter that is updated by a predefined threshold rule based on batch reward. This parameter conditions the task generator along five dimensions: tool count, interaction turns, system-prompt specificity, user expertise, and evaluation strictness. As a result, the generated tasks can follow the current capability boundary of the agent. *EvoCurr* [265] follows the same general pattern. A prompted designer agent lowers task difficulty when the solver struggles and raises it when the solver succeeds. *Learning What to Learn* [266] focuses on content selection rather than difficulty control. It shows that a carefully selected subset of tasks can approach the performance obtained from the full dataset. The method uses frozen LLMs as a generator, a reflector, and a curator to distill a compact playbook.

# 原文第 28 页 | 中文翻译初稿

28 共 70

在本小节中，自我改进（Self-Improving） 验证是指验证者、判断者或质量控制策略本身在迭代中更新的情况。该边界不包括固定的验证机制。可执行检查[362]、基于协议的过滤器[363,364]和冻结批评信号[365]可以稳定自我改进（Self-Improving）数据循环，但如果它们自己的决策规则保持固定，则它们不构成自我改进（Self-Improving）验证。

自我改进（Self-Improving）验证意味着改进发生在验证模块内。 SelfImproving Vision-Language Model (VLM) Judges [263] 使用不同质量水平的自我合成多模态数据迭代训练 VLM 法官。这个过程形成了一个用于验证的单模块 自我改进（Self-Improvement） 循环，因为判断本身是通过连续迭代进行细化的。 Escapeing Model Collapse [366] 提供了验证者引导的再训练的补充分析。它表明外部验证者可以提供有用的信息并产生短期收益。然而，重复的优化最终可能会将模型拉向验证者自己的知识边界。这一结果凸显了 自我改进（Self-Improving） 验证的一个中心风险：如果质量信号不完美，接受的数据可能会加剧验证者的偏差并缩小学习分布。

综上所述，自我改进（Self-Improving）验证提高了质量控制的准确性、覆盖率和稳健性。其主要价值是减少错误接受或拒绝，扩大故障模式的覆盖范围，并防止噪声数据通过以后的训练迭代传播。
4.2。智能体数据利用中的 自我改进（Self-Improvement）

上一小节重点介绍了智能体数据生成中的 自我改进（Self-Improvement），其中环境、任务和轨迹生成扩展了可用训练和评估数据池。本小节重点介绍智能体数据利用中的 自我改进（Self-Improvement）。它不是生成新数据，而是通过课程设计和适应来改进生成的数据用于模型训练的方式。 _课程设计_确定如何组织可用的训练数据，通常是根据任务类别的覆盖范围、难度级别或模型的能力差距来选择和排序环境、任务或轨迹。 _课程适配_重点关注在训练期间扩展、更新和细化课程的课程适配器。课程设计和课程调整共同通过使数据利用适应智能体模型不断发展的能力来确定训练数据配方。

自我改进（Self-Improving）课程改编中的方法可以分为两个层次。第一个区别是课程适配器是基于规则的还是学习的。基于规则的方法仍然具有适应性，因为它们会根据模型训练者的反馈来更新课程。学习的课程适应方法通过优化课程适配器本身而更进一步。在学习的课程适应中，第二个区别是课程适配器是在部署之前学习还是在智能体训练期间在线更新。
基于规则的课程适应。

这些方法适应课程，但适应规则本身是预先定义的

。变化的状态可能是难度参数、过去策略的记忆或精选知识的紧凑剧本。在此类中，方法的不同之处在于它们所适应的训练分布的方面。 TRUSTEE [264] 在完全模拟环境的轨迹上训练工具调用智能体。它通过标量参数来控制任务难度，该参数由基于批量奖励的预定义阈值规则进行更新。该参数从五个维度来调节任务生成器：工具数量、交互轮次、系统提示特异性、用户专业知识和评估严格性。因此，生成的任务可以遵循智能体的当前能力边界。 EvoCurr [265]遵循相同的一般模式。当解决者陷入困境时，有提示的设计智能体会降低任务难度，并在解决者成功时提高任务难度。 Learning What to Learn[266]侧重于内容选择而不是难度控制。它表明，精心选择的任务子集可以接近从完整数据集获得的性能。该方法使用冻结的法学硕士作为生成器、反射器和策展人来提炼出紧凑的剧本。

Learned Curriculum Adaptation.

Within learned curriculum adaptation, the first subtype uses an *offline meta-learned curriculum adaptor*. The curriculum adaptation strategy is optimized during a meta-training stage and is then deployed as an adaptive strategy. *AMC-TSI* [267] jointly meta-learns a curriculum adaptor and a learning algorithm. It trains three neural networks: a difficulty estimator, a meta-policy that selects operators and allocates compute, and a process reward model. After meta-training, the learned strategy adjusts its test-time decisions to observed learning dynamics. The key difference from rule-based curriculum adaptation is the source of the adaptation strategy. The curriculum adaptor is learned before deployment rather than manually predefined. The second learned subtype uses an *online learned curriculum adaptor*. In this subtype, the selector is itself a learning module inside the current training loop. *Actor-Curator* [268] trains a neural curator to select problems that maximize expected policy improvement. The actor is updated through reinforcement learning, and changes in actor performance provide feedback for refining the curator. The curriculum therefore moves with the actor's capability boundary. Meanwhile, the curriculum adaptor continues to improve during agent training through its own learning objective.

In summary, these methods improve the data curriculum for agent model training. They make the data curriculum more relevant to the current agent model's weaknesses and adjust task difficulty as the agent model improves. Curriculum adaptation increases data and computational efficiency, while selective coverage reduces redundancy without sacrificing diversity. These objectives must be balanced to avoid excessive focus on recent failures and the loss of previously acquired capabilities.

### *4.3. Multi-Module Co-Improvement in Agent Data System*

The previous subsections 4.1 and 4.2 examine cases in which each module is optimized independently. This subsection focuses on the collaborative improvement of two or more interdependent data system modules within a feedback loop, as shown in Figure 6. In other words, each module both shapes and benefits from the improvement of another module, making their improvements mutually dependent.

Collaborative improvement addresses a key limitation of optimizing each module independently. As the agent model's capabilities evolve, a data system module that remains fixed can become a bottleneck, preventing the rest of the data system from adapting effectively. Collaborative improvement allows every data system module in the feedback loop to adapt to the agent model's evolving capabilities, ensuring that training data remain appropriately challenging, experience coverage continues to expand, and low-quality data are progressively filtered out.

We organize the following discussion according to where the feedback loop is established within the agent data system. *Co-Improvement within Data Production* (Section 4.3.1) covers feedback loops among data-production modules, including environment generation or simulation, task and trajectory synthesis, and verification. In these systems, the outputs of one production module provide feedback for improving another, thereby enhancing the quality and informativeness of the generated data for subsequent agent learning. *Co-Improvement of Data Production and Utilization* (Section 4.3.2) extends these feedback loops to connect data production with data utilization, completing a closed-loop feedback cycle among data production, data utilization, and model training.

#### 4.3.1. Co-Improvement Within Data Production

Multiple modules within data production can interact through a feedback loop, where the improvement of one module provides signals or resources that facilitate the improvement of others. This subsubsection reviews co-improvement methods within the data production stage and categorizes existing approaches based on the production modules that are jointly improved. The first category characterizes collaborative improvement between synthesis and verification, where generated solutions expose limitations in the assessment mechanism and the improved verifier provides more informative feedback for subsequent synthesis. The second category examines collaborative improvement between

# 原文第 29 页 | 中文翻译初稿

29 共 70
学习课程适应。

在学习课程适应中，第一个子类型使用_离线元学习课程适配器_。课程适应策略在元训练阶段进行优化，然后部署为自适应策略。 AMC-TSI [267] 联合元学习课程适配器和学习算法。它训练三个神经网络：难度估计器、选择运算符和分配计算的元策略以及过程奖励模型。元训练后，学习策略会根据观察到的学习动态调整其测试时决策。与基于规则的课程适应的主要区别在于适应策略的来源。课程适配器是在部署之前学习的，而不是手动预定义的。第二个学习子类型使用_在线学习课程适配器_。在这种子类型中，选择器本身就是当前训练循环内的学习模块。 Actor-Curator [268] 训练神经策展人来选择最大化预期策略改进的问题。演员通过强化学习（Reinforcement Learning）进行更新，演员表演的变化为策展人的完善提供反馈。因此，课程随着演员的能力范围而变化。同时，课程适配器在智能体训练过程中通过自己的学习目标不断改进。

总之，这些方法改进了智能体模型训练的数据课程。他们使数据课程与当前智能体模型的弱点更加相关，并随着智能体模型的改进调整任务难度。课程适应提高了数据和计算效率，而选择性覆盖则在不牺牲多样性的情况下减少了冗余。必须平衡这些目标，以避免过度关注最近的失败和先前获得的能力的丧失。
4.3。 Agent数据系统多模块协同改进

前面的 4.1 和 4.2 小节研究了每个模块独立优化的情况。本小节重点关注反馈循环内两个或多个相互依赖的数据系统模块的协作改进，如图 6 所示。换句话说，每个模块都塑造另一个模块的改进并从中受益，从而使它们的改进相互依赖。

协作改进解决了独立优化每个模块的关键限制。随着智能体模型功能的发展，保持固定的数据系统模块可能会成为瓶颈，从而阻止数据系统的其余部分有效适应。协同改进使得反馈循环中的每个数据系统模块能够适应智能体模型不断发展的能力，确保训练数据保持适当的挑战性，经验覆盖范围不断扩大，低质量数据被逐步过滤掉。

我们根据反馈循环在智能体数据系统中建立的位置来组织以下讨论。 数据生产中的协同改进（第 4.3.1 节）涵盖数据生产模块之间的反馈循环，包括环境生成或模拟、任务和轨迹综合以及验证

。在这些系统中，一个生产模块的输出提供反馈以改进另一个生产模块，从而提高生成数据的质量和信息量以供后续智能体学习。 数据生产与利用的协同提升（4.3.2节）将这些反馈循环延伸，将数据生产与数据利用连接起来，完成数据生产、数据利用和模型训练之间的闭环反馈循环。
4.3.1.数据生产中的共同改进

数据生产中的多个模块可以通过反馈循环进行交互，其中一个模块的改进提供了促进其他模块改进的信号或资源。本小节回顾了数据生产阶段的共同改进方法，并根据共同改进的生产模块对现有方法进行分类。第一类的特点是综合和验证之间的协作改进，其中生成的解决方案暴露了评估机制的局限性，而改进的验证器为后续综合提供了更多信息反馈。第二类检查之间的协作改进

environment construction and task synthesis, where agent failures guide the generation of new tasks and the refinement of interactive environments.

Co-Improvement of Synthesis and Verification.

The coupled data synthesis and verification modules within the data system can jointly improve through their interactions. The agent model produces new behaviors that expose a changing error distribution. The assessment mechanism refines its judgments in response, and the improved mechanism then supplies more informative learning signals back to the agent model. The key property is that the assessment mechanism has its own learning objective and is not merely a frozen judge.

*Agent0-VL* [269] realizes this coupling within a shared vision-language model. Its solver produces increasingly challenging tool-integrated reasoning traces, while its verifier uses tool-grounded critique to provide fine-grained rewards. The two roles improve together even though they share parameters. *ACE* [270] uses a single backbone model that alternates between the roles of code solver and adversarial test generator. Unlike ground-truth tests, which contain both inputs and expected outputs, the adversarial unit tests generated by ACE contain only test inputs and are used to expose execution-level failures such as corner cases, boundary violations, and runtime fragility. Across training rounds, candidate programs that pass both ground-truth tests and adversarial tests are selected for supervised fine-tuning of the solver. In addition, adversarial tests are labeled according to their execution effects: tests that distinguish robust programs from fragile ones are treated as desirable, whereas tests that fail to reveal useful differences are treated as undesirable. These labels are then used as preference signals to optimize the adversarial test generator through Kahneman-Tversky Optimization.

These methods share a common structure: the agent model reveals limitations in the current assessment, while the refined assessment provides stronger signals for further learning. By coupling these processes, the system postpones the saturation of a static verifier and maintains supervision that tracks the agent model's shifting error distribution. The task and environment sources remain unchanged; improvement occurs specifically at the evaluation interface.

Co-Improvement of Environment and Task Synthesis.

*Agent-World* [271] jointly improves the agent policy, training tasks, and interactive environments. It builds a large collection of tool-using environments from real-world databases and web services. After each training round, the updated policy is evaluated in these environments. A diagnosis agent analyzes failure traces to identify weak environments and produces guidelines for task generation. These guidelines steer subsequent task synthesis and increase database complexity within the identified weak environments, so that new tasks and more difficult environments are produced in response to the policy's observed weaknesses. The newly generated tasks and expanded environments are then used by the reinforcement-learning trainer to update the policy, forming a feedback loop among policy learning, task generation, and environment refinement under a fixed improvement rule.

Across these systems, co-improvement within data production makes the generated data increasingly aligned with the agent's evolving training needs. Because production modules exchange feedback with each other, errors observed in trajectories, environments, or verification results can be converted into signals to improve subsequent data generation. These feedback loops help maintain an appropriate level of difficulty, expand coverage over experience and failure modes that the agent has not yet mastered, fix errors in data production modules, and reduce the propagation of low-quality data. As a result, the data production stage can provide more valid, diverse, and informative data without relying on repeated human intervention.

#### 4.3.2. Co-Improvement of Data Production and Utilization

This subsubsection focuses on closed-loop systems in which data production, data utilization, and model training mutually influence each other through continuous feedback. The data production modules synthesize tasks, environments, and trajectories. These data are then selected and incorporated into the model training by the data utilization modules. In turn, the resulting training outcomes

# 原文第 30 页 | 中文翻译初稿

70 中的 30

环境构建和任务合成，其中智能体故障指导新任务的生成和交互环境的细化。
综合与验证的共同改进。

数据系统内耦合的数据合成和验证模块可以通过它们的交互来共同改进。智能体模型产生新的行为，暴露变化的错误分布。评估机制根据响应完善其判断，然后改进的机制向智能体模型提供更多信息丰富的学习信号。关键的特性是评估机制有自己的学习目标，而不仅仅是一个冻结的法官。

Agent0-VL [269] 在共享视觉语言模型中实现了这种耦合。它的求解器产生越来越具有挑战性的工具集成推理轨迹，而它的验证器使用基于工具的批评来提供细粒度的奖励。尽管这两个角色共享参数，但它们会共同进步。 ACE [270] 使用单一骨干模型，在代码求解器和对抗性测试生成器的角色之间交替。与包含输入和预期输出的真实测试不同，ACE 生成的对抗性单元测试仅包含测试输入，并用于暴露执行级故障，例如极端情况、边界违规和运行时脆弱性。在训练轮次中，选择通过地面实况测试和对抗性测试的候选程序来对求解器进行监督微调。此外，对抗性测试根据其执行效果进行标记：区分健壮程序和脆弱程序的测试被视为可取的，而无法揭示有用差异的测试则被视为不可取的。然后，这些标签用作偏好信号，通过 Kahneman-Tversky 优化来优化对抗性测试生成器。

这些方法有一个共同的结构：智能体模型揭示了当前评估的局限性，而细化的评估为进一步学习提供了更强的信号。通过耦合这些过程，系统推迟了静态验证器的饱和，并维持跟踪智能体模型的转移误差分布的监督。任务和环境来源不变；改进特别发生在评估界面上。
环境与任务综合的共同改进。

Agent-World[271]共同改进了智能体策略、训练任务和交互环境。它从现实世界的数据库和 Web 服务构建了大量工具使用环境。每轮训练结束后，都会在这些环境中评估更新的策略。诊断智能体分析故障痕迹以识别薄弱环境并生成任务生成指南。这些指导方针指导后续的任务合成并增加已识别的薄弱环境中的数据库复杂性，以便根据策略观察到的弱点产生新的任务和更困难的环境

。然后，强化学习训练器使用新生成的任务和扩展的环境来更新策略，在固定的改进规则下形成策略学习、任务生成和环境细化之间的反馈循环。

在这些系统中，数据生产中的共同改进使得生成的数据越来越符合智能体不断变化的训练需求。由于生产模块相互交换反馈，因此在轨迹、环境或验证结果中观察到的错误可以转换为信号，以改进后续数据生成。这些反馈循环有助于保持适当的难度水平，扩大智能体尚未掌握的经验和故障模式的覆盖范围，修复数据生产模块中的错误，并减少低质量数据的传播。因此，数据生产阶段可以提供更有效、更多样化、信息量更大的数据，而无需依赖重复的人工干预。
4.3.2.数据生产与利用协同提升

本小节重点关注闭环系统，其中数据生产、数据利用和模型训练通过持续反馈相互影响。数据生产模块综合任务、环境和轨迹。然后，数据利用模块选择这些数据并将其合并到模型训练中。反过来，由此产生的训练结果

provide feedback signals that guide the collaborative improvement of subsequent data production and utilization.

We classify related work according to the mechanisms through which data production and data utilization modules collaboratively improve. *Capability boundary matching* leverages the solver agent's signals of success, failure, or uncertainty to guide data production and utilization modules in generating and selecting tasks near the current capability boundary of the solver agent. *Goal-directed curriculum bridging* uses progress on target tasks to guide the generation and selection of intermediate training data that help the agent reach externally specified goals.

Capability Boundary Matching.

Capability frontier matching uses the solver agent's current capability boundary to determine which data instances should be generated and selected for the next training round. Before training, data production modules generate candidate tasks, and data utilization modules perform multiple rollouts on these candidate tasks. The resulting rollout signals, such as correctness, uncertainty, or reward, provide feedback on whether tasks have appropriate difficulty near the solver's capability boundary. These signals then guide the improvement of both the data production and data utilization modules.

*Agent0* [272] trains a curriculum agent to generate increasingly difficult tasks, while an executor agent improves its tool-integrated reasoning on those tasks. The curriculum agent receives rewards according to whether its generated tasks are near the executor's current capability boundary. Thus, utilization outcomes from executor training directly improve the next round of task production. *R-Zero* [273] implements the same principle: a challenger is rewarded for proposing problems near the solver's current boundary, and the solver is trained on the resulting curriculum without pre-existing tasks or labels. In both systems, task production and task utilization are coupled through the agent model's changing performance profile: harder and better calibrated tasks improve the solver, and the improved solver reshapes the reward landscape for subsequent task generation.

Goal-directed Curriculum Bridging.

Goal-directed curriculum bridging uses target tasks to determine which intermediate data should be generated and selected for training. After the model is trained on the generated intermediate tasks, progress on the target tasks indicates whether these data provide useful stepping stones toward the target capability. This progress signal then guides subsequent data production and selection toward curricula that better bridge the gap between the model's current abilities and the target goal. *SOAR* [274] trains a teacher to generate synthetic stepping-stone problems and rewards it according to measurable student improvement on hard target problems. This links curriculum quality to learning progress instead of a proxy for difficulty alone. *Guided Asymmetric Self-Play (GASP)* [275] uses hard coding problems as goalposts. Its teacher generates easier lemmas and progressively harder lifts that move the student toward those goalposts, while execution feedback trains the student. In both systems, external targets provide relevance, but the learned teacher determines which intermediate data should be produced and scheduled for training. The loop therefore improves data production by measuring downstream utilization value, and improves utilization by supplying curricula that are increasingly aligned with target-task progress.

In summary, these methods share the same central mechanism, in which model training outcomes are transformed into signals for both data production and curriculum adaptation. The improved data production and utilization then jointly reshape the training distribution for the next iteration. Capability-frontier matching emphasizes calibrating task difficulty to the solver's current capability boundary, whereas goal-directed curriculum bridging emphasizes designing intermediate training data that connect current capabilities with target capabilities. Together, these patterns show how data production and data utilization can collaboratively improve within a feedback loop while keeping the generated data adaptive, informative, and aligned with model training needs.

# 原文第 31 页 | 中文翻译初稿

31 共 70

提供反馈信号，指导后续数据生产和利用的协同改进。

我们根据数据生产和数据利用模块协同改进的机制对相关工作进行分类。 _能力边界匹配_利用求解器智能体的成功、失败或不确定性信号来指导数据生产和利用模块生成和选择靠近求解器智能体的当前能力边界的任务。 _目标导向的课程桥接_使用目标任务的进展来指导中间训练数据的生成和选择，帮助智能体达到外部指定的目标。
能力边界匹配。

能力边界匹配使用求解器智能体的当前能力边界来确定应生成并选择哪些数据实例用于下一轮训练。在训练之前，数据生产模块生成候选任务，数据利用模块对这些候选任务执行多个展开轨迹（Rollouts）。生成的 展开轨迹（Rollout） 信号（例如正确性、不确定性或奖励）提供有关任务在求解器能力边界附近是否具有适当难度的反馈。这些信号然后指导数据生产和数据利用模块的改进。

Agent0 [272] 训练课程智能体来生成越来越困难的任务，而执行智能体则改进其对这些任务的工具集成推理。课程智能体根据其生成的任务是否接近执行者当前的能力边界来获得奖励。因此，执行器训练的利用结果直接改善下一轮任务的产生。 R-Zero [273] 实现了相同的原则：挑战者因在解决者当前边界附近提出问题而获得奖励，并且解决者在没有预先存在的任务或标签的情况下接受最终课程的训练。在这两个系统中，任务生产和任务利用是通过智能体模型不断变化的性能配置文件耦合的：更难和更好的校准任务改进了求解器，改进后的求解器重塑了后续任务生成的奖励景观。
目标导向的课程衔接。

目标导向的课程桥接使用目标任务来确定应生成和选择哪些中间数据进行训练。在生成的中间任务上训练模型后，目标任务的进度表明这些数据是否为实现目标能力提供了有用的垫脚石。然后，这个进度信号会指导后续的数据生产和课程选择，以更好地弥合模型当前能力和目标之间的差距。 SOAR [274] 训练教师生成综合垫脚石问题，并根据学生在硬目标问题上的可衡量的进步进行奖励。这将课程质量与学习进度联系起来，而不仅仅是难度的代表。 引导非对称自我游戏（GASP） [275]使用硬编码问题作为球门柱。它的老师生成更容易的引理和逐渐困难的提升，将学生推向那些球门柱，而执行反馈则训练学生

。在这两个系统中，外部目标提供了相关性，但有学识的教师决定应该生成和安排哪些中间数据用于训练。因此，该循环通过测量下游利用率值来提高数据生产，并通过提供与目标任务进度越来越一致的课程来提高利用率。

总之，这些方法具有相同的中心机制，即模型训练结果转化为数据生产和课程适应的信号。然后，改进的数据生产和利用共同重塑下一次迭代的训练分布。能力前沿匹配强调将任务难度校准到求解器当前的能力边界，而目标导向的课程桥接则强调设计将当前能力与目标能力联系起来的中间训练数据。这些模式共同展示了数据生产和数据利用如何在反馈循环中协作改进，同时保持生成的数据具有适应性、信息丰富并与模型训练需求保持一致。

**Table 3.** Representative self-improvement methods in the agent data system.

| Work | Self-Improvement Process | | Domain | Level |
|---|---|---|---|---|
| | Object | Evidence | | |
| **Data Production** | | | | |
| EnvGen [246] | Environment generation | Success rate | Embodied agent | L3 |
| Adaptive Env Gen [247] | Environment generation | LLM analysis + Physical check | Embodied agent | L3 |
| ACCEL [248] | Environment generation | Positive value loss | Procedural RL | L3 |
| DRED [249] | Environment generation | Value loss scoring | Procedural RL | L3 |
| SimWorld Studio [250] | Environment generation | Rule + VLM | Embodied agent | L3 |
| WebRL [252] | Task synthesis | Critic score | Web agent | L3 |
| SeRL [254] | Task synthesis | Majority-voting + Difficulty | Math reasoning | L3 |
| Self-CriTeach [255] | Task spec + Trajectory | Planner check + Reward | Robotic planning | L3 |
| SAGE [256] | Task synthesis | Code execution / LLM judge | Reasoning | L3 |
| CoEvolve [253] | Task synthesis | Env. execution + Binary reward | Tool-use agent | L3 |
| SPIN [257] | Trajectory synthesis | Self-play | General | L3 |
| Arena Learning [258] | Trajectory synthesis | LLM judge | Dialogue | L3 |
| EVOLVE [259] | Trajectory synthesis | RM | General | L3 |
| DNPO [261] | Trajectory synthesis | LLM judge | General | L3 |
| PLD [262] | Trajectory synthesis | Binary reward + Success rate | Embodied agent | L3 |
| SI VLM Judges [263] | Judge model | Accuracy | Multimodal RM | L3 |
| **Data Utilization** | | | | |
| AMC-TSI [267] | Curriculum | Accuracy + Transfer regret | Math reasoning | L3 |
| EvoCurr [265] | Curriculum + Memory | Rollout win rate | Game | L3 |
| TRUSTEE [264] | Curriculum | Eval pass rate | Tool-use agent | L3 |
| Actor-Curator [268] | Curriculum | Policy improvement | Math reasoning | L3 |
| **Co-Improvement** | | | | |
| Agent0-VL [269] | Verifier + Trajectory | Self-verification | VL reasoning | L3 |
| ACE [270] | Adversary + Trajectory + Unit tests | Adversarial testing | Code generation | L3 |
| Agent-World [271] | Synthesized env/task | Rubrics/Validators | Tool-use agent | L3 |
| *Agent0* [272] | Curriculum model + Tasks + Trajectory | Self-consistency + Majority vote | Reasoning | L3 |
| R-Zero [273] | Challenger model + Tasks + Trajectory | Majority-vote + Uncertainty | Reasoning | L3 |

## 5. Agent Trainer Self-Improvement

Building on the agent-system representation in Section 2, this section studies the trainer $\mathcal{T}$. The trainer governs how training data are turned into model updates. It consists of three internal components. The supervision design derives learning signals from the available training evidence. The optimization strategy determines how those signals change the model parameters. The training infrastructure executes and coordinates the resulting updates.

We define trainer self-improvement as an evidence-driven revision of persistent state that determines how the trainer operates. Evidence produced during training or evaluation must induce a persistent change to either the trainer or, at the meta level, the improvement mechanism that revises it. The changed state must be retained and used in subsequent training or later trainer-improvement cycles.

Figure 7 places this definition within the overall trainer framework. The left side shows a training experiment, in which the supervision design, optimization strategy, and training infrastructure work together to update the model. The right side abstracts the improvement process into four functional roles. *Diagnose* examines training and evaluation evidence to identify system bottlenecks, failure modes, or opportunities for improvement. *Propose* converts the diagnosis into concrete and runnable modifications to the trainer. *Evaluate* measures the effects of these candidates through experiments or controlled comparisons and determines which changes have adequate empirical support. *Integrate* commits an accepted modification to persistent state so that it governs subsequent training or trainer improvement. These roles may be implemented by fixed rules, learned controllers, or explicit agents. A system may combine several roles within the same procedure or component.

# 原文第 32 页 | 中文翻译初稿

32 共 70

表 3. 智能体数据系统中的代表性 自我改进（Self-Improvement） 方法。

|工作|自我|f-改进|nt流程|领域|级别|
|---|---|---|---|---|---|
||对象|数据生产|证据
行动|||
|EnvGen [246]|环境生成||成功率|体现智能体|L3|
|自适应环境生成[247]|环境生成||LLM分析+物理检查|体现智能体|L3|
|ACCEL [248]|环境生成||正值损失|程序RL|L3|
|DRED [249]|环境生成||价值损失评分|程序RL|L3|
|SimWorld Studio [250]|环境生成||规则+VLM|体现智能体|L3|
|WebRL [252]|任务综合||评论分数|Web 智能体|L3|
|SeRL [254]|任务综合||多数投票+难度|数学推理|L3|
|自我批判教学 [255]|任务规范 + 轨迹||规划器检查 + 奖励|机器人规划|L3|
|SAGE [256]|任务综合||代码执行/LLM判断|推理|L3|
|CoEvolve [253]|任务合成||Env。执行+二元奖励|工具使用智能体|L3|
|SPIN [257]|轨迹合成||自我对战|通用|L3|
|竞技场学习[258]|轨迹合成||法学硕士评判|对话|L3|
|EVOLVE [259]|轨迹合成||RM|常规|L3|
|DNPO [261]|轨迹合成||LLM判断|一般|L3|
|PLD [262]|轨迹合成||二元奖励+成功率|具体智能体|L3|
|SI VLM Judges [263]|Judge 模型||精度|多模式 RM|L3|
|||数据利用|||
|AMC-TSI [267]|课程||准确性+转移遗憾|数学推理|L3|
|EvoCurr [265]|课程 + 记忆||展开轨迹（Rollout） 胜率|游戏|L3|
|受托人[264]|课程||评估通过率|工具使用智能体|L3|
|演员策展人[268]|课程||策略改进|数学推理|L3|
|||共同改进|
ment|||
|Agent0-VL [269]|验证者+轨迹||自我验证|VL推理|L3|
|
ACE [270]|i
对手 + 轨迹 +|单元测试|i
对抗性测试|
代码生成|L3|
|Agent-World [271]|综合环境/任务||规则/验证器|工具使用智能体|L3|
|Agent0[272]|课程模型+任务|s+轨迹|自我一致性+多数投票|推理|L3|
|R-零 [273]|挑战者模型 + 任务|+ 轨迹|多数投票 + 不确定性|推理|L3|
5. 特工训练师 自我改进（Self-Improvement）

基于第 2 节中的智能体系统表示，本节研究训练器 T 。训练器控制训练数据如何转化为模型更新。它由三个内部组件组成。监督设计从可用的训练证据中得出学习信号。优化策略决定了这些信号如何改变模型参数。训练基础设施执行并协调结果更新。

我们将训练器 自我改进（Self-Improvement） 定义为持久状态的证据驱动修订版，决定了训练器的运行方式。训练或评估期间产生的证据必须引起训练师或在元层面上对其进行修改的改进机制的持续变化。更改后的状态必须保留并在后续训练或稍后的训练师改进周期中使用。

图 7 将此定义置于整个训练器框架中。左侧显示了一个训练实验，其中监督设计、优化策略和训练基础设施共同更新模型

。右侧将改进过程抽象为四个功能角色。 _诊断_检查训练和评估证据，以确定系统瓶颈、故障模式或改进机会。 Propose 将诊断结果转化为对训练器的具体且可运行的修改。 _评估_通过实验或受控比较来衡量这些候选者的效果，并确定哪些变化有足够的经验支持。 Integrate 对持久状态提交可接受的修改，以便它控制后续训练或训练师改进。这些角色可以通过固定规则、学习控制器或显式智能体来实现。系统可以在同一过程或组件中组合多个角色。

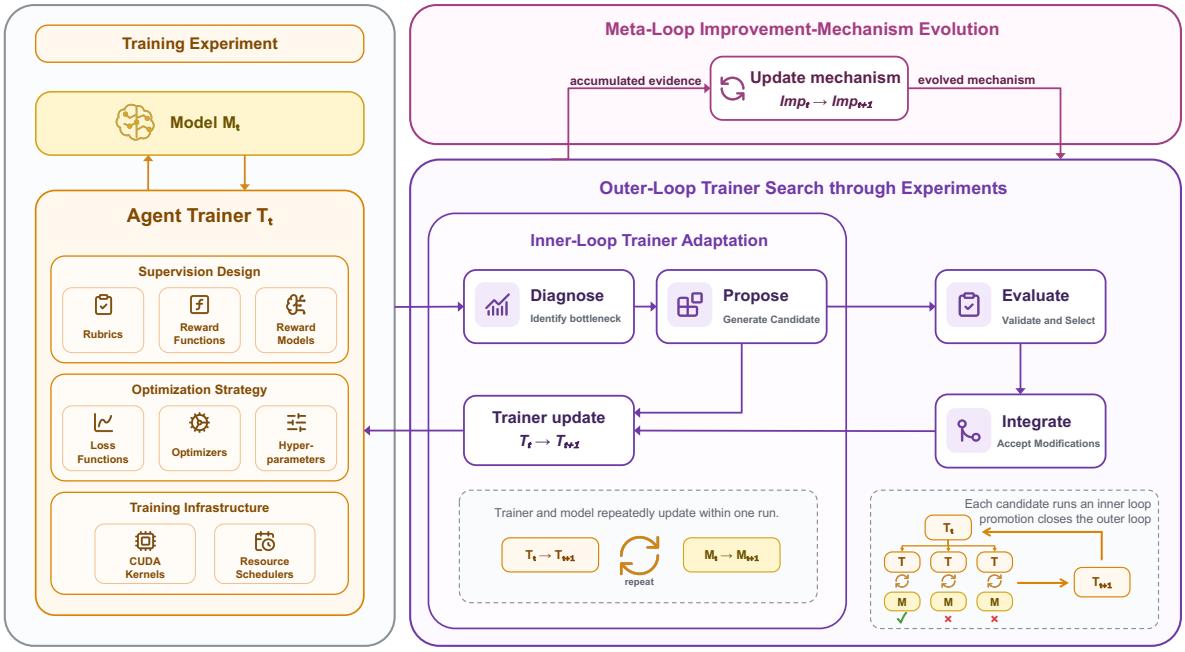



**Figure 7.** Agent trainer and the three loops of trainer self-improvement. The loops distinguish processes that adapt the active trainer within a training run, select and retain a trainer version through experiments, or revise the improvement mechanism that governs later experiments.

We organize the remainder of this section around three loops of trainer self-improvement, summarized in Table 4. The three loops are:

- *Inner-Loop Trainer Adaptation* operates within an active training lineage. Evidence from the current run directly changes persistent state in the supervision design, optimization strategy, or training infrastructure. The revised trainer state then governs subsequent model updates in the same lineage. The improvement mechanism remains fixed.
- *Outer-Loop Trainer Search through Experiments* operates across training experiments. Within each experiment, a trainer and a model interact through a training loop; across experiments, a fixed improvement mechanism uses the resulting evidence to propose runnable trainer candidates and evaluate them in bounded experiments. Based on the results, it rejects a candidate or integrates it as the retained trainer version for later experiments.
- *Meta-Loop Improvement-Mechanism Evolution* operates across successive trainer-improvement cycles. Within each cycle, the current improvement mechanism governs diagnosis, candidate proposal, evaluation, and integration. Across cycles, evidence from model-training experiments updates the persistent mechanism from $Imp_t$ to $Imp_{t+1}$. The successor mechanism then governs these processes in later trainer-improvement cycles.

**Table 4.** Representative Trainer self-improvement methods across three loops, summarized by target object, evidence, revision mechanism, and RSI level.

| Work | Self-Improvement Process | | | Level |
|---|---|---|---|---|
| | Object | Evidence | Mechanism | |
| **Inner-Loop Trainer Adaptation** | | | | |
| **ACE** [270] | Test generator | Code–test execution matrix | Preference optimization | L3 |
| **AdaLRS** [312] | Learning rate | Loss-descent velocity | Probe with retain/rollback | L3 |
| **ADMIRE** [279] | Milestone criteria | Successful trajectories | Milestone distillation | L3 |
| **ASL** [294] | Reward model | Rule-verified outcomes | Alternating role RL | L3 |
| **AI Training Manager** [311] | Training recipe | Training telemetry | Schema-bounded LLM edits | L3 |
| **ARBOR** [278] | Rubric memory | Trajectory contrasts | Rubric write–prune | L3 |
| **ARCO** [285] | Rubric generator | Binary terminal outcomes | Joint rubric–policy RL | L3 |
| **ATLAS** [310] | DPO coef.; reference anchor | Telemetry; reference KL | Bounded update; anchor swap | L3 |
| **CoNL** [295] | Critic | Critique-induced gains | Segment-rewarded RL | L3 |

# 原文第 33 页 | 中文翻译初稿

33 共 70

元环改进-机制演化
训练实验
积累证据更新机制演化机制
Imp ☐ → Imp ☐₊₁
模型 M ☐
外环训练器通过实验搜索
智能体训练器 T ☐
内环训练器适配
监督设计
诊断建议评估
评分标准 功能奖励 奖励模型 识别瓶颈 生成候选者 验证和选择
优化策略
训练器更新 集成
函数损失优化器参数Hyper- T ☐ → T ☐₊₁ 接受修改
训练基础设施 训练器和模型在一次运行中重复更新。 T ☐ 每个候选运行一次内循环提升，关闭外循环
CUDA 资源 T ☐ → T ☐₊₁ M ☐ → M ☐₊₁ T T T
内核调度程序重复 T ☐₊₁
M M M
✓ × ×

图 7. 智能体训练器和训练器 自我改进（Self-Improvement）的三个循环。这些循环区分了在训练运行中调整主动训练器的过程，通过实验选择和保留训练器版本，或者修改管理后续实验的改进机制。

我们围绕训练器 自我改进（Self-Improvement）的三个循环组织本节的其余部分，如表 4 所示。这三个循环是：

- _内环训练器适应_在活跃的训练谱系中运作。当前运行的证据直接改变监督设计、优化策略或训练基础设施中的持久状态。然后，修改后的训练器状态将管理同一谱系中的后续模型更新。改进机制保持不变。

-_通过实验进行外环训练器搜索_在训练实验中进行操作。在每个实验中，训练器和模型通过训练循环进行交互；在实验中，固定的改进机制使用结果证据来提出可运行的候选训练器，并在有界实验中对其进行评估。根据结果，它拒绝候选者或将其集成为保留的训练器版本以供以后的实验。

- _元循环改进机制进化_在连续的训练器改进周期中运行。在每个周期内，当前的改进机制控制诊断、候选提案、评估和集成。在各个周期中，模型训练实验的证据将持久机制从_Impt_更新为_Impt_+1。然后，继任者机制在以后的训练师改进周期中管理这些过程。

表 4. 跨三个循环的代表性训练器 自我改进（Self-Improvement）方法，按目标对象、证据、修订机制和 RSI 级别进行总结。

|工作||自我改进（ Self-Improvement ） 流程||级别|
|---|---|---|---|---|
||对象|证据|机制||
||Inner-Loo
|p 训练师适应
|||
|ACE[270]|测试生成器|代码-测试执行矩阵|偏好优化|L3|
|AdaLRS[312]|学习率|损失下降速度|带保留/回滚的探针|L3|
|ADMIRE[279]|里程碑标准|成功轨迹|里程碑蒸馏|L3|
|ASL[294]|奖励模型|规则验证的结果|交替角色 RL|L3|
|AI 训练管理器[311]|训练配方|训练遥测|模式限制的 LLM 编辑|L3|
|ARBOR[278]|Rubric 内存|轨迹对比|Rubric 写入-修剪|L3|
|ARCO[285]|标题生成器|二进制终端结果|联合标题-策略 RL|L3|
|ATLAS[310]|DPO 系数；参考锚|遥测；参考KL|有界更新；锚交换|L3|
|CoNL[295]|评论家|评论引发的收益|分段奖励 RL|L3|

| Work | Self-Improvement Process | | | Level |
|---|---|---|---|---|
| | Object | Evidence | Mechanism | |
| **COPUS** [314] | Batch size; parallelism | Gradient noise; throughput | Goodput-driven resharding | L3 |
| **CoVerRL** [296] | Verifier | Consensus pseudo-labels | Alternating role RL | L3 |
| **DR Tulu** [276] | Rubric memory | Trajectory contrasts | Rubric write–prune | L3 |
| **DYNAMIX** [319] | Batch-size controller | Training + system metrics | PPO controller training | L3 |
| **EARL** [321] | Parallelism config. | Context length; system load | Profile lookup; live switch | L3 |
| **ECHO** [291] | Critic | Critique-induced gains | Gain-rewarded RL | L3 |
| **EvoLM** [282] | Rubric generator | Temporal contrasts; margins | Alternating role RL | L3 |
| **EPI** [313] | Parameter mask | Per-param squared gradients | Periodic mask refresh | L3 |
| **EvoRubric** [284] | Rubric generator; archive | Rubric validity; consensus | Role RL; rubric archiving | L3 |
| **EvoRubrics** [281] | Rubric generator | Cross-evaluation scores | Adversarial GRPO update | L3 |
| **ICRL** [292] | Critic | Critique-induced gains | Alternating role RL | L3 |
| **JarvisEvo** [290] | Reward model | Human target assessments | Alternating role RL | L3 |
| **KungFu** [315] | Batch size; workers; topology | Gradient noise; throughput | Rule-based reconfiguration | L3 |
| **MagicGUI-RMS** [289] | Reward model | Reward-model disagreements | Data write-back; RM retraining | L3 |
| **Mutual-Taught** [288] | Reward model | Pre/post-update preferences | Alternating DPO updates | L3 |
| **ONES** [318] | Batch size; GPU allocation | Epoch progress; cluster state | Evolutionary scheduling | L3 |
| **Optimus** [322] | Resource allocation | Loss history; throughput | Model refit; reallocation | L3 |
| **Pollux** [316] | Batch size; GPU allocation | Gradient noise; throughput | Goodput co-optimization | L3 |
| **Q-Evolve** [301] | Value critic | Transitions; terminal reward | IQL critic training | L3 |
| **RFT-FM** [320] | Training configuration | Telemetry fault fingerprints | Diagnosis-guided repair | L3 |
| **RLAC** [293] | Critic | Validator outcomes | Alternating DPO updates | L3 |
| **RLAnything** [300] | Process reward model | Terminal outcomes; consistency | Alternating RM–policy RL | L3 |
| **RLAR** [286] | Reward-tool library | Tool verification results | Synthesize, verify, commit | L3 |
| **RLCER** [283] | Rubric generator | Rubric–answer correlation | Correlation-rewarded RL | L3 |
| **RL Tango** [299] | Verifier | Final-answer correctness | Interleaved GRPO updates | L3 |
| **rStar-Math** [302] | Process preference model | MCTS values; code execution | Pairwise preference training | L3 |
| **Rubick** [323] | Execution plan | Measured throughput | Model refit; reconfiguration | L3 |
| **RubricEM** [277] | Rubric memory; meta-policy | Trajectory contrasts | Write–prune; reflection RL | L3 |
| **SAVE** [287] | Reward model | On-policy reward–value gaps | Value-anchored self-training | L3 |
| **SCORE** [297] | Evaluator | Cross-rollout consistency | Consistency-rewarded RL | L3 |
| **Sia** [317] | Batch size; GPU allocation | Gradient noise; cluster load | Goodput co-optimization | L3 |
| **Skill-SD** [308] | Teacher state; skill bank | Trajectory rewards; utility | Skill writing; teacher sync | L3 |
| **SPARD** [306] | Reward weights | Reward progress; dispersion | Mirror-descent reweighting | L3 |
| **StepORLM** [303] | Process reward model | Solver outcomes; critiques | Iterative RM fine-tuning | L3 |
| **STRIDE** [304] | Verifier | Final-answer correctness | Verifier fine-tuning | L3 |
| **Evaluative Thinking** [280] | Evaluation rubric | Responses; RM scores | Meta-reward-model rewrite | L3 |
| **UCOB** [309] | Teacher-side skill memory | Return gaps; skill utility | Credit-aware skill write–prune | L3 |
| **UI-Genie** [305] | Reward model | Outcome/continuation labels | Iterative RM fine-tuning | L3 |
| **URPO** [298] | Reward model | Preference rankings | Rank-rewarded GRPO update | L3 |
| **ZeroCoder** [307] | Test generator; selector prior | Code–test execution matrix | Role RL; prior recalibration | L3 |
| **Outer-Loop Trainer Search through Experiments** | | | | |
| **AIDE** [353] | Training script | Validation score; run errors | LLM tree search | L3 |
| **AIRA-Design** [354] | Training script | Validation loss; executability | LLM tree search | L3 |
| **Auto Research** [328] | Training script | Eval scores; crashes; runtime | LLM code edits; promotion | L3 |
| **AutoTrainess** [326] | Fine-tuning recipe | Benchmark scores | LLM recipe assembly | L3 |
| **CARD** [335] | Reward code | Trajectory preferences | LLM code edits; gated trials | L3 |
| **CliffSearch** [329] | Optimizer code | Validation loss; reviews | Reviewer-gated evolution | L3 |
| **DiscoPOP** [348] | Loss function | Post-training scores | LLM generation; selection | L3 |
| **LLM-RL HPO** [347] | Hyperparameters | Reward/KL dynamics; cost | Multi-fidelity Bayesian opt. | L3 |
| **Eureka** [334] | Reward code | Fitness; component traces | Evolution; elite retention | L3 |
| **FT-Dojo** [327] | Fine-tuning recipe | Validation errors; loss curves | LLM proposals; gated trials | L3 |
| **Highway Reward Evolution** [351] | Reward code | Driving metrics | Evolution with RL trials | L3 |
| **LaRes** [340] | Reward code | Task success; learning curves | Evolution; elite retention | L3 |
| **LERO** [352] | Reward code | Team returns; convergence | Evolution with RL trials | L3 |
| **LLM-ALSO** [336] | Reward-shaping config. | Returns; branch stability | Fork-validated promotion | L3 |
| **LLMZero** [324] | Training configuration | Validation telemetry | LLM tree search | L3 |
| **MLEvolve** [338] | Training pipeline | Task metric; run failures | Graph search with memory | L3 |
| **OptiCo** [330] | Distributed-training config. | Throughput; runtime failures | Multi-agent search; memory | L3 |
| **POISE** [333] | Policy-optimization code | Dynamics; held-out scores | Archive-guided LLM edits | L3 |
| **R*** [341] | Reward code; coefficients | Fitness; component traces | Evolution with RL trials | L3 |
| **REvolve** [337] | Reward code | Human rollout preferences | Island evolution; migration | L3 |
| **RF-Agent** [342] | Reward code | Return; component feedback | LLM tree search | L3 |
| **RHyVE** [346] | Reward schedule | Fork margins; agreement | Fork-validated promotion | L3 |
| **ROSKA** [343] | Reward code; fusion ratio | Fitness; component traces | Evolution; tuned policy fusion | L3 |
| **Reward Synthesis** [350] | Reward code | Executability; task F1 | Ranked generation; ensemble | L3 |
| **Self-Evolving Recommender** [339] | Training + reward code | Offline loss; A/B metrics | LLM code edits; staged gates | L3 |

# 原文第 34 页 | 中文翻译初稿

34 共 70

|工作||自我改进（Self-Improvement）流程||级别|
|---|---|---|---|---|
||对象|证据|机制||
|COPUS[314]|批量大小；并行度|梯度噪声；吞吐量|Goodput驱动的重新分片|L3|
|CoVerRL[296]
|验证者
|共识伪标签
|交替角色 RL
|L3
|
|DR Tulu[276]
DYNAMIX[319]|Rubric 内存
批量大小控制器|轨迹对比
训练 + 系统指标|Rubric 写入-剪枝
PPO 控制器训练|L3
L3|
|EARL[321]|并行度配置。|上下文长度；系统加载|配置文件查找；带电开关|L3|
|ECHO[291]|评论家|评论引发的收益|增益奖励 RL|L3|
|
EvoLM[282]|Rubric 生成器|
时间对比；边距|交替角色 RL|L3|
|EPI[313]|参数掩码|每参数平方梯度|定期掩码刷新|L3|
|EvoRubric[284]|Rubric 生成器；存档|量规有效性；共识|角色RL；归档标题|L3|
|EvoRubrics[281]|Rubric 生成器|交叉评估分数|对抗性 GRPO 更新|L3|
|ICRL[292]|批评家|批评带来的收获|交替角色 RL|L3|
|
JarvisEvo[290]|奖励模型|
人类目标评估|
交替角色 RL|L3|
|KungFu[315]
|批量大小；工人；拓扑
|梯度噪声；吞吐量
|基于规则的重新配置
|L3
|
|MagicGUI-RMS[289]
互教[288]|奖励模型
奖励模型|奖励模型不一致
更新前/更新后偏好|数据回写； RM 再训练

交替 DPO 更新|L3
L3|
|ONES[318]|批量大小； GPU分配|
Epoch进展；集群状态|
进化调度|L3|
|Optimus[322]|资源分配|损失历史记录；吞吐量|车型改装；重新分配|L3|
|Pollux[316]
|批量大小； GPU分配
|梯度噪声；吞吐量
|Goodput 协同优化
|L3
|
|Q-Evolve[301]|价值批评家|过渡；终端奖励|IQL评论家训练|L3|
|RFT-FM[320]
|训练配置
|遥测故障指纹
|诊断引导修复
|L3
|
|RLAC[293]
RLAnything[300]|评论家
过程奖励模型|验证者结果
终端结果；一致性|交替 DPO 更新

交替 RM-策略 RL|L3
L3|
|RLAR[286]|奖励工具库|
工具验证结果|

合成、验证、提交|L3|
|RLCER[283]|Rubric 生成器|Rubric-答案相关性|相关奖励 RL|L3|
|RL Tango[299]
rStar-Math[302]|验证器
过程偏好模型|最终答案正确性
MCTS 值；代码执行|交错GRPO更新
成对偏好训练|L3
L3|
|Rubick[323]
|执行计划
b l|实测吞吐量
|模型改装；重新配置
fl|L3
|

|RubricEM[277]
|Ruric 内存； meta-poicy
|轨迹对比
|Write-prune;反射RL
|L3|
|SAVE[287]
SCORE[297]|奖励模型
评估器|策略奖励-价值差距
跨展开轨迹（Rollout）一致性|价值锚定自我训练
一致性奖励RL|L3
L3|
|
Sia[317]|批量大小； GPU分配|y
梯度噪声；集群负载|y
Goodput 协同优化|L3|
|技能-SD[308]|教师状态；技能库|轨迹奖励；实用|技能写作；老师同步|L3|
|
SPARD[306]|
奖励权重|
奖励进度；色散|
镜像下降重加权|L3|
|StepORLM[303]|处理奖励模型|求解器结果
;批评|迭代RM微调|L3|
|
STRIDE[304]|Vrifir|
Finl-nwr rrtn|i
Vrifir fin-tnin|L3|
|
评价性思维[280]|eie
评价量规|aase coecess
回应； RM 分数|eie ieug
元奖励模型重写|L3|
|UCOB[309]|老师端技能记忆|返回差距；技能实用程序|信用感知技能写入-修剪|L3|
|
UI-Genie[305]|
奖励模型|
结果/延续标签|
迭代RM微调|L3|
|URPO[298]|奖励模型|偏好排名|排名奖励GRPO更新|L3|
|
ZeroCoder[307]|测试生成器；选择器先验|
代码-测试执行矩阵|

角色 RL;事先重新校准|L3|
||外环训练器|搜索实验|||
|AIDE[353]|
训练脚本|
验证分数；运行错误|LLM树搜索|L3|
|AIRA-Design[354]|训练脚本|验证损失；可执行性|LLM树搜索|L3|
|汽车研究[328]|
训练脚本|
评估分数；崩溃；运行时|LLM代码编辑；促销|L3|
|AutoTrainess[326]|微调配方|基准分数|LLM配方组装|L3|
|CARD[335]|
奖励代码|轨迹偏好|
LLM 代码编辑；门控试验|L3|
|
CliffSearch[329]|
优化器代码|
验证损失；评论|
审稿人门控进化|L3|
|DiscoPOP[348]
LLM-RL HPO[347]|损失函数
超参数|训练后得分
奖励/KL动态；成本|LLM生成；选择
多保真贝叶斯选项|L3
L3|
|尤里卡[334]|奖励代码|健身；组件痕迹|进化；精英保留|L3|
|FT-Dojo[327]
|微调配方
|
验证错误；损失曲线|LLM建议；门控试验
|L3|
|高速公路奖励进化[351|]奖励代码|驾驶指标|RL试验的进化|L3|
|LaRes[340]|奖励码|任务成功；学习曲线|进化；精英保留|L3|
|
LERO[352]|
奖励码|
团队归来；收敛|
强化学习试验的进化|L3|
|LLM-ALSO[336]|奖励塑造配置。|返回；分支稳定性|分叉验证升级|L3|
|
LLMZero[324]|i
训练配置|
验证遥测|
LLM树搜索|L3|

|MLEvolve[338]
OtiCo[330]|训练管道
分布式训练配置|任务指标；运行失败
Throuhut运行时失败|内存图形搜索
多目标搜索内存|L3
L3|
|p
POISE[333]|g ig.
策略优化代码|gp;
动态;保留分数|g ; y
存档指导的 LLM 编辑|L3|
|R*[341]|奖励代码；系数|适应度；组件痕迹|强化学习试验的进化|L3|
|
REvolve[337]|i
奖励码|
人类展开轨迹（Rollout）偏好|
岛屿进化；迁移|L3|
|RF-Agent[342]|奖励码|返回；组件反馈|LLM树搜索|L3|
|
RHyVE[346]|奖励计划|
分叉利润；协议|分叉验证的促销|L3|
|ROSKA[343]
奖励合成[350]|奖励代码；融合比例
奖励码|体力；组件跟踪
可执行性；任务F1|进化；调整策略融合
排名生成；合奏|L3
L3|
|y
自我进化推荐器
[339]|
训练+奖励代码|y
离线损失； A/B 指标|g
LLM 代码编辑；阶段式登机口|L3|

| Work | Self-Improvement Process | | | Level |
|---|---|---|---|---|
| | Object | Evidence | Mechanism | |
| **SMCEvolve** [356] | Training script | Validation loss; run failures | SMC resampling; mutation | L3 |
| **TREX** [332] | Fine-tuning recipe | Validation score; run failures | LLM tree search | L3 |
| **Meta-Loop Improvement-Mechanism Evolution** | | | | |
| **AutoScientists** [331] | Role-allocation policy | Validation deltas; stagnation | Evidence-triggered reorg. | L4 |
| **Bilevel Autoresearch** [238] | Search runner code | Proposal–outcome traces | LLM-generated replacement | L4 |
| **DERL** [349] | Reward meta-optimizer | Inner-policy validation | GRPO meta-training | L4 |
| **EvoTrainer** [325] | Diagnostic harness | Diagnostic gaps; branch scores | Retained harness revision | L4 |
| **GEAR** [355] | Search controller code | Crossover failures | Source-level repair | L4 |
| **Execution-Grounded AI Research** [357] | Ideation policy | Validation score; run status | Execution-rewarded RL | L4 |

### *5.1. Inner-Loop Trainer Adaptation*

*Inner-Loop Trainer Adaptation* occupies the innermost loop in Figure 7. Evidence from the active training process changes persistent trainer state from $\mathcal{T}_t$ to $\mathcal{T}_{t+1}$. The updated state then controls subsequent model updates in the same training lineage. The current training process continues without first evaluating the proposed change as a separate trainer candidate.

We group these methods by the part of the trainer that changes. *Supervision Design Evolution* revises the labels, rewards, critiques, or teacher targets used to train the model. *Optimization Strategy Evolution* revises the objective, update algorithm, or training controls that convert supervision into parameter updates. *Training Infrastructure Evolution* revises the execution substrate that carries out those updates, including low-level CUDA kernels, distributed-training runtimes, parallel execution plans, and resource-scheduling state. In all three groups, evidence directly changes the active trainer within the ongoing training lineage, while the improvement mechanism remains fixed.

#### 5.1.1. Supervision Design Evolution

*Supervision Design Evolution* changes the persistent trainer-side state that determines how experience becomes supervision for the model. This state may define what behavior should count as successful, implement the mechanism that judges observed behavior, determine where a judgment should be assigned within a trajectory, or control how the resulting signal is delivered to and absorbed by the model. A method belongs to this category only when such a change is retained and affects later training. Producing different scores, critiques, or advantages for different samples with an otherwise fixed supervision pipeline is not sufficient. A supervisor may share parameters with the policy, but its role must receive an identifiable training signal, expose a distinct supervisory output, and use the updated function to alter later model training.

We organize this pipeline around four successive questions. *Evaluation Criteria Evolution* asks what should count as good behavior. *Evaluator and Verifier Evolution* asks how observed behavior is converted into a judgment. *Process-Level Credit Assignment Evolution* asks where within a multi-step process that judgment should be assigned. *Training-Signal Routing and Internalization* asks how the resulting supervision reaches the model and becomes available without its original supervisory support.

##### Evaluation Criteria Evolution

*Evaluation Criteria Evolution* changes the standards by which behavior is judged. It updates either the criteria themselves or the mechanism that generates them, allowing supervision to remain informative as the model's behavior evolves. Its defining effect is semantic: after the update, later behavior is evaluated against a revised conception of quality, correctness, progress, or failure.

What changes is the standard itself, not how a fixed standard is applied: updating a judge or verifier without revising the meaning of success is a matter for the next category. Criteria evolution therefore answers what should be rewarded, not yet whether a particular behavior satisfies the standard or which part of that behavior deserves credit.

# 原文第 35 页 | 中文翻译初稿

35 共 70

|工作||自我改进（Self-Improvement） 流程||级别|
|---|---|---|---|---|
||对象|证据|机制||
|SMCEvolve[356]|训练脚本|验证损失；运行失败|SMC重采样；突变|L3|
|TREX[332]|微调配方|验证分数；运行失败|LLM树搜索|L3|
||
Meta-Loop Impr
|运动机制演化
|||
|AutoScientists[331]|角色分配策略|验证增量；停滞|证据触发的重组。|L4|
|Bilevel Autoresearch[238]|搜索运行程序代码|提案-结果跟踪|LLM 生成的替换|L4|
|DERL[349]|奖励元优化器|内部策略验证|GRPO 元训练|L4|
|EvoTrainer[325]|诊断 运行支撑框架（Harness）|诊断差距；分支分数|保留运行支撑框架（Harness）修订|L4|
|GEAR[355]|搜索控制器代码|交叉故障|源码级修复|L4|
|以执行为基础的人工智能
研究[357]|构思策略|验证分数；运行状态|执行奖励RL|L4|
5.1。内环训练器适应

_内循环训练器适应_占据图 7 中最内层的循环。来自主动训练过程的证据将持久训练器状态从 Tt 更改为 Tt +1。然后，更新后的状态控制同一训练谱系中的后续模型更新。当前的训练过程将继续进行，而无需首先作为单独的训练师候选人评估拟议的变更。

我们根据训练器发生变化的部分对这些方法进行分组。 _监督设计进化_修改用于训练模型的标签、奖励、批评或教师目标。 _优化策略演化_修改目标、更新算法或将监督转换为参数更新的训练控制。 _训练基础设施演进_修改了执行这些更新的执行基础，包括低级 CUDA 内核、分布式训练运行时、并行执行计划和资源调度状态。在所有三组中，证据直接改变了正在进行的训练血统中的现役训练师，而改进机制保持不变。
5.1.1.监督设计的演变

_监督设计进化_改变了持久的训练者端状态，决定了经验如何成为模型的监督。这种状态可以定义什么行为应该被视为成功，实现判断观察到的行为的机制，确定应该在轨迹内分配判断的位置，或者控制结果信号如何传递到模型并被模型吸收。仅当这种变化被保留并影响后续训练时，该方法才属于此类。使用固定的监督管道为不同样本产生不同的分数、批评或优势是不够的。监督者可以与策略共享参数，但其角色必须接收可识别的训练信号，公开不同的监督输出，并使用更新的函数来改变以后的模型训练。

我们围绕四个连续的问题组织这个流程。 _评估标准进化_询问什么应该算作良好行为。 _评估者和验证者进化_询问观察到的行为如何转化为判断。 流程级 贡献归因（Credit Assignment） Evolution 询问应在多步骤流程中的何处分配判断

。 _训练-信号路由和内化_询问最终的监督如何到达模型并在没有原始监督支持的情况下变得可用。
评价标准的演变

_评估标准的演变_改变了判断行为的标准。它更新标准本身或生成标准的机制，从而允许监督随着模型行为的发展而保持信息丰富。它的定义效果是语义的：更新后，根据修订后的质量、正确性、进度或失败概念来评估后来的行为。

改变的是标准本身，而不是如何应用固定标准：更新法官或验证者而不修改成功的含义是下一个类别的问题。因此，标准的演变回答了应该奖励什么，而不是回答特定行为是否满足标准或者该行为的哪一部分值得赞扬。

One route treats criteria as editable memory rather than model parameters. DR Tulu contrasts on-policy research responses to add positive and negative criteria, then retains the items whose rewards remain discriminative for later GRPO batches [276]. RubricEM applies a related write–score–prune cycle to stage-local rubric buffers, while ARBOR adds cross-query consolidation, reuse statistics, and retirement to maintain a longer-lived process-rubric memory [277,278].

Criteria can also encode progress through a trajectory. Adaptive Milestone Reward distills only a superior successful GUI trajectory into a revised per-instruction milestone sequence; a fixed matching rule then turns progress through those milestones into dense rewards for later successful and failed rollouts [279]. Toward Evaluative Thinking instead rewrites the textual rubric used by a reward model: periodic meta-analysis of recent responses and scores revises its criteria, examples, and scoring ranges before the next policy batches are evaluated [280]. In both cases, the semantic specification changes while the mechanism that applies it remains fixed.

A second route trains the producer of criteria. EvoRubrics gives the rubric generator a dedicated adapter and updates it from cross-evaluation evidence, so the successor adapter generates the criteria used to reward later policy batches [281]. EvoLM trains a shared-backbone rubric role from temporal response contrasts and frozen-judge score margins, whereas RLCER gives its shared-parameter rubricator a separate reward based on rubric validity and discrimination against answer correctness [282,283]. These systems share a criterion-generation interface, although their parameter sharing and external anchors differ.

Generator evolution can be coupled to additional persistent state. EvoRubric verifies response-conditioned atomic criteria, removes uninformative items, and archives accepted rubrics for later generation and reward construction [284]. ARCO jointly updates a per-step criterion generator and a criterion-conditioned scorer from terminal outcomes; the successor rubric model then supplies new step-specific criteria and reward-to-go signals to later policy batches [285]. Its scoring head participates in the loop, but the adaptive supervision first enters through the generated criteria.

### Evaluator and Verifier Evolution

*Evaluator and Verifier Evolution* changes the mechanism that converts behavior and available evidence into an evaluative judgment. Evaluators may produce learned assessments, preferences, critiques, or scalar rewards. Verifiers may instead ground judgments in execution results, environment feedback, formal checks, or reusable reward tools. In either case, the evolving object is a retained model, program, tool collection, test generator, or verification state that changes how subsequent behavior is judged.

What evolves is the judge, not the standard: when the meaning of success itself changes, the method belongs to *Evaluation Criteria Evolution*.

Executable evidence can update verifier state without learning a scalar reward model. ACE trains an adversarial unit-test generator from a code–test execution matrix; the retained adversary adapter produces later tests that determine which solver programs enter supervised fine-tuning [270]. RLAR instead detects missing reward coverage, synthesizes or retrieves a callable checker or reward-model wrapper, verifies it, and commits the accepted tool to a library used on later training samples [286]. The first mechanism evolves a test generator, whereas the second evolves the available verification toolkit.

Separate reward models can follow the changing policy distribution directly. SAVE updates a persistent reward-and-value model from value-anchored on-policy feedback before using the successor model to score later response groups [287]. Mutual-Taught constructs pseudo-preferences between responses sampled before and after a policy update, MagicGUI-RMS refluxes disagreements with a fixed general-purpose verifier into a domain reward model, and JarvisEvo aligns an evaluator role to human target assessments before reusing it as the editor's reward source [288–290]. Their evidence sources differ, but each closes the same write-back path from current policy behavior to a successor evaluator and then to later policy supervision.

# 原文第 36 页 | 中文翻译初稿

36 共 70

一种方法将标准视为可编辑的内存而不是模型参数。 DR Tulu 对比策略研究响应，添加正面和负面标准，然后保留其奖励对后续 GRPO 批次仍然具有歧视性的项目 [276]。 RubricEM 将相关的写入-评分-修剪循环应用于阶段本地 rubric 缓冲区，而 ARBOR 添加了跨查询整合、重用统计数据和退役以维护寿命更长的进程 rubric 内存 [277,278]。

标准还可以通过轨迹编码进展。自适应里程碑奖励仅将卓越的成功 GUI 轨迹提炼为修订后的每指令里程碑序列；然后，固定的匹配规则将这些里程碑的进展转化为对后来成功和失败的 展开轨迹（Rollouts） [279] 的密集奖励。相反，走向评估性思维重写了奖励模型使用的文本标题：对最近的响应和分数进行定期元分析，在评估下一批策略之前修改其标准、示例和评分范围[280]。在这两种情况下，语义规范都会发生变化，而应用它的机制保持不变。

第二条路线是训练标准的制定者。 EvoRubrics 为评分标准生成器提供了一个专用适配器，并根据交叉评估证据对其进行更新，因此后继适配器生成用于奖励后续策略批次的标准 [281]。 EvoLM 根据时间响应对比和冻结判断分数裕度训练共享主干评分角色，而 RLCER 根据评分有效性和对答案正确性的区分为其共享参数评分器提供单独的奖励 [282,283]。这些系统共享一个标准生成接口，尽管它们的参数共享和外部锚点不同。

生成器演化可以耦合到额外的持久状态。 EvoRubric 验证响应条件原子标准，删除无信息的项目，并存档已接受的规则以供后代和奖励构建[284]。 ARCO 根据最终结果联合更新每步标准生成器和标准条件评分器；然后，后继的标题模型为后续的策略批次提供新的具体步骤标准和奖励信号[285]。它的评分头参与循环，但自适应监督首先通过生成的标准进入。
评估者和验证者的演变

_评估者和验证者进化_改变了将行为和可用证据转化为评估判断的机制。评估者可能会产生学习评估、偏好、批评或标量奖励。验证者可以根据执行结果、环境反馈、正式检查或可重复使用的奖励工具进行判断。在任何一种情况下，演变的对象都是保留的模型、程序、工具集合、测试生成器或验证状态，它们改变了后续行为的判断方式。

进化的是判断，而不是标准：当成功本身的含义发生变化时，方法就属于_评价标准进化_。

可执行证据可以更新验证者状态，而无需学习标量奖励模型。 ACE 从代码测试执行矩阵训练对抗性单元测试生成器

;保留的对手适配器会生成后续测试，以确定哪些求解器程序进入监督微调[270]。相反，RLAR 会检测缺失的奖励覆盖范围，合成或检索可调用的检查器或奖励模型包装器，对其进行验证，并将接受的工具提交到用于后续训练样本的库中 [286]。第一个机制发展了测试生成器，而第二个机制发展了可用的验证工具包。

单独的奖励模型可以直接遵循不断变化的策略分配。 SAVE 在使用后继模型对后来的响应组进行评分之前，根据价值锚定的策略反馈更新持久的奖励和价值模型[287]。 Mutual-Taught 在策略更新前后采样的响应之间构建伪偏好，MagicGUI-RMS 将与固定 通用（General-Purpose） 验证者的分歧回流到域奖励模型中，JarvisEvo 在将评估者角色重新用作编辑者的奖励源之前将评估者角色与人类目标评估保持一致 [288-290]。他们的证据来源各不相同，但每个人都关闭了从当前策略行为到继任评估者，然后到后来的策略监督的相同回写路径。

Other critics learn from the downstream usefulness of their judgments. ECHO rewards a separate critic by the improvement obtained when the policy follows its critique [291]. ICRL uses the same causal test with a shared backbone: revision success supplies a role-specific critic reward, and the updated critic conditions later training trajectories while the token-calibration rule remains fixed [292]. RLAC grounds an adversarial critic in fixed fact-checking or execution outcomes, training it to surface rubrics that the current generator is likely to violate and thereby changing where later verification is applied [293].

Several systems encode an explicit supervisor role in the same checkpoint as the policy. Agentic Self-Learning trains a generative reward-model role from verifiable labels, CoNL assigns distinct rewards to critique and ranking segments, and CoVerRL explicitly trains a verifier role that filters pseudo-labels for later updates [294–296]. Self-Evolving Deep Research updates an evaluator role from report-consistency evidence, changing the rubric dimensions and scores supplied to subsequent solver optimization [297]. URPO is the limiting shared-state case: a Kendall-$\tau$ reward trains an identifiable referee interface from ranked preferences, and the same updated model later scores open-ended response groups [298]. In these methods, the evolving object is the trained supervision function and its downstream interface, not generic improvement of the shared policy.

Process-Level Credit Assignment Evolution

*Process-Level Credit Assignment Evolution* changes how outcome-level or trajectory-level evidence is attributed to intermediate decisions. It converts an available judgment into supervision for steps, actions, stages, turns, or tokens, thereby identifying which parts of a process should be reinforced or corrected. The evolving object may be an attribution rule, a process evaluator, a decomposition mechanism, or another retained structure that improves the localization of credit under later model behavior.

The defining question is where evidence should be assigned, not whether the overall behavior is successful: a mechanism belongs here when its main evolving output is the placement of credit over a process, even when a learned model implements that mechanism. A system whose primary change instead improves the underlying correctness or quality judgment belongs to *Evaluator and Verifier Evolution*.

Outcome-grounded process verifiers learn to replace a terminal verdict with denser judgments. RL Tango alternates a generator with a separately trained generative verifier: known final-answer correctness updates the verifier, whose successor step judgments become rewards for later generator updates [299]. RLAnything similarly trains a generative process reward model from terminal outcomes and self-consistency evidence, then combines its revised step evaluations with outcome feedback in subsequent policy updates [300].

Credit can also be represented through values or process preferences. Q-Evolve trains a persistent Q/V critic on hybrid trajectories and converts its value estimates through generalized advantage estimation into step-wise supervision for the next policy phase [301]. rStar-Math derives correct–incorrect step pairs from verified search trajectories to train a process-preference model, whose successor scores guide later MCTS nodes and policy-training targets [302]. StepORLM refines a generative process reward model from outcome-labeled trajectories; its revised scores determine both the preference pairs and pair weights used by the next policy update [303].

Explicit localization methods identify where a trajectory can still be corrected. STRIDE trains a verifier from final correctness, uses its language feedback to identify the first erroneous step, and constructs redirected prefixes for subsequent generator training [304]. UI-Genie instead launches continuation rollouts from intermediate GUI states to label whether completion remains possible; the resulting process labels update its reward model, whose successor judgments guide later trajectory selection and supervision [305].

# 原文第 37 页 | 中文翻译初稿

37 共 70

其他批评家从他们的判断的下游有用性中学习。 ECHO 通过当策略遵循其批评时所获得的改进来奖励单独的批评者 [291]。 ICRL 使用具有共享主干的相同因果测试：修订成功提供了特定于角色的评论家奖励，更新后的评论家条件后来的训练轨迹，而令牌校准规则保持固定[292]。 RLAC 将对抗性批评者置于固定的事实检查或执行结果中，训练其暴露当前生成器可能违反的规则，从而改变以后验证的应用范围 [293]。

多个系统在与策略相同的检查点中编码明确的主管角色。智能体自学习根据可验证的标签训练生成奖励模型角色，CoNL 为批评和排名部分分配不同的奖励，CoVerRL 明确训练一个验证者角色，过滤伪标签以供以后更新[294-296]。自我进化深度研究根据报告一致性证据更新了评估者角色，改变了为后续求解器优化提供的标题维度和分数[297]。 URPO 是限制共享状态的情况：Kendall-τ 奖励根据排名偏好训练可识别的裁判界面，并且相同的更新模型后来对开放式响应组进行评分 [298]。在这些方法中，演化的对象是经过训练的监督函数及其下游接口，而不是共享策略的一般改进。

过程级贡献归因（Credit Assignment）演进

_过程级贡献归因（Credit Assignment）进化_改变了结果级或轨迹级证据如何归因于中间决策。它将可用的判断转化为对步骤、行动、阶段、回合或标记的监督，从而确定流程的哪些部分应该加强或纠正。演化的对象可能是一个归因规则、一个过程评估器、一个分解机制，或者是另一个在以后的模型行为下改善信用定位的保留结构。

决定性的问题是证据应该分配在哪里，而不是整体行为是否成功：当一个机制的主要演变输出是在一个过程中放置信用时，即使一个学习模型实现了该机制，该机制就属于这里。其主要变化反而提高了底层正确性或质量判断的系统属于_Evaluator和Verifier Evolution_。

以结果为基础的过程验证者学会用更密集的判断来取代最终判决。 RL Tango 将生成器与单独训练的生成验证器交替使用：已知的最终答案正确性更新验证器，其后续步骤判断将成为后续生成器更新的奖励[299]。RLanything 类似地根据最终结果和自我一致性证据训练生成过程奖励模型，然后将其修订的步骤评估与后续策略更新中的结果反馈相结合[300]。

信用也可以通过价值观或流程偏好来表示。 Q-Evolve 在混合轨迹上训练持续的 Q/V 批评家，并通过广义优势估计将其价值估计转换为下一策略阶段的逐步监督 [301]

。 rStar-Math 从经过验证的搜索轨迹中导出正确-错误的步骤对来训练过程偏好模型，其后继分数指导后续的 MCTS 节点和策略训练目标 [302]。 StepORLM 根据结果标记的轨迹完善生成过程奖励模型；其修订后的分数决定了下一次策略更新所使用的偏好对和对权重[303]。

显式定位方法可识别仍可纠正轨迹的位置。 STRIDE 从最终正确性开始训练验证器，使用其语言反馈来识别第一个错误步骤，并为后续生成器训练构建重定向前缀[304]。 UI-Genie 相反从中间 GUI 状态启动延续展开轨迹（Rollouts）来标记是否仍然可以完成；由此产生的过程标签更新其奖励模型，其后继判断指导后来的轨迹选择和监督[305]。

Training-Signal Routing and Internalization

*Training-Signal Routing and Internalization* changes how already available supervision is selected, weighted, transferred, or made usable by the model. Routing determines which supervision channels, examples, teacher views, critiques, or reward components reach a learning decision and with what relative influence. Internalization transfers information available through a teacher, critic, privileged view, or temporary scaffold into a model that can later act without that auxiliary source. This category begins where the previous three end: the standard, the judgment, and its placement are already fixed, and what changes is how the resulting signal reaches the model and outlives its original source.

Routing state determines which already defined signals dominate the next update. SPARD maintains moving estimates of progress and dispersion for fixed reward dimensions, then updates their scalarization weights so later responses receive a different reward mixture [306]. ZeroCoder's Dynamic B4 mechanism recalibrates Bayesian selector priors from current-policy execution matrices and a small labeled set; the active priors then change which consensus outputs become coder and tester rewards in the next update [307]. The former continuously reweights reward channels, while the latter changes which proxy labels are admitted as supervision.

Internalization methods change the privileged teacher state from which training targets are drawn. Skill-SD writes successful strategies, mistakes, and workflows into a utility-tracked skill bank; a selected skill conditions a refreshed teacher whose token distribution supervises the unaugmented student [308]. UCOB maintains task- and state-level skill memories, compares skilled and unskilled returns at matched states, and distills from the locally stronger view while updating each skill's future retrieval utility [309]. In both systems, the persistent skill-and-teacher state changes later training targets; an inference-only memory would leave those targets unchanged.

Together, these four interfaces trace the supervision path from defining a standard, through producing and localizing a judgment, to delivering that judgment to the model. The next subsection turns from the supervision path to the mechanisms that use its output to form parameter updates.

### 5.1.2. Optimization Strategy Evolution

Optimization-strategy evolution begins after supervision has been produced. It changes persistent trainer state that determines either what objective is optimized or how that objective becomes a parameter update. We therefore separate two interfaces: *Objective Adaptation* changes the mathematical target assembled from existing supervision, whereas *Parameter-Update Adaptation* changes the controls that realize this target as parameter changes. In both cases, evidence from the active training process is written into an explicit objective or update-control state, and the revised state governs later updates in the same model lineage. Ordinary optimizer moments and schedules fixed by step count remain part of ordinary training because their transitions are already specified by a fixed update rule.

Objective Adaptation

Objective adaptation changes how available supervision is combined, normalized, or anchored before an optimizer consumes it. One route revises the relative influence of existing signals. SPARD tracks progress and dispersion for fixed reward dimensions and updates a persistent scalarization vector, so subsequent GRPO steps optimize a different reward mixture [306]. RL Tango maintains separate exponential-moving-average statistics for correct and incorrect generator outcomes; these statistics rescale the advantages used in later verifier updates and thereby rebalance the two outcome classes [299]. The two methods adapt objective weights at different resolutions: SPARD operates across reward dimensions, while RL Tango operates across outcome classes. Their separate data-routing and verifier-learning mechanisms belong to Supervision Design Evolution.

Reference-based objectives expose a second control surface. In ATLAS v3, phase telemetry updates the DPO coefficient $\beta_{\mathrm{DPO}}$, and a fixed utility–proxy-KL gate decides whether a policy checkpoint should become the reference for the next EvoDPO phase [310]. The coefficient controls the strength of reference-relative optimization, while the accepted checkpoint changes the anchor used in later

# 原文第 38 页 | 中文翻译初稿

38 共 70
训练-信号路由和内部化

_训练-信号路由和内化_改变了模型如何选择、加权、传输或使可用的监督可用。路由决定哪些监督渠道、例子、教师观点、批评或奖励成分达成学习决策以及具有何种相对影响力。内化将通过教师、批评家、特权观点或临时支架获得的信息转移到模型中，该模型随后可以在没有辅助来源的情况下发挥作用。这一类别的起点是前三者的终点：标准、判断及其位置已经固定，改变的是结果信号如何到达模型并比其原始信号源寿命更长。

路由状态决定哪些已定义的信号主导下一次更新。 SPARD 保持对固定奖励维度的进展和离散度的移动估计，然后更新其标化权重，以便后续响应收到不同的奖励混合物 [306]。 ZeroCoder 的动态 B4 机制根据当前策略执行矩阵和小型标记集重新校准贝叶斯选择器先验；然后，活跃的先验会改变哪些共识输出在下一次更新中成为编码器和测试人员的奖励[307]。前者不断重新调整奖励渠道的权重，后者则改变哪些智能体标签被接纳为监管。

内化方法改变了制定训练目标的教师特权状态。 Skill-SD 将成功的策略、错误和工作流程写入实用程序跟踪的技能库；选定的技能条件是一位焕然一新的教师，其代币分配监督未增强的学生[308]。 UCOB 维护任务级和状态级技能记忆，比较匹配状态下的技能和非技能回报，并从局部更强的视图中提取，同时更新每个技能的未来检索效用 [309]。在这两个系统中，持久的技能和教师状态会改变以后的训练目标；仅推理记忆将使这些目标保持不变。

这四个接口共同追踪从定义标准到生成和本地化判断，再到将该判断传递给模型的监督路径。下一小节从监督路径转向使用其输出形成参数更新的机制。
5.1.2.优化策略演化

优化策略的演化在监督产生后开始。它改变持久的训练器状态，确定优化哪个目标或该目标如何成为参数更新。因此，我们分离了两个接口：_目标适应_改变从现有监督组装的数学目标，而_参数更新适应_随着参数变化改变实现该目标的控制。在这两种情况下，来自主动训练过程的证据都被写入明确的目标或更新控制状态，并且修订后的状态控制同一模型谱系中的后续更新。由步数固定的普通优化器时刻和时间表仍然是普通训练的一部分，因为它们的转换已经由固定更新规则指定。
客观适应

目标适应改变了优化器使用可用监督之前的组合、标准化或锚定方式。一种方法是修改现有信号的相对影响。 SPARD 跟踪固定奖励维度的进展和分散并更新持久的标量化向量，因此后续的 GRPO 步骤优化不同的奖励混合物 [306]。 RL Tango 为正确和错误的生成器结果维护单独的指数移动平均统计数据；这些统计数据重新调整了以后验证者更新中使用的优势，从而重新平衡了两个结果类别[299]。这两种方法在不同的分辨率下调整客观权重：SPARD 跨奖励维度运行，而 RL Tango 跨结果类别运行。它们各自的数据路由和验证者学习机制属于监督设计演化。

基于参考的目标暴露了第二个控制面。在 ATLAS v3 中，阶段遥测更新 DPO 系数_β_ DPO，并且固定实用程序-智能体-KL 门决定策略检查点是否应成为下一个 EvoDPO 阶段的参考[310]。该系数控制参考相对优化的强度，而接受的检查点会改变后面使用的锚点

likelihood ratios. This is objective-state adaptation within one continuing lineage; preference-pair admission is an upstream supervision decision.

Some systems expose several bounded objective controls through one recipe interface. AI Training Manager reads active-run telemetry and prior intervention outcomes, proposes schema-constrained changes to loss weights or regularization, and persists only changes that pass deterministic range, freshness, and safety checks [311]. The accepted recipe governs later updates, while the manager model, action schema, verifier, and application protocol remain fixed.

Parameter-Update Adaptation

Parameter-update adaptation preserves the established target and changes how it is converted into parameter movement. AdaLRS provides the clearest example of evidence-conditioned step-size control: when loss-descent velocity slows, it probes a higher learning rate, keeps the probe if the velocity improves, and otherwise rolls back and applies a lower rate [312]. The retained learning-rate state controls later steps in the same run, while the probe and rollback rules themselves remain fixed.

The legal support of an update can evolve independently of its magnitude. Evolving Parameter Isolation accumulates squared-gradient evidence into a sensitivity estimate and periodically converts it into a protection mask. The mask suppresses AdamW updates and weight decay on newly important parameters, while parameters whose estimated importance has faded can re-enter the trainable set [313]. The persistent mask therefore determines which coordinates later updates may modify.

Batch adaptation changes the stochastic gradient estimator and often couples that change to the effective step size. COPUS and KungFu use online gradient-noise evidence to revise the batch regime used by subsequent updates; COPUS additionally writes micro-batch size, accumulation state, and a coupled Adam learning rate [314,315]. Pollux and Sia mediate the same optimizer-facing decision through learned statistical-efficiency or goodput models, selecting later batch sizes and associated learning-rate scaling [316,317]. These placements concern only batch and learning-rate state. Their independent changes to GPU allocation, parallel execution, placement, and worker topology belong to Training Infrastructure Evolution.

ONES and DYNAMIX provide cross-domain precedents for broader batch controllers. ONES uses online evolutionary scheduling to select a batch assignment and applies a coupled learning-rate scaling to later gradient steps [318]. DYNAMIX trains a PPO controller from training and systems evidence; the retained controller then chooses per-worker batch sizes for later update windows [319]. The former evolves candidate schedules, whereas the latter learns the update controller itself, but both leave their governing schedule-search or PPO procedure fixed.

Finally, RFT-FM shows how a diagnosis can feed a bounded update-control interface: its intervention stage receives an identified fault, writes a constrained corrective configuration, and resumes or restarts reinforcement fine-tuning under it [320]. The fault analysis itself belongs to Experiment Diagnosis in Section 5.2.1; what changes here is the configuration that controls subsequent updates.

Across these methods, the evolving object is a bounded objective or parameter-update state. The next subsection moves to the executable substrate: training code, parallel layout, runtime, placement, and resource scheduling.

### 5.1.3. Training Infrastructure Evolution

After supervision has been shaped into a parameter update, the training infrastructure determines how that update is actually executed: which workers participate, where model shards and computation are placed, how devices communicate, and how the workload is decomposed and scheduled across the cluster. *Training Infrastructure Evolution* occurs when an active training lineage rewrites this persistent execution state on the basis of its own measurements, so that later updates run under a configuration the run itself has selected. The evidence is typically gradient noise, throughput, or cluster load; the writeback is a new parallel layout, a resized worker set, or a revised resource allocation, often mediated by online performance estimates that training continues to refine. The boundary of the category lies in the origin of the decision rather than the form of the controller: reconfiguration that follows

# 原文第 39 页 | 中文翻译初稿

39 共 70

似然比。这是一个连续谱系内的客观状态适应；偏好对准入是上游监督决策。

一些系统通过一个配方接口公开多个有界目标控制。 AI Training Manager 读取主动运行的遥测数据和先前的干预结果，提出对减肥或正则化进行模式约束的更改，并仅保留通过确定性范围、新鲜度和安全检查的更改[311]。接受的配方控制以后的更新，而管理器模型、操作模式、验证器和应用程序协议保持固定。

参数更新适配

参数更新自适应保留既定目标并改变其转换为参数移动的方式。 AdaLRS 提供了证据条件步长控制的最清晰示例：当损失下降速度减慢时，它会探测更高的学习速率，如果速度提高则保持探测，否则回滚并应用较低的速率 [312]。保留的学习率状态控制同一运行中的后续步骤，而探测和回滚规则本身保持固定。

更新的法律支持可以独立于其规模而发展。进化参数隔离将平方梯度证据累积为灵敏度估计，并定期将其转换为保护掩模。该掩模抑制了 AdamW 更新和新重要参数的权重衰减，而估计重要性已减弱的参数可以重新进入可训练集[313]。因此，持久掩码确定稍后更新可以修改哪些坐标。

批量适应会改变随机梯度估计器，并且通常会耦合到有效步长的变化。 COPUS 和 KungFu 使用在线梯度噪声证据来修改后续更新所使用的批处理机制； COPUS 还记录了微批量大小、累积状态和耦合的 Adam 学习率 [314,315]。 Pollux 和 Sia 通过学习的统计效率或良好输出模型来调节相同的面向优化器的决策，选择稍后的批量大小和相关的学习率缩放[316,317]。这些放置仅涉及批次和学习率状态。它们对 GPU 分配、并行执行、放置和工作拓扑的独立更改属于训练基础设施演进。

ONES 和 DYNAMIX 为更广泛的批量控制器提供了跨域先例。 ONES 使用在线进化调度来选择批量分配，并将耦合学习率缩放应用于后续的梯度步骤 [318]。 DYNAMIX 根据训练和系统证据来训练 PPO 控制器；然后，保留的控制器为以后的更新窗口选择每个工作线程的批量大小[319]。前者发展候选调度，而后者学习更新控制器本身，但两者都保持其管理调度搜索或 PPO 程序固定。

最后，RFT-FM 展示了诊断如何提供有界更新控制接口：其干预阶段接收已识别的故障，写入受约束的纠正配置，并在其下恢复或重新启动强化微调[320]。故障分析本身属于5.2.1节的实验诊断

;这里改变的是控制后续更新的配置。

在这些方法中，演化的对象是有界目标或参数更新状态。下一小节转向可执行基础：训练代码、并行布局、运行时、布局和资源调度。
5.1.3.训练基础设施的演变

在监督形成参数更新后，训练基础设施决定如何实际执行更新：哪些工作人员参与，模型分片和计算放置在哪里，设备如何通信，以及如何在集群中分解和调度工作负载。当活动的训练沿袭根据其自己的测量重写此持久执行状态时，就会发生_训练基础设施演进_，以便以后的更新在运行本身选择的配置下运行。证据通常是梯度噪声、吞吐量或集群负载；写回是新的并行布局、调整大小的工作集或修订的资源分配，通常由训练不断完善的在线性能估计来调节。类别的边界在于决策的起源而不是控制器的形式：随后的重新配置

a predetermined schedule remains ordinary training, whereas even a fixed rule qualifies once live measurements decide which configuration is written back. Because a single reconfiguration often adjusts batch size or learning rate as well, Section 5.1.2 treats the optimization semantics of those changes, and this subsection follows their execution-side effects.

COPUS provides the clearest direct LLM example. During one pretraining run, it combines an online, 3D-parallel-aware gradient-noise estimate with a measured throughput table and ranks feasible global-batch, micro-batch, and data-, tensor-, and pipeline-parallel configurations under a fixed Goodput objective. When another topology offers sufficient projected gain after accounting for switching cost, COPUS pauses at an optimizer-step boundary, rebuilds process groups, reshards model and optimizer state, and resumes the same run under the selected layout [314]. The infrastructure-side successor is therefore an enacted parallel configuration that governs later updates in the same model lineage.

Earlier systems expose narrower run-internal control surfaces. KungFu embeds monitoring and control operators in the training dataflow, allowing a fixed user-authored adaptation policy to use live gradient or throughput measurements to resize the worker set or replace the collective-communication topology [315]. The policy remains fixed, while its evidence-dependent output becomes the distributed state used by subsequent iterations. EARL specializes this pattern to agentic LLM reinforcement learning: startup profiling maps context-length ranges to parallel configurations, and the current context length and system load select the layout used by later rollout and experience-preparation stages [321]. Its selector provides bounded infrastructure adaptation; the lookup rule and layout-aware dispatcher remain fixed, and EARL does not dynamically reconfigure the gradient-update stage.

At cluster scope, training progress can be combined with the state of several active jobs. Optimus repeatedly fits per-job convergence and resource–speed models from observed loss and throughput, then changes worker and parameter-server allocations according to their predicted effect on completion time [322]. ONES adds an online evolutionary scheduler: epoch-level progress and cluster state score a population of job–GPU mappings, and the selected mapping changes worker topology and placement before the affected jobs continue [318]. These systems establish predictive and evolutionary resource scheduling as broader model-training precedents, while keeping the scheduling objective and update rule fixed.

Pollux couples the job and cluster levels through Goodput. Each job continually estimates statistical efficiency and throughput from gradient and timing measurements; a cluster scheduler uses the resulting functions to revise GPU allocations, after which the same job continues under the new resource state [316]. Sia extends this feedback path to heterogeneous clusters by learning per-job performance across GPU types and selecting GPU type, count, and placement at successive scheduling rounds [317]. Both systems also coordinate batch-related controls, but their infrastructure contribution is the persistent reassignment of workers and devices from evidence collected during ongoing training.

Rubick broadens the mutable object from resource quantity to the execution plan itself: runtime measurements refine a joint model of resource allocations and parallelization strategies, and a fixed scheduler applies the selected resource–plan pair through checkpoint and resume while preserving the global batch size [323]. Subsequent updates may therefore use a different organization of computation, communication, and memory without changing the learning objective.

Across these systems, measured evidence selects an infrastructure state that is written directly into an active training lineage. Profiling, analytical enumeration, or surrogate search inside the controller does not create the experiment boundary used in Section 5.2. That boundary appears when an infrastructure change becomes a distinct runnable trainer candidate, receives downstream evidence from a declared training experiment, and is then promoted, rejected, or rolled back. We next examine this experimental search loop in *Outer-Loop Trainer Search through Experiments*.

### *5.2. Outer-Loop Trainer Search Through Experiments*

*Outer-Loop Trainer Search through Experiments* organizes trainer self-improvement as a cross-experiment process of candidate construction, evaluation, and retention, as shown in Figure 7. A

# 原文第 40 页 | 中文翻译初稿

70 中的 40

预定的时间表仍然是普通的训练，而一旦实时测量决定写回哪种配置，即使是固定的规则也符合资格。由于单个重新配置通常也会调整批量大小或学习率，因此第 5.1.2 节将讨论这些更改的优化语义，本小节将介绍它们的执行副作用。

COPUS 提供了最清晰直接的 LLM 示例。在一次预训练运行期间，它将在线、3D 并行感知梯度噪声估计与测量的吞吐量表相结合，并在固定的 Goodput 目标下对可行的全局批量、微批量以及数据、张量和管道并行配置进行排名。当另一个拓扑在考虑切换成本后提供足够的预计增益时，COPUS 在优化器步骤边界处暂停，重建进程组，重新分片模型和优化器状态，并在所选布局下恢复相同的运行[314]。因此，基础设施端的后继者是一个制定的并行配置，它管理同一模型谱系中的后续更新。

早期的系统暴露出较窄的运行内部控制面。 KungFu 在训练数据流中嵌入监视和控制操作符，允许固定的用户编写的适应策略使用实时梯度或吞吐量测量来调整工作集的大小或替换集体通信拓扑[315]。该策略保持固定，而其依赖证据的输出成为后续迭代使用的分布式状态。 EARL 将这种模式专门用于智能体 LLM 强化学习（Reinforcement Learning）：启动分析将上下文长度范围映射到并行配置，当前上下文长度和系统负载选择后面的 展开轨迹（Rollout） 和体验准备阶段使用的布局[321]。它的选择器提供有限的基础设施适应；查找规则和布局感知调度程序保持固定，并且 EARL 不会动态重新配置梯度更新阶段。

在集群范围内，训练进度可以与多个活动作业的状态相结合。 Optimus 根据观察到的损失和吞吐量反复拟合每个作业的收敛和资源速度模型，然后根据对完成时间的预测影响更改工作人员和参数服务器分配[322]。ONES 添加了一个在线进化调度程序：纪元级进度和集群状态对作业-GPU 映射群体进行评分，并且所选映射会在受影响的作业继续之前更改工作节点拓扑和布局 [318]。这些系统将预测和进化资源调度建立为更广泛的模型训练先例，同时保持调度目标和更新规则固定。

Pollux 通过 Goodput 将作业和集群级别耦合起来。每项工作都会根据梯度和时序测量不断估计统计效率和吞吐量；集群调度程序使用结果函数来修改 GPU 分配，然后在新的资源状态下继续执行相同的作业 [316]。Sia 通过学习跨 GPU 类型的每个作业性能并在连续的调度轮次中选择 GPU 类型、数量和布局，将此反馈路径扩展到异构集群 [317]

。这两个系统还协调与批次相关的控制，但它们的基础设施贡献是根据持续训练期间收集的证据持续重新分配工作人员和设备。

Rubick 将可变对象从资源数量扩展到执行计划本身：运行时测量细化了资源分配和并行化策略的联合模型，固定调度程序通过检查点和恢复应用选定的资源计划对，同时保留全局批量大小[323]。因此，后续更新可能会使用不同的计算、通信和内存组织，而不改变学习目标。

在这些系统中，测量的证据选择直接写入活动训练谱系的基础设施状态。控制器内部的分析、分析枚举或智能体搜索不会创建第 5.2 节中使用的实验边界。当基础设施变更成为一个独特的可运行训练器候选者，从已声明的训练实验接收下游证据，然后被提升、拒绝或回滚时，就会出现该边界。接下来，我们在 _Outer-Loop Trainer Search through Experiments_中检查这个实验性搜索循环。

5.2。通过实验搜索外环训练器

_通过实验进行外环训练器搜索_将训练器 自我改进（Self-Improvement） 组织为候选构建、评估和保留的交叉实验过程，如图 7 所示。

proposed change is instantiated as a runnable trainer candidate and evaluated through a training experiment. Evidence from the experiment determines whether the candidate is rejected, evaluated further, or promoted. A promoted candidate becomes the retained trainer version used in subsequent experiments.

We organize this literature around three functions. *Experiment Diagnosis* interprets experiment evidence and explains the observed result. *Trainer Candidate Search* constructs runnable alternatives and organizes their version relationships. *Trainer Candidate Evaluation* gathers evidence and decides whether a candidate should be rejected, continued, or promoted. Together, these functions use evidence from training experiments to evaluate trainer candidates and determine which version is retained for subsequent experiments.

#### 5.2.1. Experiment Diagnosis

Experiment Diagnosis converts artifacts from a completed or bounded training experiment into an evidence-linked account of its outcome. The account identifies the observed failure, a likely cause, the affected trainer-side object, the strength and uncertainty of the evidence, and a hypothesis that a later experiment could falsify. The affected object may be supervision code, an objective or optimizer, a training configuration, or execution state. A scalar score can rank experiments or trigger inspection, but ranking alone does not explain a result. Diagnosis reads the observation surface produced by the experiment. Changes to future instrumentation belong to Meta-Loop Improvement-Mechanism Evolution, while runnable alternatives and promotion decisions belong to Trainer Candidate Search and Trainer Candidate Evaluation.

We organize diagnosis as three successive transformations. *Diagnostic Evidence Synthesis* structures heterogeneous artifacts while preserving their experimental provenance. *Trainer Fault Localization* maps the resulting evidence to a failure mechanism and an affected trainer-side object. *Diagnostic Hypothesis Validation* qualifies that account through historical or controlled comparison and records what later evidence could refute it.

##### Diagnostic Evidence Synthesis

Diagnostic Evidence Synthesis turns heterogeneous run artifacts into a representation from which a failure can be inferred. Useful evidence records when an observation occurred, which component produced it, which behavior it describes, and which experiment or trainer version it belongs to. This provenance is essential because the same terminal outcome can arise from sparse supervision, ineffective updates, invalid output formatting, evaluator failure, or broken execution.

Run-local synthesis assembles optimization telemetry and behavior from the same experiment. Reward, KL, entropy, policy-loss, and response-length traces provide a temporal fingerprint of RFT failure modes [320]. Validation, gradient-norm, and clipping trajectories add checkpoint-level evidence for adaptive-strategy diagnosis [324]. Post-run views add losses and failed-output examples from the same experiment [327], so a terminal regression is represented through temporal and behavioral signatures rather than as a single scalar event. These signatures remain correlational: when a controller changes several settings together, they bound plausible explanations but do not identify a unique cause.

Version-bound artifact packets add provenance across code and execution. Metrics, rollouts, configurations, logs, diffs, legality outcomes, and runtime traces are attached to the candidate or distributed configuration that produced them, separating a valid negative result from an invalid run or measurement failure [325,328,330]. Within this family, EvoTrainer interprets its artifact inputs through four diagnostic layers—score, signal, behavior, and version [325].

Persistent lineages add comparative context by preserving parent deltas, bad-case attributions, effect estimates, failed ranges, and dead ends around an incumbent. Their common role is to relate the current outcome to known version changes and contrary outcomes, not to claim causal identification from history alone [331–333].

# 原文第 41 页 | 中文翻译初稿

41 共 70

提议的更改被实例化为可运行的候选训练器，并通过训练实验进行评估。实验的证据决定了候选人是否被拒绝、进一步评估或晋升。升级的候选者将成为后续实验中使用的保留训练器版本。

我们围绕三个功能来组织这些文献。 _实验诊断_解释实验证据并解释观察到的结果。 Trainer Candidate Search 构造可运行的替代方案并组织它们的版本关系。 _训练师候选人评估_收集证据并决定是否应拒绝、继续或晋升候选人。这些函数一起使用训练实验的证据来评估候选训练器并确定为后续实验保留哪个版本。

5.2.1.实验诊断

实验诊断将已完成或有界训练实验中的工件转换为其结果的证据相关说明。该描述确定了观察到的失败、可能的原因、受影响的训练者端对象、证据的强度和不确定性，以及后续实验可能证伪的假设。受影响的对象可能是监督代码、目标或优化器、训练配置或执行状态。标量分数可以对实验进行排名或触发检查，但排名本身并不能解释结果。诊断读取实验产生的观察面。未来仪器的改变属于元循环改进机制进化，而可运行的替代方案和升级决策属于训练师候选人搜索和训练师候选人评估。

我们将诊断组织为三个连续的转变。 _诊断证据合成_构建异构工件，同时保留其实验来源。 _训练器故障定位_将所得证据映射到故障机制和受影响的训练器端对象。 _诊断假设验证_通过历史或受控比较来验证该说法，并记录后来的证据可以反驳它。
诊断证据综合

诊断证据合成将异构运行工件转换为可以推断故障的表示形式。有用的证据记录了观察发生的时间、哪个组件产生了它、它描述了哪些行为以及它属于哪个实验或训练器版本。这种来源至关重要，因为稀疏的监督、无效的更新、无效的输出格式、评估器故障或执行中断可能会产生相同的最终结果。

运行本地综合汇集了来自同一实验的优化遥测和行为。奖励、KL、熵、策略损失和响应长度轨迹提供了 RFT 故障模式的时间指纹[320]。验证、梯度范数和裁剪轨迹为自适应策略诊断添加了检查点级证据[324]。运行后视图添加来自同一实验的损失和失败输出示例 [327]，因此终端回归通过时间和行为签名表示，而不是作为单个标量事件表示

。这些签名保持相关性：当控制器一起更改多个设置时，它们会给出合理的解释，但无法识别唯一的原因。

版本绑定的工件数据包添加了跨代码和执行的来源。指标、展开轨迹（Rollouts）、配置、日志、差异、合法性结果和运行时跟踪附加到生成它们的候选或分布式配置，将有效的负面结果与无效的运行或测量失败分开[325,328,330]。在这个系列中，EvoTrainer 通过四个诊断层（分数、信号、行为和版本）解释其工件输入 [325]。

持久的血统通过保留父代增量、坏情况归因、效果估计、失败范围和现任者周围的死胡同来增加比较背景。他们的共同作用是将当前结果与已知版本更改和相反结果联系起来，而不是仅从历史中得出因果关系[331-333]。

Trainer Fault Localization

Evidence becomes diagnosis when it is mapped to an affected trainer-side object and a failure mechanism. A useful localization has the form *artifact pattern* → *failure mode* → *affected object* → *predicted consequence*. The output remains an explanatory claim: constructing the corresponding modification begins in Trainer Candidate Search.

Learning-side localization is most defensible when it maps a repeatable observation pattern to a bounded trainer object. Fine-grained fault schemas and layered evidence connect temporal anomalies or behavioral failures to reward computation, generation, optimization, credit assignment, tool interaction, exposed training controls, or harness code [320,324,325]. These mappings define a plausible intervention surface, but LLMZero's coordinated multi-parameter transitions do not isolate a single causal control. OptiCo extends the same schema-based localization to distributed-runtime failures, mapping configuration and execution evidence to a root-cause category, implicated fields, constraints, and an uncalibrated, agent-reported confidence used to route later search [330].

Known version deltas provide another narrowing device. Parent–child trajectories, sibling and historical comparisons, and champion-relative dead-end records restrict a hypothesis to what changed while preserving counterevidence; they still leave simultaneous edits and inherited state as possible confounders [331–333].

At the supervision interface, cross-domain reward-design systems connect policy behavior to specific signal defects. Component trajectories expose inactive or dominant terms, trajectory preferences expose ordering violations, and evidence-keyed critiques name a bounded shaping failure before candidate generation [334–336]. These are transferable localization interfaces, not direct evidence of general LLM trainer diagnosis. Conversely, a free-form reflection produced jointly with its mutation does not by itself establish localization; the affected trainer object and its supporting observations must remain separable from the next candidate.

Diagnostic Hypothesis Validation

Diagnostic Hypothesis Validation asks whether a localized explanation is sufficiently discriminating to guide a later test. A complete hypothesis states the suspected fault, the affected trainer-side object, the supporting and contradicting observations, and an outcome that would refute the proposed cause. This stage interprets available comparisons and specifies the required test. The execution of a new branch and the decision to promote it remain in Trainer Candidate Evaluation.

Evidence strength increases from historical recurrence to matched intervention. Version histories can reveal whether a proposed relation recurs or encounters counterexamples across parent–child changes and sibling branches, but they do not remove confounding from seed, checkpoint state, horizon, or coupled edits [325,332,333].

Matched-state probes provide sharper local tests. A same-checkpoint fork can compare a shaping update with a no-change continuation and reject a near tie [336]. A richer matched-state design clones policy, critic, and optimizer state across reward hypotheses, then uses winner margin, repeated-fork agreement, entropy, and abstention to express when the contrast is non-discriminating [346]. These cross-domain probes qualify a hypothesis under a bounded horizon; their execution, budget allocation, and promotion remain in Trainer Candidate Evaluation.

Implementation and measurement claims require a separate falsification surface. Single-factor branches, historical-rollout backtests, evaluator or harness audits, legality checks, code review, and progressive mini-runs can expose leakage, invalid code, constraint violations, or a failure that does not improve when its implicated object changes [325,327–329]. These checks can reject an explanation without turning a successful repair into proof of a general causal mechanism.

Experiment Diagnosis therefore ends with an evidence-linked, uncertainty-qualified, and falsifiable trainer hypothesis. It records what failed, why a particular trainer-side object is implicated, which observations support or contradict the account, and what result would disconfirm it. Section 5.2.2

# 原文第 42 页 | 中文翻译初稿

42 共 70
训练器故障定位

当证据映射到受影响的训练者端对象和故障机制时，证据就成为诊断。有用的定位具有以下形式：工件模式→故障模式→受影响的对象→预测的后果。输出仍然是一个解释性声明：构建相应的修改从训练师候选人搜索开始。

当学习端定位将可重复的观察模式映射到有界训练对象时，它是最可靠的。细粒度的故障模式和分层证据将时间异常或行为失败连接到奖励计算、生成、优化、贡献归因（Credit Assignment）、工具交互、暴露的训练控制或运行支撑框架（Harness）代码[320,324,325]。这些映射定义了一个合理的干预面，但 LLMZero 的协调多参数转换不会孤立单个因果控制。 OptiCo 将相同的基于模式的本地化扩展到分布式运行时故障，将配置和执行证据映射到根本原因类别、隐含字段、约束以及用于路由后续搜索的未经校准的智能体报告置信度[330]。

已知版本的三角洲提供了另一种缩小装置。亲子轨迹、兄弟姐妹和历史比较以及冠军相关的死胡同记录将假设限制在发生变化的情况下，同时保留反证据；他们仍然将同时编辑和继承状态作为可能的混杂因素[331-333]。

在监督界面，跨域奖励设计系统将策略行为与特定信号缺陷联系起来。组件轨迹暴露了不活跃或占主导地位的术语，轨迹偏好暴露了顺序违规，而基于证据的批评在候选生成之前命名了有限的塑造失败[334-336]。这些是可转移的本地化界面，而不是一般法学硕士训练师诊断的直接证据。相反，与其突变共同产生的自由形式反射本身并不建立定位；受影响的训练对象及其支持观察结果必须与下一个候选对象保持分离。
诊断假设验证

诊断假设验证询问本地化解释是否足以指导后续测试。完整的假设说明了可疑的故障、受影响的训练者端对象、支持和矛盾的观察结果以及反驳所提出原因的结果。此阶段解释可用的比较并指定所需的测试。新分支的执行和提升它的决定仍保留在训练师候选人评估中。

从历史复发到匹配干预，证据强度不断增强。版本历史可以揭示提议的关系是否重复出现或在父子更改和同级分支中遇到反例，但它们不能消除种子、检查点状态、范围或耦合编辑的混淆[325,332,333]。

匹配状态探针提供更清晰的本地测试。相同检查点分叉可以将整形更新与无更改延续进行比较，并拒绝接近平局 [336]

。更丰富的匹配状态设计克隆了奖励假设中的策略、批评者和优化器状态，然后使用获胜者利润、重复分叉协议、熵和弃权来表达对比是非歧视性的[346]。这些跨域探测在有限的范围内验证了假设；他们的执行、预算分配和晋升仍保留在训练师候选人评估中。

实施和测量声明需要单独的伪造表面。单因素分支、历史 展开轨迹（Rollout） 回测、评估器或 运行支撑框架（Harness） 审计、合法性检查、代码审查和渐进式小运行可能会暴露泄漏、无效代码、约束违规或在其所涉及的对象发生变化时无法改善的故障[325,327-329]。这些检查可以拒绝解释，而不将成功的修复转化为一般因果机制的证据。

因此，实验诊断以与证据相关、不确定性合格且可证伪的训练师假设结束。它记录了失败的原因、为什么某个特定的训练者端对象受到牵连、哪些观察结果支持或反驳该帐户，以及什么结果会反驳该帐户。第 5.2.2 节

materializes that hypothesis as a runnable trainer candidate. Section 5.2.3 executes discriminating tests and decides whether the candidate should be rejected, continued, or promoted.

#### 5.2.2. Trainer Candidate Search

*Trainer Candidate Search* turns a diagnosed hypothesis into a runnable alternative to the retained trainer: a candidate specification that states what changes, which parent version and training state it starts from, and how to execute it as a training experiment. Whether the candidate should be kept is not decided here; that is the task of Trainer Candidate Evaluation.

We distinguish three classes of candidates by the depth of the required change. *Training-Configuration Search* stays within the existing trainer interface and chooses values, schedules, or pre-implemented options. *Learning-Objective and Signal Search* changes what the model is trained to optimize—rewards, losses, targets, or credit assignment—while the surrounding training procedure remains intact. *Executable Training-Procedure Search* rewrites the training computation itself, from update rules and control flow to complete training pipelines. A candidate that spans several levels is classified by the deepest change it requires; the finer boundaries are drawn within each class below.

##### Training-Configuration Search

*Training-Configuration Search* constructs candidates by assigning values, schedules, or named alternatives already exposed by a fixed trainer interface. Its scope includes hyperparameters, static mixtures over available data sources, batch and sampling settings, checkpoint anchors, fixed reward coefficients, resource allocations, stopping conditions, and schedules over already defined training stages. The candidate may coordinate many fields and may define a multi-stage recipe, but the meaning of every field and the executable implementation behind every option remain fixed.

The category is bounded by the exposed interface, not by the apparent size or file format of the intervention: selecting a pre-implemented optimizer from a registry, retuning existing reward coefficients, or scheduling already defined stages are all configuration choices, whereas a candidate leaves the category as soon as it changes what a field means or how an option is implemented. A horizon or data allocation belongs to the candidate only when it is retained as part of the trainer recipe; a temporary budget used to screen candidates belongs to the evaluation contract, and generating or curating the underlying data belongs to the data system.

At its narrowest, candidate construction is conventional hyperparameter search over a declared schema. Efficient HPO for LLM RL uses a cost-aware acquisition rule to propose GRPO configurations from a fixed parameter space [347]. The inner loop of Bilevel Autoresearch instead asks an LLM search operator to make incumbent-relative parameter edits, but each first-order candidate is still an assignment within the unchanged training program [238]. Both systems can instantiate a candidate without introducing a new learning objective or executable operator.

Natural-language recipe construction remains configuration search when the backend defines the available training operations. AutoTrainess and FT-Dojo both assemble runnable post-training recipes within fixed execution interfaces [326,327]. AutoTrainess selects supported stages and methods and emits a complete LlamaFactory configuration, whereas FT-Dojo revises an incumbent SFT or LoRA recipe through exposed controls such as the update mode, optimizer settings, formatting, and duration. The proposal language is open-ended, but the candidate vocabulary is bounded by the backend.

Configuration candidates may also bind parameter choices to inherited training state. LLMZero constructs each child phase from a coordinated configuration, a parent checkpoint or scratch initialization, and a phase budget, so successive children form an adaptive multi-stage recipe within the same GRPO pipeline [324].

The same boundary extends beyond single-model recipes. OptiCo applies diagnosis-guided, rule-constrained transformations to distributed-training configurations while leaving the underlying runtime implementation fixed [330]. As a cross-domain boundary case, LLM-ALSO searches a predefined potential-based reward-shaping space [336]. Selecting terms and coefficients from that fixed basis is a configuration choice, even though the configured fields affect reward shaping.

# 原文第 43 页 | 中文翻译初稿

43 共 70

将这一假设具体化为可运行的训练器候选者。第 5.2.3 节执行区分测试并决定是否应拒绝、继续或晋升候选人。

5.2.2.训练师候选人搜索

_训练器候选搜索_将诊断的假设转化为保留训练器的可运行替代方案：候选规范，说明发生了什么变化、从哪个父版本和训练状态开始，以及如何将其作为训练实验来执行。是否保留该候选人不是这里决定的；这就是训练师候选人评估的任务。

我们根据所需变革的深度来区分三类候选人。 TrainingConfiguration Search 保留在现有的训练师界面中，并选择值、计划或预实施的选项。 _学习目标和信号搜索_改变模型训练优化的内容--奖励、损失、目标或贡献归因（Credit Assignment）--而周围的训练程序保持不变。 _可执行训练过程搜索_重写了训练计算本身，从更新规则和控制流到完整的训练管道。跨越多个级别的候选者按其所需的最深变化进行分类；下面的每个类别中都划出了更精细的界限。

训练-配置搜索

_训练配置搜索_通过分配已由固定训练器界面公开的值、时间表或命名替代方案来构造候选者。其范围包括超参数、可用数据源的静态混合、批次和采样设置、检查点锚点、固定奖励系数、资源分配、停止条件以及已定义训练阶段的时间表。候选人可以协调许多字段并可以定义多阶段配方，但每个字段的含义以及每个选项背后的可执行实现保持固定。

该类别受公开接口的限制，而不是受干预的表观大小或文件格式的限制：从注册表中选择预先实现的优化器、重新调整现有的奖励系数或调度已经定义的阶段都是配置选择，而候选者一旦更改字段的含义或选项的实现方式，就会离开该类别。仅当范围或数据分配作为训练器配方的一部分保留时才属于候选者；用于筛选候选人的临时预算属于评估合约，生成或整理底层数据属于数据系统。

从最狭义的角度来说，候选构建是对已声明模式的传统超参数搜索。 LLM RL 的高效 HPO 使用成本感知采集规则从固定参数空间提出 GRPO 配置[347]。 Bilevel Autoresearch 的内部循环要求 LLM 搜索操作员进行现有相关参数编辑，但每个一阶候选者仍然是未更改的训练计划中的分配 [238]。两个系统都可以实例化候选者，而无需引入新的学习目标或可执行运算符。

当后端定义可用的训练操作时，自然语言配方构建仍然是配置搜索

。 AutoTrainess 和 FT-Dojo 都在固定的执行接口中组装可运行的训练后配方 [326,327]。 AutoTrainess 选择支持的阶段和方法，并发出完整的 LlamaFactory 配置，而 FT-Dojo 通过公开的控件（例如更新模式、优化器设置、格式和持续时间）修改现有的 SFT 或 LoRA 配方。提案语言是开放式的，但候选词汇受后端限制。

配置候选者还可以将参数选择绑定到继承的训练状态。 LLMZero 根据协调配置、父检查点或暂存初始化以及阶段预算构建每个子阶段，因此连续的子阶段在同一 GRPO 管道内形成自适应多阶段配方 [324]。

同样的界限也超出了单一模型配方的范围。 OptiCo 将诊断引导、规则约束的转换应用于分布式训练配置，同时保持底层运行时实现固定 [330]。作为跨域边界情况，LLM-ALSO 搜索预定义的基于潜力的奖励塑造空间 [336]。从该固定基础中选择项和系数是一种配置选择，即使配置的字段会影响奖励塑造。

Learning-Objective and Signal Search

*Learning-Objective and Signal Search* changes what the learner is trained to optimize or how supervision is assigned to its behavior, while the surrounding training scaffold stays in place. Its scope includes objective and loss definitions, reward and shaping functions, target construction rules, advantage transformations, supervision weights, and mechanisms that distribute credit across examples, tokens, actions, or trajectories.

Classification follows the functional role of the signal, not its surface form: a signal may be written declaratively, symbolically, in natural language, or as executable reward or loss code, and it belongs here as long as an otherwise fixed learner consumes it through a stable interface. Selecting among pre-implemented objectives or tuning their coefficients remains *Training-Configuration Search*, and a candidate stops being a signal candidate once realizing it requires changing the learner's own computation or control flow. This category also concerns the objective used to train the model, not the criterion used to certify the candidate: a metric that only compares completed candidates belongs to Trainer Candidate Evaluation, whereas any quantity consumed during optimization belongs to the candidate, even when it is also reported as an experiment metric.

Executable syntax alone does not make a candidate a general training procedure. DiscoPOP uses an LLM to write and revise preference-optimization losses as Python functions [348], while POISE instantiates policy-optimization candidates through a shared interface for losses, advantage transformations, and regularization [333]. In both cases, the code changes the mathematical signal consumed by an otherwise stable training scaffold.

Signal candidates differ in how tightly their semantics are constrained. Differentiable Evolutionary RL searches compositions of predefined symbolic reward modules and their parameters [349]. Search-Driven Reward Synthesis instead generates free-form reward code and may combine selected rewards into an ensemble [350]. The former searches a structured signal grammar; the latter enlarges the space of expressible reward functions while preserving a fixed reward-call interface.

Cross-domain reward-design systems can be grouped by how completed policy evidence produces the next signal candidate. Incumbent-centered methods feed trajectory ordering, reward-component behavior, or downstream task metrics back into revision of a retained reward program [334,335,351]. Population methods maintain several reward programs and generate descendants from selected or retained elites [337,340,352]. Their diversification mechanisms range from crossover and mutation to island-based migration. Structured variants preserve richer cross-candidate state [341–343]. They use this state to recombine reward modules and tune their coefficients, revisit alternative tree paths, or carry both reward and policy state into the next search round. These mechanisms organize alternatives over a stable signal interface; they do not by themselves turn the candidate into a general executable training procedure.

Executable Training-Procedure Search

*Executable Training-Procedure Search* treats the training computation itself as the mutable candidate. Rather than filling fields in a fixed interface or swapping a bounded signal function, it writes or revises executable operators and their composition: update rules, sampling and rollout control, data flow and stage transitions, training-state management, and entire training scripts or pipelines. The category turns on which logic must change, not on whether code appears: code that only computes a reward remains a signal candidate and a script that only serializes settings remains a configuration candidate, whereas a candidate belongs here once its effect requires new control flow, a changed update implementation, or a recomposition of trainer components. Throughout, the improvement mechanism that generates and validates these patches remains fixed; when experiment evidence begins to revise that mechanism itself, the mutable object belongs to Section 5.3.

Procedure search begins when a candidate must change the executable training computation. AIDE and the Autoresearch task in AIRA-Design both organize draft, debug, and improve operations over runnable training scripts [353,354]. AIDE grows a tree through atomic script revisions, while

# 原文第 44 页 | 中文翻译初稿

44 共 70
学习目标和信号搜索

_学习目标和信号搜索_改变了学习者被训练来优化的内容或如何将监督分配给其行为，而周围的训练支架保持不变。其范围包括目标和损失定义、奖励和塑造函数、目标构建规则、优势转换、监督权重以及跨示例、代币、行动或轨迹分配信用的机制。

分类遵循信号的功能角色，而不是其表面形式：信号可以用自然语言以声明性、象征性的方式编写，或者作为可执行的奖励或损失代码编写，只要固定的学习者通过稳定的接口使用它，它就属于这里。在预先实现的目标中进行选择或调整其系数仍然是_训练-配置搜索_，一旦意识到需要改变学习者自己的计算或控制流程，候选者就不再是信号候选者。此类别还涉及用于训练模型的目标，而不是用于验证候选者的标准：仅比较已完成的候选者的指标属于训练师候选者评估，而优化期间消耗的任何数量都属于候选者，即使它也被报告为实验指标。

仅可执行语法并不能使候选程序成为通用训练程序。 DiscoPOP 使用 LLM 将偏好优化损失编写为 Python 函数 [348]，而 POISE 通过共享接口实例化策略优化候选对象，以实现损失、优势转换和正则化 [333]。在这两种情况下，代码都会改变稳定的训练支架所消耗的数学信号。

候选信号的不同之处在于其语义的限制程度。可微进化强化学习搜索预定义符号奖励模块及其参数的组合 [349]。 SearchDriven Reward Synthesis 而是生成自由形式的奖励代码，并且可以将选定的奖励组合成一个整体 [350]。前者搜索结构化信号语法；后者扩大了可表达奖励函数的空间，同时保留了固定的奖励调用接口。

跨域奖励设计系统可以根据完整的策略证据如何产生下一个候选信号进行分组。以现有者为中心的方法将轨迹排序、奖励组件行为或下游任务指标反馈到保留奖励计划的修订中[334,335,351]。种群方法维持多种奖励计划，并从选定或保留的精英中产生后代[337,340,352]。它们的多样化机制包括从交叉和突变到基于岛屿的迁移。结构化变体保留了更丰富的跨候选状态[341-343]。他们使用此状态来重新组合奖励模块并调整其系数，重新访问替代树路径，或将奖励和策略状态带入下一轮搜索。这些机制通过稳定的信号接口组织替代方案；他们本身并没有将候选人变成通用的可执行训练程序。
可执行训练-程序搜索

_可执行训练过程搜索_将训练计算本身视为可变候选者。它不是在固定接口中填充字段或交换有界信号函数，而是编写或修改可执行运算符及其组合：更新规则、采样和 展开轨迹（Rollout） 控制、数据流和阶段转换、训练状态管理以及整个训练脚本或管道。该类别取决于必须更改的逻辑，而不是代码是否出现：仅计算奖励的代码仍然是信号候选，仅序列化设置的脚本仍然是配置候选，而一旦候选的效果需要新的控制流、更改的更新实现或训练器组件的重组，则候选属于此处。自始至终，生成和验证这些补丁的改进机制保持不变；当实验证据开始修改该机制本身时，可变对象属于第 5.3 节。

当候选人必须更改可执行的训练计算时，过程搜索开始。 AIDE 和 AIRA-Design 中的自动研究任务都通过可运行的训练脚本来组织草稿、调试和改进操作 [353,354]。 AIDE 通过原子脚本修订来长出一棵树，同时

AIRA-Design explores branches that can alter training-loop and optimizer logic. TREX extends this pattern from local script edits to a Researcher–Executor workflow that turns a training plan into an executable fine-tuning scheme [332]; we discuss its trainer-side implementations here, while its data-only branches belong to the data system (Section 4).

Repository-scale systems retain an executable incumbent and construct later procedures relative to it. Auto Research and AutoScientists both branch program edits from a current champion and share the resulting lineage across later experiments [328,331]. EvoTrainer materializes each proposed intervention in an isolated worktree and records it as a versioned trainer branch [325]. The candidate is therefore a versioned implementation whose inherited code state is fixed before evaluation begins.

Population operators offer different ways to generate such implementations. GEAR applies genetic mutation and crossover to a bounded frontier of complete trainer programs [355]. SMCEvolve treats complete pretraining scripts as particles and constructs descendants through sequential local edits or full rewrites [356]. Towards Execution-Grounded Automated AI Research separates ideation from implementation, using an executor to materialize population proposals as runnable pre-training or post-training procedures [357]. The search operator differs across these systems, but each candidate is executable before its training outcome is judged.

A procedure candidate need not span an entire repository: CliffSearch evolves executable optimizer classes whose update equations and internal state transitions replace the incumbent optimizer implementation within an otherwise fixed training stack [329]. The changed object is the optimization operator itself, not a registry choice or a coefficient within a fixed operator.

Cross-domain pipeline systems provide secondary precedents for broader procedure representations. ML-Agent edits versioned ML scripts, while MLE-Ideator separates a strategic proposal from the agent that implements it as an executable solution [344,345]. MLEvolve represents complete ML pipelines as nodes in a progressively expanded graph [338], and the Self-Evolving Recommendation System uses specialized personas to materialize optimizer and reward-definition code changes [339]. These works demonstrate transferable candidate representations and search operators.

Across all three classes, Trainer Candidate Search ends once a runnable candidate has been materialized together with its parent version, inherited state, and declared scope of change. These outputs specify what differs between candidates; Trainer Candidate Evaluation supplies the comparison contract and determines whether that difference is beneficial.

#### 5.2.3. Trainer Candidate Evaluation

*Trainer Candidate Evaluation* decides, from measured training consequences, whether a runnable candidate is rejected, evaluated further, or promoted. The measurement itself is simple in form: train the candidate from a declared initial state under a specified budget, then score the resulting model on a designated evaluation set. What gives the score meaning is the evaluation contract—the initial state, budget, evaluation data, comparison baseline, and promotion rule; change any of these and the score supports a different claim.

We distinguish three evaluation regimes by the role their evidence plays. *Full-Run Evaluation* compares candidates at target fidelity under the declared contract. *Multi-Fidelity Evaluation* uses cheaper approximations to decide which candidates deserve more compute. *Robust Validation* checks that an apparent advantage is really caused by the candidate and survives beyond the conditions where it was first observed. A system may combine these regimes or skip unnecessary ones; what characterizes it is the strongest evidence it requires before promotion.

##### Full-Run Evaluation

*Full-Run Evaluation* judges a candidate by what it produces: the candidate is trained to completion under the target evaluation contract, and the resulting model or policy is assessed by a criterion external to the candidate itself. Internal rewards, training losses, or self-reported rationales may guide learning, but they do not certify the candidate; promotion follows the measured utility of the trained system. “Full-run” is relative to the declared contract rather than to the largest experiment one could

# 原文第 45 页 | 中文翻译初稿

45 共 70

AIRA-Design 探索了可以改变训练循环和优化器逻辑的分支。 TREX 将这种模式从本地脚本编辑扩展到研究者-执行者工作流程，将训练计划转变为可执行的微调方案 [332]；我们在这里讨论它的训练器端实现，而它的纯数据分支属于数据系统（第 4 节）。

存储库规模的系统保留可执行的现有程序并构建与其相关的后续程序。 Auto Research 和 AutoScientists 都对当前冠军进行分支程序编辑，并在以后的实验中共享所得的谱系 [328,331]。 EvoTrainer 在孤立的工作树中具体化每个建议的干预，并将其记录为版本化的训练器分支 [325]。因此，候选者是一个版本化的实现，其继承的代码状态在评估开始之前就已固定。

群体运算符提供了不同的方法来生成此类实现。 GEAR 将基因突变和交叉应用于完整训练程序的边界 [355]。 SMCEvolve 将完整的预训练脚本视为粒子，并通过顺序本地编辑或完全重写构建后代[356]。走向以执行为基础的自动化人工智能研究将构思与实施分开，使用执行器将总体提案具体化为可运行的预训练或训练后程序[357]。这些系统中的搜索运算符有所不同，但每个候选者在判断其训练结果之前都是可执行的。

候选过程不需要跨越整个存储库：CliffSearch 发展了可执行的优化器类，其更新方程和内部状态转换取代了其他固定训练堆栈中的现有优化器实现[329]。更改的对象是优化算子本身，而不是注册表选择或固定算子内的系数。

跨域管道系统为更广泛的过程表示提供了次要先例。 ML-Agent 编辑版本化的 ML 脚本，而 MLE-Ideator 将战略提案与将其实现为可执行解决方案的智能体分开 [344,345]。 MLEvolve 将完整的机器学习管道表示为逐渐扩展的图中的节点[338]，而自我进化推荐系统使用专门的角色来实现优化器和奖励定义代码更改 [339]。这些作品展示了可转移的候选表示和搜索运算符。

在所有三个类别中，一旦可运行候选者及其父版本、继承状态和声明的变更范围被具体化，训练师候选者搜索就会结束。这些输出指定了候选人之间的差异；训练师候选人评估提供比较合同并确定这种差异是否有益。

5.2.3.训练师候选人评估

_训练师候选人评估_根据测量的训练结果决定是否拒绝、进一步评估或晋升可运行的候选人。测量本身的形式很简单：在指定的预算下从声明的初始状态训练候选者，然后在指定的评估集上对结果模型进行评分

。赋予分数意义的是评估契约--初始状态、预算、评估数据、对比基线、晋升规则；更改其中任何一项，分数都会支持不同的主张。

我们根据证据所起的作用来区分三种评估机制。 _全面评估_根据已声明的合同以目标保真度对候选人进行比较。 _多保真度评估_使用更便宜的近似值来决定哪些候选者值得更多计算。 _稳健验证_检查明显的优势确实是由候选者引起的，并且在首次观察到的条件下仍然存在。系统可以组合这些制度或跳过不必要的制度；它的特点是晋升前需要最有力的证据。

全面运行评估

_全面运行评估_根据候选者产生的结果来判断候选者：根据目标评估合同对候选者进行训练直至完成，并根据候选者本身外部的标准对生成的模型或策略进行评估。内部奖励、训练损失或自我报告的理由可能会指导学习，但它们并不能证明候选人；晋升遵循经过训练的系统的衡量效用。 “全面运行”是相对于已宣布

的合同而言的，而不是相对于可以进行的最大实验而言。

conduct: a bounded experiment still counts when its completed outcome directly decides rejection or promotion under the declared rule.

*Fix an evaluator-owned target contract.* A target contract keeps mutable code separate from evaluator-owned data, budgets, legality checks, and metric extraction. Whether candidates are generated through a specialist lineage, a particle population, or a general proposal–executor split, each must materialize as a runnable training recipe and return measured status and utility under the same external contract [328,356,357]. Search topology therefore changes how candidates are proposed, not what Full-Run evidence means.

*Measure the candidate through its downstream learner.* For bounded rewards or optimizers, utility still passes through the learner they produce. A reward candidate can be scored by validation performance of its trained inner policy, with the selected reward later training a fresh policy for held-out evaluation [349]. More generally, reward programs and optimizer implementations are compared through policies or models trained under a common scaffold [329,334]. The reward-program case is a cross-domain precedent; the transferable principle is that internal reward values and proposal rationales do not certify candidate utility.

*Write the completed outcome into version state.* Completed evidence changes trainer evolution only when subsequent experiments inherit a new baseline or an explicit negative record. Champion replacement, best-configuration retention, keep-or-discard decisions, and branch dispositions such as keep, prune, revert, or merge write measured results into version state, with consequential EvoTrainer promotion remaining human-gated [238,325,327,331]. Full-run evidence therefore establishes utility only under the declared contract; cheaper evidence determines whether that contract should be reached, and additional controls determine how strongly its outcome should be trusted.

Multi-Fidelity Evaluation

*Multi-Fidelity Evaluation* compares or filters candidates using evidence collected at systematically related levels of cost and approximation to the target training contract. Fidelity may vary with training horizon, data or model scale, rollout count, environment realism, the amount of newly collected experience, or the directness of the evaluation signal. Lower-fidelity observations provide inexpensive but imperfect estimates; their defining role is to decide whether a candidate should be rejected, continued, or granted a more expensive evaluation.

Replay-based proxies, partial training, checkpoint-based short runs, smaller-scale experiments, learning-curve prediction, successive halving, and adaptive budget allocation all qualify when they forecast target-fidelity utility. Low cost alone is not the criterion: a uniformly short experiment without such gating is simply a full run under a smaller contract, and a static validity check establishes executability rather than likely training benefit.

*Reuse existing evidence.* Candidate screening can begin before a new training run. EvoTrainer backtests proposed reward and filtering changes on historical rollouts, using group-level reward behavior and output audits to expose dead groups, reward leakage, or undesirable behavior before launching a new branch [325]. As a cross-domain reward-design precedent, CARD rejects reward candidates whose scores fail to preserve the ordering of stored high- and low-quality trajectories [335]. FT-Dojo combines schema and data checks with a reduced mini-run before full fine-tuning [327]. The static gates establish that a candidate can run; replay or reduced-run evidence becomes multi-fidelity evaluation only when it forecasts training value and controls whether the candidate receives a larger budget.

*Run controlled low-cost probes.* When retained evidence cannot rank candidates, RHyVE clones the same policy, critic, and optimizer state into short PPO forks, changes the reward hypothesis, and compares the resulting trajectories before committing to a longer phase [346]. The shared checkpoint reduces both repeated training cost and variation in the candidates' starting states. Related cross-domain systems strengthen this comparison in different ways: LLM-ALSO adds a no-change branch from the same checkpoint and abstains when the short-run advantage is unstable, while ROSKA gives each reward–policy candidate approximately 200 PPO updates before extending selected candi-

# 原文第 46 页 | 中文翻译初稿

46 共 70

行为：当其完成的结果直接决定在所声明的规则下的拒绝或升级时，有界实验仍然有效。

_修复评估者拥有的目标合约。_目标合约将可变代码与评估者拥有的数据、预算、合法性检查和指标提取分开。无论候选者是通过专业谱系、粒子群还是一般提案-执行者分裂生成的，每个候选人都必须具体化为可运行的训练配方，并在同一外部合同下返回测量的状态和效用[328,356,357]。因此，搜索拓扑改变了候选者的提议方式，而不是完整运行证据的含义。

_通过其下游学习器来衡量候选者。_对于有界奖励或优化器，效用仍然通过它们产生的学习器传递。奖励候选者可以通过其经过训练的内部策略的验证性能进行评分，然后选择的奖励训练新的策略以进行保留评估[349]。更一般地说，奖励计划和优化器实现是通过在通用支架下训练的策略或模型进行比较的[329,334]。奖励计划案例是一个跨领域的先例；可转让原则是内部奖励价值和提案理由不能证明候选人的效用。

_将完成的结果写入版本状态。_仅当后续实验继承新的基线或明确的负面记录时，完成的证据才会改变训练器的进化。冠军替换、最佳配置保留、保留或丢弃决策以及分支处置（例如保留、修剪、恢复或将测量结果合并写入版本状态），从而导致 EvoTrainer 升级仍然由人为控制 [238,325,327,331]。因此，完整的证据仅在所声明的合同下确定效用；更便宜的证据决定了是否应该达成该合同，而额外的控制则决定了其结果的可信度。
多重保真度评估

_多重保真度评估_使用在系统相关的成本水平和目标训练合同的近似水平上收集的证据来比较或筛选候选人。保真度可能随训练范围、数据或模型规模、展开轨迹（Rollout）计数、环境真实性、新收集的经验量或评估信号的直接性而变化。保真度较低的观测提供了廉价但不完善的估计；他们的决定性作用是决定候选人是否应该被拒绝、继续或给予更昂贵的评估。

基于重放的智能体、部分训练、基于检查点的短期运行、小规模实验、学习曲线预测、连续减半和自适应预算分配在预测目标保真度效用时都符合条件。低成本本身并不是标准：没有这种门控的统一短实验只是在较小合同下的完整运行，静态有效性检查建立了可执行性而不是可能的训练收益。

_重用现有证据。_可以在新的训练运行之前开始候选人筛选

。 EvoTrainer 对历史 展开轨迹（Rollouts）上提出的奖励和过滤更改进行回测，使用组级奖励行为和输出审核来在启动新分支之前暴露死组、奖励泄漏或不良行为 [325]。作为跨领域奖励设计的先例，CARD 拒绝其分数无法保持存储的高质量和低质量轨迹的顺序的奖励候选者 [335]。 FT-Dojo 在完全微调之前将模式和数据检查与减少的迷你运行结合起来[327]。静态门确定候选人可以参选；仅当重播或减少运行的证据预测训练价值并控制候选者是否获得更大的预算时，它才成为多保真度评估。

_运行受控的低成本探针。_当保留的证据无法对候选者进行排名时，RHyVE 将相同的策略、批评者和优化器状态克隆到短 PPO 分叉中，更改奖励假设，并在投入较长阶段之前比较结果轨迹 [346]。共享检查点减少了重复训练成本和候选人起始状态的变化。相关的跨域系统以不同的方式加强了这种比较：
LLM-ALSO 从同一检查点添加了一个无变化分支，并在短期优势不稳定时放弃，而 ROSKA 在扩展选定的候选者之前为每个奖励策略候选者提供了大约 200 个 PPO 更新。

dates [336,343]. Efficient HPO for LLM RL treats model scale and training budget as fidelity variables, using proxy trials to decide which configuration–checkpoint pairs merit target-model training [347]. A probe belongs to this regime because its result gates the next allocation of compute, not merely because the run is short.

*Allocate budget as evidence arrives.* LLMZero evaluates a branch while it trains: at fixed intervals, a frozen early-stopper compares the branch's validation trajectory and diagnostic curves with those of the incumbent, stopping the branch once further compute is unlikely to change the ordering [324]. A stopped branch writes its score back to the search tree, discouraging repeated allocation to the same low-potential direction. Efficient HPO for LLM RL similarly applies fixed gates to reward and KL dynamics, terminates failing trials, and resumes surviving checkpoints at higher fidelity [347].

*Account for fidelity risk.* The goal of these approximations is to preserve the candidate ordering that matters at target fidelity, rather than to predict every final score precisely. That ordering can still invert: a slow-starting candidate may be pruned too early, a configuration that succeeds on a proxy model may fail to transfer, and a short-fork gain may reflect noise or a favorable checkpoint. Multi-fidelity evaluation therefore decides where additional compute should go; robust validation determines whether the resulting advantage is reliable enough for promotion.

Robust Validation

*Robust Validation* asks whether a candidate's apparent advantage is attributable, reproducible, and persistent enough to support promotion. Controlled comparison first limits confounding: candidate and reference start from matched or explicitly comparable states, receive comparable budgets, and are scored under the same data and metric conditions. Repeated seeds, shared-checkpoint forks, no-change controls, uncertainty-aware thresholds, and abstention rules then separate a repeatable intervention effect from experimental noise.

Validation may further require the advantage to survive a second evaluation boundary, such as held-out tasks or seeds, a different horizon or model scale, a shifted data distribution, human assessment, or deployment traffic. "Robust" therefore describes the reliability of the evaluation conclusion, not only adversarial robustness of the trained model. The boundary with the previous regime lies in purpose rather than technique: a scale change or held-out score that economizes screening belongs to *Multi-Fidelity Evaluation*, while the same instrument counts as robust evidence only when it adds a confirmation boundary beyond the evidence used to select the candidate.

*Construct a matched counterfactual.* LLM-ALSO creates every proposed reward-shaping branch together with a mandatory no-change branch from the same policy checkpoint. Its stability-aware comparison penalizes transient peaks, promotes a new shaping configuration and endpoint only when the margin is clear, and otherwise preserves the incumbent [336]. RHyVE likewise clones policy, critic, and optimizer state across reward-hypothesis forks, then examines winner margins, repeated-fork agreement, and winner entropy before committing to a reward or phase schedule; ambiguous evidence produces abstention rather than forced promotion [346]. The short forks allocate compute in Multi-Fidelity Evaluation, while their matched starting state, stability tests, and abstention rule support attribution here. EvoTrainer applies the same logic at version scale through single-factor branches and targeted audits, including rejection of a score gain that disappeared after Git-history leakage was removed [325].

*Calibrate promotion to experimental noise.* AutoScientists estimates a seed-induced noise floor from paired executions of identical code and compares each candidate's validation gain with a calibrated band [331]. A candidate with a clear gain may replace the shared champion directly; one with a marginal gain must improve under a second seed as well, and a candidate that fails this confirmation is retained as a near miss rather than promoted. This rule turns repeated outcomes into a candidate-level decision and prevents one favorable stochastic run from becoming the baseline for all later experiments.

*Cross an independent confirmation boundary.* Efficient HPO for LLM RL uses proxy models and partial budgets to allocate search compute, then trains the selected configuration on the target model to test whether the proxy ordering transfers at the intended scale [347]. As a production-ML precedent,

# 原文第 47 页 | 中文翻译初稿

47 共 70

日期[336,343]。 LLM RL 的高效 HPO 将模型规模和训练预算视为保真度变量，使用智能体试验来决定哪些配置-检查点对值得目标模型训练 [347]。探测器属于这种机制，因为它的结果决定了下一次的计算分配，而不仅仅是因为运行时间很短。

_在证据到达时分配预算。_LLMZero 在训练时评估分支：以固定的时间间隔，冻结的早期停止者将分支的验证轨迹和诊断曲线与现有分支的验证轨迹和诊断曲线进行比较，一旦进一步计算就停止分支不太可能改变顺序 [324]。停止的分支将其分数写回搜索树，阻止重复分配到相同的低潜力方向。 LLM RL 的高效 HPO 类似地应用固定门来奖励和 KL 动态，终止失败的试验，并以更高的保真度恢复幸存的检查点 [347]。

_考虑保真度风险。_这些近似的目标是保留在目标保真度方面重要的候选排序，而不是精确预测每个最终分数。这种顺序仍然可能颠倒：启动缓慢的候选者可能被过早修剪，在智能体模型上成功的配置可能无法传输，短分叉增益可能反映噪音或有利的检查点。因此，多保真度评估决定了额外的计算应该去哪里；稳健的验证决定了所产生的优势是否足够可靠以进行推广。
稳健的验证

_稳健验证_询问候选人的明显优势是否可归因、可重复且持久，足以支持晋升。受控比较首先限制了混淆：候选者和参考者从匹配或明确可比较的州开始，接收可比较的预算，并在相同的数据和指标条件下进行评分。然后，重复种子、共享检查点分叉、不变控制、不确定性感知阈值和弃权规则将可重复的干预效果与实验噪声分开。

验证可能还需要在第二个评估边界中生存的优势，例如保留的任务或种子、不同的范围或模型规模、变化的数据分布、人工评估或部署流量。因此，“鲁棒性”描述了评估结论的可靠性，而不仅仅是训练模型的对抗鲁棒性。与之前制度的边界在于目的而不是技术：节省筛选的量表变化或保留分数属于_多重保真度评估_，而相同的仪器只有当它在用于选择候选人的证据之外添加确认边界时才算作强有力的证据。

构建匹配的 反事实（Counterfactual）. LLM-ALSO 创建每个提议的奖励塑造分支以及来自同一策略检查点的强制无更改分支。它的稳定性感知比较会惩罚瞬态峰值，仅在余量明确时才促进新的整形配置和端点，否则保留现有的[336]

。 RHyVE 同样克隆了奖励假设分叉中的策略、批评者和优化器状态，然后在提交奖励或阶段计划之前检查获胜者利润、重复分叉协议和获胜者熵；模棱两可的证据会导致弃权而不是强制晋升[346]。短分叉在多重保真度评估中分配计算，而它们匹配的起始状态、稳定性测试和弃权规则支持此处的归因。
EvoTrainer 通过单因素分支和有针对性的审计在版本规模上应用相同的逻辑，包括拒绝在 Git 历史泄漏被删除后消失的分数增益 [325]。

校准实验噪声的提升。 AutoScientists 通过相同代码的配对执行来估计种子引起的本底噪声，并将每个候选者的验证增益与校准带进行比较 [331]。获得明显收益的候选人可以直接取代并列冠军；具有边际收益的候选人也必须在第二种子的领导下有所提高，而未能通过此确认的候选人将被视为差点错过而不是晋升。该规则将重复的结果转变为候选级决策，并防止一次有利的随机运行成为所有后续实验的基线。

_跨越独立的确认边界。_LLM RL 的高效 HPO 使用智能体模型和部分预算来分配搜索计算，然后在目标模型上训练选定的配置，以测试智能体排序是否按预期规模传输 [347]。作为生产机器学习的先例，

the Self-Evolving Recommendation System sends offline survivors through versioned training and guarded online A/B evaluation; safety violations can abort the candidate, while delayed north-star metrics and run status are written to the experiment journal [339]. Such a second boundary can confirm the apparent advantage, restrict it to the conditions where it survives, or leave the incumbent unchanged. Robust validation therefore strengthens a promotion claim through attribution, repetition, or transfer, while leaving the validation and promotion rules themselves fixed.

Robust Validation supplies the strongest candidate-level basis for promotion while keeping the improvement mechanism fixed. Evaluating whether a mutable mechanism will produce better future candidates instead crosses into Meta-Loop Improvement-Mechanism Evolution.

### *5.3. Meta-Loop Improvement-Mechanism Evolution*

*Meta-Loop Improvement-Mechanism Evolution* begins when evidence from actual model-training experiments changes the persistent decision mechanism that governs later trainer experiments. A qualifying cycle evaluates a trainer candidate, observes the resulting training outcome, and transforms $Imp_t$ into a successor $Imp_{t+1}$ whose changed decisions affect a later cycle. The writeback may alter diagnosis, proposal and search, evaluation, or promotion. Accumulating logs, context, memories, archives, populations, posteriors, or tree statistics under unchanged decision rules leaves the improvement mechanism fixed.

The successor mechanism may be realized through persistent change at two non-exclusive implementation levels. In *Improvement-Model Evolution*, model-training evidence updates the parameters of a learned model that makes later improvement decisions; this model is distinct from the downstream model trained within each experiment. In *Improvement-Harness Evolution*, the evidence revises the persistent operational logic that structures and executes those decisions, including executable analyzers, reusable skills, runner or controller code, and role-allocation policies.

#### 5.3.1. Improvement-Model Evolution

An Improvement Model evolves when outcomes from trainer experiments become learning signals for the model that conducts later improvement. Differentiable Evolutionary Reinforcement Learning provides a direct bilevel example: its Meta-Optimizer composes symbolic reward functions from a fixed primitive vocabulary, and each candidate reward governs a separate inner GRPO run [349]. Held-out performance of the trained inner policy is the outer reward that updates the Meta-Optimizer, so the successor proposes different reward structures for later policy-training experiments while the primitive vocabulary and training protocol remain fixed.

The same writeback can train a research policy that proposes configurations or executable trainer changes. In *Towards Execution-Grounded Automated AI Research*, an executor turns proposed pre-training or post-training ideas into code patches and obtains rewards from bounded model-training runs. Its RL branch uses these rewards to train the Ideator with GRPO, so the successor Ideator governs later proposals; its archive-conditioned evolutionary branch keeps the proposal procedure fixed and remains Outer-Loop Trainer Search through Experiments [357].

#### 5.3.2. Improvement-Harness Evolution

An Improvement Harness evolves when training evidence revises persistent operational logic around the Improvement Model. The successor may be executable search code, an analysis procedure, a reusable skill, or an organizational policy that changes how later experiments are interpreted, generated, or allocated.

*Executable controller revision.* Bilevel Autoresearch makes the writeback explicit at the runner boundary. Its inner loop edits the active GPT training program and evaluates each edit through a bounded pre-training run. The outer level then reads the accumulated trace and current

`runner.py`, generates a replacement search procedure, validates that procedure, and installs it or restores the incumbent. An accepted runner changes the exploration operators applied to subsequent

# 原文第 48 页 | 中文翻译初稿

48 共 70

自进化推荐系统通过版本化训练和受保护的在线A/B评估来发送离线幸存者；安全违规可能会中止候选，而延迟的北极星指标和运行状态会写入实验日志[339]。这样的第二个边界可以确认明显的优势，将其限制在其生存的条件下，或者保持现有优势不变。因此，稳健的验证通过归因、重复或转移来强化促销主张，同时保持验证和促销规则本身固定。

稳健的验证为晋升提供了最强有力的候选人级别基础，同时保持改进机制的固定。评估可变机制是否会产生更好的未来候选者，而不是跨入元循环改进机制进化。

5.3。元循环改进-机制进化

当来自实际模型训练实验的证据改变了控制后续训练器实验的持久决策机制时，_元循环改进机制演化_就开始了。资格周期评估训练师候选人，观察训练结果，并将 Impt 转换为后继者 Impt +1，其更改的决策会影响后续周期。写回可能会改变诊断、建议和搜索、评估或升级。在未改变的决策规则下累积日志、上下文、记忆、档案、群体、后验或树统计数据使改进机制保持固定。

后继机制可以通过两个非排他性实现级别上的持久变化来实现。在_Improvement-Model Evolution_中，模型训练证据会更新学习模型的参数，从而做出后续的改进决策；该模型与每个实验中训练的下游模型不同。在_Improvement-运行支撑框架（Harness） Evolution_中，证据修改了构建和执行这些决策的持久操作逻辑，包括可执行分析器、可重用技能、运行程序或控制器代码以及角色分配策略。

5.3.1.改进-模型演化

当训练师实验的结果成为进行后续改进的模型的学习信号时，改进模型就会发展。可微进化 强化学习（Reinforcement Learning） 提供了一个直接的双层示例：其元优化器从固定的原始词汇组成符号奖励函数，每个候选奖励控制一个单独的内部 GRPO 运行 [349]。训练后的内部策略的表现是更新元优化器的外部奖励，因此后继者为以后的策略训练实验提出了不同的奖励结构，而原始词汇和训练协议保持不变。

相同的写回可以训练提出配置或可执行训练器更改的研究策略。在_走向以执行为基础的自动化人工智能研究_中，执行者将提出的训练前或训练后想法转化为代码补丁，并从有界模型训练运行中获得奖励。它的 RL 分支使用这些奖励来训练具有 GRPO 的 Ideator，因此后继的 Ideator 可以管理后续的提案

;其存档条件进化分支保持提案程序固定，并通过实验保持外环训练器搜索[357]。

5.3.2.改进-运行支撑框架（Harness）进化

当训练证据修改围绕改进模型的持久操作逻辑时，改进 运行支撑框架（Harness） 就会发展。后继者可以是可执行的搜索代码、分析过程、可重用的技能或改变后来的实验如何解释、生成或分配的组织策略。

_可执行控制器修订版。_双层自动研究使写回在运行器边界处明确。其内循环编辑活动的 GPT 训练程序，并通过有界的预训练运行来评估每次编辑。然后外层读取累积的迹线和当前的迹线

runner.py ，生成替换搜索过程，验证该过程，然后安装它或恢复现有的。接受的跑步者会更改应用于后续的探索运算符

trainer experiments; the outer schedule, import gate, validation metric, and inner keep-or-discard rule remain fixed [238]. GEAR-Evolve performs a narrower controller repair: observed crossover failures produce retained source-level changes to parent eligibility, ancestry traversal, exhausted-pair blocking, and expansion rules. The elite frontier and ancestry records remain ordinary search state; the controller patches are the successor mechanism that governs later language-model training candidates [355].

*Diagnostic and organizational revision. EvoTrainer* provides a representative instantiation of this pattern. Each isolated RL branch binds scores, training signals, behaviors, configurations, code differences, and rollouts to an explicit version. The harness interprets this packet through score, signal, behavior, and version layers, while a version ledger, case memory, skill library, and search trace preserve what was tried and why it succeeded or failed. Candidate interventions are materialized in isolated worktrees and normally change one factor—such as reward, data, hyperparameters, or filtering—so branch evidence can support keep, prune, revert, or merge decisions. When the available evidence cannot explain an outcome, distinguish competing hypotheses, or justify the next intervention, EvoTrainer records a diagnostic gap. The retained response may expand metrics, specialize an analyzer, revise a procedure, retrieve external evidence, or add a validated reusable skill. These changes alter how later branches are diagnosed and constructed; the ledger and memories remain evidence stores rather than the evolving mechanism itself, and costly runs or consequential promotions remain human-gated [325]. Figure 8 maps this distinction onto the system architecture.

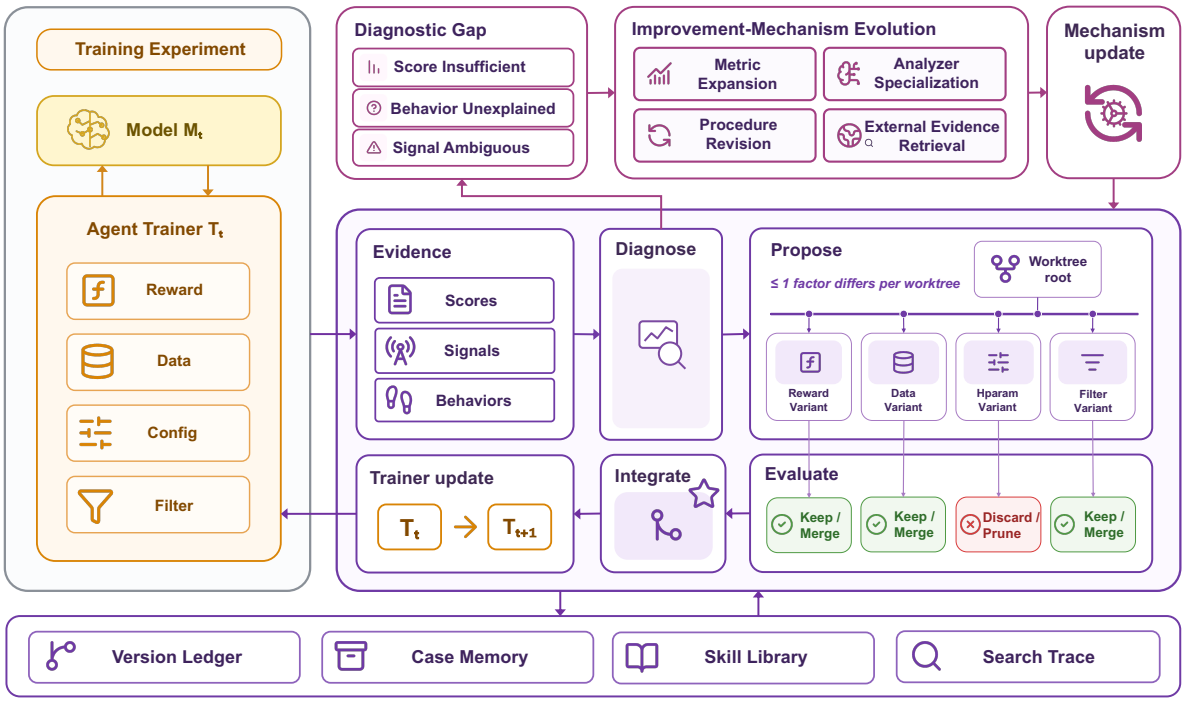



**Figure 8.** EvoTrainer as a representative coupling between trainer-version search and improvement-mechanism evolution. Versioned training branches expose scores, signals, behaviors, configurations, code differences, and rollouts for evaluating single-factor trainer candidates that may be retained, merged, pruned, or reverted. Diagnostic gaps can persistently revise metrics, analyzers, procedures, retrieval, and reusable skills, thereby changing how later branches are interpreted and constructed; consequential runs and promotions remain human-gated [325].

AutoScientists revises the organizational interface through which improvement work is assigned. Model-training evidence can trigger persistent creation, merging, splitting, retirement, or rebalancing of specialist teams, after which the revised roster and allocation policy control who investigates each search axis and launches later experiments. Its queues, champion history, and dead-end records supply evidence but do not independently constitute mechanism evolution [331].

# 原文第 49 页 | 中文翻译初稿

49 共 70

训练师实验；外部时间表、导入门、验证指标和内部保留或丢弃规则保持不变[238]。 GEAR-Evolve 执行范围更窄的控制器修复：观察到的交叉故障会对父资格、祖先遍历、耗尽对阻塞和扩展规则产生保留的源级别更改。精英边疆和血统记录保持普通搜索状态；控制器补丁是管理后来的语言模型训练候选者的后继机制[355]。

诊断和组织修订。 EvoTrainer 提供了此模式的代表性实例。每个独立的 RL 分支将分数、训练信号、行为、配置、代码差异和 展开轨迹（Rollouts） 绑定到显式版本。 运行支撑框架（Harness） 通过分数、信号、行为和版本层解释此数据包，而版本分类帐、案例内存、技能库和搜索跟踪则保留尝试过的内容以及成功或失败的原因。候选干预措施在孤立的工作树中具体化，通常会改变一个因素（例如奖励、数据、超参数或过滤），因此分支证据可以支持保留、修剪、恢复或合并决策。当现有证据无法解释结果、区分相互竞争的假设或证明下一次干预的合理性时，EvoTrainer 会记录诊断差距。保留的响应可以扩展指标、专门化分析器、修改程序、检索外部证据或添加经过验证的可重用技能。这些变化改变了后来的分支的诊断和构建方式；账本和记忆仍然是证据存储，而不是进化机制本身，并且昂贵的运行或相应的晋升仍然是人为控制的[325]。图 8 将这种区别映射到系统架构上。

诊断差距改进机制进化机制
训练实验分数不足指标分析器更新
扩展专业化
行为无法解释
模型M□程序外部证据
信号模糊修订检索
智能体训练器T□
证据诊断建议工作树
奖励≤1个因子不同每个工作树根
分数
数据信号
行为奖励变体VariantData HparamVariant VariantFilter
配置
训练器更新集成评估
过滤器
T □ T □₊₁ 保留/合并保留/合并丢弃/修剪保留/合并
版本分类帐案例记忆技能库搜索Trace

图 8. EvoTrainer 作为训练器版本搜索和改进机制演化之间的代表性耦合。版本化训练分支公开分数、信号、行为、配置、代码差异和 展开轨迹（Rollouts），用于评估可能保留、合并、修剪或恢复的单因素训练器候选者。诊断差距可以持续修改指标、分析器、程序、检索和可重用技能，从而改变后续分支的解释和构建方式；随之而来的竞选和晋升仍然是人为的[325]。

AutoScientists 修改了分配改进工作的组织界面

。模型训练证据可以触发专家团队的持续创建、合并、分裂、退休或重新平衡，之后修改后的名册和分配策略控制谁调查每个搜索轴并启动后续实验。它的队列、冠军历史和死胡同记录提供了证据，但并不独立构成机制进化[331]。

Taken together, the three loops trace a single progression: persistent change moves from the active trainer, to retained trainer versions, to the improvement mechanism itself. Their maturity declines along the same path. Inner-Loop Trainer Adaptation rests on the most direct evidence, since each revision acts and is observed within the same training lineage. Outer-Loop Trainer Search has runnable systems in all three candidate classes, but its bottleneck is the evaluation contract: target-fidelity experiments are expensive, cheap proxies can invert the candidate ordering, and few systems implement the matched controls, noise calibration, and confirmation boundaries that Robust Validation requires, so promotion evidence lags behind candidate generation. Meta-Loop Improvement-Mechanism Evolution remains the least developed: current systems evolve bounded parts of *Imp* under fixed higher-order boundaries—a protected evaluation signal, a permitted modification interface, an execution budget, or human release authority—that keep successor mechanisms comparable and auditable. Trainer self-improvement today is therefore predominantly self-improvement under a fixed mechanism, with recursion realized only in bounded and audited forms.

The trainer loop also does not close the full self-improvement cycle on its own: its supervision consumes the tasks, trajectories, and verification signals supplied by the data system, its revisions take effect only through the model they update, and a promoted trainer version changes which future experience is worth collecting. Section 6 therefore moves from this component-level view to a unified treatment in which the model, harness, data system, trainer, and improvement mechanism improve as one coupled system.

## 6. Co-Improvement of Multiple System Components

Sections 3, 4, and 5 review how the agent harness, data system, and trainer improve individually. The component-level view is useful, but it is not sufficient for sustained improvement in agent systems. In a coupled system, any component can become the main constraint on overall performance. If one component improves while the others remain fixed, further improvement of that component may produce diminishing returns. System-level bottlenecks are dynamic rather than static, and they can shift as the capabilities and components of the agent system change. Therefore, single-component self-improvement cannot fully identify or address the moving constraints that arise in an evolving agent system.

Following the unified research framework in Section 2.3, we further study collaborative improvement of agent systems. From a system-level perspective, co-improvement refers to the joint improvement of multiple system components through their dependency relationships. Based on the components involved in the collaborative improvement, this section organizes related work into three categories.

### *6.1. Harness-Trainer Co-Improvement*

Harness-trainer co-improvement couples the improvement of harness modules with the training of the agent model. In this setting, improved skills, tools, workflows, or scaffolds make training signals more informative, while training outcomes reveal which harness modules should be revised next.

Training Conditioned on Agent Skills and Experience.

Accumulated skills or experience records can serve as conditioning signals, curriculum guides, or distillation targets during agent model training. SkillRL and ARISE maintain skill libraries that condition policy learning. After training, the updated policy generates new trajectories that are used to expand or refine the skill library [367,368].

Harness Refinement Driven by Training Feedback.

Trainer-side evidence, including policy performance, failure patterns, and reward signals, can guide revisions to harness code, workflow logic, or agent architecture. SIA places scaffold updates and weight updates into a single feedback-driven improvement loop [369]. HarnessForge models the agent

# 原文第 50 页 | 中文翻译初稿

50 共 70

总而言之，这三个循环跟踪一个单一的进程：持续的变化从活跃的训练师到保留的训练师版本，再到改进机制本身。他们的成熟度也沿着同样的路径下降。内环训练器适应依赖于最直接的证据，因为每次修订都会在相同的训练谱系中起作用并被观察到。外环训练器搜索在所有三个候选类别中都有可运行的系统，但其瓶颈是评估契约：目标保真度实验昂贵，廉价的智能体可以反转候选排序，并且很少有系统实现鲁棒验证所需的匹配控制、噪声校准和确认边界，因此升级证据落后于候选生成。元循环改进机制进化仍然是最不发达的：当前系统在固定的高阶边界下进化_Imp_的有界部分--受保护的评估信号、允许的修改接口、执行预算或人类发布权限--保持后继机制的可比较性和可审计性。因此，今天的训练器 自我改进（Self-Improvement） 主要是固定机制下的 自我改进（Self-Improvement），仅以有界和审核的形式实现递归。

训练器循环本身也不会关闭完整的 自我改进（Self-Improvement） 循环：它的监督消耗数据系统提供的任务、轨迹和验证信号，其修订仅通过它们更新的模型生效，并且升级的训练器版本更改值得未来收集经验。因此，第 6 节从组件级视图转向统一处理，其中模型、运行支撑框架（Harness）、数据系统、训练器和改进机制作为一个耦合系统进行改进。
6. 多个系统组件的共同改进

第 3、4 和 5 节回顾了智能体 运行支撑框架（Harness）、数据系统和训练师如何单独改进。组件级视图很有用，但不足以持续改进智能体系统。在耦合系统中，任何组件都可能成为整体性能的主要约束。如果一个组件得到改进而其他组件保持不变，则该组件的进一步改进可能会产生收益递减。系统级瓶颈是动态的而不是静态的，并且它们可以随着智能体系统的功能和组件的变化而变化。因此，单组件 自我改进（Self-Improvement） 无法完全识别或解决不断发展的智能体系统中出现的移动约束。

遵循2.3节的统一研究框架，我们进一步研究智能体系统的协同改进。从系统层面来看，协同改进是指多个系统组件通过依赖关系共同改进。根据协同改进所涉及的组成部分，本节将相关工作分为三类。
6.1。 运行支撑框架（Harness）-训练师协同提升

运行支撑框架（Harness）-trainer协同改进将运行支撑框架（Harness）模块的改进与智能体模型的训练结合起来。在这种情况下，改进的技能、工具、工作流程或支架可以使训练信号提供更多信息，而训练结果则揭示下一步应该修改哪些 运行支撑框架（Harness） 模块。

以智能体技能和经验为条件的训练。

积累的技能或经验记录可以作为智能体模型训练过程中的条件信号、课程指南或蒸馏目标。 SkillRL 和 ARISE 维护着影响策略学习的技能库。训练后，更新后的策略会生成新的轨迹，用于扩展或完善技能库[367,368]。

运行支撑框架（Harness） 由训练反馈驱动的细化。

训练者方面的证据，包括策略性能、失败模式和奖励信号，可以指导对 运行支撑框架（Harness） 代码、工作流逻辑或智能体架构的修改。 SIA 将脚手架更新和权重更新放入单个反馈驱动的改进循环中[369]。
HarnessForge 为智能体建模

system as an interdependent harness-policy system, where fault-guided harness tailoring is combined with harness-conditioned policy adaptation [360].

### *6.2. Harness-Data Co-Improvement*

Harness-data co-improvement couples the improvement of harness modules with the data pipeline. Harness modules change the data that the agent system can produce or access, while data-side signals drive the creation and refinement of those harness modules.

Co-Improving Skills and Trajectory Generation.

Agent skills extracted by the harness can become the source of new training tasks or environments. Task outcomes then provide evidence for further skill refinement. CODESKILL converts historical trajectories into procedural skills and feeds the resulting library into code-repair task construction and solving loops [370]. VOYAGER demonstrates this pattern in an embodied setting, where an executable code-skill library enables exploration of progressively harder environments in Minecraft without changing model weights [371].

Co-Improving Harness Code and Trajectory Generation.

Harness code improvement and trajectory generation influence each other through mutual dependencies. Better tools and harness code expand the agent's action space and improve execution trajectories. Execution failures and debugging records from trajectory synthesis reveal capability gaps and guide the creation and refinement of new tools and harness code. Live-SWE-agent creates, debugs, and reuses executable custom tools during software-engineering tasks. The resulting tools change later interaction traces, and failures in those traces guide further tool debugging or tool creation [372]. AutoHarness synthesizes executable code harnesses, including action filters, verifiers, and code policies, from environment feedback. The accepted harness modifications alter the agent's future interaction capabilities, thereby affecting the distribution of trajectories collected in subsequent episodes [373].

### *6.3. Data-Trainer Co-Improvement*

Data-trainer co-improvement creates a feedback loop where better training produces better data, which in turn improves training. The environment simulator and policy can improve each other through iterative interaction. Some existing studies train world models to serve as environment simulators. Real trajectories refine the world model, and the improved world model helps synthesize experience in regions exposed by the current policy. WebEvolver trains a world model that predicts web observations and produces imagined trajectories for policy learning [374]. VLAW establishes an analogous loop for robotic manipulation, where real rollouts improve an action-conditioned video world model that then generates synthetic rollouts for further policy training [375].

## 7. Open Problems and Future Research Directions

Despite rapid progress in self-improving agent systems, many fundamental problems on the path toward recursive self-improvement remain unsolved. These open problems concern the conditions that enable agent systems to become reliable, scalable, general, safe, and beneficial to humans as they progress toward recursive self-improvement.

### *7.1. Long-Horizon Evaluation of Improvements in Real-World Production*

Current agent evaluations primarily measure task performance within a single evaluation run. In contrast, evaluating self-improving agent systems requires assessing whether repeated self-modifications consistently improve the reliability, robustness, and overall usefulness of subsequent system versions. To measure the effects of multi-round self-modification, future benchmarks should simulate long-horizon real-world workloads, such as maintaining large code repositories, conducting iterative scientific research, and operating long-running services. Beyond measuring capability gains,

# 原文第 51 页 | 中文翻译初稿

51 共 70

系统作为一个相互依赖的 运行支撑框架（Harness） 策略系统，其中故障引导的 运行支撑框架（Harness） 剪裁与 运行支撑框架（Harness） 条件策略适应相结合[360]。

6.2。 运行支撑框架（Harness）-数据协同改进

运行支撑框架（Harness）-数据协同改进将运行支撑框架（Harness）模块的改进与数据管道结合起来。运行支撑框架（Harness） 模块更改智能体系统可以生成或访问的数据，而数据侧信号驱动这些 运行支撑框架（Harness） 模块的创建和细化。

共同提高技能和轨迹生成。

运行支撑框架（Harness）提取的智能体技能可以成为新训练任务或环境的来源。任务结果为进一步改进技能提供证据。 CODESKILL 将历史轨迹转换为程序技能，并将生成的库提供给代码修复任务构建和求解循环 [370]。 VOYAGER 在具体环境中演示了这种模式，其中可执行代码技能库可以在 Minecraft 中探索逐渐困难的环境，而无需更改模型权重 [371]。

共同改进 运行支撑框架（Harness） 代码和轨迹生成。

运行支撑框架（Harness） 代码改进和轨迹生成通过相互依赖相互影响。更好的工具和 运行支撑框架（Harness） 代码扩展了智能体的操作空间并改善了执行轨迹。轨迹综合的执行失败和调试记录揭示了能力差距，并指导新工具和 运行支撑框架（Harness） 代码的创建和完善。 Live-SWE-agent 在软件工程任务期间创建、调试和重用可执行的自定义工具。由此产生的工具会更改后来的交互跟踪，并且这些跟踪中的失败会指导进一步的工具调试或工具创建[372]。 AutoHarness 根据环境反馈合成可执行代码 运行支撑框架（Harnesses），包括操作过滤器、验证器和代码策略。已接受的 运行支撑框架（Harness） 修改改变了智能体未来的交互能力，从而影响后续事件中收集的轨迹的分布[373]。

6.3。数据训练师共同改进

数据与训练师的共同改进创建了一个反馈循环，更好的训练会产生更好的数据，从而改进训练。环境模拟器和策略可以通过迭代交互相互改进。一些现有的研究训练 世界模型（World Models） 作为环境模拟器。真实轨迹对世界模型（World Model）进行了细化，改进后的世界模型（World Model）有助于综合当前策略所暴露区域的经验。 WebEvolver 训练 世界模型（World Model） 来预测网络观察并生成用于策略学习的想象轨迹 [374]。 VLAW 为机器人操作建立了一个类似的循环，其中真实的 展开轨迹（Rollouts） 改进了动作条件视频 世界模型（World Model），然后生成合成的 展开轨迹（Rollouts） 用于进一步的策略训练[375]。

7. 未解决的问题和未来的研究方向

尽管自我改进（Self-Improving）智能体系统取得了快速进展，但通往递归自我改进（Recursive Self-Improvement）的道路上的许多基本问题仍未解决。这些悬而未决的问题涉及到使智能体系统在朝着递归自我改进（Recursive Self-Improvement） 方向发展时变得可靠、可扩展、通用、安全且对人类有益的条件。

7.1。 长时程（Long-Horizon） 实际生产改进评估

当前的智能体评估主要衡量单次评估运行中的任务绩效。相比之下，评估 自我改进（Self-Improving） 智能体系统需要评估重复的自我修改是否持续提高后续系统版本的可靠性、稳健性和整体实用性。为了衡量多轮自我修改的效果，未来的基准测试应该模拟 长时程（Long-Horizon） 现实世界的工作负载，例如维

护大型代码存储库、进行迭代科学研究以及运行长时间运行的服务。除了衡量能力增益之外，

these benchmarks should assess whether agent systems preserve previously acquired capabilities, learn from unsuccessful modifications, and remain effective as goals, budgets, tools, and environments evolve. They should further evaluate the improvement process itself by examining whether systems become better at diagnosing their own bottlenecks, selecting beneficial modifications, and carrying improvements forward to future tasks.

### *7.2. Observable, Scalable, and Modifiable Training and Inference Infrastructure*

RSI requires infrastructure that exposes the agent system's behavior and update process through representations that agents can inspect and improve. Existing infrastructure is typically designed and implemented as isolated components. Although many systems already record useful information, such as execution traces, training records, and branch histories, these artifacts are distributed across different storage systems, accessed through diverse interfaces, and governed by inconsistent access control mechanisms. Consequently, they do not provide a unified, agent-friendly interface through which inference-time behavior, data generation, model training, validation, and self-modification can be fully observed, reproduced, compared, and revised.

Future infrastructure should tightly integrate agents with the training and inference infrastructure by making the entire improvement process observable through standardized system representations, enabling agents to dynamically allocate and reconfigure computational resources, and allowing them to safely analyze, evaluate, and revise different parts of the training and inference infrastructure. This creates a positive feedback loop in which agents improve the infrastructure, and the improved infrastructure, in turn, enables agents to improve themselves more efficiently and reliably in subsequent iterations.

### *7.3. From Bounded RSI to General RSI*

The capability grading standard in Section 2.2 distinguishes bounded RSI at L4 from general RSI at L5. Recent systems have demonstrated self-improvement within specific domains, such as automatically repairing code, optimizing training infrastructure, or refining task-specific improvement procedures. However, these results do not yet constitute general RSI, as the effectiveness of their improvement mechanisms remains closely tied to particular tasks, environments, tools, and evaluation criteria. The key challenge in advancing from L4 to L5 is to achieve transferable self-improvement, where the system can preserve and adapt its improvement capabilities across diverse domains. Future research should identify the fundamental principles underlying reusable improvement mechanisms while preventing the inappropriate transfer of domain-specific strategies that may perform well in one setting but fail when applied to different scenarios.

### *7.4. Safety and Controllability Under Recursive Self-Modification*

Safety and controllability become substantially more challenging when an RSI system can modify not only task-level policies, but also the infrastructure and improvement mechanisms that govern future self-modification. As an RSI system becomes more capable, it may propose modifications to evaluators, permission rules, and version management policies. The challenge is to ensure that governance mechanisms can evolve while preventing successive system versions from weakening the safeguards that keep continued self-improvement trustworthy.

Future research should distinguish capability-oriented components from protected governance components. Capability-oriented components that primarily affect performance, efficiency, or adaptability should remain modifiable. By contrast, modifications to components that enforce safety, legal, or privacy requirements must undergo a stricter verification before acceptance. In addition, RSI systems should continuously monitor performance degradation, evaluator failures, reward hacking, privacy leakage, unsafe tool generation, and system failures. They should also preserve immutable records of rollback versions, safety alerts, and incident logs to maintain traceability across successive system versions and support human review.

# 原文第 52 页 | 中文翻译初稿

52 共 70

这些基准应该评估智能体系统是否保留以前获得的功能，从不成功的修改中学习，并随着目标、预算、工具和环境的发展而保持有效。他们应该通过检查系统是否能够更好地诊断自身瓶颈、选择有益的修改以及将改进推进到未来的任务来进一步评估改进过程本身。

7.2。可观察、可扩展、可修改的训练和推理基础设施

RSI 需要基础设施，通过智能体可以检查和改进的表示来公开智能体系统的行为和更新过程。现有基础设施通常被设计和实现为独立的组件。尽管许多系统已经记录了有用的信息，例如执行跟踪、训练记录和分支历史记录，但这些工件分布在不同的存储系统中，通过不同的接口访问，并受到不一致的访问控制机制的管理。因此，它们没有提供一个统一的、智能体友好的界面，通过该界面可以充分观察、再现、比较和修改推理时间行为、数据生成、模型训练、验证和自我修改。

未来的基础设施应该将智能体与训练和推理基础设施紧密集成，通过标准化系统表示使整个改进过程可观察，使智能体能够动态分配和重新配置计算资源，并允许它们安全地分析、评估和修改训练和推理基础设施的不同部分。这创建了一个积极的反馈循环，在这个循环中，智能体改进了基础设施，而改进后的基础设施反过来又使智能体能够在后续迭代中更有效、更可靠地改进自己。

7.3。从有界 RSI 到一般 RSI

2.2 节中的能力分级标准将 L4 的有界 RSI 与 L5 的一般 RSI 区分开来。最近的系统已经在特定领域展示了 自我改进（Self-Improvement），例如自动修复代码、优化训练基础设施或细化特定于任务的改进程序。然而，这些结果尚不构成一般的 RSI，因为其改进机制的有效性仍然与特定任务、环境、工具和评估标准密切相关。从 L4 升级到 L5 的关键挑战是实现可转移的 自我改进（Self-Improvement），其中系统可以在不同领域保留和调整其改进能力。未来的研究应该确定可重用改进机制的基本原则，同时防止特定领域策略的不当转移，这些策略可能在一种环境中表现良好，但在应用于不同场景时会失败。

7.4。递归自修改下的安全可控

当 RSI 系统不仅可以修改任务级策略，还可以修改管理未来自我修改的基础设施和改进机制时，安全性和可控性将变得更具挑战性

。随着 RSI 系统变得更加强大，它可能会建议对评估器、权限规则和版本管理策略进行修改。面临的挑战是确保治理机制能够不断发展，同时防止连续的系统版本削弱使 自我改进（Self-Improvement） 持续值得信赖的保障措施。

未来的研究应该区分面向能力的组件和受保护的治理组件。主要影响性能、效率或适应性的面向能力的组件应该保持可修改性。相比之下，对强制执行安全、法律或隐私要求的组件的修改必须在接受之前经过更严格的验证。此外，RSI 系统应持续监控性能下降、评估器故障、奖励黑客、隐私泄露、不安全工具生成和系统故障。他们还应该保留回滚版本、安全警报和事件日志的不可变记录，以保持连续系统版本之间的可追溯性并支持人工审查。

### 7.5. Human-Expert and Agent Co-Improvement

Self-improvement should not be studied only as the replacement of human iteration by autonomous iteration. In many high-impact domains, a more practical and valuable objective is to establish a process of co-improvement between human and agent systems. Human experts contribute domain knowledge, value judgments, contextual understanding, and accountability, whereas agent systems offer scalable exploration, persistent memory, automated experimentation, and efficient modification. The central research challenge is therefore to develop mechanisms that enable agents to improve through expert interaction while enhancing human capabilities through better evidence, more effective tools, and stronger decision support.

Future self-improving agent systems should learn how to identify situations that require expert involvement, determine the appropriate form of interaction, provide interpretable evidence for human review, and convert expert feedback into reusable improvement resources. Accordingly, the evaluation of agent systems should move beyond task-level performance metrics and examine whether human-agent collaboration leads to higher-quality decisions, greater transparency, and stronger generalization to previously unseen scenarios.

## 8. Conclusion

This survey studies self-improving agent systems, which autonomously transform experience and evaluation feedback into persistent updates to their own components. We begin by formalizing the problem and introducing a capability grading standard ranging from manual improvement to general RSI. Furthermore, we propose a unified research framework that analyzes these agent systems as a whole, characterizing both the scope of autonomous improvement and the improvement process. We organize existing work into a taxonomy covering self-improvement within the agent harness, data system, and trainer, as well as co-improvement across components. Additionally, we identify open problems that motivate progress toward more autonomous, reliable, and general self-improving agent systems. We hope that this survey lays the foundation for studying this emerging area and provides a roadmap for future research.

## References


1. METR. Measuring AI Ability to Complete Long Tasks. https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/, 2025.
2. Kwa, T.; West, B.; Becker, J.; Deng, A.; Garcia, K.; Hasin, M.; Jawhar, S.; Kinniment, M.; Rush, N.; Arx, S.V.; et al. Measuring AI Ability to Complete Long Software Tasks, 2026, [arXiv:cs.AI/2503.14499].
3. Qwen Team. Qwen3.7: The Agent Frontier, 2026.
4. Xu, A.; Lin, B.; Xue, B.; Wang, B.; Xu, B.; Wu, B.; Zhang, B.; Lin, C.; Dong, C.; Ling, C.; et al. Deepseek-v4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348* **2026**.
5. Blakeman, A.; Thomas, A.; Jhunjhunwala, A.; Gupta, A.; Khattar, A.; Rajfer, A.; Renduchintala, A.; Asif, A.; Vavre, A.; Miranda, A.F.; et al. Nemotron 3 Ultra: Open, Efficient Mixture-of-Experts Hybrid Mamba-Transformer Model for Agentic Reasoning. *arXiv preprint arXiv:2606.15007* **2026**.
6. Jimenez, C.E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; Narasimhan, K. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?, 2024, [arXiv:cs.CL/2310.06770].
7. Deng, X.; Da, J.; Pan, E.; He, Y.Y.; Ide, C.; Garg, K.; Lauffer, N.; Park, A.; Pasari, N.; Rane, C.; et al. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?, 2025, [arXiv:cs.SE/2509.16941].
8. Merrill, M.A.; Shaw, A.G.; Carlini, N.; Li, B.; Raj, H.; Bercovich, I.; Shi, L.; Shin, J.Y.; Walshe, T.; Buchanan, E.K.; et al. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces, 2026, [arXiv:cs.SE/2601.11868].
9. Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models, 2023, [arXiv:cs.CL/2210.03629].
10. Yang, J.; Jimenez, C.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; Press, O. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* **2024**, *37*, 50528–50652.

# 原文第 53 页 | 中文翻译初稿

53 共 70

7.5。人类专家和智能体共同改进

自我改进（Self-Improvement） 不应仅仅作为自主迭代替代人工迭代来研究。在许多高影响力的领域，更实际和更有价值的目标是建立人类和智能体系统之间的共同改进过程。人类专家贡献领域知识、价值判断、情境理解和责任，而智能体系统提供可扩展的探索、持久记忆、自动化实验和高效修改。因此，研究的核心挑战是开发机制，使智能体人能够通过专家互动进行改进，同时通过更好的证据、更有效的工具和更有力的决策支持来增强人类的能力。

未来的 自我改进（Self-Improving） 智能体系统应该学习如何识别需要专家参与的情况，确定适当的交互形式，为人工审查提供可解释的证据，并将专家反馈转化为可重用的改进资源。因此，对智能体系统的评估应该超越任务级绩效指标，并检查人工智能体协作是否会带来更高质量的决策、更大的透明度以及对以前未见过的场景的更强的泛化能力。

八、结论

这项调查研究了 自我改进（Self-Improving） 智能体系统，该系统自动将经验和评估反馈转化为对其自身组件的持续更新。我们首先将问题形式化，并引入从手动改进到一般 RSI 的能力分级标准。此外，我们提出了一个统一的研究框架，将这些智能体系统作为一个整体进行分析，描述自主改进的范围和改进过程。我们将现有工作组织成一个分类法，涵盖智能体 运行支撑框架（Harness）、数据系统和训练器中的 自我改进（Self-Improvement），以及跨组件的共同改进。此外，我们还发现了一些悬而未决的问题，这些问题推动了更加自主、可靠和通用的 自我改进（Self-Improving） 智能体系统的发展。我们希望这项调查为研究这一新兴领域奠定基础，并为未来的研究提供路线图。

参考文献

1. 公制。衡量人工智能完成长时间任务的能力。 https://metr.org/blog/2025-03-19-measuring-aiability-to-complete-long-tasks/，2025 年。

2. Kwa, T.；韦斯特，B.；贝克尔，J.；邓，A.；加西亚，K.；哈辛，M.；乔哈尔，S.；金尼门特，M.；拉什，N.；阿尔克斯，S.V.；等人。衡量 AI 完成长时间软件任务的能力，2026 年，[arXiv:cs.AI/2503.14499]。

3.Qwen团队。 Qwen3.7：特工前沿，2026 年。

4. 徐A.；林，B.；薛，B.；王，B.；徐，B。吴，B.；张，B。林，C.；董，C.；凌，C.；等人。Deepseek-v4：迈向高效的百万令牌上下文智能。 arXiv 预印本 arXiv:2606.19348 2026 。

5.布莱克曼，A.；托马斯，A.； Jhunjhunwala，A.；古普塔，A.；哈塔尔，A.；拉杰弗，A.；伦杜金塔拉，A.；阿西夫，A.；瓦夫尔，A.；米兰达，AF；等人。 Nemotron 3 Ultra：开放、高效的专家混合 MambaTransformer 模型，用于智能体推理。 arXiv 预印本 arXiv:2606.15007 2026 。

6.希门尼斯，C.E.；杨，J。韦蒂格，A.；姚，S。佩，K.；新闻，O.；纳拉辛汗，K

。 SWE-bench：语言模型能否解决现实世界的 GitHub 问题？，2024 年，[arXiv:cs.CL/2310.06770]。

7. 邓X.；达，J。潘，E.；他，Y.Y.；伊德，C.；加尔格，K.；劳弗，N.；帕克，A.；帕萨里，N.；雷恩，C.；等人。 SWE-Bench Pro：AI 智能体能否解决 长时程（Long-Horizon） 软件工程任务？，2025 年，[arXiv:cs.SE/2509.16941]。

8. 梅里尔，文学硕士；肖，A.G.；卡利尼，N.；李，B。拉杰，H.；贝尔科维奇，I.；施L.；辛，J.Y.；沃尔什，T.；布坎南，E.K.；等人。 Terminal-Bench：在命令行界面中针对艰巨、现实的任务对智能体进行基准测试，2026 年，[arXiv:cs.SE/2601.11868]。

9.姚S.；赵，J。于，D。杜，N。沙夫兰，I。纳拉西姆汗，K.； Cao, Y. ReAct：在语言模型中协同推理和行动，2023，[arXiv:cs.CL/2210.03629]。

10.杨，J.；希门尼斯，C.；韦蒂格，A.；利埃特，K.；姚，S。纳拉西姆汗，K.； Press, O. Swe-agent：智能体计算机接口支持自动化软件工程。 _神经信息处理系统的进展_2024，37，50528-50652。

11. Anthropic. Claude Code. https://docs.anthropic.com/en/docs/claude-code, 2025. AI coding agent harness.
12. OpenAI. OpenAI Codex CLI. https://github.com/openai/codex, 2025. Lightweight coding agent running in the terminal.
13. Blakeman, A.; Grattafiori, A.; Basant, A.; Gupta, A.; Khattar, A.; Renduchintala, A.; Vavre, A.; Shukla, A.; Bercovich, A.; Ficek, A.; et al. NVIDIA Nemotron 3: Efficient and Open Intelligence. *arXiv preprint arXiv:2512.20856* **2025**.
14. Zhao, J.; Chen, G.; Meng, F.; Li, M.; Chen, J.; Xu, H.; Sun, Y.; Zhao, W.X.; Song, R.; Zhang, Y.; et al. Immersion in the github universe: Scaling coding agents to mastery. *arXiv preprint arXiv:2602.09892* **2026**.
15. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* **2017**.
16. Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Kunc, Y.; et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300* **2024**.
17. Sheng, G.; Zhang, C.; Ye, Z.; Wu, X.; Zhang, W.; Zhang, R.; Peng, Y.; Lin, H.; Wu, C. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv preprint arXiv: 2409.19256* **2024**.
18. Zhu, Z.; Xie, C.; Lv, X.; slime Contributors. slime: An LLM post-training framework for RL Scaling. https://github.com/THUDM/slime, 2025. GitHub repository. Corresponding author: Xin Lv.
19. Zhang, L.; Chen, M.; Cao, R.; Chen, J.; Zhou, F.; Xu, Y.; Yang, J.; Ma, Z.; Chen, L.; Luo, C.; et al. MegaFlow: Large-Scale Distributed Orchestration System for the Agentic Era, 2026, [arXiv:cs.DC/2601.07526].
20. Wu, R.; Wang, X.; Mei, J.; Cai, P.; Fu, D.; Yang, C.; Wen, L.; Yang, X.; Shen, Y.; Wang, Y.; et al. EvolveR: Self-Evolving LLM Agents through an Experience-Driven Lifecycle, 2025, [arXiv:cs.CL/2510.16079].
21. Novikov, A.; Vu, N.; Eisenberger, M.; Dupont, E.; Huang, P.S.; Wagner, A.Z.; Shirobokov, S.; Kozlovskii, B.; Ruiz, F.J.R.; Mehrabian, A.; et al. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery, 2025, [arXiv:cs.AI/2506.13131].
22. Lin, J.; Liu, S.; Pan, C.; Lin, L.; Dou, S.; Huang, X.; Yan, H.; Han, Z.; Gui, T. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses. *CoRR* **2026**, *abs/2604.25850*, [2604.25850]. https://doi.org/10.48550/ARXIV.2604.25850.
23. Zhang, J.; Zhao, B.; Yang, W.; Foerster, J.N.; Clune, J.; Jiang, M.; Devlin, S.; Shavrina, T. Hyperagents. *CoRR* **2026**, *abs/2603.19461*, [2603.19461]. https://doi.org/10.48550/ARXIV.2603.19461.
24. Good, I.J. Speculations Concerning the First Ultraintelligent Machine; Elsevier, 1966; Vol. 6, *Advances in Computers*, pp. 31–88. https://doi.org/https://doi.org/10.1016/S0065-2458(08)60418-0.
25. Schmidhuber, J. Gödel Machines: Fully Self-referential Optimal Universal Self-improvers. In *Artificial General Intelligence*; Goertzel, B.; Pennachin, C., Eds.; Cognitive Technologies, Springer, 2007; pp. 199–226. https://doi.org/10.1007/978-3-540-68677-4_7.
26. Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; Yao, S. Reflexion: language agents with verbal reinforcement learning. In Proceedings of the Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023; Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; Levine, S., Eds., 2023.
27. Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.; Huang, G. ExpeL: LLM Agents Are Experiential Learners. *CoRR* **2023**, *abs/2308.10144*, [2308.10144]. https://doi.org/10.48550/ARXIV.2308.10144.
28. Chen, S.; Lin, S.; Gu, X.; Shi, Y.; Lian, H.; Yun, L.; Chen, D.; Sun, W.; Cao, L.; Wang, Q. SWE-Exp: Experience-Driven Software Issue Resolution. *CoRR* **2025**, *abs/2507.23361*, [2507.23361]. https://doi.org/10.48550/ARXIV.2507.23361.
29. Ouyang, S.; Yan, J.; Hsu, I.; Chen, Y.; Jiang, K.; Wang, Z.; Han, R.; Le, L.T.; Daruki, S.; Tang, X.; et al. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. *CoRR* **2025**, *abs/2509.25140*, [2509.25140]. https://doi.org/10.48550/ARXIV.2509.25140.
30. Cai, Z.; Guo, X.; Pei, Y.; Feng, J.; Chen, J.; Zhang, Y.; Ma, W.; Wang, M.; Zhou, H. FLEX: Continuous Agent Evolution via Forward Learning from Experience. *CoRR* **2025**, *abs/2511.06449*, [2511.06449]. https://doi.org/10.48550/ARXIV.2511.06449.
31. Shen, M.; Zha, K.; He, Z.; Hong, Z.; Ouyang, S.; Ryu, J.J.; Sattigeri, P.; Diggavi, S.N.; Wornell, G.W. Decocted Experience Improves Test-Time Inference in LLM Agents. *CoRR* **2026**, *abs/2604.04373*, [2604.04373]. https://doi.org/10.48550/ARXIV.2604.04373.
32. Packer, C.; Fang, V.; Patil, S.G.; Lin, K.; Wooders, S.; Gonzalez, J.E. MemGPT: Towards LLMs as Operating Systems. *CoRR* **2023**, *abs/2310.08560*, [2310.08560]. https://doi.org/10.48550/ARXIV.2310.08560.

# 原文第 54 页 | 中文翻译初稿

54 共 70

11.人择。克劳德·代码。 https://docs.anthropic.com/en/docs/claude-code，2025。AI 编码智能体 运行支撑框架（Harness）。

12.开放人工智能。 OpenAI Codex CLI。 https://github.com/openai/codex，2025。在终端中运行的轻量级编码智能体。

13.布莱克曼，A.；格拉塔菲奥里，A.；巴桑特，A.；古普塔，A.；哈塔尔，A.；伦杜金塔拉，A.；瓦夫尔，A.；舒克拉，A.；贝尔科维奇，A.；菲切克，A.；等人。 NVIDIA Nemotron 3：高效、开放的智能。 arXiv 预印本 arXiv:2512.20856 2025 。

14.赵杰；陈，G。孟，F。李，M。陈，J。徐，H。孙，Y。赵W.X.；宋，R。张，Y。等人。沉浸在 github 世界中：将编码智能体扩展到精通。 arXiv preprint arXiv:2602.09892 2026 .

15. 舒尔曼，J.；沃尔斯基，F.；达里瓦尔，P.；雷德福，A.； Klimov, O. 近端策略优化算法。 arXiv 预印本 arXiv:1707.06347 2017 。

16. 邵Z.；王，P。朱Q。徐，R。宋，J。毕，X。张，H。张，M。昆克，Y.；等人。 DeepSeekMath：突破开放语言模型中数学推理的极限。 arXiv 预印本 arXiv:2402.03300 2024 。

17.盛，G.；张，C.；叶，Z。吴，X。张，W。张，R。彭，Y。林，H。 Wu, C. HybridFlow：灵活高效的 RLHF 框架。 arXiv 预印本 arXiv：2409.19256 2024 。

18.朱Z.；谢，C.； Lv，X.；史莱姆贡献者。 slime：用于 RL 扩展的 LLM 后训练框架。 https://github.com/THUDM/slime，2025。GitHub 存储库。通讯作者: 吕鑫.

19.张L.；陈，M。曹，R。陈，J。周，F。徐，Y。杨，J。马，Z。陈L.；罗，C.；等人。 MegaFlow: Large-Scale Distributed Orchestration System for the Agentic Era, 2026, [arXiv:cs.DC/2601.07526].

20.吴R.；王X；梅，J。蔡，P。傅，D。杨，C.；文L.；杨X。沉，Y。王，Y。等人。 EvolveR：通过经验驱动的生命周期自我进化法学硕士智能体人，2025 年，[arXiv:cs.CL/2510.16079]。

21.诺维科夫，A.；武，N.；艾森伯格，M.；杜邦，E.；黄，附注；瓦格纳，AZ；希罗博科夫，S.；科兹洛夫斯基，B.；鲁伊斯，F.J.R.；梅赫拉比安，A.；等人。 AlphaEvolve：科学和算法发现的编码智能体，2025，[arXiv:cs.AI/2506.13131]。

22.林，J.；刘，S。潘，C.；林，L。窦，S。黄X。严，H。汉，Z。 Gui, T. Agentic 运行支撑框架（Harness） Engineering: Observability-Driven Automatic Evolution of Coding-Agent 运行支撑框架（Harnesses）. CoRR 2026，abs/2604.25850，[2604.25850]。 https://doi.org/10.48550/ARXIV.2604.25850。

23.张杰；赵，B。杨，W。福斯特，J.N.；克鲁恩，J.；江，M。德夫林，S.； Shavrina，T. Hyperagents。 CoRR 2026，abs/2603.19461，[2603.19461]。 https://doi.org/10.48550/ARXIV.2603.19461。

24. 很好，I.J. Speculations Concerning the First Ultraintelligent Machine;爱思唯尔，1966；卷。 6，计算机的进展，第 31-88 页。 https://doi.org/https://doi.org/10.1016/S0065-2458(08)60418-0.

25. Schmidhuber, J. Gödel Machines: Fully Self-referential Optimal Universal Self-improvers.在_人工智能_中；戈策尔，B.； Pennachin，C.，编辑；认知技术，施普林格，2007 年；第 199-226 页。https://doi.org/10.1007/978-3-540-68677-4_7。

26.辛恩，N.；卡萨诺，F.；戈皮纳特，A.；纳拉西姆汗，K.； Yao, S. Reflexion: language agents with verbal 强化学习（Reinforcement Learning）

。神经信息处理系统进展论文集第 36 期：2023 年神经信息处理系统年会，NeurIPS 2023，美国路易斯安那州新奥尔良，2023 年 12 月 10 日至 16 日；哦，A.；瑙曼，T.；格洛伯森，A.；萨恩科，K.；哈特，M.；莱文，S.，编辑，2023 年。

27.赵，A.；黄，D.；徐Q。林，M。刘，Y。 Huang, G. ExpeL：法学硕士智能体人是体验式学习者。 CoRR 2023 , abs/2308.10144 , [2308.10144]. https://doi.org/10.48550/ARXIV.2308.10144.

28.陈S.；林，S。顾X。石，Y。连，H.；云，L。陈，D。孙W.；曹L.； Wang, Q. SWE-Exp：体验驱动的软件问题解决。 CoRR 2025 , abs/2507.23361 , [2507.23361]. https://doi.org/10.48550/ARXIV.2507.23361.

29.欧阳S.；严，J。许，I。陈，Y。江，K。王，Z。汉，R。勒，L.T.；达鲁基，S.；唐X.；等人。 ReasoningBank：使用推理内存进行自我进化的缩放智能体。 CoRR 2025 , abs/2509.25140 , [2509.25140]. https://doi.org/10.48550/ARXIV.2509.25140.

30.蔡Z.；郭X。裴，Y。冯，J。陈，J。张，Y。马，W。王，M。 Zhou, H. FLEX：通过从经验中进行前向学习来实现持续智能体进化。 CoRR 2025 , abs/2511.06449 , [2511.06449]. https://doi.org/10.48550/ARXIV.2511.06449.

31.沉，M.；查，K。他，Z。洪，Z。欧阳，S。柳，J.J.；萨蒂杰里，P.；迪加维，S.N.；沃内尔，G.W. 煎煮经验可改善 LLM 智能体的测试时推理。 CoRR 2026 , abs/2604.04373 , [2604.04373]. https://doi.org/10.48550/ARXIV.2604.04373.

32.帕克，C.；方，V。帕蒂尔，S.G.；关联。;伍德斯，S.； Gonzalez, J.E. MemGPT：将法学硕士作为操作系统。 CoRR 2023 , abs/2310.08560 , [2310.08560]. https://doi.org/10.48550/ARXIV.2310.08560.

33. Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; Zhang, Y. A-MEM: Agentic Memory for LLM Agents. *CoRR* **2025**, *abs/2502.12110*, [2502.12110]. https://doi.org/10.48550/ARXIV.2502.12110.
34. Liu, J.; Su, Y.; Xia, P.; Han, S.; Zheng, Z.; Xie, C.; Ding, M.; Yao, H. SimpleMem: Efficient Lifelong Memory for LLM Agents. *CoRR* **2026**, *abs/2601.02553*, [2601.02553]. https://doi.org/10.48550/ARXIV.2601.02553.
35. Dai, Z.; Deng, S.; Guan, S.; Tian, Y.; Yao, X.; Yan, X.; Cheng, J. RecMem: Recurrence-based Memory Consolidation for Efficient and Effective Long-Running LLM Agents. *CoRR* **2026**, *abs/2605.16045*, [2605.16045]. https://doi.org/10.48550/ARXIV.2605.16045.
36. Jin, Y.; Zhang, S.; Wang, H.; Qin, L.; Zhang, Y.; Zhang, W. EXG: Self-Evolving Agents with Experience Graphs. *CoRR* **2026**, *abs/2605.17721*, [2605.17721]. https://doi.org/10.48550/ARXIV.2605.17721.
37. Fang, J.; Xu, B.; Wang, Z.; Cao, H.; Deng, X.; Dong, B.; Zhu, H.; Huang, R.; Yu, G.; Wei, Y.; et al. Rethinking Memory as Continuously Evolving Connectivity. *CoRR* **2026**, *abs/2605.28773*, [2605.28773]. https://doi.org/10.48550/ARXIV.2605.28773.
38. Ji, S.; Wu, B.; Wang, Z.; Xia, L.; Li, Q.; Wang, R.; Ding, W.; Zhu, Z.; Li, B.; Dai, G.; et al. Infini Memory: Maintainable Topic Documents for Long-Term LLM Agent Memory. 2026.
39. Fei, T.; Song, M.; Zheng, M.; Yu, X. Memory Beyond Recall: A Dual-Process Cognitive Memory System for Self-Evolving LLM Agents. 2026.
40. Ye, C.; Liu, Y.; Wang, Y.; Yu, H.; Zhao, Y.; Liu, G.; McAuley, J.J.; You, J. Auto-Dreamer: Learning Offline Memory Consolidation for Language Agents. *CoRR* **2026**, *abs/2605.20616*, [2605.20616]. https://doi.org/10.48550/ARXIV.2605.20616.
41. Zhang, Y.; Wu, Y.; Yu, Y.; Wu, Q.; Wang, H. Live-Evo: Online Evolution of Agentic Memory from Continuous Feedback. *CoRR* **2026**, *abs/2602.02369*, [2602.02369]. https://doi.org/10.48550/ARXIV.2602.02369.
42. Liao, J.; Shi, H.; Zhou, R.; Wang, J.; Zhang, S.; Zhang, W.; Wen, Y.; Li, Z.; Xiong, F.; Tang, B.; et al. MemQ: Integrating Q-Learning into Self-Evolving Memory Agents over Provenance DAGs. *CoRR* **2026**, *abs/2605.08374*, [2605.08374]. https://doi.org/10.48550/ARXIV.2605.08374.
43. Wang, X.; Mao, W.; Wu, J.; Wang, X.; He, X. Rˆ2-Mem: Reflective Experience for Memory Search. *CoRR* **2026**, *abs/2605.13486*, [2605.13486]. https://doi.org/10.48550/ARXIV.2605.13486.
44. Zhang, Y.; Li, Y.; Payani, A.; Wang, L. AdaMEM: Test-Time Adaptive Memory for Language Agents. 2026.
45. Kim, K.; Kang, M.; Kim, T.; Yang, Y.; Ren, M.; Hwang, S.J. Memory Transfer Learning: How Memories are Transferred Across Domains in Coding Agents. *CoRR* **2026**, *abs/2604.14004*, [2604.14004]. https://doi.org/10.48550/ARXIV.2604.14004.
46. Cheng, Y.; Zhou, J.; Hu, Y.; Chen, Y.; Zhou, H.; Chen, M.; Zhang, Z.; Shao, K.; Xie, Y.; Yin, Z. TAME: A Trustworthy Test-Time Evolution of Agent Memory with Systematic Benchmarking. *CoRR* **2026**, *abs/2602.03224*, [2602.03224]. https://doi.org/10.48550/ARXIV.2602.03224.
47. Wang, S.; Brahma, D.; Henao, R. SAGE: A Novelty Gate for Efficient Memory Evolution in Agentic LLMs. *CoRR* **2026**, *abs/2605.30711*, [2605.30711]. https://doi.org/10.48550/ARXIV.2605.30711.
48. Song, Y.; Xin, Q. D-MEM: Dopamine-Gated Agentic Memory via Reward Prediction Error Routing. *CoRR* **2026**, *abs/2603.14597*, [2603.14597]. https://doi.org/10.48550/ARXIV.2603.14597.
49. Zhang, G.; Ren, H.; Zhan, C.; Zhou, Z.; Wang, J.; Zhu, H.; Zhou, W.; Yan, S. MemEvolve: Meta-Evolution of Agent Memory Systems. *CoRR* **2025**, *abs/2512.18746*, [2512.18746]. https://doi.org/10.48550/ARXIV.2512.18746.
50. Xiong, Y.; Hu, S.; Clune, J. Learning to Continually Learn via Meta-learning Agentic Memory Designs. *CoRR* **2026**, *abs/2602.07755*, [2602.07755]. https://doi.org/10.48550/ARXIV.2602.07755.
51. Pan, W.; Liu, S.; Zhou, X.; Zhang, S.; Shi, W.; Xu, M.; Jia, X. M$^*$: Every Task Deserves Its Own Memory Harness. *CoRR* **2026**, *abs/2604.11811*, [2604.11811]. https://doi.org/10.48550/ARXIV.2604.11811.
52. Liu, J.; Ye, X.; Xia, P.; Zheng, Z.; Xie, C.; Ding, M.; Yao, H. EvolveMem:Self-Evolving Memory Architecture via AutoResearch for LLM Agents. *CoRR* **2026**, *abs/2605.13941*, [2605.13941]. https://doi.org/10.48550/ARXIV.2605.13941.
53. Liu, Q.; Wang, G.; Wu, W.; Huang, J.; Tao, X.; Song, D.; Zhou, J.; He, L. MemPro: Agentic Memory Systems as Evolvable Programs. 2026.
54. Zhang, H.; Long, Q.; Bao, J.; Feng, T.; Zhang, W.; Yue, H.; Wang, W. MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents. *CoRR* **2026**, *abs/2602.02474*, [2602.02474]. https://doi.org/10.48550/ARXIV.2602.02474.
55. Yang, Y.; Liu, T.; Zhu, W.B.; Shi, T.; Song, L.; Jia, R. Self-Evolving LLM Memory Extraction Across Heterogeneous Tasks. *CoRR* **2026**, *abs/2604.11610*, [2604.11610]. https://doi.org/10.48550/ARXIV.2604.11610.

# 原文第 55 页 | 中文翻译初稿

55 共 70

33.徐W.；梁，Z。梅，K.；高，H。谭，J。张，Y.A-MEM：法学硕士智能体的智能体记忆。 CoRR 2025，abs/2502.12110，[2502.12110]。 https://doi.org/10.48550/ARXIV.2502.12110。

34.刘，J.；苏，Y。夏，P。汉，S。郑Z.；谢，C.；丁，M。 Yao, H. SimpleMem：法学硕士智能体人的高效终身记忆。 CoRR 2026，abs/2601.02553，[2601.02553]。 https://doi.org/10.48550/ARXIV.2601.02553。

35.戴Z.；邓，S。关，S。田，Y。姚X.；严X； Cheng, J. RecMem：基于循环的内存整合，实现高效且有效的长期运行的 LLM 智能体。 CoRR 2026，abs/2605.16045，[2605.16045]。 https://doi.org/10.48550/ARXIV.2605.16045。

36.金，Y.；张，S。王，H。秦L.；张，Y。张，W. EXG：具有经验图的自我进化智能体。 CoRR 2026，abs/2605.17721，[2605.17721]。 https://doi.org/10.48550/ARXIV.2605.17721。

37.方，J.；徐，B。王，Z。曹H.；邓X.；董，B.；朱，H.；黄，R。于，G。魏，Y。等人。重新思考记忆作为不断发展的连接性。 CoRR 2026，abs/2605.28773，[2605.28773]。 https://doi.org/10.48550/ARXIV.2605.28773。

38. 吉，S.；吴，B.；王，Z。夏L.；李，Q。王，R。丁，W。朱，Z.；李，B。戴，G.；等人。 Infini Memory：长期 LLM 智能体内存的可维护主题文档。 2026 年。

39.费，T.；宋，M.；郑，M。 Yu, X. 无法回忆的记忆：用于自我进化 LLM 智能体的双进程认知记忆系统。 2026 年。

40. 叶，C.；刘，Y。王，Y。于，H.；赵，Y。刘，G.；麦考利，J.J.； You, J. Auto-Dreamer：学习语言智能体的离线内存整合。 CoRR 2026，abs/2605.20616，[2605.20616]。 https://doi.org/10.48550/ARXIV.2605.20616。

41.张，Y.；吴，Y。于，Y。吴，Q。 Wang, H. Live-Evo：来自连续反馈的智能体记忆的在线进化。 CoRR 2026，abs/2602.02369，[2602.02369]。 https://doi.org/10.48550/ARXIV.2602.02369。

42.廖J.；石，H。周，R。王，J。张，S。张，W。温，Y。李，Z。熊F.；唐，B.；等人。 MemQ：将 Q-Learning 集成到起源 DAG 上的自我进化内存智能体中。 CoRR 2026，abs/2605.08374，[2605.08374]。 https://doi.org/10.48550/ARXIV.2605.08374。

43.王X.；毛，W.；吴，J。王X； He，X.Rˆ2-Mem：记忆搜索的反思体验。 CoRR 2026，abs/2605.13486，[2605.13486]。 https://doi.org/10.48550/ARXIV.2605.13486。

44.张，Y.；李，Y。帕亚尼，A.； Wang, L. AdaMEM：语言智能体的测试时自适应记忆。 2026 年。

45.金，K.；康，M.；金，T。杨，Y。任，M。黄，S.J.内存迁移学习：如何在编码智能体中跨域传输内存。 CoRR 2026，abs/2604.14004，[2604.14004]。 https://doi.org/10.48550/ARXIV.2604.14004。

46.程Y.；周，J。胡，Y。陈，Y。周，H.；陈，M。张，Z。邵，K.；谢，Y。 Yin, Z. TAME：具有系统基准测试的智能体内存的值得信赖的测试时演变。 CoRR 2026，abs/2602.03224，[2602.03224]。 https://doi.org/10.48550/ARXIV.2602.03224。

47.王S.；布拉马，D.； Henao, R. SAGE：智能体法学硕士中高效内存进化的新颖之门。 CoRR 2026，abs/2605.30711，[2605.30711]。 https://doi.org/10.48550/ARXIV.2605.30711。

48.宋，Y.；辛琪

。 D-MEM: Dopamine-Gated Agentic Memory via Reward Prediction Error Routing. CoRR 2026，abs/2603.14597，[2603.14597]。 https://doi.org/10.48550/ARXIV.2603.14597。

49.张，G.；任，H.；詹，C。周，Z.；王，J。朱，H.；周，W。 Yan, S. MemEvolve：智能体内存系统的元进化。 CoRR 2025，abs/2512.18746，[2512.18746]。 https://doi.org/10.48550/ARXIV.2512.18746。

50.熊Y.；胡，S。 Clune, J. Learning to Continually Learn via Meta-learning Agentic Memory Designs. CoRR 2026，abs/2602.07755，[2602.07755]。 https://doi.org/10.48550/ARXIV.2602.07755。

51.潘W.；刘，S。周X；张，S。施W.；徐，M。 Jia, X. M*：每个任务都值得拥有自己的记忆 运行支撑框架（Harness）。 CoRR 2026，abs/2604.11811，[2604.11811]。 https://doi.org/10.48550/ARXIV.2604.11811。

52.刘，J.；叶，X。夏，P。郑Z.；谢，C.；丁，M。 Yao, H. EvolveMem：通过 LLM 智能体的自动研究实现自我进化内存架构。 CoRR 2026，abs/2605.13941，[2605.13941]。 https://doi.org/10.48550/ARXIV.2605.13941。

53.刘Q.；王，G。吴，W。黄，J。陶，X.；宋，D.；周，J。 He, L. MemPro：作为可进化程序的智能体内存系统。 2026 年。

54.张H.；长，Q。鲍，J.；冯，T。张，W。岳，H.； Wang, W. MemSkill: Learning and Evolving Memory Skills for Self-Evolving Agents. CoRR 2026，abs/2602.02474，[2602.02474]。 https://doi.org/10.48550/ARXIV.2602.02474。

55.杨，Y.；刘，T。朱文波；石，T。宋L.； Jia, R. 跨异构任务的自我进化 LLM 内存提取。 CoRR 2026，abs/2604.11610，[2604.11610]。 https://doi.org/10.48550/ARXIV.26 04.11610。

56. Qian, C.; Han, C.; Fung, Y.R.; Qin, Y.; Liu, Z.; Ji, H. CREATOR: Disentangling Abstract and Concrete Reasonings of Large Language Models through Tool Creation. *CoRR* **2023**, *abs/2305.14318*, [2305.14318]. https://doi.org/10.48550/ARXIV.2305.14318.
57. Cai, T.; Wang, X.; Ma, T.; Chen, X.; Zhou, D. Large Language Models as Tool Makers. *CoRR* **2023**, *abs/2305.17126*, [2305.17126]. https://doi.org/10.48550/ARXIV.2305.17126.
58. Ding, H.; Tao, S.; Pang, L.; Wei, Z.; Gao, J.; Ding, B.; Shen, H.; Cheng, X. ToolCoder: A Systematic Code-Empowered Tool Learning Framework for Large Language Models. *CoRR* **2025**, *abs/2502.11404*, [2502.11404]. https://doi.org/10.48550/ARXIV.2502.11404.
59. Liu, X.; Yin, D.; Wu, Z.; Feng, Y. RefTool: Enhancing Model Reasoning with Reference-Guided Tool Creation. *CoRR* **2025**, *abs/2505.21413*, [2505.21413]. https://doi.org/10.48550/ARXIV.2505.21413.
60. Zheng, B.; Fatemi, M.Y.; Jin, X.; Wang, Z.Z.; Gandhi, A.; Song, Y.; Gu, Y.; Srinivasa, J.; Liu, G.; Neubig, G.; et al. SkillWeaver: Web Agents can Self-Improve by Discovering and Honing Skills. *CoRR* **2025**, *abs/2504.07079*, [2504.07079]. https://doi.org/10.48550/ARXIV.2504.07079.
61. Wang, C.; Yu, Z.; Xie, X.; Yao, W.; Fang, R.; Qiao, S.; Cao, K.; Zheng, G.; Qi, X.; Zhang, P.; et al. SkillX: Automatically Constructing Skill Knowledge Bases for Agents. *CoRR* **2026**, *abs/2604.04804*, [2604.04804]. https://doi.org/10.48550/ARXIV.2604.04804.
62. Zhang, Y.; Han, X.; Jiang, X.; Wang, R. Workflow-to-Skill: Skill Creation via Routing-Workflow-Semantics-Attachments Decomposition. 2026.
63. Xiao, C.; Jiao, Z.; Wang, S.; Wang, W.; Zhao, B.; Wei, H.; Zhang, L.; Qu, L. Socratic-SWE: Self-Evolving Coding Agents via Trace-Derived Agent Skills. 2026.
64. Pan, Q.; Yang, Y.; Li, J.; Zhou, J.; Chen, K.; Li, X.; Chen, Q.; He, L. Anything2Skill: Compiling External Knowledge into Reusable Skills for Agents. 2026.
65. Yan, Z.; Song, D.; Zhang, H.; Liang, W.; Zhang, Y.; Dai, Y.; He, L.; Yu, P.S.; Xu, R.; Li, X.; et al. OpenSkill: Open-World Self-Evolution for LLM Agents. 2026.
66. Shen, S.; Cheng, W.; Ma, M.; Turcan, A.; Zhang, M.J.; Ma, J. SKILLFOUNDRY: Building Self-Evolving Agent Skill Libraries from Heterogeneous Scientific Resources. *CoRR* **2026**, *abs/2604.03964*, [2604.03964]. https://doi.org/10.48550/ARXIV.2604.03964.
67. Qiu, Z.; Song, K.; Tang, S.; Qiao, S.; Liang, L.; Chen, H.; Deng, S. Unsupervised Skill Discovery for Agentic Data Analysis. 2026.
68. Lin, H.; Li, P.; Song, J.; Jiang, F.; Zhang, T. MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation. *CoRR* **2026**, *abs/2605.27366*, [2605.27366]. https://doi.org/10.48550/ARXIV.2605.27366.
69. Ouyang, S.; Yan, J.; Chen, Y.; Han, R.; Wang, Z.; Mishra, B.D.; Meng, R.; Li, C.; Jiao, Y.; Zha, K.; et al. SkillOS: Learning Skill Curation for Self-Evolving Agents. *CoRR* **2026**, *abs/2605.06614*, [2605.06614]. https://doi.org/10.48550/ARXIV.2605.06614.
70. Ma, Z.; Yang, S.; Ji, Y.; Wang, X.; Wang, Y.; Hu, Y.; Huang, T.; Chu, X. SkillClaw: Let Skills Evolve Collectively with Agentic Evolver. *CoRR* **2026**, *abs/2604.08377*, [2604.08377]. https://doi.org/10.48550/ARXIV.2604.08377.
71. Yue, M.; Liu, Z.; Yang, L.; Zhang, J.; Liu, Z.; Chen, H.; Yao, Z.; Savarese, S.; Xiong, C.; Heinecke, S.; et al. ToolLibGen: Scalable Automatic Tool Creation and Aggregation for LLM Reasoning. *CoRR* **2025**, *abs/2510.07768*, [2510.07768]. https://doi.org/10.48550/ARXIV.2510.07768.
72. Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. *CoRR* **2026**, *abs/2605.23904*, [2605.23904]. https://doi.org/10.48550/ARXIV.2605.23904.
73. Wang, Y.; Zhou, Y.; Liang, Y.; Zhang, C.; Liu, F.; Zhou, J.; Yao, H. Not All Skills Help: Measuring and Repairing Agent Knowledge. 2026.
74. Zhang, Q.; Feng, Z.; Shi, X.; Hu, X.; Liu, C.; Xie, P.; Wang, X.; Ye, J.; Hooi, B.; Wang, H.; et al. SkillComposer: Learning to Evolve Agent Skills for Specification and Generalization. 2026.
75. Shi, H.; Yuan, X.; Liu, B. Evolving Programmatic Skill Networks. *CoRR* **2026**, *abs/2601.03509*, [2601.03509]. https://doi.org/10.48550/ARXIV.2601.03509.
76. Gautam, S.; Radhakrishna, A.; Gulwani, S. SkillAxe: Sharpening LLM-Authored Agent Skills Through Evaluation-Guided Self-Refinement. 2026.
77. Gao, H.; Chen, H.; Wang, C.; Guo, S.; Pang, L.; Liu, Z.; Shen, H.; Cheng, X. SkillAudit: Ground-Truth-Free Skill Evolution via Paired Trajectory Auditing. 2026.

# 原文第 56 页 | 中文翻译初稿

56 共 70

56.钱，C.；汉，C.；冯，Y.R.；秦，Y。刘，Z.； Ji, H. CREATOR：通过工具创建解开大型语言模型的抽象和具体推理。 CoRR 2023，abs/2305.14318，[2305.14318]。 https://doi.org/10.48550/ARXIV.2305.14318。

57.蔡T.；王X；马，T。陈X.； Zhou, D. 作为工具制造者的大型语言模型。 CoRR 2023， abs/2305.17126，[2305.17126]。 https://doi.org/10.48550/ARXIV.2305.17126。

58.丁H.；陶，S。庞L.；魏，Z.；高，J。丁，B.；沉H.； Cheng, X. ToolCoder：用于大型语言模型的系统性 CodeEmpowered 工具学习框架。 CoRR 2025，abs/2502.11404，[2502.11404]。 https://doi.org/10.48550/ARXIV.2502.11404。

59.刘X.；尹，D。吴，Z。 Feng, Y. RefTool：通过参考引导工具创建增强模型推理。 CoRR 2025， abs/2505.21413，[2505.21413]。 https://doi.org/10.48550/ARXIV.2505.21413。

60.郑B.；法特米，M.Y.；金，X。王ZZ；甘地，A.；宋，Y。顾，Y。斯里尼瓦萨，J.；刘，G.；纽比格，G.；等人。 SkillWeaver：网络智能体可以通过发现和磨练技能来自我提高。 CoRR 2025， abs/2504.07079，[2504.07079]。 https://doi.org/10.48550/ARXIV.2504.07079。

61.王，C.；于，Z。谢X。姚，W.；方，R。乔，S。曹K.；郑，G。齐，X。张，P。等人。 SkillX：自动构建座席技能知识库。 CoRR 2026，abs/2604.04804，[2604.04804]。 https://doi.org/10.48550/ARXIV.2604.04804。

62.张，Y.；韩X.；蒋X； Wang, R. 工作流到技能：通过路由-工作流-语义附件分解创建技能。 2026 年。

63.肖，C.；焦，Z.；王，S。王，W。赵，B。魏，H。张L。 Qu, L. Socratic-SWE：通过跟踪衍生智能体技能进行自我进化编码智能体。 2026 年。

64.潘Q.；杨，Y。李，J。周，J。陈，K。李，X。陈Q。 He, L. Anything2Skill：将外部知识编译为智能体可重用的技能。 2026 年。

65.严Z.；宋，D.；张，H。梁，W。张，Y。戴，Y。他，L.；余，附注；徐，R。李，X。等人。 OpenSkill：LLM 智能体的开放世界自我进化。 2026 年。

66.沉S.；郑，W.；马，M。图尔坎，A.；张，M.J.； Ma, J. SKILLFOUNDRY：从异构科学资源构建自我进化智能体技能库。 CoRR 2026，abs/2604.03964，[2604.03964]。 https://doi.org/10.48550/ARXIV.2604.03964。

67.邱Z.；宋，K.；唐，S。乔，S。梁，L.；陈，H。 Deng, S. 智能体数据分析的无监督技能发现。 2026 年。

68.林H.；李，P。宋，J。江，F。张，T. MUSE-Autoskill：通过技能创建、记忆、管理和评估进行自我进化智能体。 CoRR 2026，abs/2605.27366，[2605.27366]。 https://doi.org/10.48550/ARXIV。 2605.27366。

69.欧阳S.；严，J。陈，Y。汉，R。王，Z。米什拉，B.D.；孟，R。李，C。焦，Y。查，K。等人。 SkillOS：自我进化智能体的学习技能管理。 CoRR 2026，abs/2605.06614，[2605.06614]。 https://doi.org/10.48550/ARXIV.2605.06614。

70.马Z.；杨，S。吉，Y。王X；王，Y。胡，Y。黄，T。 Chu, X. SkillClaw：让技能与 Agentic Evolver 一起进化。 CoRR 2026，abs/2604.08377，[2604.08377]。 https://doi.org/10.48550/ARXIV.2604.08377。

71.岳，M.；刘，Z.；杨，L。张，J。刘，Z.；陈，H.

;姚，Z.；萨瓦雷塞，S.；熊，C.；海内克，S.；等人。 ToolLibGen：用于 LLM 推理的可扩展自动工具创建和聚合。 CoRR 2025，abs/2510.07768，[2510.07768]。 https://doi.org/10.48550/ARXIV.2510.07768。

72.杨，Y.；龚，Z。黄W.；杨，Q。周，Z.；黄，Z。李，Y。高X.；戴，Q。刘，B.；等人。 SkillOpt：自我进化智能体技能的执行策略。 CoRR 2026，abs/2605.23904，[2605.23904]。 https://doi.org/10.48550/ARXIV.2605.23904。

73.王，Y.；周，Y。梁，Y。张，C.；刘，F。周，J。 Yao, H. 并非所有技能都有帮助：测量和修复智能体知识。 2026 年。

74.张Q.；冯，Z.；施X.；胡X.；刘，C.；谢，P。王X；叶，J。胡伊，B.；王，H。等人。 SkillComposer：学习发展智能体技能以进行规范和泛化。 2026 年。

75.石H.；袁X.； Liu, B. 不断发展的程序化技能网络。 CoRR 2026，abs/2601.03509，[2601.03509]。 https://doi.org/10.48550/ARXIV.2601.03509。

76.高塔姆，S.；拉达克里希纳，A.； Gulwani, S. SkillAxe：通过评估引导的自我完善提高法学硕士创作的智能体技能。 2026 年。

77.高H.；陈，H。王，C.；郭，S。庞L.；刘，Z.；沉H.； Cheng, X. SkillAudit：通过配对轨迹审核实现无地面真实性的技能进化。 2026 年。

78. Ma, Y.; Huang, Y.; Bao, H.; Zhuang, H.; Shukla, S.; Galley, M.; Zhang, X.; Feuerriegel, S. SkillGen: Verified Inference-Time Agent Skill Synthesis. *CoRR* **2026**, *abs/2605.10999*, [2605.10999]. https://doi.org/10.48550/ARXIV.2605.10999.
79. Zhang, H.; Fan, S.; Zou, H.P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.; Yao, Y.; Zheng, K.; et al. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. *CoRR* **2026**, *abs/2604.01687*, [2604.01687]. https://doi.org/10.48550/ARXIV.2604.01687.
80. Yang, Y.; Bhatt, N.P.; Wang, K.; Tetteh, S.; Wang, Z.; Topcu, U. VASO: Formally Verifiable Self-Evolving Skills for Physical AI Agents. 2026.
81. Lu, J.; Kong, Z.; Wang, Y.; Fu, R.; Wan, H.; Yang, C.; Lou, W.; Sun, H.; Wang, L.; Jiang, Y.; et al. Beyond Static Tools: Test-Time Tool Evolution for Scientific Reasoning. *CoRR* **2026**, *abs/2601.07641*, [2601.07641]. https://doi.org/10.48550/ARXIV.2601.07641.
82. Wei, S.; Min, H.S.; Dong, X.; Lin, X.; Cui, S.; Jiang, B.; Dai, Z.; Kuang, K.; Xu, G.; Wu, F.; et al. MetaForge: A Self-Evolving Multimodal Agent that Retrieves, Adapts, and Forges Tools On Demand. 2026.
83. Wei, Y.; Huang, Z.; Lu, S.; Qian, J.; Wang, Q.; Wu, C.; He, L. SkillSmith: Co-Evolving Skills and Tools for Self-Improving Agent Systems. 2026.
84. Wang, J.; Yan, Q.; Wang, Y.; Tian, Y.; Mishra, S.S.; Xu, Z.; Gandhi, M.; Xu, P.; Cheong, L.L. Reinforcement Learning for Self-Improving Agent with Skill Library. *CoRR* **2025**, *abs/2512.17102*, [2512.17102]. https://doi.org/10.48550/ARXIV.2512.17102.
85. Wong, S.; Qi, Z.; Wang, Z.; Hu, N.; Lin, S.; Ge, J.; Gao, E.; Chen, W.; Du, Y.; Yu, M.; et al. Confucius Code Agent: Scalable Agent Scaffolding for Real-World Codebases. *CoRR* **2025**, *abs/2512.10398*, [2512.10398]. https://doi.org/10.48550/ARXIV.2512.10398.
86. Yang, C.; Wang, X.; Lu, Y.; Liu, H.; Le, Q.V.; Zhou, D.; Chen, X. Large Language Models as Optimizers. *CoRR* **2023**, *abs/2309.03409*, [2309.03409]. https://doi.org/10.48550/ARXIV.2309.03409.
87. Fernando, C.; Banarse, D.; Michalewski, H.; Osindero, S.; Rocktäschel, T. Promptbreeder: Self-Referential Self-Improvement Via Prompt Evolution. *CoRR* **2023**, *abs/2309.16797*, [2309.16797]. https://doi.org/10.48550/ARXIV.2309.16797.
88. Xiang, J.; Zhang, J.; Yu, Z.; Teng, F.; Tu, J.; Liang, X.; Hong, S.; Wu, C.; Luo, Y. Self-Supervised Prompt Optimization. *CoRR* **2025**, *abs/2502.06855*, [2502.06855]. https://doi.org/10.48550/ARXIV.2502.06855.
89. Peng, D.; Zhou, Y.; Chen, Q.; Liu, J.; Chen, J.; Qin, L. DLPO: Towards a Robust, Efficient, and Generalizable Prompt Optimization Framework from a Deep-Learning Perspective. *CoRR* **2025**, *abs/2503.13413*, [2503.13413]. https://doi.org/10.48550/ARXIV.2503.13413.
90. Lin, Z.J.; Letham, B.; Dooley, S.; Balandat, M.; Bakshy, E. Embedding by Elicitation: Dynamic Representations for Bayesian Optimization of System Prompts. *CoRR* **2026**, *abs/2605.19093*, [2605.19093]. https://doi.org/10.48550/ARXIV.2605.19093.
91. Singhal, R.; Tambwekar, P.; Maamari, K. PrefPO: Pairwise Preference Prompt Optimization. *CoRR* **2026**, *abs/2603.19311*, [2603.19311]. https://doi.org/10.48550/ARXIV.2603.19311.
92. Wang, F.; Si, S.; Hsieh, C.J.; Dhillon, I.S. APEX: Automated Prompt Engineering eXpert with Dynamic Data Selection. 2026.
93. Kang, E.H.; Yoganarasimhan, H. Bayesian Optimization in Language Space: An Eval-Efficient AI Self-Improvement Framework. *CoRR* **2025**, *abs/2511.12063*, [2511.12063]. https://doi.org/10.48550/ARXIV.2511.12063.
94. Lee, Y.; Boen, J.; Finn, C. Feedback Descent: Open-Ended Text Optimization via Pairwise Comparison. *ArXiv* **2025**, *abs/2511.07919*.
95. Lu, M.; Feng, C.; Han, H.; Lu, G.; Sun, Y.; Ding, X.; Long, S.; Li, F.; Motwani, T. SPEAR: Code-Augmented Agentic Prompt Optimization. *CoRR* **2026**, *abs/2605.26275*, [2605.26275]. https://doi.org/10.48550/ARXIV.2605.26275.
96. Fernandes, R.C.; Fehring, L.; Eimer, T.; Lindauer, M.; Feurer, M. Environment-Grounded Automated Prompt Optimization for LLM Game Agents. *arXiv preprint arXiv:2606.17838* **2026**.
97. Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; Joshi, T.T.; Moazam, H.; et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *CoRR* **2023**, *abs/2310.03714*, [2310.03714]. https://doi.org/10.48550/ARXIV.2310.03714.
98. Opsahl-Ong, K.; Ryan, M.J.; Purtell, J.; Broman, D.; Potts, C.; Zaharia, M.; Khattab, O. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. *CoRR* **2024**, *abs/2406.11695*, [2406.11695]. https://doi.org/10.48550/ARXIV.2406.11695.

# 原文第 57 页 | 中文翻译初稿

57 共 70

78.马，Y.；黄，Y。鲍，H.；庄H.；舒克拉，S.；加利，M.；张X； Feuerriegel, S. SkillGen：经过验证的推理时间智能体技能综合。 CoRR 2026，abs/2605.10999，[2605.10999]。 https://doi.org/10.48550/ARXIV.2605.10999。

79.张H.；范，S。邹，H.P.；陈，Y。王，Z。周，J。李，C。黄W.；姚，Y。郑，K。等人。 CoEvoSkills：通过共同进化验证自我进化智能体技能。 CoRR 2026，abs/2604.01687，[2604.01687]。 https://doi.org/10.48550/ARXIV.2604.01687。

80.杨，Y.；巴特，N.P.；王，K。泰特，S.；王，Z。 Topcu，U. VASO：物理人工智能智能体的可正式验证的自我进化技能。 2026 年。

81. 卢杰；孔，Z.；王，Y。毛皮。;万，H.；杨，C.；卢，W.；孙，H.；王L.；蒋，Y。等人。超越静态工具：科学推理的测试时工具演变。 CoRR 2026，abs/2601.07641，[2601.07641]。 https://doi.org/10.48550/ARXIV.2601.07641。

82.魏S.；敏，H.S.；董X.；林，X。崔，S。江，B.；戴，Z。邝，K.；徐，G。吴，F。等人。 MetaForge：一种自我进化的多模式智能体，可按需检索、适应和锻造工具。 2026 年。

83.魏，Y.；黄，Z。卢，S。钱，J。王Q。吴，C.； He, L. SkillSmith：自我改进（Self-Improving） 智能体系统的共同进化技能和工具。 2026 年。

84.王杰；严Q。王，Y。田，Y。米什拉，S.S.；徐，Z。甘地，M.；徐，P。 Cheong, L.L. 强化学习（Reinforcement Learning），用于具有技能库的 自我改进（Self-Improving） 智能体。 CoRR 2025，abs/2512.17102，[2512.17102]。 https://doi.org/10.48550/ARXIV.2512.17102。

85.黄S.；齐，Z.；王，Z。胡，N。林，S。葛，J。高，E。陈，W。杜，Y。于，M。等人。孔子代码智能体：现实世界代码库的可扩展智能体脚手架。 CoRR 2025，abs/2512.10398，[2512.10398]。 https://doi.org/10.48550/ARXIV.2512.10398。

86.杨，C.；王X；卢，Y。刘，H.；勒，Q.V.；周，D。 Chen, X。作为优化器的大型语言模型。 CoRR 2023，abs/2309.03409，[2309.03409]。 https://doi.org/10.48550/ARXIV.2309.03409。

87.费尔南多，C.；巴纳斯，D.；米卡莱夫斯基，H.；奥辛德罗，S.； Rocktäschel, T. Promptbreeder：通过即时进化进行自我参考 自我改进（Self-Improvement）。 CoRR 2023，abs/2309.16797，[2309.16797]。 https://doi.org/10.485 50/ARXIV.2309.16797。

88.向，J.；张，J。于，Z。滕，F.；涂，J。梁X.；洪，S。吴，C.；罗，Y。自监督提示优化。 CoRR 2025，abs/2502.06855，[2502.06855]。 https://doi.org/10.48550/ARXIV.2502.06855。

89.彭，D.；周，Y。陈Q。刘，J。陈，J。秦 L. DLPO：从深度学习的角度迈向稳健、高效和可泛化的即时优化框架。 CoRR 2025，abs/2503.13413，[2503.13413]。 https://doi.org/10.48550/ARXIV.2503.13413。

90.林ZJ；莱瑟姆，B.；杜利，S.；巴兰达特，M.； Bakshy, E. 通过启发嵌入：系统提示贝叶斯优化的动态表示。 CoRR 2026，abs/2605.19093，[2605.19093]。 https://doi.org/10.48550/ARXIV.2605.19093。

91. Singhal，R.；坦布韦卡尔，P.； Maamari, K. PrefPO：成对偏好提示优化。 CoRR 2026，abs/2603.19311，[2603.19311]。 https://doi.org/10.48550/ARXIV.2603.19311。

92.王F.；西，S。谢，C.J.；爱达荷州迪隆

。 APEX：具有动态数据选择功能的自动化提示工程专家。 2026 年。

93.康，E.H.； Yoganarasimhan，H. 语言空间中的贝叶斯优化：一种有效评估的人工智能自我改进框架。 CoRR 2025，abs/2511.12063，[2511.12063]。 https://doi.org/10.48550/ARXIV.2511 .12063。

94.李，Y.；伯恩，J.； Finn, C. 反馈下降：通过成对比较进行开放式文本优化。 ArXiv 2025，abs/2511.07919。

95. 卢，M.；冯，C.；韩，H。卢，G。孙，Y。丁X。朗，S。李，F。 Motwani, T. SPEAR：代码增强智能体提示优化。 CoRR 2026，abs/2605.26275，[2605.26275]。 https://doi.org/10.48550/ARXIV。2605.26275。

96.费尔南德斯，R.C.；费林，L.；艾默，T.；林道尔，M.； Feurer, M. LLM 游戏智能体的基于环境的自动提示优化。 arXiv 预印本 arXiv:2606.17838 2026 。

97.卡塔布，O.；辛格维，A.；马赫什瓦里，P.；张，Z。桑塔南，K.；瓦尔达马南，S.；哈克，S.；夏尔马，A.；乔希，T.T.；莫阿扎姆，H.；等人。 DSPy：将声明性语言模型调用编译到 自我改进（Self-Improving）管道中。 CoRR 2023，abs/2310.03714，[2310.03714]。 https://doi.org/10.48550/ARXIV.2310.03714。

98.Opsahl-Ong，K.；瑞安，M.J.；珀特尔，J.；布罗曼，D.；波茨，C.；扎哈里亚，M.； Khattab, O. 多阶段语言模型程序的优化说明和演示。 CoRR 2024，abs/2406.11695，[2406.11695]。 https://doi.org/10.48550/ARXIV.2406.11695。

99. Spiess, C.; Vaziri, M.; Mandel, L.; Hirzel, M. AutoPDL: Automatic Prompt Optimization for LLM Agents. *CoRR* **2025**, *abs/2504.04365*, [2504.04365]. https://doi.org/10.48550/ARXIV.2504.04365.
100. Lin, S.; Hua, W.; Li, L.; Wang, Z.; Zhang, Y. ADO: Automatic Data Optimization for Inputs in LLM Prompts. *CoRR* **2025**, *abs/2502.11436*, [2502.11436]. https://doi.org/10.48550/ARXIV.2502.11436.
101. Shankaranarayanan, A.; Venkataraman, A.N.; Nikolakopoulos.; Kumaraswamy, V.S.; Zhang, T.; Chander, S.; Saboo, R.R.; Khan, S.A. A FRAMEWORK FOR PROMPT OPTIMIZATION AND TRANSLATION A CROSS FOUNDATION MODELS.
102. Yüksekgönül, M.; Bianchi, F.; Boen, J.; Liu, S.; Huang, Z.; Guestrin, C.; Zou, J. TextGrad: Automatic "Differentiation" via Text. *CoRR* **2024**, *abs/2406.07496*, [2406.07496]. https://doi.org/10.48550/ARXIV.2406.07496.
103. Agrawal, L.A.; Tan, S.; Soylu, D.; Ziems, N.; Khare, R.; Opsahl-Ong, K.; Singhvi, A.; Shandilya, H.; Ryan, M.J.; Jiang, M.; et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *ArXiv* **2025**, *abs/2507.19457*.
104. Chen, M.; Deng, W.; Zou, J.; Yu, H.; Li, X. Textual Equilibrium Propagation for Deep Compound AI Systems. *CoRR* **2026**, *abs/2601.21064*, [2601.21064]. https://doi.org/10.48550/ARXIV.2601.21064.
105. Rishav, R.; Pujari, P.; Rastogi, P. ContraPrompt: Contrastive Prompt Optimization via Dyadic Reasoning Trace Analysis. *CoRR* **2026**, *abs/2604.17937*, [2604.17937]. https://doi.org/10.48550/ARXIV.2604.17937.
106. Zhu, T.; Yao, T.; Kuwaranancharoen, K.; Singh, A.; Lai, Y.; Mohan, D.A.; Bhargava, S. Graph-based Target Back-Propagation for Context Adaptation in Multi-LLM Agentic Systems. 2026.
107. Li, W.; Song, Y.; Zhao, M.; Jin, B.; Li, W. Unifying Temporal and Structural Credit Assignment in LLM-Based Multi-Agent Prompt Optimization. *CoRR* **2026**, *abs/2605.30227*, [2605.30227]. https://doi.org/10.48550/ARXIV.2605.30227.
108. Xu, W.; Liu, S.; Wang, M. EEVEE: Towards Test-time Prompt Learning in the Real World for Self-Improving Agents. 2026.
109. Li, H.; He, R.; Zhang, Q.; Ji, C.; Mang, Q.; Chen, X.; Agrawal, L.A.; Liao, W.; Yang, E.; Cheung, A.; et al. Combee: Scaling Prompt Learning for Self-Improving Language Model Agents. *CoRR* **2026**, *abs/2604.04247*, [2604.04247]. https://doi.org/10.48550/ARXIV.2604.04247.
110. Suzgun, M.; Yüksekgönül, M.; Bianchi, F.; Jurafsky, D.; Zou, J. Dynamic Cheatsheet: Test-Time Learning with Adaptive Memory. *CoRR* **2025**, *abs/2504.07952*, [2504.07952]. https://doi.org/10.48550/ARXIV.2504.07952.
111. Zhang, Q.; Hu, C.; Upasani, S.; Ma, B.; Hong, F.; Kamanuru, V.; Rainton, J.; Wu, C.; Ji, M.; Li, H.; et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. *CoRR* **2025**, *abs/2510.04618*, [2510.04618]. https://doi.org/10.48550/ARXIV.2510.04618.
112. Pei, Z.; Zhen, H.; Kai, S.; Pan, S.J.; Wang, Y.; Yuan, M.; Yu, B. SCOPE: Prompt Evolution for Enhancing Agent Effectiveness. *CoRR* **2025**, *abs/2512.15374*, [2512.15374]. https://doi.org/10.48550/ARXIV.2512.15374.
113. Vassilyev, N.; Berrios, W.; Zhang, R.; Han, B.; Kiela, D.; Mehri, S. Reflective Context Learning: Studying the Optimization Primitives of Context Space. *CoRR* **2026**, *abs/2604.03189*, [2604.03189]. https://doi.org/10.48550/ARXIV.2604.03189.
114. Zhu, Z.; Hu, Y.; Dai, Y.; Fang, J.; Jiang, C.; Hu, S.; Zhao, Y. Unified Context Evolution for LLM Agents. 2026.
115. Parashar, J.; Bhandarkar, S. KACE: Knowledge-Adaptive Context Engineering for Mathematical Reasoning. 2026.
116. Huang, Z.; Kuncoro, A.; Feng, Q.; Shen, J.; Dery, L.M.; Szlam, A.; Ranzato, M. Context Training with Active Information Seeking. *CoRR* **2026**, *abs/2605.13050*, [2605.13050]. https://doi.org/10.48550/ARXIV.2605.13050.
117. Xie, Y.; Wang, K.; Cheng, B.; Yao, J.; Sha, Z.; Duffy, A.; Xi, Y.; Mei, H.; Tan, C.; Wei, C.; et al. MEMO: Memory-Augmented Model Context Optimization for Robust Multi-Turn Multi-Agent LLM Games. *CoRR* **2026**, *abs/2603.09022*, [2603.09022]. https://doi.org/10.48550/ARXIV.2603.09022.
118. Wu, Y.; Long, W.; Nguyen, C.T.; Wang, X.; et al. Contrastive Self-Refinement for Low-Cost Adaptation in Real-World Text-to-SQL **2026**.
119. Zha, J.; Wang, J.; Zhou, C.; Song, X. Trace2Policy: From Expert Behavior Traces to Self-Evolving Decision Agents. 2026.
120. Tao, W.; Wu, H.; Wong, W.F. SePO: Self-Evolving Prompt Agent for System Prompt Optimization, 2026, [arXiv:cs.CL/2606.04465].
121. Chen, X.; Xu, C.; Wang, Y.; Liu, B.; Yao, Z.; He, Y. Learning to Self-Evolve. *CoRR* **2026**, *abs/2603.18620*, [2603.18620]. https://doi.org/10.48550/ARXIV.2603.18620.

# 原文第 58 页 | 中文翻译初稿

58 共 70

99.斯皮斯，C.；瓦齐里，M.；曼德尔，L.； Hirzel, M. AutoPDL：LLM 智能体的自动提示优化。 CoRR 2025，abs/2504.04365，[2504.04365]。 https://doi.org/10.48550/ARXIV.2504.04365。

100.林，S.；华，W。李，L。王，Z。张，Y. ADO：LLM 提示中输入的自动数据优化。 CoRR 2025，abs/2502.11436，[2502.11436]。 https://doi.org/10.48550/ARXIV.2502.11436。

101.尚卡拉纳拉亚南，A.；文卡塔拉曼，A.N.；尼古拉科普洛斯。库马拉斯瓦米，V.S.；张，T。钱德，S.；萨布，R.R.； Khan, S.A. 一个跨 基础模型（Foundation Models） 的快速优化和翻译框架。

102.Yüksekgönül，M.；比安奇，F.；伯恩，J.；刘，S。黄，Z。盖斯特林，C.； Zou, J. TextGrad：通过文本自动“区分”。 CoRR 2024，abs/2406.07496，[2406.07496]。 https://doi.org/10.48550/ARXIV.2406.0 7496。

103.阿格拉沃尔，洛杉矶；谭，S。索伊卢，D.；齐姆斯，N.；卡雷，R.；奥普萨尔-翁，K.；辛格维，A.；尚迪利亚，H.；瑞安，M.J.；江，M。等人。 GEPA：反射提示进化可以优于 强化学习（Reinforcement Learning）。 ArXiv 2025，abs/2507.19457。

104.陈，M.；邓，W.；邹，J。于，H.； Li, X. 深度复合人工智能系统的文本平衡传播。 CoRR 2026，abs/2601.21064，[2601.21064]。 https://doi.org/10.48550/ARXIV.2601.21064。

105. 里沙夫，R.；普贾里，P.； Rastogi, P. ContraPrompt：通过二元推理跟踪分析进行对比提示优化。 CoRR 2026，abs/2604.17937，[2604.17937]。 https://doi.org/10.48550/ARXIV.2604.17937。

106.朱T.；姚，T。库瓦拉纳查罗恩，K.；辛格，A.；赖，Y。莫汉，D.A.； Bhargava, S. 基于图的目标反向传播，用于多 LLM 智能体系统中的上下文适应。 2026 年。

107.李W.；宋，Y。赵，M。金，B.； Li, W. 在基于 LLM 的多智能体提示优化中统一时间和结构 贡献归因（Credit Assignment）。 CoRR 2026，abs/2605.30227，[2605.30227]。 https://doi.org/10.48550/ARXIV.2605.30227。

108.徐W.；刘，S。 Wang, M. EEVEE：面向 自我改进（Self-Improving） 智能体在现实世界中进行测试时即时学习。 2026 年。

109.李H.；她。;张，Q。吉，C.；芒，Q。陈X.；阿格拉沃尔，洛杉矶；廖，W.；杨，E。张，A.；等人。 Combee：自我改进（Self-Improving） 语言模型智能体的扩展即时学习。 CoRR 2026，abs/2604.04247，[2604.04247]。 https://doi.org/10.48550/ARXIV.2604.04247。

110.苏兹贡，M.；于克塞克努尔，M.；比安奇，F.；尤拉夫斯基，D.； Zou, J. 动态备忘单：使用自适应记忆进行测试时学习。 CoRR 2025，abs/2504.07952，[2504.07952]。 https://doi.org/10.48550/ARXIV.2504.07952。

111.张Q.；胡，C.；乌帕萨尼，S.；马，B。洪，F。卡马努鲁，V.；雷顿，J.；吴，C.；吉，M。李，H。等人。智能体上下文工程：自我改进（Self-Improving） 语言模型的不断发展的上下文。 CoRR 2025，abs/2510.04618，[2510.04618]。 https://doi.org/10.48550/ARXIV.2510.04618。

112.裴Z.；珍，H.；凯，S。潘，S.J.；王，Y。袁，M.； Yu, B. 范围：增强智能体有效性的快速演变。 CoRR 2025，abs/2512.15374，[2512.15374]。 https://doi.org/10.48550/ARXIV.2512.15374。

113.瓦西里耶夫，N.；贝里奥斯，W.；张，R。汉，B.；基拉，D.； Mehri, S. 反思性上下文学习：研究上下文空间的优化原语。 CoRR 2026，abs/2604.03189，[2604.03189]

。 https://doi.org/10.485 50/ARXIV.2604.03189。

114.朱Z.；胡，Y。戴，Y。方，J。江，C.；胡，S。赵，Y。LLM 智能体的统一上下文演化。 2026 年。

115.帕拉沙尔，J.； Bhandarkar，S. KACE：数学推理的知识自适应上下文工程。 2026 年。

116.黄Z.；昆科罗，A.；冯，Q。沉，J。德里，L.M.；斯兹拉姆，A.； Ranzato, M. 主动信息搜索的情境训练。 CoRR 2026，abs/2605.13050，[2605.13050]。 https://doi.org/10.48550/ARXIV.2605.13050。

117.谢Y.；王，K。程，B.；姚，J。沙，Z。达菲，A.；习，Y。梅，H.；谭，C.；魏，C.；等人。备忘录：用于鲁棒多回合多智能体 LLM 游戏的记忆增强模型上下文优化。 CoRR 2026，abs/2603.09022，[2603.09022]。 https://doi.org/10.48550/ARXIV.2603.09022。

118.吴，Y.；长，W。阮，C.T.；王X；等人。现实世界文本到 SQL 2026 中低成本适应的对比自我完善。

119.查J.；王，J。周，C.； Song, X. Trace2Policy：从专家行为跟踪到自我进化决策智能体。 2026 年。

120.陶，W.；吴，H。黄W.F. SePO：用于系统提示优化的自我进化提示智能体，2026，[arXiv:cs.CL/2606.04465]。

121.陈X.；徐，C.；王，Y。刘，B.；姚，Z.；他，Y。学习自我进化。 CoRR 2026，abs/2603.18620，[2603.18620]。 https://doi.org/10.48550/ARXIV.2603.18620。

122. Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T.L.; Cao, Y.; Narasimhan, K. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *CoRR* **2023**, *abs/2305.10601*, [2305.10601]. https://doi.org/10.48550/ARXIV.2305.10601.
123. Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; Wang, Y. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. *CoRR* **2023**, *abs/2310.04406*, [2310.04406]. https://doi.org/10.48550/ARXIV.2310.04406.
124. Zhu, K.; Li, H.; Wu, S.; Xing, T.; Ma, D.; Tang, X.; Liu, M.; Yang, J.; Liu, J.; Jiang, Y.E.; et al. Scaling Test-time Compute for LLM Agents. *CoRR* **2025**, *abs/2506.12928*, [2506.12928]. https://doi.org/10.48550/ARXIV.2506.12928.
125. Antoniades, A.; Örwall, A.; Zhang, K.; Xie, Y.; Goyal, A.; Wang, W.Y. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. *CoRR* **2024**, *abs/2410.20285*, [2410.20285]. https://doi.org/10.48550/ARXIV.2410.20285.
126. Kim, J.; Yang, W.; Niu, K.; Zhang, H.; Zhu, Y.; Helenowski, E.; Silva, R.; Chen, Z.; Iyer, S.; Zaheer, M.; et al. Scaling Test-Time Compute for Agentic Coding. *CoRR* **2026**, *abs/2604.16529*, [2604.16529]. https://doi.org/10.48550/ARXIV.2604.16529.
127. Zeng, G.; Shen, M.; Chen, D.; Qi, Z.; Das, S.; Gutfreund, D.; Cox, D.; Wornell, G.W.; Lu, W.; Hong, Z.; et al. Satori-SWE: Evolutionary Test-Time Scaling for Sample-Efficient Software Engineering. *CoRR* **2025**, *abs/2505.23604*, [2505.23604]. https://doi.org/10.48550/ARXIV.2505.23604.
128. Lin, J.; Guo, Y.; Han, Y.; Hu, S.; Ni, Z.; Wang, L.; Chen, M.; Liu, H.; Chen, R.; He, Y.; et al. SE-Agent: Self-Evolution Trajectory Optimization in Multi-Step Reasoning with LLM-Based Agents. *CoRR* **2025**, *abs/2508.02085*, [2508.02085]. https://doi.org/10.48550/ARXIV.2508.02085.
129. Tan, D.Y.Y.; Chin, K.; Zhang, J. AgentGA: Evolving Code Solutions in Agent-Seed Space. *CoRR* **2026**, *abs/2604.14655*, [2604.14655]. https://doi.org/10.48550/ARXIV.2604.14655.
130. Fattha, A.D.; Chua, K.Y.; Jiang, L.; Wynter, L. Exploration Structure in LLM Agents for Multi-File Change Localization. 2026.
131. Zhang, S.; Wang, M.; Shi, Y.; Wang, Y.; Gu, X.; Yao, Y.; Fu, R.; Fu, S. FastContext: Training Efficient Repository Explorer for Coding Agents. 2026.
132. Han, H.; Xie, J.; Ma, X.; Zhu, W.; Zhang, Z.; Long, Z.; Chen, H.; Ye, Q. SWE-TRACE: Optimizing Long-Horizon SWE Agents Through Rubric Process Reward Models and Heuristic Test-Time Scaling. *CoRR* **2026**, *abs/2604.14820*, [2604.14820]. https://doi.org/10.48550/ARXIV.2604.14820.
133. Mao, C.; Lei, Y.; Wei, Z.; Liang, M.; Wang, Z.; Xu, J.; Chen, D.; Jiang, W.; Li, Y. EGSS: Entropy-guided Stepwise Scaling for Reliable Software Engineering. *CoRR* **2026**, *abs/2602.05242*, [2602.05242]. https://doi.org/10.48550/ARXIV.2602.05242.
134. Tan, B.; Deng, H.; Zhang, J.; Xu, J.; He, P.; Sun, Y. SWE-Manager: Selecting and Synthesizing Golden Proposals Before Coding. *CoRR* **2026**, *abs/2601.22956*, [2601.22956]. https://doi.org/10.48550/ARXIV.2601.22956.
135. Liu, M.; Chen, Z.; Pei, Z.; Wang, Z.; Wang, Y.; Zheng, Z. Architecture-Aware Multi-Design Generation for Repository-Level Feature Addition. *CoRR* **2026**, *abs/2603.01814*, [2603.01814]. https://doi.org/10.48550/ARXIV.2603.01814.
136. Ding, Y.; Zhang, L. SWE-Replay: Efficient Test-Time Scaling for Software Engineering Agents. *CoRR* **2026**, *abs/2601.22129*, [2601.22129]. https://doi.org/10.48550/ARXIV.2601.22129.
137. Zhang, T.; Popa, A.; Xu, Y.; Song, R.; Dimitriadis, D. PIVOT: Bridging Planning and Execution in LLM Agents via Trajectory Refinement. *CoRR* **2026**, *abs/2605.11225*, [2605.11225]. https://doi.org/10.48550/ARXIV.2605.11225.
138. Pan, R.; Wang, J.; Zhang, Q.; Zhu, Y.; Wu, L.; Yang, Z.; Zhang, Y.; Zhang, L.; Zhang, H. Persistent Cross-Attempt State Optimization for Repository-Level Code Generation. *CoRR* **2026**, *abs/2604.03632*, [2604.03632]. https://doi.org/10.48550/ARXIV.2604.03632.
139. Zhou, Z.; Cao, C.; Feng, X.; Li, X.; Li, Z.; Lu, X.; Yao, J.; Huang, W.; Cheng, T.; Zhang, J.; et al. AlphaApollo: A System for Deep Agentic Reasoning. 2025.
140. Chen, P.B.; Zhang, Y.; Roth, D.; Madden, S.; Andreas, J.; Cafarella, M.J. Log-Augmented Generation: Scaling Test-Time Reasoning with Reusable Computation. *CoRR* **2025**, *abs/2505.14398*, [2505.14398]. https://doi.org/10.48550/ARXIV.2505.14398.
141. Tran, Q.M.; Huang, Z.; Zhang, W.; Han, B.; Yatani, K.; Sugiyama, M.; Liu, T. Bifrost: Steering Strategic Trajectories to Bridge Contextual Gaps for Self-Improving Agents. *CoRR* **2026**, *abs/2602.05810*, [2602.05810]. https://doi.org/10.48550/ARXIV.2602.05810.

# 原文第 59 页 | 中文翻译初稿

59 共 70

122.姚S.；于，D。赵，J。沙夫兰，I。格里菲斯，T.L.；曹，Y。 Narasimhan, K. 思想之树：使用大型语言模型有意识地解决问题。 CoRR 2023，abs/2305.10601，[2305.10601]。 https://doi.org/10.48550/ARXIV.2305.10601。

123.周，A.；严K.；什拉彭托克-罗斯曼，M.；王，H。 Wang, Y. 语言智能体树搜索统一了语言模型中的推理、行动和规划。 CoRR 2023，abs/2310.04406，[2310.04406]。 https://doi.org/10.48550/ARXIV.2310.04406。

124.朱K.；李，H。吴，S。邢，T。疯狂的。;唐X.；刘，M.；杨，J。刘，J。江，Y.E.；等人。 LLM 智能体的扩展测试时计算。 CoRR 2025，abs/2506.12928，[2506.12928]。 https://doi.org/10.48550/ARXIV.2506 .12928。

125.安东尼亚德斯，A.；奥沃尔，A.；张，K。谢，Y。戈亚尔，A.；王 W.Y. SWE-Search：通过蒙特卡罗树搜索和迭代细化增强软件智能体。 CoRR 2024，abs/2410.20285，[2410.20285]。 https://doi.org/10.48550/ARXIV.2410.20285。

126.金，J.；杨，W。牛K.；张，H。朱，Y。海伦诺夫斯基，E.；席尔瓦，R.；陈，Z.；艾耶，S.；扎希尔，M.；等人。缩放智能体编码的测试时间计算。 CoRR 2026，abs/2604.16529，[2604.16529]。 https://doi.org/10.48550/ARXIV.2604.16529。

127.曾G.；沉，M。陈，D。齐，Z.；达斯，S。古特弗罗因德，D.；考克斯，D.；沃内尔，G.W.；卢，W。洪，Z。等人。 Satori-SWE：样本高效软件工程的进化测试时间缩放。 CoRR 2025，abs/2505.23604，[2505.23604]。 https://doi.org/10.48550/ARXIV.2505.23604。

128.林，J.；郭，Y。韩，Y。胡，S。尼，Z。王L.；陈，M。刘，H.；陈，R。嘿。;等人。 SE-Agent：基于 LLM 的智能体的多步推理中的自我进化轨迹优化。 CoRR 2025，abs/2508.02085，[2508.02085]。 https://doi.org/10.48550/ARXIV.2508.02085。

129. 谭，D.Y.Y.；钦，K。张，J. AgentGA：智能体种子空间中不断发展的代码解决方案。 CoRR 2026，abs/2604.14655，[2604.14655]。 https://doi.org/10.48550/ARXIV.2604.14655。

130.法塔，公元；蔡，K.Y.；蒋L.； Wynter, L. 用于多文件更改本地化的 LLM 智能体的探索结构。 2026 年。

131.张S.；王，M。石，Y。王，Y。顾X。姚，Y。毛皮。；Fu, S. FastContext：为编码智能体训练高效的存储库资源管理器。 2026 年。

132. 汉，H.；谢，J.；最大限度。;朱W.；张，Z。朗，Z.；陈，H。 Ye, Q. SWE-TRACE：通过 Rubric 过程奖励模型和启发式测试时间缩放优化 LongHorizon SWE 智能体。 CoRR 2026，abs/2604.14820，[2604.14820]。 https://doi.org/10.48550/ARXIV.2604.14820。

133.毛，C.；雷，Y。魏，Z.；梁，M。王，Z。徐，J。陈，D。姜W。 Li, Y. EGSS：用于可靠软件工程的熵引导逐步扩展。 CoRR 2026，abs/2602.05242，[2602.05242]。 https://doi.org/10.48550/ARXIV.2602.05242。

134.Tan，B.；邓，H.；张，J。徐，J。他，P。 Sun, Y. SWE 经理：编码前选择和综合黄金提案。 CoRR 2026，abs/2601.22956，[2601.22956]。 https://doi.org/10.48550/ARXIV.2601.22956。

135.刘，M.；陈，Z.；裴，Z.；王，Z。王，Y。郑 Z. 用于存储库级功能添加的架构感知多重设计生成。 CoRR 2026，abs/2603.01814，[2603.01814]

。 https://doi.org/10.48550/ARXIV.2603.01814。

136.丁，Y.；张 L. SWE-Replay：软件工程智能体的高效测试时间扩展。 CoRR 2026，abs/2601.22129，[2601.22129]。 https://doi.org/10.48550/ARXIV.2601.22129。

137.张T.；波帕，A.；徐，Y。宋，R。 Dimitriadis, D. PIVOT：通过轨迹细化在 LLM 智能体中桥接规划和执行。 CoRR 2026，abs/2605.11225，[2605.11225]。 https://doi.org/10.48550/ARXIV.2605 .11225。

138.潘，R.；王，J。张，Q。朱，Y。吴L.；杨，Z。张，Y。张L。张 H. 存储库级代码生成的持久交叉尝试状态优化。 CoRR 2026，abs/2604.03632，[2604.03632]。 https://doi.org/10.48550/ARXIV.2604.03632。

139.周Z.；曹，C.；冯X。李，X。李，Z。卢X。姚，J。黄W.；程，T。张，J。等人。 AlphaApollo：深度智能体推理系统。 2025 年。

140.陈，P.B.；张，Y。罗斯，D.；马登，S。安德烈亚斯，J.； Cafarella, M.J. 对数增强生成：通过可重用计算扩展测试时间推理。 CoRR 2025，abs/2505.14398，[2505.14398]。 https://doi.org/10.48550/ARXIV.2505.14398。

141.Tran，Q.M.；黄，Z。张，W。汉，B.；亚塔尼，K.；杉山，M.； Liu, T. Bifrost：引导战略轨迹以弥合 自我改进（Self-Improving） 智能体的上下文差距。 CoRR 2026，abs/2602.05810，[2602.05810]。 https://doi.org/10.48550/ARXIV.2602.05810。

142. Yang, X.; Zhou, J.; Pacheco, M.; Zhu, W.; He, P.; Wang, S.; Liu, K.; Pan, R. Lingxi: Repository-Level Issue Resolution Framework Enhanced by Procedural Knowledge Guided Scaling. *CoRR* **2025**, *abs/2510.11838*, [2510.11838]. https://doi.org/10.48550/ARXIV.2510.11838.
143. Ma, R.; Jiang, Y.; Zhang, S.; Ma, Z.; Feng, Y.; Ng, V.; Wang, Z.; Yue, X.; Li, C.; Lu, L. FailureMem: A Failure-Aware Multimodal Framework for Autonomous Software Repair. *CoRR* **2026**, *abs/2603.17826*, [2603.17826]. https://doi.org/10.48550/ARXIV.2603.17826.
144. Hu, H.; Xie, G.; Zhang, Q.; Liu, J.; Yu, S.; Fang, C.; Chen, Z.; Xiao, L. EvoRepair: Enhancing Vulnerability Repair Agents Through Experience-Based Self-Evolution. *CoRR* **2026**, *abs/2605.30105*, [2605.30105]. https://doi.org/10.48550/ARXIV.2605.30105.
145. Yu, S.; Chong, D.; Nandi, A.; Soylu, D.; Sun, J.; Manning, C.D.; Shi, W. Shepherd: A Runtime Substrate Empowering Meta-Agents with a Formalized Execution Trace. *CoRR* **2026**, *abs/2605.10913*, [2605.10913]. https://doi.org/10.48550/ARXIV.2605.10913.
146. Dong, Y.; He, J.; Hou, Y.; Du, D.; Xu, Z.; Yu, S.; Xia, Y.; Chen, H. DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback. *CoRR* **2026**, *abs/2605.22781*, [2605.22781]. https://doi.org/10.48550/ARXIV.2605.22781.
147. Zhang, X.; Wang, D.; Xu, K.; Zhu, Q.; Che, W. Scaling Laws for Agent Harnesses via Effective Feedback Compute. *CoRR* **2026**, *abs/2605.29682*, [2605.29682]. https://doi.org/10.48550/ARXIV.2605.29682.
148. Shang, Y.; Li, Y.; Zhao, K.; Ma, L.; Liu, J.; Xu, F.; Li, Y. AgentSquare: Automatic LLM Agent Search in Modular Design Space. *CoRR* **2024**, *abs/2410.06153*, [2410.06153]. https://doi.org/10.48550/ARXIV.2410.06153.
149. Li, Y.; Li, L.; Wu, Z.; Liao, Q.; Hao, J.; Shao, K.; Xu, F.; Li, Y. AgentSwift: Efficient LLM Agent Design via Value-guided Hierarchical Search. *CoRR* **2025**, *abs/2506.06017*, [2506.06017]. https://doi.org/10.48550/ARXIV.2506.06017.
150. Chen, J.; Shen, J.; Kang, H.; Hong, Z.; Jiang, Q.; Bose, S.; Zhang, Y.; Leng, L.; Vyas, A.K.; Mao, L.; et al. AgentSpec: Understanding Embodied Agent Scaffolds Through Controlled Composition. 2026.
151. Chen, G.; Dong, S.; Shu, Y.; Zhang, G.; Sesay, J.; Karlsson, B.F.; Fu, J.; Shi, Y. AutoAgents: A Framework for Automatic Agent Generation. *CoRR* **2023**, *abs/2309.17288*, [2309.17288]. https://doi.org/10.48550/ARXIV.2309.17288.
152. Yuan, S.; Song, K.; Chen, J.; Tan, X.; Li, D.; Yang, D. EvoAgent: Towards Automatic Multi-Agent Generation via Evolutionary Algorithms. *CoRR* **2024**, *abs/2406.14228*, [2406.14228]. https://doi.org/10.48550/ARXIV.2406.14228.
153. Hu, S.; Lu, C.; Clune, J. Automated Design of Agentic Systems. *CoRR* **2024**, *abs/2408.08435*, [2408.08435]. https://doi.org/10.48550/ARXIV.2408.08435.
154. Ye, R.; Tang, S.; Ge, R.; Du, Y.; Yin, Z.; Chen, S.; Shao, J. MAS-GPT: Training LLMs to Build LLM-based Multi-Agent Systems. *CoRR* **2025**, *abs/2503.03686*, [2503.03686]. https://doi.org/10.48550/ARXIV.2503.03686.
155. Xu, A.; Tai, Y. Meta-Agent: From Task Descriptions to Verified Multi-Agent Systems. *CoRR* **2026**, *abs/2605.25233*, [2605.25233]. https://doi.org/10.48550/ARXIV.2605.25233.
156. Chen, X.; Liu, Y.; Wei, H.; Ding, K. LEMON: Learning Executable Multi-Agent Orchestration via Counterfactual Reinforcement Learning. *CoRR* **2026**, *abs/2605.14483*, [2605.14483]. https://doi.org/10.48550/ARXIV.2605.14483.
157. Feng, Y.; Luo, H.; Lin, Z.; Sun, Y.; Wei, P.; Hsieh, L.B.; Luu, A.T. OrchMAS: Orchestrated Reasoning with Multi Collaborative Heterogeneous Scientific Expert Structured Agents. *CoRR* **2026**, *abs/2603.03005*, [2603.03005]. https://doi.org/10.48550/ARXIV.2603.03005.
158. Hu, Y.; Zhang, Y.; Trager, M.; Zhang, Y.E.; Yang, S.; Xia, W.; Soatto, S.. Evolutionary Generation of Multi-Agent Systems. *ArXiv* **2026**, *abs/2602.06511*.
159. Zhang, Y.; Xu, T.; Dai, S.; Shao, Z.; Wu, Q.; Wang, H. EVOCHAMBER: Test-Time Co-evolution of Multi-Agent System at Individual, Team, and Population Scales. *CoRR* **2026**, *abs/2605.11136*, [2605.11136]. https://doi.org/10.48550/ARXIV.2605.11136.
160. Zhang, G.; Niu, L.; Fang, J.; Wang, K.; Bai, L.; Wang, X. Multi-agent Architecture Search via Agentic Supernet. *CoRR* **2025**, *abs/2502.04180*, [2502.04180]. https://doi.org/10.48550/ARXIV.2502.04180.
161. Ma, B.; Li, H.; Hu, Z.; Gui, X.; Liu, L.; Liu, S. AutoMaAS: Self-Evolving Multi-Agent Architecture Search for Large Language Models. *CoRR* **2025**, *abs/2510.02669*, [2510.02669]. https://doi.org/10.48550/ARXIV.2510.02669.
162. Yao, T.; Li, Z.; Shen, Z. HieraMAS: Optimizing Intra-Node LLM Mixtures and Inter-Node Topology for Multi-Agent Systems. *CoRR* **2026**, *abs/2602.20229*, [2602.20229]. https://doi.org/10.48550/ARXIV.2602.20229.

# 原文第 60 页 | 中文翻译初稿

60 共 70

142.杨X.；周，J。帕切科，M.；朱W.；他，P。王，S。刘，K.； Pan, R. Lingxi：通过程序知识引导扩展增强存储库级问题解决框架。 CoRR 2025，abs/2510.11838，[2510.11838]。 https://doi.org/10.48550/ARXIV.2510.11838。

143.马，R.；蒋，Y。张，S。马，Z。冯，Y。吴，V。王，Z。岳，X。李，C。 Lu, L. FailureMem：用于自主软件修复的故障感知多模式框架。 CoRR 2026，abs/2603.17826，[2603.17826]。 https://doi.org/10.48550/ARXIV.2603.17826。

144.胡，H.；谢G.；张，Q。刘，J。于，S。方，C.；陈，Z.；肖，L. EvoRepair：通过基于经验的自我进化增强漏洞修复智能体。 CoRR 2026，abs/2605.30105，[2605.30105]。 https://doi.org/10.48550/ARXIV.2605.30105。

145.余S.；冲，D。南迪，A.；索伊卢，D.；孙，J。曼宁，C.D.； Shi, W. Shepherd：运行时基质为元智能体提供形式化的执行跟踪。 CoRR 2026，abs/2605.10913，[2605.10913]。 https://doi.org/10.48550/ARXIV.2605.10913。

146.董Y.；他，J。侯，Y。杜，D。徐，Z。于，S。夏，Y。 Chen, H. DeltaBox：使用毫秒级沙箱检查点/回滚扩展有状态的 AI 智能体。 CoRR 2026，abs/2605.22781，[2605.22781]。 https://doi.org/10.48550/ARXIV.2605.22781。

147.张X.；王，D。徐K.；朱Q。 Che, W. 通过有效反馈计算的智能体 运行支撑框架（Harnesses） 的缩放定律。 CoRR 2026，abs/2605.29682，[2605.29682]。 https://doi.org/10.48550/ARXIV.2605.29682。

148.尚，Y.；李，Y。赵，K。马，L。刘，J。徐，F。 Li, Y. AgentSquare：模块化设计空间中的自动 LLM 智能体搜索。 CoRR 2024，abs/2410.06153，[2410.06153]。 https://doi.org/10.48550/ARXIV.2410.06153。

149.李，Y.；李，L。吴，Z。廖，Q。郝，J。邵，K.；徐，F。 Li, Y. AgentSwift：通过价值引导的分层搜索进行高效的 LLM 智能体设计。 CoRR 2025，abs/2506.06017，[2506.06017]。 https://doi.org/10.48550/ARXIV.2506.06017。

150.陈，J.；沉，J。康，H.；洪，Z。姜Q。博斯，S。张，Y。冷，L.；维亚斯，A.K.；毛，L.；等人。 AgentSpec：通过受控组合了解具体智能体支架。 2026 年。

151.陈，G.；董，S。舒，Y。张，G。塞赛，J.；卡尔森，B.F.；傅，J。 Shi, Y. AutoAgents：自动智能体生成框架。 CoRR 2023，abs/2309.17288，[2309.17288]。 https://doi.org/10.48550/ARXIV.2309.17288。

152.袁S.；宋，K.；陈，J。谭，X。李，D。 Yang, D. EvoAgent：通过进化算法实现自动多智能体生成。 CoRR 2024，abs/2406.14228，[2406.14228]。 https://doi.org/10.48550/ARXIV.2406.14228。

153.胡S.；卢，C。 Clune, J. 智能体系统的自动化设计。 CoRR 2024，abs/2408.08435，[2408.08435]。 https://doi.org/10.48550/ARXIV.2408.08435。

154.叶，R.；唐，S。葛，R。杜，Y。尹，Z.；陈，S。 Shao, J. MAS-GPT：训练法学硕士构建基于法学硕士的多智能体系统。 CoRR 2025，abs/2503.03686，[2503.03686]。

https://doi.org/10.48550/ARXIV.2503.03686。

155.徐，A.； Tai, Y. 元智能体：从任务描述到经过验证的多智能体系统。 CoRR 2026，abs/2605.25233，[2605.25233]。 https://doi.org/10.48550/ARXIV.2605.25233。

156.陈X.；刘，Y。魏，H。丁坤

。 LEMON：通过 反事实（Counterfactual） 强化学习（Reinforcement Learning） 学习可执行的多智能体编排。 CoRR 2026，abs/2605.14483，[2605.14483]。 https://doi.org/10.48550/ARXIV.26 05.14483。

157.冯，Y.；罗，H.；林，Z。孙，Y。魏，P。谢，L.B.；卢，A.T. OrchMAS：利用多协作异构科学专家结构化智能体进行精心策划的推理。 CoRR 2026，abs/2603.03005，[2603.03005]。 https://doi.org/10.48550/ARXIV.2603.03005。

158.胡，Y.；张，Y。特拉格，M.；张，Y.E.；杨，S。夏，W。 Soatto, S. 多智能体系统的进化一代。 ArXiv 2026，abs/2602.06511。

159.张，Y.；徐，T。戴，S。邵，Z。吴，Q。 Wang, H. EVOCHAMBER：个人、团队和群体规模的多智能体系统的测试时协同进化。 CoRR 2026，abs/2605.11136，[2605.11136]。 https://doi.org/10.48550/ARXIV.2605.11136。

160.张，G.；牛L.；方，J。王，K。白，L.； Wang, X. 通过智能体超网进行多智能体架构搜索。 CoRR 2025，abs/2502.04180，[2502.04180]。 https://doi.org/10.48550/ARXIV.2502.04180。

161.马，B.；李，H。胡，Z。桂，X。刘，L。 Liu, S. AutoMaAS：大型语言模型的自进化多智能体架构搜索。 CoRR 2025，abs/2510.02669，[2510.02669]。 https://doi.org/10.48550/ARXIV.2510.0 2669。

162.姚，T.；李，Z。 Shen, Z. HieraMAS：优化 MultiAgent 系统的节点内 LLM 混合和节点间拓扑。 CoRR 2026，abs/2602.20229，[2602.20229]。 https://doi.org/10.48550/ARXIV.2602.20229。

163. Guo, J.; Xue, X.; Zhang, L.; Xu, W.; Chen, S.; Torr, P.; Ouyang, W.; Bai, L.; Yin, Z. SciOrch: Learning to Orchestrate Expert LLMs for Solving Frontier Multimodal Scientific Reasoning Tasks. 2026.
164. Fang, W.; Yuan, L.; Lan, G.; Han, D.; Brinton, C.G. Iterative Critique-and-Routing Controller for Multi-Agent Systems with Heterogeneous LLMs. *CoRR* **2026**, *abs/2605.08686*, [2605.08686]. https://doi.org/10.48550/ARXIV.2605.08686.
165. Ye, R.; Liu, X.; Wu, Q.; Pang, X.; Yin, Z.; Bai, L.; Chen, S. X-MAS: Towards Building Multi-Agent Systems with Heterogeneous LLMs. *CoRR* **2025**, *abs/2505.16997*, [2505.16997]. https://doi.org/10.48550/ARXIV.2505.16997.
166. Feng, Y.; Du, J.; Hong, Y.; Wang, Q.; Yu, L. PASS: Probabilistic Agentic Supernet Sampling for Interpretable and Adaptive Chest X-Ray Reasoning. *CoRR* **2025**, *abs/2508.10501*, [2508.10501]. https://doi.org/10.48550/ARXIV.2508.10501.
167. Su, J.; Xia, Y.; Lan, Q.; Song, X.; Chen, C.; Yang, J.; He, L.; Shi, T. Difficulty-Aware Agent Orchestration in LLM-Powered Workflows. *CoRR* **2025**, *abs/2509.11079*, [2509.11079]. https://doi.org/10.48550/ARXIV.2509.11079.
168. Zhuge, M.; Wang, W.; Kirsch, L.; Faccio, F.; Khizbullin, D.; Schmidhuber, J. Language Agents as Optimizable Graphs. *CoRR* **2024**, *abs/2402.16823*, [2402.16823]. https://doi.org/10.48550/ARXIV.2402.16823.
169. Li, Z.; Xu, S.; Mei, K.; Hua, W.; Rama, B.; Raheja, O.; Wang, H.; Zhu, H.; Zhang, Y. AutoFlow: Automated Workflow Generation for Large Language Model Agents. *CoRR* **2024**, *abs/2407.12821*, [2407.12821]. https://doi.org/10.48550/ARXIV.2407.12821.
170. Zhang, J.; Xiang, J.; Yu, Z.; Teng, F.; Chen, X.; Chen, J.; Zhuge, M.; Cheng, X.; Hong, S.; Wang, J.; et al. AFlow: Automating Agentic Workflow Generation. *CoRR* **2024**, *abs/2410.10762*, [2410.10762]. https://doi.org/10.48550/ARXIV.2410.10762.
171. Wang, Y.; Yang, L.; Li, G.; Wang, M.; Aragam, B. ScoreFlow: Mastering LLM Agent Workflows via Score-based Preference Optimization. *CoRR* **2025**, *abs/2502.04306*, [2502.04306]. https://doi.org/10.48550/ARXIV.2502.04306.
172. Niu, B.; Song, Y.; Lian, K.; Shen, Y.; Yao, Y.; Zhang, K.; Liu, T. Flow: A Modular Approach to Automated Agentic Workflow Generation. *CoRR* **2025**, *abs/2501.07834*, [2501.07834]. https://doi.org/10.48550/ARXIV.2501.07834.
173. Zheng, C.; Chen, J.; Lyu, Y.; Ng, W.Z.T.; Zhang, H.; Ong, Y.; Tsang, I.W.; Yin, H. MermaidFlow: Redefining Agentic Workflow Generation via Safety-Constrained Evolutionary Programming. *CoRR* **2025**, *abs/2505.22967*, [2505.22967]. https://doi.org/10.48550/ARXIV.2505.22967.
174. Wang, Y.; Liu, S.; Fang, J.; Meng, Z. EvoAgentX: An Automated Framework for Evolving Agentic Workflows. *CoRR* **2025**, *abs/2507.03616*, [2507.03616]. https://doi.org/10.48550/ARXIV.2507.03616.
175. Zhang, G.; Chen, K.; Wan, G.; Chang, H.; Cheng, H.; Wang, K.; Hu, S.; Bai, L. EvoFlow: Evolving Diverse Agentic Workflows On The Fly. *CoRR* **2025**, *abs/2502.07373*, [2502.07373]. https://doi.org/10.48550/ARXIV.2502.07373.
176. Liu, S.; Fang, J.; Zhou, H.; Wang, Y.; Meng, Z. SEW: Self-Evolving Agentic Workflows for Automated Code Generation. *CoRR* **2025**, *abs/2505.18646*, [2505.18646]. https://doi.org/10.48550/ARXIV.2505.18646.
177. Wei, Y.; Huang, Z.; Li, H.; Xing, W.W.; Lin, T.; He, L. VFlow: Discovering Optimal Agentic Workflows for Verilog Generation. *CoRR* **2025**, *abs/2504.03723*, [2504.03723]. https://doi.org/10.48550/ARXIV.2504.03723.
178. Hou, Z.; Tang, J.; Wang, Y. HALO: Hierarchical Autonomous Logic-Oriented Orchestration for Multi-Agent LLM Systems. *CoRR* **2025**, *abs/2505.13516*, [2505.13516]. https://doi.org/10.48550/ARXIV.2505.13516.
179. Xu, B.; Ye, Y.; Shen, C.; Zhou, Y.; Chen, C.; Chen, M. HyEvo: Self-Evolving Hybrid Agentic Workflows for Efficient Reasoning. *CoRR* **2026**, *abs/2603.19639*, [2603.19639]. https://doi.org/10.48550/ARXIV.2603.19639.
180. Zhu, R.; Jiang, B.; Mei, L.; Yang, F.; Wang, L.; Gao, H.; Bai, F.; Zhao, P.; Lin, Q.; Rajmohan, S.; et al. AdaptFlow: Adaptive Workflow Optimization via Meta-Learning. *CoRR* **2025**, *abs/2508.08053*, [2508.08053]. https://doi.org/10.48550/ARXIV.2508.08053.
181. Wang, J.; Xu, S.; Liu, H.; Wang, J.; Luo, Y.; Di, S.; Zhang, M.; Chen, L. Learning to Compose for Cross-domain Agentic Workflow Generation. *CoRR* **2026**, *abs/2602.11114*, [2602.11114]. https://doi.org/10.48550/ARXIV.2602.11114.
182. Yuan, B.; Zhou, Y.; Xu, Z.; Ramnath, K.; Feng, A.; Srinivasan, B. BayesFlow: A Probability Inference Framework for Meta-Agent Assisted Workflow Generation. *CoRR* **2026**, *abs/2601.22305*, [2601.22305]. https://doi.org/10.48550/ARXIV.2601.22305.
183. Kong, M.; Qu, Z.; Zhou, Z.; Liang, P.; Li, X.; Shang, Z.; Hong, Z.; Huang, K.; Wang, Z.; Dai, Z. Workflow-R1: Group Sub-sequence Policy Optimization for Multi-turn Workflow Construction. *CoRR* **2026**, *abs/2602.01202*, [2602.01202]. https://doi.org/10.48550/ARXIV.2602.01202.

# 原文第 61 页 | 中文翻译初稿

61 共 70

163.郭杰；薛，X。张L。徐，W。陈，S。托尔，P.；欧阳W.；白，L.； Yin, Z. SciOrch：学习协调专家法学硕士以解决前沿多模式科学推理任务。 2026 年。

164.方，W.；袁L.；兰，G。手。;布林顿，C.G.具有异构 LLM 的多智能体系统的迭代批评和路由控制器。 CoRR 2026，abs/2605.08686，[2605.08686]。 https://doi.org/10.48550/ARXIV.2605.08686。

165.叶，R.；刘X。吴，Q。庞X。尹，Z.；白，L.； Chen, S. X-MAS：利用异构法学硕士构建多智能体系统。 CoRR 2025，abs/2505.16997，[2505.16997]。 https://doi.org/10.48550/ARXIV.2505 .16997。

166.冯，Y.；杜，J。洪，Y。王Q。 Yu, L. PASS：用于可解释和自适应胸部 X 射线推理的概率智能体超网采样。 CoRR 2025，abs/2508.10501，[2508.10501]。 https://doi.org/10.48550/ARXIV.2508.10501。

167.苏，J.；夏，Y。兰，Q。宋X.；陈，C.；杨，J。他，L.； Shi, T. LLMPowered 工作流程中的难度感知智能体编排。 CoRR 2025，abs/2509.11079，[2509.11079]。 https://doi.org/10.48550/ARXIV.2509.11079。

168.诸葛，M.；王，W。基尔希，L.；法西奥，F.；希兹布林，D.； Schmidhuber, J. 语言智能体作为可优化图。 CoRR 2024，abs/2402.16823，[2402.16823]。 https://doi.org/10.48550/ARXIV.2402.16823。

169.李Z.；徐，S。梅，K.；华，W。拉玛，B.；拉赫贾，O.；王，H。朱，H.；张，Y。AutoFlow：大型语言模型智能体的自动工作流生成。 CoRR 2024，abs/2407.12821，[2407.12821]。 https://doi.org/10.48550/ARXIV.2407.12821。

170.张杰；向，J。于，Z。滕，F.；陈X.；陈，J。诸葛，M。程X；洪，S。王，J。等人。 AFlow：自动生成智能体工作流。 CoRR 2024，abs/2410.10762，[2410.10762]。 https://doi.org/10.48550/ARXIV.2410.10762。

171.王，Y.；杨，L。李，G。王，M。 Aragam, B. ScoreFlow：通过基于分数的偏好优化掌握 LLM 智能体工作流程。 CoRR 2025，abs/2502.04306，[2502.04306]。 https://doi.org/10.48550/ARXIV。 2502.04306。

172.牛B.；宋，Y。连，K.；沉，Y。姚，Y。张，K。 Liu, T. Flow：自动化智能体工作流生成的模块化方法。 CoRR 2025，abs/2501.07834，[2501.07834]。 https://doi.org/10.48550/ARXIV。 2501.07834。

173.郑，C.；陈，J。吕，Y。吴，W.Z.T.；张，H。翁，Y。曾I.W.； Yin, H. MermaidFlow：通过安全约束进化编程重新定义智能体工作流生成。 CoRR 2025，abs/2505.22967，[2505.22967]。 https://doi.org/10.48550/ARXIV.2505.22967。

174.王，Y.；刘，S。方，J。 Meng, Z. EvoAgentX：用于不断发展的智能体工作流程的自动化框架。 CoRR 2025，abs/2507.03616，[2507.03616]。 https://doi.org/10.48550/ARXIV.2507.03616。

175.张，G.；陈，K。万，G。张，H.；程，H。王，K。胡，S。 Bai, L. EvoFlow：动态发展多样化的智能体工作流程。 CoRR 2025，abs/2502.07373，[2502.07373]。 https://doi.org/10.48550/ARXIV。 2502.07373。

176.刘S.；方，J。周，H.；王，Y。 Meng, Z. SEW：用于自动代码生成的自我进化智能体工作流程。 CoRR 2025，abs/2505.18646，[2505.18646]。 https://doi.org/10.48550/ARXIV.2505.18646。

177.魏，Y.；黄Z.

;李，H。邢W.W.；林，T。 He, L. VFlow：发现 Verilog 生成的最佳智能体工作流程。 CoRR 2025，abs/2504.03723，[2504.03723]。 https://doi.org/10.48550/ARXIV.2504.03723。

178.侯Z.；唐，J。 Wang, Y. HALO：多智能体 LLM 系统的分层自治面向逻辑编排。 CoRR 2025，abs/2505.13516，[2505.13516]。 https://doi.org/10.48550/ARXIV.2505.13516。

179.徐，B.；叶，Y。沉，C.；周，Y。陈，C.； Chen, M. HyEvo：用于高效推理的自我进化混合智能体工作流程。 CoRR 2026，abs/2603.19639，[2603.19639]。 https://doi.org/10.48550/ARXIV.2603.19639。

180.朱R.；江，B.；梅，L.；杨，F。王L.；高，H。白，F。赵，P。林，Q。拉杰莫汉，S.；等人。 AdaptFlow：通过元学习进行自适应工作流程优化。 CoRR 2025，abs/2508.08053，[2508.08053]。 https://doi.org/10.48550/ARXIV.2508.08053。

181.王杰；徐，S。刘，H.；王，J。罗，Y。迪，S。张，M。 Chen, L. 学习构建跨域智能体工作流生成。 CoRR 2026，abs/2602.11114，[2602.11114]。 https://doi.org/10.48550/ARXIV。 2602.11114。

182.袁，B.；周，Y。徐，Z。拉姆纳斯，K.；冯，A.； Srinivasan, B. BayesFlow：元智能体辅助工作流生成的概率推理框架。 CoRR 2026，abs/2601.22305，[2601.22305]。 https://doi.org/10.48550/ARXIV.2601.22305。

183.孔，M.；曲，Z。周，Z.；梁，P。李，X。尚，Z。洪，Z。黄K.；王，Z。 Dai, Z. Workflow-R1：多轮工作流构建的组子序列策略优化。 CoRR 2026，abs/2602.01202，[2602.01202]。 https://doi.org/10.48550/ARXIV.2602.01202。

184. Ma, Z.; Zhao, Z.; Hua, C.; Berto, F.; Park, J. JudgeFlow: Agentic Workflow Optimization via Block Judge. *CoRR* **2026**, *abs/2601.07477*, [2601.07477]. https://doi.org/10.48550/ARXIV.2601.07477.
185. Xu, S.; Zhang, J.; Di, S.; Luo, Y.; Yao, L.; Liu, H.; Zhu, J.; Liu, F.; Zhang, M. RobustFlow: Towards Robust Agentic Workflow Generation. *CoRR* **2025**, *abs/2509.21834*, [2509.21834]. https://doi.org/10.48550/ARXIV.2509.21834.
186. Shi, X.; Zheng, M.; Lou, Q. Learning Latency-Aware Orchestration for Parallel Multi-Agent Systems. *CoRR* **2026**, *abs/2601.10560*, [2601.10560]. https://doi.org/10.48550/ARXIV.2601.10560.
187. Li, J.; Hong, Z.; Shen, M.; Zhang, Y.; Gan, C. FlowCompile: An Optimizing Compiler for Structured LLM Workflows. *CoRR* **2026**, *abs/2605.13647*, [2605.13647]. https://doi.org/10.48550/ARXIV.2605.13647.
188. Li, A.; Yang, S.; Chen, F.; Xu, T.; Li, P.; Su, Z. GraphFlow: A Graph-Based Workflow Management for Efficient LLM-Agent Serving. *CoRR* **2026**, *abs/2605.22566*, [2605.22566]. https://doi.org/10.48550/ARXIV.2605.22566.
189. Li, J.; Zhang, E.; Zhou, D.; Chen, E.; Yan, Y. Learning to Hand Off: Provably Convergent Workflow Learning under Interface Constraints. *CoRR* **2026**, *abs/2605.19140*, [2605.19140]. https://doi.org/10.48550/ARXIV.2605.19140.
190. Liu, S.; Li, M.; Fu, D.; Wang, H.P.; Xia, Y.; Li, H.; Yan, H.; Li, P. Towards Direct Latent-Space Synthesis for Parallel Branches in LLM-Agent Workflows. 2026.
191. Hu, Y.; Cai, Y.; Du, Y.; Zhu, X.; Liu, X.; Yu, Z.; Hou, Y.; Tang, S.; Chen, S. Self-Evolving Multi-Agent Collaboration Networks for Software Development. *CoRR* **2024**, *abs/2410.16946*, [2410.16946]. https://doi.org/10.48550/ARXIV.2410.16946.
192. Zhang, G.; Yue, Y.; Sun, X.; Wan, G.; Yu, M.; Fang, J.; Wang, K.; Cheng, D. G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks. *CoRR* **2024**, *abs/2410.11782*, [2410.11782]. https://doi.org/10.48550/ARXIV.2410.11782.
193. Zhang, G.; Yue, Y.; Li, Z.; Yun, S.; Wan, G.; Wang, K.; Cheng, D.; Yu, J.X.; Chen, T. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. *CoRR* **2024**, *abs/2410.02506*, [2410.02506]. https://doi.org/10.48550/ARXIV.2410.02506.
194. Li, B.; Zhao, Z.; Lee, D.; Wang, G. Adaptive Graph Pruning for Multi-Agent Communication. *CoRR* **2025**, *abs/2506.02951*, [2506.02951]. https://doi.org/10.48550/ARXIV.2506.02951.
195. Zhang, R.; Zhao, X.; Wang, R.; Chen, S.; Zhang, G.; Zhang, A.; Wang, K.; Wen, Q. SafeSieve: From Heuristics to Experience in Progressive Pruning for LLM-based Multi-Agent Communication. *CoRR* **2025**, *abs/2508.11733*, [2508.11733]. https://doi.org/10.48550/ARXIV.2508.11733.
196. Leong, H.Y.; Li, Y.; Wu, Y.; Ouyang, W.; Zhu, W.; Gao, J.; Han, W. AMAS: Adaptively Determining Communication Topology for LLM-based Multi-Agent System. *CoRR* **2025**, *abs/2510.01617*, [2510.01617]. https://doi.org/10.48550/ARXIV.2510.01617.
197. Jiang, E.H.; Wan, G.; Yin, S.; Li, M.; Wu, Y.; Liang, X.; Li, X.; Sun, Y.; Wang, W.; Chang, K.; et al. Dynamic Generation of Multi-LLM Agents Communication Topologies with Graph Diffusion Models. *CoRR* **2025**, *abs/2510.07799*, [2510.07799]. https://doi.org/10.48550/ARXIV.2510.07799.
198. Li, S.; Liu, Y.; Zheng, Y.; Li, M.; Nguyen, Q.V.H.; Pan, S. OFA-MAS: One-for-All Multi-Agent System Topology Design based on Mixture-of-Experts Graph Generative Models. *CoRR* **2026**, *abs/2601.12996*, [2601.12996]. https://doi.org/10.48550/ARXIV.2601.12996.
199. Wu, X.; Liu, X.; Lu, J.; Wang, S.; Qiu, X.; Shu, Y.; Hu, J.; Guo, C.; Yang, B. ST-EVO: Towards Generative Spatio-Temporal Evolution of Multi-Agent Communication Topologies. *CoRR* **2026**, *abs/2602.14681*, [2602.14681]. https://doi.org/10.48550/ARXIV.2602.14681.
200. Jiang, E.H.; Li, L.; Sun, R.; Liang, X.; Li, Y.; Wu, Y.; Luo, H.; Li, H.; Zhang, Z.; Kang, Z.; et al. Agent Q-Mix: Selecting the Right Action for LLM Multi-Agent Systems through Reinforcement Learning. *CoRR* **2026**, *abs/2604.00344*, [2604.00344]. https://doi.org/10.48550/ARXIV.2604.00344.
201. Zhang, Z.; Zhou, W.; Li, J.; Fei, H.; Wen, J.; Ji, W. RADAR: Redundancy-Aware Diffusion for Multi-Agent Communication Structure Generation. *CoRR* **2026**, *abs/2605.09907*, [2605.09907]. https://doi.org/10.48550/ARXIV.2605.09907.
202. Wu, X.; Lu, J.; Yan, S.; Qiu, X.; Hu, J.; Guo, C.; Yang, B. Differentiable Mixture-of-Agents Incentivizes Swarm Intelligence of Large Language Models. *CoRR* **2026**, *abs/2605.15706*, [2605.15706]. https://doi.org/10.48550/ARXIV.2605.15706.
203. Wang, S.; Lu, R.; Yang, Z.; Wang, Y.; Zhang, Y.; Xu, L.; Xu, Q.; Yin, G.; Chen, C.; Guan, X. AgentConductor: Topology Evolution for Multi-Agent Competition-Level Code Generation. *CoRR* **2026**, *abs/2602.17100*, [2602.17100]. https://doi.org/10.48550/ARXIV.2602.17100.

# 原文第 62 页 | 中文翻译初稿

62 共 70

184.马Z.；赵，Z。华，C.；贝尔托，F.； Park, J. JudgeFlow：通过 Block Judge 进行智能体工作流程优化。 CoRR 2026，abs/2601.07477，[2601.07477]。 https://doi.org/10.48550/ARXIV.2601.07477。

185.徐S.；张，J。迪，S。罗，Y。姚，L.；刘，H.；朱，J。刘，F。张，M。RobustFlow：迈向稳健的智能体工作流生成。 CoRR 2025，abs/2509.21834，[2509.21834]。 https://doi.org/10.48550/ARXIV。2509.21834。

186.施X.；郑，M。 Lou, Q. 学习并行多智能体系统的延迟感知编排。 CoRR 2026，abs/2601.10560，[2601.10560]。 https://doi.org/10.48550/ARXIV.2601.10560。

187.李，J.；洪，Z。沉，M。张，Y。 Gan, C. FlowCompile：结构化 LLM 工作流程的优化编译器。 CoRR 2026，abs/2605.13647，[2605.13647]。 https://doi.org/10.48550/ARXIV.2605.13647。

188.李，A.；杨，S。陈，F。徐，T。李，P。 Su, Z. GraphFlow：基于图形的工作流管理，用于高效的 LLM 智能体服务。 CoRR 2026，abs/2605.22566，[2605.22566]。 https://doi.org/10.48550/ARXIV.2605.22566。

189.李，J.；张，E。周，D。陈，E。 Yan, Y. 学习交接：接口约束下可证明收敛的工作流学习。 CoRR 2026，abs/2605.19140，[2605.19140]。 https://doi.org/10.48550/ARXIV.26 05.19140。

190.刘S.；李，M。傅，D。王H.P.；夏，Y。李，H。严，H。 Li, P. 面向 LLM-Agent 工作流程中并行分支的直接潜在空间合成。 2026 年。

191.胡，Y.；蔡，Y。杜，Y。朱X。刘X。于，Z。侯，Y。唐，S。 Chen, S. 用于软件开发的自我进化多智能体协作网络。 CoRR 2024，abs/2410.16946，[2410.16946]。 https://doi.org/10.48550/ARXIV.2410.16946。

192.张，G.；岳，Y。孙，X.；万，G。于，M。方，J。王，K。 Cheng, D. G-设计师：通过图神经网络构建多智能体通信拓扑。 CoRR 2024，abs/2410.11782，[2410.11782]。 https://doi.org/10.48550/ARXIV.2410.11782。

193.张，G.；岳，Y。李，Z。云，S。万，G。王，K。程，D。于J.X.； Chen, T. 废话少说：基于 LLM 的多智能体系统的经济通信管道。 CoRR 2024，abs/2410.02506，[2410.02506]。 https://doi.org/10.48550/ARXIV.2410.02506。

194.李，B.；赵，Z。李，D.； Wang, G. 用于多智能体通信的自适应图修剪。 CoRR 2025，abs/2506.02951，[2506.02951]。 https://doi.org/10.48550/ARXIV.2506.02951。

195.张R.；赵X。王，R。陈，S。张，G。张，A。王，K。 Wen, Q. SafeSieve：从启发式到基于 LLM 的多智能体通信的渐进修剪经验。 CoRR 2025，abs/2508.11733，[2508.11733]。 https://doi.org/10.48550/ARXIV.2508.11733。

196.Leong，H.Y.；李，Y。吴，Y。欧阳W.；朱W.；高，J。 Han, W. AMAS：自适应确定基于 LLM 的多智能体系统的通信拓扑。 CoRR 2025，abs/2510.01617，[2510.01617]。 https://doi.org/10.48550/ARXIV.2510.01617。

197.蒋，E.H.；万，G。尹，S。李，M。吴，Y。梁X.；李，X。孙，Y。王，W。张，K。等人。使用图扩散模型动态生成多 LLM 智能体通信拓扑。 CoRR 2025，abs/2510.07799，[2510.07799]。 https://doi.org/10.48550/ARXIV.2510.07799。

198.李S.；刘，Y.

;郑，Y。李，M。阮，Q.V.H.； Pan, S. OFA-MAS：基于专家混合图生成模型的一对多多智能体系统拓扑设计。 CoRR 2026，abs/2601.12996，[2601.12996]。 https://doi.org/10.48550/ARXIV.2601.12996。

199.吴X.；刘X。卢，J。王，S。邱X。舒，Y。胡，J。郭，C.； Yang, B. ST-EVO：迈向多智能体通信拓扑的生成时空演化。 CoRR 2026，abs/2602.14681，[2602.14681]。 https://doi.org/10.48550/ARXIV.2602.14681。

200.蒋，E.H.；李，L。孙，R。梁X.；李，Y。吴，Y。罗，H.；李，H。张，Z。康，Z.；等人。 Agent Q-Mix：通过 强化学习（Reinforcement Learning）为 LLM 多智能体系统选择正确的操作。 CoRR 2026，abs/2604.00344，[2604.00344]。 https://doi.org/10.48550/ARXIV.2604.00344。

201.张Z.；周，W。李，J。费，H.；文，J。 Ji, W. RADAR：用于多智能体通信结构生成的冗余感知扩散。 CoRR 2026，abs/2605.09907，[2605.09907]。 https://doi.org/10.48550/ARXIV.2605.09907。

202.吴X.；卢，J。严S.；邱X。胡，J。郭，C.； Yang, B. 可微的智能体混合激励大型语言模型的群体智能。 CoRR 2026，abs/2605.15706，[2605.15706]。 https://doi.org/10.48550/ARXIV.2605.15706。

203.王S.；卢，R。杨，Z。王，Y。张，Y。徐L.；徐Q。尹，G.；陈，C.； guan, X. AgentConductor：多智能体竞争级代码生成的拓扑演化。 CoRR 2026，abs/2602.17100，[2602.17100]。 https://doi.org/10.48550/ARXIV.2602.17100。

204. Xu, C.; Hu, Y.; Wang, R.; Lin, X.; Wang, W.; Liu, D.; Feng, F. TacoMAS: Test-Time Co-Evolution of Topology and Capability in LLM-based Multi-Agent Systems. *CoRR* **2026**, *abs/2605.09539*, [2605.09539]. https://doi.org/10.48550/ARXIV.2605.09539.
205. Zhang, T.; Zhou, Z.; Wan, J.; Hu, T.; Wang, C.; He, X.; Hong, R. Learning Transferable Topology Priors for Multi-Agent LLM Collaboration Across Domains. *CoRR* **2026**, *abs/2605.17359*, [2605.17359]. https://doi.org/10.48550/ARXIV.2605.17359.
206. Wang, X.; Wang, J.; Zhang, F.; Hu, Y.; Zhang, D.; Ye, Y.; Ban, Y.; Han, J.; Wang, R. MasFACT: Continual Multi-Agent Topology Learning via Geometry-Aware Posterior Transfer. *CoRR* **2026**, *abs/2605.17361*, [2605.17361]. https://doi.org/10.48550/ARXIV.2605.17361.
207. Gou, W.; Liu, Z. Dynamic Trust-Aware Sparse Communication Topology for LLM-Based Multi-Agent Consensus. 2026.
208. Talluri, A.; Anne, P.; Pendiyala, B.C.; Chilukuri, R. Retrieval-Conditioned Topology Selection with Provable Budget Conservation for Multi-Agent Code Generation. *CoRR* **2026**, *abs/2605.05657*, [2605.05657]. https://doi.org/10.48550/ARXIV.2605.05657.
209. Tastan, N.; Iacob, A.; Sani, L.; Kurmanji, M.; Lane, N.D.; Horvath, S.; Nandakumar, K. Response-Conditioned Parallel-to-Sequential Orchestration for Multi-Agent Systems. *CoRR* **2026**, *abs/2605.15573*, [2605.15573]. https://doi.org/10.48550/ARXIV.2605.15573.
210. Wang, D.; Yin, D.; Desai, R.; Li, L.; Celikyilmaz, A.; Ni, A. Learning to Interrupt in Language-based Multi-agent Communication. *CoRR* **2026**, *abs/2604.06452*, [2604.06452]. https://doi.org/10.48550/ARXIV.2604.06452.
211. Yu, Y.; Liu, H.; Jin, H.; Yuan, X.; Kuang, P.; Wang, H. Learning to Communicate: Toward End-to-End Optimization of Multi-Agent Language Systems. *CoRR* **2026**, *abs/2604.21794*, [2604.21794]. https://doi.org/10.48550/ARXIV.2604.21794.
212. Xu, T.; Wen, H.; Li, M. Adapting the Interface, Not the Model: Runtime Harness Adaptation for Deterministic LLM Agents. *CoRR* **2026**, *abs/2605.22166*, [2605.22166]. https://doi.org/10.48550/ARXIV.2605.22166.
213. Chen, M.; Wang, J.; Liu, Z.; Wang, Y.; Wang, Q. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. 2026.
214. Wei, C.; Gao, M.; Han, Z.; Chen, K.; Zhuang, Y.; Guan, H.; Zhang, Y.; Cheng, Y.; He, J.; Chen, H.; et al. The World Leaks the Future: Harness Evolution for Future Prediction Agents. *CoRR* **2026**, *abs/2604.15719*, [2604.15719]. https://doi.org/10.48550/ARXIV.2604.15719.
215. Zhang, H.; Zhang, S.; Li, K.; Zhang, C.; Chen, Y.; Zhang, Y.; Bai, L.; Hu, S. Self-Harness: Harnesses That Improve Themselves. 2026.
216. Kakade, A.; Srivastava, V.; Karande, S.S. Polaris: A Gödel Agent Framework for Small Language Models through Experience-Abstracted Policy Repair. *CoRR* **2026**, *abs/2603.23129*, [2603.23129]. https://doi.org/10.48550/ARXIV.2603.23129.
217. Lee, Y.; Nair, R.; Zhang, Q.; Lee, K.; Khattab, O.; Finn, C. Meta-Harness: End-to-End Optimization of Model Harnesses. *CoRR* **2026**, *abs/2603.28052*, [2603.28052]. https://doi.org/10.48550/ARXIV.2603.28052.
218. Liu, H.; Shou, C.; Liu, X.; Wen, H.; Chen, Y.; Fang, R.J.; Feng, Y. Synthesizing Multi-Agent Harnesses for Vulnerability Discovery. *CoRR* **2026**, *abs/2604.20801*, [2604.20801]. https://doi.org/10.48550/ARXIV.2604.20801.
219. Zhang, D. AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering. *CoRR* **2026**, *abs/2601.04620*, [2601.04620]. https://doi.org/10.48550/ARXIV.2601.04620.
220. Nanda, R.; Maddila, C.; Jha, S.; Khan, E.M.; Paltenghi, M.; Chandra, S. Wink: Recovering from Misbehaviors in Coding Agents. *CoRR* **2026**, *abs/2602.17037*, [2602.17037]. https://doi.org/10.48550/ARXIV.2602.17037.
221. Mulian, H.; Zeltyn, S.; Levy, I.; Galanti, L.; Yaeli, A.; Shlomov, S. AgentFixer: From Failure Detection to Fix Recommendations in LLM Agentic Systems. *CoRR* **2026**, *abs/2603.29848*, [2603.29848]. https://doi.org/10.48550/ARXIV.2603.29848.
222. Bonagiri, A.; Borkar, D.; Anderias, G.J.; Rafatirad, S.; Homayoun, H. CausalFlow: Causal Attribution and Counterfactual Repair for LLM Agent Failures. *CoRR* **2026**, *abs/2605.25338*, [2605.25338]. https://doi.org/10.48550/ARXIV.2605.25338.
223. Zhang, G.; Wang, J.; Chen, J.; Zhou, W.; Wang, K.; Yan, S. AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems? *CoRR* **2025**, *abs/2509.03312*, [2509.03312]. https://doi.org/10.48550/ARXIV.2509.03312.
224. Zhang, S.; Yin, M.; Zhang, J.; Liu, J.; Han, Z.; Zhang, J.; Li, B.; Wang, C.; Wang, H.; Chen, Y.; et al. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. *CoRR* **2025**, *abs/2505.00212*, [2505.00212]. https://doi.org/10.48550/ARXIV.2505.00212.

# 原文第 63 页 | 中文翻译初稿

63 共 70

204.徐，C.；胡，Y。王，R。林，X。王，W。刘，D.； Feng, F. TacoMAS：基于 LLM 的多智能体系统中拓扑和功能的测试时协同演化。 CoRR 2026，abs/2605.09539，[2605.09539]。 https://doi.org/10.48550/ARXIV.2605.09539。

205.张T.；周，Z.；万，J。胡，T。王，C.；他，X。 Hong, R. 学习可转移拓扑先验以实现跨领域的多智能体 LLM 协作。 CoRR 2026，abs/2605.17359，[2605.17359]。 https://doi.org/10.48550/ARXIV.2605.17359。

206.王X.；王，J。张，F。胡，Y。张，D。叶，Y。班，Y。韩，J。 Wang, R. MasFACT：通过几何感知后向迁移进行持续多智能体拓扑学习。 CoRR 2026，abs/2605.17361，[2605.17361]。 https://doi.org/10.48550/ARXIV.2605.17361。

207.郭，W.； Liu, Z. 基于 LLM 的多智能体共识的动态信任感知稀疏通信拓扑。 2026 年。

208.塔鲁里，A.；安妮，P。不列颠哥伦比亚省彭迪亚拉； Chilukuri, R. 用于多智能体代码生成的具有可证明预算守恒的检索条件拓扑选择。 CoRR 2026，abs/2605.05657，[2605.05657]。 https://doi.org/10.48550/ARXIV.2605.05657。

209.塔斯坦，N.；雅各布，A.；萨尼，L.；库尔曼吉，M.；北达科他州莱恩；霍瓦斯，S.； Nandakumar, K. 多智能体系统的响应条件并行到顺序编排。 CoRR 2026，abs/2605.15573，[2605.15573]。 https://doi.org/10.48550/ARXIV.2605.15573。

210.王D.；尹，D。德赛，R.；李，L。塞利基尔马兹，A.； Ni, A. 学习中断基于语言的多主体通信。 CoRR 2026，abs/2604.06452，[2604.06452]。 https://doi.org/10.48550/ARXIV.26 04.06452。

211.余，Y.；刘，H.；金，H。袁X.；匡，P。 Wang, H. 学习沟通：走向多智能体语言系统的端到端优化。 CoRR 2026，abs/2604.21794，[2604.21794]。 https://doi.org/10.48550/ARXIV.2604.21794。

212.徐T.；温，H。 Li, M. 调整接口，而不是模型：确定性 LLM 智能体的运行时 运行支撑框架（Harness） 调整。 CoRR 2026，abs/2605.22166，[2605.22166]。 https://doi.org/10.48550/ARXIV.2605.22166。

213.陈，M.；王，J。刘，Z.；王，Y。 Wang, Q. 从失败的轨迹到可靠的 LLM 智能体：诊断和修复 运行支撑框架（Harness） 缺陷。 2026 年。

214.魏，C.；高，M。汉，Z。陈，K。庄，Y。关，H。张，Y。程，Y。他，J。陈，H。等人。世界泄露未来：运行支撑框架（Harness） 未来预测智能体的演变。 CoRR 2026，abs/2604.15719，[2604.15719]。 https://doi.org/10.48550/ARXIV.2604.15719。

215.张H.；张，S。李，K。张，C.；陈，Y。张，Y。白，L.； Hu, S. 自我 运行支撑框架（Harness）：自我提升的 运行支撑框架（Harnesses）。 2026 年。

216.卡卡德，A.；斯里瓦斯塔瓦，V.； Karande，S.S. Polaris：通过经验抽象策略修复实现小语言模型的哥德尔智能体框架。 CoRR 2026，abs/2603.23129，[2603.23129]。 https://doi.org/10.48550/ARXIV.2603.23129。

217.李，Y.；奈尔，R.；张，Q。韭葱。;哈塔布，O.； Finn, C. Meta-运行支撑框架（Harness）：模型运行支撑框架（Harnesses） 的端到端优化。 CoRR 2026，abs/2603.28052，[2603.28052]。

https://doi.org/10.48550/ARXIV.2603.28052。

218.刘H.；寿，C.；刘X。温，H。陈，Y。方，R.J.； Feng, Y. 综合多智能体 运行支撑框架（Harnesses） 用于漏洞发现

。 CoRR 2026，abs/2604.20801，[2604.20801]。 https://doi.org/10.48550/ARXIV.2604.2 0801。

219.Zhang, D. AgentDevel：将自我进化的 LLM 智能体重新构建为发布工程。 CoRR 2026，abs/2601.04620，[2601.04620]。 https://doi.org/10.48550/ARXIV.2601.04620。

220.南达，R.；玛迪拉，C.；贾，S.；汗，E.M.；帕尔滕吉，M.； Chandra, S. Wink：从编码智能体的不当行为中恢复。 CoRR 2026，abs/2602.17037，[2602.17037]。 https://doi.org/10.48550/ARXIV.2602.17037。

221.木莲，H.；泽尔廷，S.；利维，I。加兰蒂，L.；耶利，A.； Shlomov, S. AgentFixer：从故障检测到 LLM 智能体系统中的修复建议。 CoRR 2026，abs/2603.29848，[2603.29848]。 https://doi.org/10.48550/ARXIV.2603.29848。

222.博纳吉里，A.；博卡，D.；安德烈亚斯，G.J.；拉法蒂拉德，S.； Homayoun, H. CausalFlow：LLM 智能体故障的因果归因和 反事实（Counterfactual） 修复。 CoRR 2026，abs/2605.25338，[2605.25338]。 https://doi.org/10.48550/ARXIV.2605.25338。

223.张，G.；王，J。陈，J。周，W。王，K。 Yan, S. AgenTracer：谁在导致 LLM 智能体系统失败？ CoRR 2025，abs/2509.03312，[2509.03312]。 https://doi.org/10.48550/ARXIV.2509.03312。

224.张S.；尹，M.；张，J。刘，J。汉，Z。张，J。李，B。王，C.；王，H。陈，Y。等人。哪个智能体导致任务失败以及何时？关于LLM多智能体系统的自动故障归因。 CoRR 2025，abs/2505.00212，[2505.00212]。 https://doi.org/10.48550/ARXIV.2505.00212。

225. Zelikman, E.; Lorch, E.; Mackey, L.; Kalai, A.T. Self-Taught Optimizer (STOP): Recursively Self-Improving Code Generation. *CoRR* **2023**, *abs/2310.02304*, [2310.02304]. https://doi.org/10.48550/ARXIV.2310.02304.
226. Yin, X.; Wang, X.; Pan, L.; Lin, L.; Wan, X.; Wang, W.Y. Gödel Agent: A Self-Referential Agent Framework for Recursive Self-Improvement. In Proceedings of the Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, 2025.
227. Robeyns, M.; Szummer, M.; Aitchison, L. A Self-Improving Coding Agent. *CoRR* **2025**, *abs/2504.15228*, [2504.15228]. https://doi.org/10.48550/ARXIV.2504.15228.
228. Cai, Q.; Zhang, Y.; Jia, X.; Zheng, H.; Xue, W.; Song, J.; Tian, X.; Guo, Y. MOSS: Self-Evolution through Source-Level Rewriting in Autonomous Agent Systems. *CoRR* **2026**, *abs/2605.22794*, [2605.22794]. https://doi.org/10.48550/ARXIV.2605.22794.
229. Robol, M.; Giorgini, P. Self-Evolving Software Agents. *CoRR* **2026**, *abs/2604.27264*, [2604.27264]. https://doi.org/10.48550/ARXIV.2604.27264.
230. Che, L.; Yang, Y.; Lin, P.; Wang, C.; Wang, X.; Su, J. DemoEvolve: Overcoming Sparse Feedback in Agentic Harness Evolution with Demonstrations. *CoRR* **2026**, *abs/2605.24539*, [2605.24539]. https://doi.org/10.48550/ARXIV.2605.24539.
231. Ho, M.; Liu, B.; Chen, J.; Wang, A.X.; Qin, L. SIGA: Self-Evolving Coding-Agent Adapters for Scientific Simulation. 2026.
232. Wang, R.; Huang, J.; Wang, P.; Liu, X.; Kong, L.; Zhang, T. Lean4Agent: Formal Modeling and Verification for Agent Workflow and Trajectory. 2026.
233. Zhang, S.; Yuan, C.; Guo, R.; Yu, X.; Xu, R.; Chen, Z.; Li, Z.; Yang, Z.; Guan, S.; Tang, Z.; et al. EvoFSM: Controllable Self-Evolution for Deep Research with Finite State Machines. *CoRR* **2026**, *abs/2601.09465*, [2601.09465]. https://doi.org/10.48550/ARXIV.2601.09465.
234. Zhang, J.; Hu, S.; Lu, C.; Lange, R.T.; Clune, J. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. In Proceedings of the The Fourteenth International Conference on Learning Representations, 2026.
235. Wang, W.; Piekos, P.; Li, N.; Laakom, F.; Chen, Y.; Ostaszewski, M.; Zhuge, M.; Schmidhuber, J. Huxley-Gödel Machine: Human-Level Coding Agent Development by an Approximation of the Optimal Self-Improving Machine. *CoRR* **2025**, *abs/2510.21614*, [2510.21614]. https://doi.org/10.48550/ARXIV.2510.21614.
236. Weng, Z.; Antoniades, A.; Nathani, D.; Zhang, Z.; Pu, X.; Wang, X.E. Group-Evolving Agents: Open-Ended Self-Improvement via Experience Sharing. *CoRR* **2026**, *abs/2602.04837*, [2602.04837]. https://doi.org/10.48550/ARXIV.2602.04837.
237. Zhang, J.; Gu, Y.; Ruan, J.; Song, M.; Peng, Y.; Han, Z.; Xiang, J.; Wang, Z.; Yang, C.; Ouyang, Y.; et al. Harnessing Agentic Evolution. *CoRR* **2026**, *abs/2605.13821*, [2605.13821]. https://doi.org/10.48550/ARXIV.2605.13821.
238. Qu, Y.; Lu, M. Bilevel Autoresearch: Meta-Autoresearching Itself. *CoRR* **2026**, *abs/2603.23420*, [2603.23420]. https://doi.org/10.48550/ARXIV.2603.23420.
239. Liu, Z.; Shi, Z.; Sang, Y.; He, B.; Lin, M.; Wei, T.; Wang, D.; Dumoulin, B.; Jin, W.; Lu, H. Adaptive Auto-Harness: Sustained Self-Improvement for Agentic System Deployment on Open-Ended Task Streams. 2026.
240. Liu, S.; Agarwal, S.; Maheswaran, M.; Cemri, M.; Li, Z.; Mang, Q.; Naren, A.; Boneh, E.; Cheng, A.; Pan, M.Z.; et al. EvoX: Meta-Evolution for Automated Discovery. *CoRR* **2026**, *abs/2602.23413*, [2602.23413]. https://doi.org/10.48550/ARXIV.2602.23413.
241. Wu, X.; Yin, S.; Kang, Y.; Zhang, X.; Xu, Q.; Chen, Z.; Zhang, W. SGM: A Statistical Godel Machine for Risk-Controlled Recursive Self-Modification. *CoRR* **2025**, *abs/2510.10232*, [2510.10232]. https://doi.org/10.48550/ARXIV.2510.10232.
242. Garralda-Barrio, M. Governed Evolution of Agent Runtimes through Executable Operational Cognition. *CoRR* **2026**, *abs/2605.27328*, [2605.27328]. https://doi.org/10.48550/ARXIV.2605.27328.
243. Hakim, S.B.; Guo, K.; Tan, W.; Velasquez, A.; Xu, S.; Song, H.H. ANNEAL: Adapting LLM Agents via Governed Symbolic Patch Learning. *CoRR* **2026**, *abs/2605.16309*, [2605.16309]. https://doi.org/10.48550/ARXIV.2605.16309.
244. Sahoo, S.; Chadha, A.; Jain, V.; Chaudhary, D. SAHOO: Safeguarded Alignment for High-Order Optimization Objectives in Recursive Self-Improvement. *CoRR* **2026**, *abs/2603.06333*, [2603.06333]. https://doi.org/10.48550/ARXIV.2603.06333.
245. Shi, D.; He, J.; Chen, J.; Wang, B.; Nakashima, Y. Towards Healthy Evolution: Exploring the Role and Mechanisms of Human-Agent Interaction in Self-Evolving Systems. 2026.

# 原文第 64 页 | 中文翻译初稿

64 共 70

225.泽利克曼，E.；洛奇，E.；麦基，L.；卡莱，A.T.自学优化器（STOP）：递归 自我改进（Self-Improving）代码生成。 CoRR 2023，abs/2310.02304，[2310.02304]。https://doi.org/10.48550/ARXIV.2310.02304。

226.尹X.；王X；潘，L.；林，L。万X；王 W.Y.哥德尔智能体：递归自我改进（Recursive Self-Improvement）的自指智能体框架。计算语言学协会第 63 届年会论文集，2025 年。

227.罗宾斯，M.；苏默，M.； Aitchison, L.A 自我改进（Self-Improving）编码剂。 CoRR 2025，abs/2504.15228，[2504.15228]。 https://doi.org/10.48550/ARXIV.2504.15228。

228.蔡Q.；张，Y。贾X.；郑，H。薛，W.；宋，J。田，X。郭，Y. MOSS：自主智能体系统中通过源代码级重写的自我进化。 CoRR 2026，abs/2605.22794，[2605.22794]。https://doi.org/10.48550/ARXIV.2605.22794。

229.罗博尔，M.； Giorgini, P. 自我进化的软件智能体。 CoRR 2026，abs/2604.27264，[2604.27264]。https://doi.org/10.48550/ARXIV.2604.27264。

230.车，L.；杨，Y。林，P。王，C.；王X； Su, J. DemoEvolve：通过演示克服 Agentic 运行支撑框架（Harness）进化中的稀疏反馈。 CoRR 2026，abs/2605.24539，[2605.24539]。https://doi.org/10.48550/ARXIV.2605.24539。

231.何，M.；刘，B.；陈，J。王A.X.； Xin, L. SIGA：用于科学模拟的自进化编码智能体适配器。 2026 年。

232.王R.；黄，J。王，P。刘X。孔，L.；张，T. Lean4Agent：智能体工作流程和轨迹的形式化建模和验证。 2026 年。

233.张S.；袁，C.；郭，R。于X。徐，R。陈，Z.；李，Z。杨，Z。关，S。唐，Z。等人。 EvoFSM：利用有限状态机进行深度研究的可控自进化。 CoRR 2026，abs/2601.09465，[2601.09465]。https://doi.org/10.48550/ARXIV.2601.09465。

234.张杰；胡，S。卢，C。兰格，R.T.； Clune, J. Darwin Gödel Machine：自我改进智能体的开放式进化。 2026 年第十四届国际学习表征会议论文集。

235.王W.；皮耶科斯，P.；李娜；拉科姆，F.；陈，Y。奥斯塔泽夫斯基，M.；诸葛，M。 Schmidhuber，J. Huxley-Gödel 机器：通过近似最佳 自我改进（Self-Improving）机器进行人类级编码智能体开发。 CoRR 2025，abs/2510.21614，[2510.21614]。 https://doi.org/10.48550/ARXIV.2510.21614。

236.翁Z.；安东尼亚德斯，A.；纳萨尼，D.；张，Z。濮，X。王X.E.团体进化智能体：通过经验分享开放式 自我改进（Self-Improvement）。 CoRR 2026，abs/2602.04837，[2602.04837]。https://doi.org/10.485 50/ARXIV.2602.04837。

237.张杰；顾，Y。阮，J.；宋，M.；彭，Y。汉，Z。向，J。王，Z。杨，C.；欧阳，Y。等人。利用智能体进化。 CoRR 2026，abs/2605.13821，[2605.13821]。 https://doi.org/10.48550/ARXIV。2605.13821。

238.曲，Y.； Lu, M. Bilevel Autoresearch：元自动研究本身。 CoRR 2026，abs/2603.23420，[2603.23420]。 https://doi.org/10.48550/ARXIV.2603.23420。

239.刘Z.；石，Z。桑，Y。他，B.；林，M。魏，T。王，D。杜穆兰，B.；金，W。 Lu, H. 自适应 Auto-运行支撑框架（Harness）：用于开放式任务流上的智能体系统部署的持续 自我改进（Self-Improvement）。 2026 年。

240.刘S.；阿加瓦尔，S.

;马赫斯瓦兰，M.；塞姆里，M.；李，Z。芒，Q。纳伦，A.；博内，E.；程，A.；潘，M.Z.；等人。 EvoX：自动发现的元进化。 CoRR 2026，abs/2602.23413，[2602.23413]。 https://doi.org/10.48550/ARXIV.2602.23413。

241.吴X.；尹，S。康，Y。张X；徐Q。陈，Z.；张，W. SGM：用于风险控制递归自我修改的统计哥德尔机。 CoRR 2025，abs/2510.10232，[2510.10232]。 https://doi.org/10.4 8550/ARXIV.2510.10232。

242. Garralda-Barrio, M。通过可执行的操作认知控制智能体运行时的演化。 CoRR 2026，abs/2605.27328，[2605.27328]。 https://doi.org/10.48550/ARXIV.2605.27328。

243.哈基姆，S.B.；郭，K.；谭，W。维拉斯奎兹，A.；徐，S。 Song，H.H. ANNEAL：通过受控符号补丁学习调整 LLM 智能体。 CoRR 2026，abs/2605.16309，[2605.16309]。 https://doi.org/10.48550/ARXIV.2605.16309。

244.萨胡，S.；查达，A.；贾恩，V.； Chaudhary, D. SAHOO：递归自我改进（Recursive Self-Improvement）中高阶优化目标的保障对齐。 CoRR 2026，abs/2603.06333，[2603.06333]。 https://doi.org/10.485 50/ARXIV.2603.06333。

245.石D.；他，J。陈，J。王，B.； Nakashima, Y. 迈向健康进化：探索自我进化系统中人与智能体交互的作用和机制。 2026 年。

246. Zala, A.; Cho, J.; Lin, H.; Yoon, J.; Bansal, M. EnvGen: Generating and Adapting Environments via LLMs for Training Embodied Agents, 2024, [arXiv:cs.CL/2403.12014].
247. Yeo, T.; Weerakoon, D.; Weerakoon, D.; Misra, A. Towards Adaptive Environment Generation for Training Embodied Agents, 2026, [arXiv:cs.RO/2602.06366].
248. Parker-Holder, J.; Jiang, M.; Dennis, M.; Samvelyan, M.; Foerster, J.; Grefenstette, E.; Rocktäschel, T. Evolving Curricula with Regret-Based Environment Design, 2022, [arXiv:cs.LG/2203.01302].
249. Garcin, S.; Doran, J.; Guo, S.; Lucas, C.G.; Albrecht, S.V. DRED: Zero-Shot Transfer in Reinforcement Learning via Data-Regularised Environment Design, 2024, [arXiv:cs.LG/2402.03479].
250. Kang, H.; Ye, X.; Liu, Y.; Mantri, S.H.; Mao, L.; Fleming, J.; Regmi, D.; Qin, L. SimWorld Studio: Automatic Environment Generation with Evolving Coding Agent for Embodied Agent Learning, 2026, [arXiv:cs.AI/2605.09423].
251. Chen, Z.; Zhao, Z.; Zhang, K.; Liu, B.; Qi, Q.; Wu, Y.; Kalluri, T.; Cao, S.; Xiong, Y.; Tong, H.; et al. Scaling Agent Learning via Experience Synthesis, 2025, [arXiv:cs.AI/2511.03773].
252. Qi, Z.; Liu, X.; Iong, I.L.; Lai, H.; Sun, X.; Zhao, W.; Yang, Y.; Yang, X.; Sun, J.; Yao, S.; et al. WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning, 2024, [arXiv:cs.CL/2411.02337].
253. Yang, S.; Ma, Z.; Huang, T.; Hu, Y.; Wang, Y.; Chu, X. CoEvolve: Training LLM Agents via Agent-Data Mutual Evolution. In Proceedings of the Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), San Diego, California, United States, 2026; pp. 23015–23036. https://doi.org/10.18653/v1/2026.acl-long.1055.
254. Fang, W.; Liu, S.; Zhou, Y.; Zhang, K.; Zheng, T.; Chen, K.; Song, M.; Tao, D. SeRL: Self-Play Reinforcement Learning for Large Language Models with Limited Data, 2026, [arXiv:cs.CL/2505.20347].
255. Huang, J.; Li, Z.; Hu, Y.; Zhang, Z.; Coates, M.; Quan, X.; Zhang, Y. Self-CriTeach: LLM Self-Teaching and Self-Critiquing for Improving Robotic Planning via Automated Domain Generation, 2026, [arXiv:cs.RO/2509.21543].
256. Amjith, S.; Wang, M.X.; Lynch, J.; Gundlach, H.; Thompson, N. SAGE: Self-play Adversarial Games Enhance Large Language Model Reasoning Capabilities.
257. Chen, Z.; Deng, Y.; Yuan, H.; Ji, K.; Gu, Q. Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models, 2024, [arXiv:cs.LG/2401.01335].
258. Luo, H.; Sun, Q.; Xu, C.; Zhao, P.; Lin, Q.; Lou, J.; Chen, S.; Tang, Y.; Chen, W. Arena Learning: Build Data Flywheel for LLMs Post-training via Simulated Chatbot Arena, 2024, [arXiv:cs.CL/2407.10627].
259. Zeng, Y.; Cui, X.; Jin, X.; Mi, Q.; Liu, G.; Sun, Z.; Yang, M.; Li, D.; Ma, W.; Yang, N.; et al. Evolving LLMs' Self-Refinement Capability via Synergistic Training-Inference Optimization, 2025, [arXiv:cs.CL/2502.05605].
260. Qu, Y.; Zhang, T.; Garg, N.; Kumar, A. Recursive Introspection: Teaching Language Model Agents How to Self-Improve, 2024, [arXiv:cs.LG/2407.18219].
261. Yang, H.; Le, K.; Hua, T.; Gao, S.; Xu, B.; Tang, Z.; Xu, J.; Chawla, N.V.; Jin, H.; Srinivasan, V. Dynamic Noise Preference Optimization: Self-Improvement of Large Language Models with Self-Synthetic Data, 2026, [arXiv:cs.CL/2502.05400].
262. Xiao, W.; Lin, H.; Peng, A.; Xue, H.; He, T.; Xie, Y.; Hu, F.; Wu, J.; Luo, Z.; Fan, L.J.; et al. Self-Improving Vision-Language-Action Models with Data Generation via Residual RL, 2025, [arXiv:cs.CV/2511.00091].
263. Lin, I.W.; Hu, Y.; Li, S.S.; Geng, S.; Koh, P.W.; Zettlemoyer, L.; Althoff, T.; Ghazvininejad, M. Self-Improving VLM Judges Without Human Annotations, 2025, [arXiv:cs.CV/2512.05145].
264. Tang, C.; Huang, H.Y.; Liu, W.; Zheng, J.; Yang, S.; Wu, Y. Democratizing Tool Learning with Environments Fully Simulated by a Free 8B Language Model. *arXiv preprint arXiv:2604.17739* **2026**.
265. Cheng, Y.; Wang, Z.; Ma, W.; Zhu, W.; Deng, Y.; Zhao, J. EvoCurr: Self-evolving Curriculum with Behavior Code Generation for Complex Decision-making, 2025, [arXiv:cs.AI/2508.09586].
266. Zhang, Q.; Ruan, S.; Upasani, S.; Hong, F.; Ji, C.; Hu, C.; Li, B.; Li, H.; Olukotun, K. Learning What to Learn: Curriculum Curation for Test-Time Agent Learning. In Proceedings of the ICLR 2026 Workshop on AI with Recursive Self-Improvement (RSI), 2026.
267. Bukkapatnam, K.; Lala, A.; Patel, L. Adaptive Meta-Curriculum for Test-Time Self-Improvement. In Proceedings of the ICLR 2026 Workshop on AI with Recursive Self-Improvement (RSI), 2026.
268. Gu, Z.; Light, J.; Astudillo, R.; Ye, Z.; He, L.; Zou, H.P.; Cheng, W.; Paternain, S.; Yu, P.S.; Yue, Y. Actor-Curator: Co-adaptive Curriculum Learning via Policy-Improvement Bandits for RL Post-Training, 2026, [arXiv:cs.LG/2602.20532].

# 原文第 65 页 | 中文翻译初稿

65 共 70

246.扎拉，A.；曹，J。林，H。尹，J。 Bansal, M. EnvGen：通过法学硕士生成和调整环境以训练具体智能体，2024 年，[arXiv:cs.CL/2403.12014]。

247.杨，T.；威拉孔，D.；威拉孔，D.； Misra, A. 为训练实体智能体构建自适应环境，2026 年，[arXiv:cs.RO/2602.06366]。

248.帕克-霍尔德，J.；江，M。丹尼斯，M.；萨姆维利安，M.；福斯特，J.；格雷芬斯特特，E.；Rocktäschel, T. 基于遗憾的环境设计不断发展的课程，2022 年，[arXiv:cs.LG/2203.01302]。

249.加尔辛，S.；多兰，J.；郭，S。卢卡斯，C.G.；阿尔布雷希特，S.V. DRED：通过数据规范化环境设计在 强化学习（Reinforcement Learning） 中实现零样本传输，2024 年，[arXiv:cs.LG/2402.03479]。

250.康，H.；叶，X。刘，Y。曼特里，S.H.；毛，L.；弗莱明，J.；雷格米，D.；秦 L. SimWorld Studio：使用进化编码智能体自动生成环境以实现具体智能体学习，2026，[arXiv:cs.AI/2605.09423]。

251.陈Z.；赵，Z。张，K。刘，B.；齐，Q。吴，Y。卡鲁里，T.；曹S.；熊，Y。童，H.；等人。通过经验合成扩展智能体学习，2025，[arXiv:cs.AI/2511.03773]。

252.齐，Z.；刘X。 Iong，I.L.；赖，H。孙，X.；赵W。杨，Y。杨X。孙，J。姚，S。等人。 WebRL：通过自我进化在线课程训练 LLM Web 智能体 强化学习（Reinforcement Learning），2024，[arXiv:cs.CL/2411.02337]。

253.杨S.；马，Z。黄，T。胡，Y。王，Y。 Chu, X. CoEvolve：通过智能体数据相互进化训练 LLM 智能体。计算语言学协会第 64 届年会论文集（第一卷：长论文），美国加利福尼亚州圣地亚哥，2026 年；第 23015-23036 页。 https://doi.org/10.18653/v1/2026.acl-long.1055。

254.方，W.；刘，S。周，Y。张，K。郑，T。陈，K。宋，M.； Tao, D. SeRL：针对数据有限的大型语言模型的自玩 强化学习（Reinforcement Learning），2026，[arXiv:cs.CL/2505.20347]。

255.黄，J.；李，Z。胡，Y。张，Z。科茨，M.；全，X.；张，Y. Self-CriTeach：法学硕士自我教学和自我批评，通过自动域生成改进机器人规划，2026，[arXiv:cs.RO/2509.21543]。

256.阿姆吉斯，S.；王，M.X.；林奇，J.；冈拉克，H.； Thompson, N. SAGE：自玩对抗游戏增强大型语言模型推理能力。

257.陈Z.；邓，Y。袁，H.；吉，K。 Gu, Q. 自玩微调将弱语言模型转换为强语言模型，2024，[arXiv:cs.LG/2401.01335]。

258.罗H.；孙，Q。徐，C.；赵，P。林，Q。卢，J。陈，S。唐，Y。 Chen, W. Arena Learning：通过模拟聊天机器人 Arena 为法学硕士后期训练构建数据飞轮，2024 年，[arXiv:cs.CL/2407.10627]。

259.曾，Y.；崔X；金，X。米，Q。刘，G.；孙Z.；杨，M。李，D。马，W。杨，N。等人。通过协同训练推理优化发展法学硕士的自我完善能力，2025，[arXiv:cs.CL/2502.05605]。

260.曲，Y.；张，T。加尔格，N.； Kumar, A. 递归内省：教授语言模型智能体如何自我改进，2024 年，[arXiv:cs.LG/2407.18219]。

261.杨H.；勒，K。华，T.；高，S。徐，B。唐，Z。徐，J。内华达州乔拉；金，H。斯里尼瓦桑，V

。动态噪声偏好优化：具有自合成数据的大型语言模型的 自我改进（Self-Improvement），2026，[arXiv:cs.CL/2502.05400]。

262.肖W.；林，H。彭，A.；薛，H.；他，T。谢，Y。胡，F。吴，J。罗，Z.；范L.J.；等人。通过残差 RL 生成数据的 自我改进（Self-Improving）视觉语言动作（Vision-Language-Action）模型，2025，[arXiv:cs.CV/2511.00091]。

263.林，I.W.；胡，Y。李，S.S.；耿S.； Koh，P.W.；泽特尔莫耶，L.；阿尔托夫，T.； Ghazvininejad, M. 自我改进（Self-Improving）VLM 没有人工注释的判断，2025，[arXiv:cs.CV/2512.05145]。

264.唐，C.；黄H.Y.；刘，W.；郑，J。杨，S。 Wu, Y。通过免费 8B 语言模型完全模拟的环境使工具学习民主化。 arXiv 预印本 arXiv:2604.17739 2026 。

265.程，Y.；王，Z。马，W。朱W.；邓，Y。赵，J. EvoCurr：用于复杂决策的行为代码生成的自我进化课程，2025，[arXiv:cs.AI/2508.09586]。

266.张Q.；阮S.；乌帕萨尼，S.；洪，F。吉，C.；胡，C.；李，B。李，H。 Olukotun, K.《学习该学什么：测试时智能体学习的课程策划》。 ICLR 2026 人工智能研讨会论文集，递归自我改进（Recursive Self-Improvement）(RSI)，2026 年。

267.布卡帕特南，K.；拉拉，A.； Patel, L. 测试时自适应元课程 自我改进（Self-Improvement）。 ICLR 2026 人工智能研讨会论文集，递归自我改进（Recursive Self-Improvement）(RSI)，2026 年。

268.顾Z.；光，J.；阿斯图迪略，R.；叶，Z。他，L.；邹，H.P.；郑，W.；帕特南，S.；余，附注；Yue, Y. ActorCurator：通过策略改进 Bandits 进行 RL 后训练的自适应课程学习，2026 年，[arXiv:cs.LG/2602.20532]。

269. Liu, J.; Xiong, K.; Xia, P.; Zhou, Y.; Ji, H.; Feng, L.; Han, S.; Ding, M.; Yao, H. Agent0-VL: Exploring Self-Evolving Agent for Tool-Integrated Vision-Language Reasoning, 2025, [arXiv:cs.CV/2511.19900].
270. Huang, Y.; Yu, X.; Wei, Z. ACE: Self-Evolving LLM Coding Framework via Adversarial Unit Test Generation and Preference Optimization, 2026, [arXiv:cs.SE/2605.16299].
271. Dong, G.; Lu, J.; Huang, J.; Zhong, W.; Liu, L.; Huang, S.; Li, Z.; Zhao, Y.; Song, X.; Li, X.; et al. Agent-World: Scaling Real-World Environment Synthesis for Evolving General Agent Intelligence, 2026, [arXiv:cs.AI/2604.18292].
272. Xia, P.; Zeng, K.; Liu, J.; Qin, C.; Wu, F.; Zhou, Y.; Xiong, C.; Yao, H. Agent0: Unleashing Self-Evolving Agents from Zero Data via Tool-Integrated Reasoning, 2025, [arXiv:cs.LG/2511.16043].
273. Huang, C.; Yu, W.; Wang, X.; Zhang, H.; Li, Z.; Li, R.; Huang, J.; Mi, H.; Yu, D. R-Zero: Self-Evolving Reasoning LLM from Zero Data, 2025, [arXiv:cs.LG/2508.05004].
274. Sundaram, S.; Quan, J.; Kwiatkowski, A.; Ahuja, K.; Ollivier, Y.; Kempe, J. Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability, 2026, [arXiv:cs.LG/2601.18778].
275. Jana, S.; Sancaktar, C.; Daniš, T.; Martius, G.; Orvieto, A.; Kolev, P. GASP: Guided Asymmetric Self-Play For Coding LLMs, 2026, [arXiv:cs.LG/2603.15957].
276. Shao, R.; Asai, A.; Shen, S.Z.; Ivison, H.; Kishore, V.; Zhuo, J.; Zhao, X.; Park, M.; Finlayson, S.G.; Sontag, D.; et al. DR Tulu: Reinforcement Learning with Evolving Rubrics for Deep Research, 2025, [arXiv:cs.CL/2511.19399].
277. Li, G.; Mishra, B.D.; Wang, Z.; Yan, J.; Chen, Y.; Li, C.L.; Le, L.T.; Han, R.; Lee, G.; Tong, H.; et al. RubricEM: Meta-RL with Rubric-guided Policy Decomposition beyond Verifiable Rewards, 2026, [arXiv:cs.CL/2605.10899].
278. Liu, Z.; Zhang, L.; Wang, X.; Xu, Z.; Zhan, S.; Shan, X.; Huang, W.; Dai, T.; Xia, S.T.; Huo, C.; et al. ARBOR: Online Process Rewards via a Reusable Rubric Buffer for Search Agents, 2026, [arXiv:cs.CL/2606.03239].
279. Zheng, C.; Mo, X.; Ma, X.; Lin, Q.; Zhao, Y.; Zhu, J.; Lou, X.; Wang, J.; Wang, Z.; Liu, W.; et al. Adaptive Milestone Reward for GUI Agents. *arXiv preprint arXiv:2602.11524* **2026**, [arXiv:cs.LG/2602.11524].
280. Kim, Z.M.; Park, C.; Raheja, V.; Kim, S.; Kang, D. Toward Evaluative Thinking: Meta Policy Optimization with Evolving Reward Models. *arXiv preprint arXiv:2504.20157* **2025**, [arXiv:cs.CL/2504.20157].
281. Ding, H.; Huang, B.; Fang, Y.; Liao, W.; Li, Z.; Zhang, J.; Wu, Z.; Zhao, J.; Wang, Y. EvoRubrics: Dynamic Rubrics as Rewards via Adversarial Co-Evolution for LLM Reinforcement Learning, 2026, [arXiv:cs.CL/2606.23038].
282. Li, S.S.; Xin, R.; Xiao, T.; Wang, Y.; Shao, R.; Hao, Z.; Sclar, M.; Oh, S.; Brahman, F.; Koh, P.W.; et al. EvoLM: Self-Evolving Language Models through Co-Evolved Discriminative Rubrics, 2026, [arXiv:cs.CL/2605.03871].
283. Sheng, L.; Ma, W.; Hong, R.; Wang, X.; Zhang, A.; Chua, T.S. Reinforcing Chain-of-Thought Reasoning with Self-Evolving Rubrics, 2026, [arXiv:cs.AI/2602.10885].
284. Guan, X.; Hu, X.; Huang, S.; Wang, Z.; Zhang, B.; Li, Z.; Xie, P.; Liu, B.; Cao, J. EvoRubric: Self-Evolving Rubric-Driven RL for Open-Ended Generation, 2026, [arXiv:cs.CL/2605.29847].
285. Tian, Z.; Zhang, J.; Li, R.; Bo, X.; Li, Y.; Chen, X. ARCO: Adaptive Rubric with Co-Evolution for Multi-Step LLM-Based Agents, 2026, [arXiv:cs.CL/2606.21262].
286. Feng, A.Z.; Wang, C.; Wen, B.; Wang, Y.; Luo, Y.; Wang, H.; Huang, M. RLAR: An Agentic Reward System for Multi-task Reinforcement Learning on Large Language Models, 2026, [arXiv:cs.CL/2603.00724].
287. Wang, X.; Wu, T.; Tang, M.; Li, J.; Liu, Q.; Zheng, Z. The Flip Side of RLHF: On-Policy Feedback for Reward Model Self-Supervised Improvement, 2026, [arXiv:cs.CL/2605.30888].
288. Shi, T.; Huang, C.; Wan, F.; Zhong, L.; Yang, Z.; Shen, W.; Quan, X.; Yan, M. Mutual-Taught for Co-adapting Policy and Reward Models, 2025, [arXiv:cs.CL/2506.06292].
289. Li, Z.; Cao, Z.; Huang, W.; Zhang, Y.; Qi, K.; Wang, R.; Zheng, Z.; Zhao, J.; Zhu, H.; Wu, H.; et al. MagicGUI-RMS: A Multi-Agent Reward Model System for Self-Evolving GUI Agents via Automated Feedback Reflux, 2026, [arXiv:cs.AI/2601.13060].
290. Lin, Y.; Wang, L.; Lin, K.; Lin, Z.; Gong, K.; Li, W.; Lin, B.; Li, Z.; Zhang, S.; Peng, Y.; et al. JarvisEvo: Towards a Self-Evolving Photo Editing Agent with Synergistic Editor-Evaluator Optimization, 2025, [arXiv:cs.CV/2511.23002].
291. Li, Z.; Jiang, L.; Hu, Y.; Zeng, X.; Li, Y.; Zhang, X.; Chen, G.; Pan, Z.; Li, X.; Liu, Y. No More Stale Feedback: Co-Evolving Critics for Open-World Agent Learning, 2026, [arXiv:cs.AI/2601.06794].
292. Lin, J.; Yu, X.; Xin, Y.; Guo, Y.; Jiang, Z.; Yue, Z.; Wang, W.; Zou, H.; Qin, C.; Xiong, H. ICRL: Learning to Internalize Self-Critique with Reinforcement Learning, 2026, [arXiv:cs.AI/2605.15224].

# 原文第 66 页 | 中文翻译初稿

66 共 70

269.刘，J.；熊K.；夏，P。周，Y。吉，H。冯，L。汉，S。丁，M。 Yao, H. Agent0-VL：探索工具集成视觉语言推理的自我进化智能体，2025，[arXiv:cs.CV/2511.19900]。

270.黄，Y.；于X。 Wei, Z. ACE：通过对抗性单元测试生成和偏好优化的自我进化 LLM 编码框架，2026，[arXiv:cs.SE/2605.16299]。

271.董，G.；卢，J。黄，J。钟W。刘，L。黄，S。李，Z。赵，Y。宋X.；李，X。等人。智能体世界：扩展现实世界环境综合以发展通用智能体智能，2026，[arXiv:cs.AI/2604.18292]。

272.夏，P.；曾，K.；刘，J。秦，C.；吴，F。周，Y。熊，C.； Yao, H. Agent0：通过工具集成推理从零数据中释放自我进化智能体，2025，[arXiv:cs.LG/2511.16043]。

273.黄，C.；于，W.；王X；张，H。李，Z。李，R。黄，J。米，H。 Yu, D. R-Zero：来自零数据的自我进化推理法学硕士，2025 年，[arXiv:cs.LG/2508.05004]。

274.孙达拉姆，S.；全，J.；科维亚特科夫斯基，A.；阿胡贾，K.；奥利维尔，Y.； Kempe, J. 自学教学模型：可学习性边缘的推理，2026，[arXiv:cs.LG/2601.18778]。

275.贾娜，S.；桑卡塔尔，C.；丹尼什，T.；马修斯，G.；奥维多，A.； Kolev, P. GASP：编码法学硕士的引导非对称自我游戏，2026 年，[arXiv:cs.LG/2603.15957]。

276.邵，R.；浅井，A.；沉S.Z.；艾维森，H.；基肖尔，V.；卓，J.；赵X。帕克，M。芬利森，S.G.；桑塔格，D.；等人。 Tulu 博士：强化学习（Reinforcement Learning）以及不断发展的深度研究标准，2025 年，[arXiv:cs.CL/2511.19399]。

277.李，G.；米什拉，B.D.；王，Z。严，J。陈，Y。李，C.L.；勒，L.T.；汉，R。李，G。童，H.；等人。 RubricEM：Meta-RL 与 Rubric 引导的策略分解超越可验证的奖励，2026 年，[arXiv:cs.CL/2605.10899]。

278.刘Z.；张L。王X；徐，Z。詹，S。单，X。黄W.；戴，T。夏，S.T.；霍，C.；等人。 ARBOR：通过可重复使用的 Rubric 缓冲区为搜索智能体提供在线流程奖励，2026 年，[arXiv:cs.CL/2606.03239]。

279.郑，C.；莫，X。最大限度。;林，Q。赵，Y。朱，J。卢，X。王，J。王，Z。刘，W.；等人。 GUI 智能体的自适应里程碑奖励。 arXiv 预印本 arXiv:2602.11524 2026 ，[arXiv:cs.LG/2602.11524]。

280.金，Z.M.；帕克，C.；拉赫贾，V.；金，S。 Kang, D. 走向评价性思维：通过不断发展的奖励模型进行元策略优化。 arXiv 预印本 arXiv:2504.20157 2025 ，[arXiv:cs.CL/2504.20157]。

281.丁，H.；黄，B.；方，Y。廖，W.；李，Z。张，J。吴，Z。赵，J。 Wang, Y. EvoRubrics：通过 LLM 强化学习（Reinforcement Learning）的对抗性共同进化动态评分作为奖励，2026，[arXiv:cs.CL/2606.23038]。

282.李S.S.；辛，R。肖，T。王，Y。邵，R。郝，Z。斯克拉尔，M.；哦，S。婆罗门，F.； Koh，P.W.；等人。 EvoLM：通过共同进化的判别规则进行自我进化的语言模型，2026，[arXiv:cs.CL/2605.03871]。

283.盛L.；马，W。洪，R。王X；张，A。蔡，T.S.用自我进化的规则强化思想链推理，2026，[arXiv:cs.AI/2602.10885]。

284.关X.；胡X.；黄，S。王，Z。张，B。李，Z。谢，P。刘，B.； Cao, J. EvoRubric：用于开放式生成的自进化 Rubric 驱动的强化学习，2026，[arXiv:cs.CL/2605.29847]。

285.田，Z。

;张，J。李，R。盒子。;李，Y。 Chen, X. ARCO：基于 LLM 的多步智能体的自适应评分标准和协同进化，2026，[arXiv:cs.CL/2606.21262]。

286.冯，A.Z.；王，C.；温，B.；王，Y。罗，Y。王，H。 Huang, M. RLAR：大型语言模型上多任务 强化学习（Reinforcement Learning） 的智能体奖励系统，2026，[arXiv:cs.CL/2603.00724]。

287.王X.；吴，T。唐，M。李，J。刘，Q。 Cheng, Z. RLHF 的另一面：奖励模型自我监督改进的策略反馈，2026，[arXiv:cs.CL/2605.30888]。

288.石T.；黄，C.；万，F。钟L.；杨，Z。沉W.；全，X.； Yan, M. 共同适应策略和奖励模型的互教，2025，[arXiv:cs.CL/2506.06292]。

289.李Z.；曹，Z。黄W.；张，Y。齐，K。王，R。郑Z.；赵，J。朱，H.；吴，H。等人。 MagicGUIRMS：通过自动反馈回流实现 GUI 智能体自我进化的多智能体奖励模型系统，2026 年，[arXiv:cs.AI/2601.13060]。

290.林，Y.；王L.；关联。;林，Z。龚，K.；李，W。林，B.；李，Z。张，S。彭，Y。等人。 JarvisEvo：通过协同编辑器-评估器优化实现自我进化的照片编辑智能体，2025 年，[arXiv:cs.CV/2511.23002]。

291.李Z.；蒋L.；胡，Y。曾X.；李，Y。张X；陈，G。潘，Z。李，X。 Liu, Y.不再有陈旧的反馈：开放世界智能体学习的共同进化批评，2026，[arXiv:cs.AI/2601.06794]。

292.林，J.；于X。辛，Y。郭，Y。蒋Z.；岳，Z。王，W。邹，H.；秦，C.； Xiong, H. ICRL：通过 强化学习（Reinforcement Learning） 学习内化自我批评，2026，[arXiv:cs.AI/2605.15224]。

293. Wu, M.; Zhang, G.; Min, S.; Levine, S.; Kumar, A. RLAC: Reinforcement Learning with Adversarial Critic for Free-Form Generation Tasks. *arXiv preprint arXiv:2511.01758* **2025**, [arXiv:cs.LG/2511.01758].
294. Sun, W.; Cheng, X.; Fan, J.; Yu, X.; Xu, Y.; He, S.; Zhao, J.; Liu, K. Towards Agentic Self-Learning LLMs in Search Environment, 2025, [arXiv:cs.AI/2510.14253].
295. Sui, Y.; Hooi, B. Conversation for Non-verifiable Learning: Self-Evolving LLMs through Meta-Evaluation, 2026, [arXiv:cs.CL/2601.21464].
296. Pan, T.; Yan, Y.; Wang, Z.; Zhang, R.; Hou, G.; Zhang, W.; Lu, W.; Xiao, J.; Shen, Y. CoVerRL: Breaking the Consensus Trap in Label-Free Reasoning via Generator-Verifier Co-Evolution. *arXiv preprint arXiv:2603.17775* **2026**, [arXiv:cs.CL/2603.17775].
297. Zhu, H.; Cai, C.; Song, Y.; Chen, X.; Han, S.; Guo, Y. Self-Evolving Deep Research via Joint Generation and Evaluation. *arXiv preprint arXiv:2606.04507* **2026**, [arXiv:cs.CL/2606.04507].
298. Lu, S.; Wang, H.; Chen, Z.; Tang, Y. URPO: A Unified Reward & Policy Optimization Framework for Large Language Models. *arXiv preprint arXiv:2507.17515* **2025**, [arXiv:cs.CV/2507.17515].
299. Zha, K.; Gao, Z.; Shen, M.; Hong, Z.W.; Boning, D.S.; Katabi, D. RL Tango: Reinforcing Generator and Verifier Together for Language Reasoning. *arXiv preprint arXiv:2505.15034* **2025**, [arXiv:cs.LG/2505.15034].
300. Wang, Y.; Xie, T.; Shen, K.; Wang, M.; Yang, L. RLAnything: Forge Environment, Policy, and Reward Model in Completely Dynamic RL System, 2026, [arXiv:cs.LG/2602.02488].
301. Zhang, Y.; Fang, M.; Chen, Z.; Pechenizkiy, M. Self-evolving LLM agents with in-distribution Optimization, 2026, [arXiv:cs.LG/2606.07367].
302. Guan, X.; Zhang, L.L.; Liu, Y.; Shang, N.; Sun, Y.; Zhu, Y.; Yang, F.; Yang, M. rStar-Math: Small LLMs Can Master Math Reasoning with Self-Evolved Deep Thinking. *arXiv preprint arXiv:2501.04519* **2025**, [arXiv:cs.CL/2501.04519].
303. Zhou, C.; Xu, T.; Lin, J.; Ge, D. StepORLM: A Self-Evolving Framework With Generative Process Supervision For Operations Research Language Models. *arXiv preprint arXiv:2509.22558* **2025**, [arXiv:cs.AI/2509.22558].
304. Zhang, J.; Ma, G.; Liu, S.; Hu, Z.; Jing, Y.; Lin, T.E.; Li, Y.; Tao, D. STRIDE: Learnable Stepwise Language Feedback for LLM Reasoning. *arXiv preprint arXiv:2605.18851* **2026**, [arXiv:cs.LG/2605.18851].
305. Xiao, H.; Wang, G.; Chai, Y.; Lu, Z.; Lin, W.; He, H.; Fan, L.; Bian, L.; Hu, R.; Liu, L.; et al. UI-Genie: A Self-Improving Approach for Iteratively Boosting MLLM-based Mobile GUI Agents, 2025, [arXiv:cs.CL/2505.21496].
306. Zhi, X.; zhou, P.; Lu, C.; Lv, H.; Liang, Y.; Zhang, R.; Gao, Y.; WU, Y.; Hu, Y.; Gu, H.; et al. SPARD: Self-Paced Curriculum for RL Alignment via Integrating Reward Dynamics and Data Utility. *arXiv preprint arXiv:2604.07837* **2026**, [arXiv:cs.AI/2604.07837].
307. Fan, L.; Chen, M.; Zhu, T.; Liu, K.; Xia, X.; Li, S.; Liu, Z. ZeroCoder: Can LLMs Improve Code Generation Without Ground-Truth Supervision? *arXiv preprint arXiv:2604.07864* **2026**, [arXiv:cs.SE/2604.07864].
308. Wang, H.; Wang, G.; Xiao, H.; Zhou, Y.; Pan, Y.; Wang, J.; Xu, K.; Wen, Y.; Ruan, X.; Chen, X.; et al. Skill-SD: Skill-Conditioned Self-Distillation for Multi-turn LLM Agents, 2026, [arXiv:cs.LG/2604.10674].
309. Tu, S.; Xu, C.; Zhang, Q.; Ma, Y.; Zhang, Y.; Li, L.; Li, D.; Lan, X.; Zhao, D. UCOB: Learning to Utilize and Evolve Agentic Skills via Credit-Aware On-Policy Bidirectional Self-Distillation, 2026, [arXiv:cs.AI/2606.29502].
310. Jeon, U.; Kwon, J.; Sullivan, M.A.; Lee, C.E.; Lin, G. ATLAS: Adaptive Self-Evolutionary Research Agent with Task-Distributed Multi-LLM Supporters, 2026, [arXiv:cs.AI/2602.02709].
311. Rao, A.; Advani, N.K. AI Training Manager: Bounded Closed-Loop Control of Adaptive Training Recipes, 2026, [arXiv:cs.AI/2606.29871].
312. Dong, H.; Yang, D.; Liang, X.; Feng, C.; Ran, J. AdaLRS: Loss-Guided Adaptive Learning Rate Search for Efficient Foundation Model Pretraining. In Proceedings of the Advances in Neural Information Processing Systems, 2025, [arXiv:cs.LG/2506.13274].
313. Lin, Z.; Xue, C.; Liang, D.; Han, X.; Liu, P.; Wu, X.; Jiang, L.; Lu, Y.; Shi, H.; Liang, S.; et al. Parameter Importance is Not Static: Evolving Parameter Isolation for Supervised Fine-Tuning. *arXiv preprint arXiv:2604.14010* **2026**, [arXiv:cs.LG/2604.14010].
314. Sakip, A.; Fuadi, E.H.; Sayedelahl, O.; Li, Z.; She, J.; Aji, A.F.; Liu, S.; Xing, E.; Ho, Q. COPUS: Co-adaptive Parallelism and Batch Size Selection in Large Language Model Training, 2026, [arXiv:cs.DC/2604.26687].
315. Mai, L.; Li, G.; Wagenländer, M.; Fertakis, K.; Brabete, A.O.; Pietzuch, P. KungFu: Making Training in Distributed Machine Learning Adaptive. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). USENIX Association, 2020, pp. 937–954.

# 原文第 67 页 | 中文翻译初稿

67 共 70

293.吴，M.；张，G。敏，S。莱文，S.； Kumar, A. RLAC：强化学习（Reinforcement Learning） 与自由形式生成任务的对抗性批评。 arXiv 预印本 arXiv:2511.01758 2025，[arXiv:cs.LG/2511.01758]。

294.孙W.；程X；范，J。于X。徐，Y。他，S。赵，J。 Liu, K. 在搜索环境中迈向智能体自学法学硕士，2025 年，[arXiv:cs.AI/2510.14253]。

295.隋，Y.； Hooi, B. 不可验证学习的对话：通过元评估自我进化法学硕士，2026，[arXiv:cs.CL/2601.21464]。

296.潘，T.；严，Y。王，Z。张，R。侯，G.；张，W。卢，W。肖，J。 Shen, Y. CoVerRL：通过生成器-验证器协同进化打破无标签推理中的共识陷阱。 arXiv 预印本 arXiv:2603.17775 2026 ，[arXiv:cs.CL/2603.17775]。

297.朱H.；蔡，C.；宋，Y。陈X.；汉，S。郭，Y。通过联合生成和评估进行自我进化的深度研究。 arXiv 预印本 arXiv:2606.04507 2026 ，[arXiv:cs.CL/2606.04507]。

298.陆S.；王，H。陈，Z.； Tang, Y. URPO：大型语言模型的统一奖励和策略优化框架。 arXiv 预印本 arXiv:2507.17515 2025 ，[arXiv:cs.CV/2507.17515]。

299.查，K.；高，Z。沉，M。洪Z.W.；博宁，D.S.； Katabi, D. RL Tango：共同强化生成器和验证器以进行语言推理。 arXiv 预印本 arXiv:2505.15034 2025 ，[arXiv:cs.LG/2505.15034]。

300.王，Y.；谢T.；沉K.；王，M。 Yang, L. RLAnything：在完全动态 RL 系统中打造环境、策略和奖励模型，2026 年，[arXiv:cs.LG/2602.02488]。

301.张，Y.；方，M.；陈，Z.； Pechenizkiy, M. 具有分布内优化的自我进化 LLM 智能体，2026，[arXiv:cs.LG/2606.07367]。

302.关X.；张，L.L.；刘，Y。尚，N。孙，Y。朱，Y。杨，F。 Yang, M. rStar-Math：小型法学硕士可以通过自我进化的深度思考掌握数学推理。 arXiv 预印本 arXiv:2501.04519 2025 ，[arXiv:cs.CL/2501.04519]。

303.周，C.；徐，T。林，J。 Ge, D. StepORLM：一种具有运筹学语言模型生成过程监督的自我演化框架。 arXiv 预印本 arXiv:2509.22558 2025 ，[arXiv:cs.AI/2509.22558]。

304.张杰；马，G。刘，S。胡，Z。静，Y。林，T.E.；李，Y。 Tao, D. STRIDE：用于法学硕士推理的可学习逐步语言反馈。 arXiv 预印本 arXiv:2605.18851 2026 ，[arXiv:cs.LG/2605.18851]。

305.肖H.；王，G。柴，Y。卢，Z。林，W。他，H.；范，L.；卞，L.；胡，R。刘，L。等人。 UIGenie：用于迭代增强基于 MLLM 的移动 GUI 智能体的 自我改进（Self-Improving） 方法，2025 年，[arXiv:cs.CL/2505.21496]。

306. 志X.；周，P。卢，C。吕，H.；梁，Y。张，R。高，Y。吴，Y。胡，Y。顾，H。等人。 SPARD：通过集成奖励动态和数据实用程序来调整 RL 的自定进度课程。 arXiv 预印本 arXiv:2604.07837 2026 ，[arXiv:cs.AI/2604.07837]。

307.范L.；陈，M。朱，T。刘，K.；夏，X。李，S。 Liu, Z. ZeroCoder：法学硕士能否在没有地面实况监督的情况下改进代码生成？ arXiv 预印本 arXiv:2604.07864 2026 ，[arXiv:cs.SE/2604.07864]。

308.王H.；王，G。肖，H。周，Y。潘，Y。王，J。徐K.；温，Y。阮X.；陈X.；等人

。 Skill-SD：多轮 LLM 智能体的技能条件自蒸馏，2026 年，[arXiv:cs.LG/2604.10674]。

309.图，S.；徐，C.；张，Q。可能。;张，Y。李，L。李，D。兰，X。赵，D. UCOB：通过信用感知的策略双向自蒸馏学习利用和发展智能体技能，2026，[arXiv:cs.AI/2606.29502]。

310.全，美国；权，J。沙利文，文学硕士；李，C.E.； Lin, G. ATLAS：具有任务分布式多法学硕士支持者的自适应自我进化研究智能体，2026，[arXiv:cs.AI/2602.02709]。

311.拉奥，A.；阿德瓦尼，N.K. AI 训练管理器：自适应训练方案的有界闭环控制，2026，[arXiv:cs.AI/2606.29871]。

312.董H.；杨，D。梁X.；冯，C.； Ran, J. AdaLRS：用于高效 基础模型（Foundation Model） 预训练的损失引导自适应学习率搜索。神经信息处理系统进展论文集，2025 年，[arXiv:cs.LG/2506.13274]。

313.林Z.；薛，C.；梁，D。韩X.；刘，P。吴，X。蒋L.；卢，Y。石，H。梁，S。等人。参数重要性不是静态的：不断发展的用于监督微调的参数隔离。 arXiv 预印本 arXiv:2604.14010 2026 ，[arXiv:cs.LG/2604.14010]。

314.萨基普，A.；福阿迪，E.H.；萨耶德拉尔，O.；李，Z。她，J。阿吉，A.F.；刘，S。邢，E。 Ho, Q. COPUS：大型语言模型训练中的自适应并行性和批量大小选择，2026，[arXiv:cs.DC/2604.26687]。

315.麦，L.；李，G。瓦根兰德，M.；费尔塔基斯，K.；布拉贝特，A.O.； Pietzuch, P. KungFu：使分布式机器学习的训练具有适应性。第 14 届 USENIX 操作系统设计与实现研讨会 (OSDI 20) 论文集。USENIX 协会，2020 年，第 937-954 页。

316. Qiao, A.; Choe, S.K.; Subramanya, S.J.; Neiswanger, W.; Ho, Q.; Zhang, H.; Ganger, G.R.; Xing, E.P. Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning. In Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21). USENIX Association, 2021, pp. 1–18.
317. Subramanya, S.J.; Arfeen, D.; Lin, S.; Qiao, A.; Jia, Z.; Ganger, G.R. Sia: Heterogeneity-aware, Goodput-optimized ML-cluster Scheduling. In Proceedings of the Proceedings of the 29th Symposium on Operating Systems Principles, 2023, pp. 642–657. https://doi.org/10.1145/3600006.3613175.
318. Bian, Z.; Li, S.; Wang, W.; You, Y. Online Evolutionary Batch Size Orchestration for Scheduling Deep Learning Workloads in GPU Clusters. In Proceedings of the Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, 2021, pp. 1–15, [2108.03645]. https://doi.org/10.1145/3458817.3480859.
319. Dai, Y.; He, K.; Wang, A. DYNAMIX: RL-based Adaptive Batch Size Optimization in Distributed Machine Learning Systems, 2025, [arXiv:cs.LG/2510.08522].
320. Zhang, L.; Jia, T.; Zhai, Y.; Fang, L.; Zheng, K.; Liu, H.; Huang, X.; Yu, P.S.; Li, Y. Towards Robust LLM Post-Training: Automatic Failure Management for Reinforcement Fine-Tuning. *arXiv preprint arXiv:2605.04431* **2026**, [arXiv:cs.LG/2605.04431].
321. Tan, Z.; Abdullahi, M.; Shi, T.; Yuan, H.; Xu, Z.; Yu, C.; Li, B.; Zhao, B. EARL: Efficient Agentic Reinforcement Learning Systems for Large Language Models, 2025, [arXiv:cs.DC/2510.05943].
322. Peng, Y.; Bao, Y.; Chen, Y.; Wu, C.; Guo, C. Optimus: An Efficient Dynamic Resource Scheduler for Deep Learning Clusters. In Proceedings of the Proceedings of the Thirteenth EuroSys Conference, 2018. https://doi.org/10.1145/3190508.3190517.
323. Zhang, X.; Zhao, H.; Xiao, W.; Jia, X.; Xu, F.; Li, Y.; Lin, W.; Liu, F. Rubick: Exploiting Job Reconfigurability for Deep Learning Cluster Scheduling, 2024, [arXiv:cs.DC/2408.08586].
324. Fang, H.; Zhu, W.; Han, B.; Zhang, A.; Pan, Z.; Yang, S.; Zhang, S.; Gai, J.; Tang, P.; Hu, C.; et al. LLMZero: Discovering Adaptive Training Strategies for RL Post-Training via LLM Agents. *arXiv preprint arXiv:2606.18388* **2026**, [arXiv:cs.LG/2606.18388].
325. Chen, G.; Shi, Y.; Li, Y.; Li, B.; Xu, X.; Wei, H.; Ni, S.; Yang, M.; Ye, J. EvoTrainer: Co-Evolving LLM Policies and Training Harnesses for Autonomous Agentic Reinforcement Learning, 2026, [arXiv:cs.AI/2606.03108].
326. Yu, Z.; Yin, P.; Gao, S.; He, S.; Cai, K.; Zhang, X.P. AutoTrainess: Teaching Language Models to Improve Language Models Autonomously. *arXiv preprint arXiv:2606.31551* **2026**, [arXiv:cs.CL/2606.31551].
327. Li, Q.; Zhang, Y.; Yang, X.; Yang, X.; Wang, Z.; Liu, W.; Bian, J. FT-Dojo: Towards Autonomous LLM Fine-Tuning with Language Agents. *arXiv preprint arXiv:2603.01712* **2026**, [arXiv:cs.AI/2603.01712].
328. Ning, J.; Li, X.; Zeng, J.; Kang, H.; Xiong, C. Auto Research with Specialist Agents Develops Effective and Non-Trivial Training Recipes, 2026, [arXiv:cs.MA/2605.05724].
329. Mroueh, Y.; Fonseca, C.; Belgodere, B.; Cox, D. CliffSearch: Structured Agentic Co-Evolution over Theory and Code for Scientific Algorithm Discovery, 2026, [arXiv:cs.LG/2604.01210].
330. Chen, S.; Tang, Z.; Zhang, W.; Yang, F.; Wang, Y.; Li, T.; Liu, Y. OptiCo: Adaptive Distributed Training Optimization via Collaborative Agent Reasoning. In Proceedings of the Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics, 2026.
331. Gao, S.; Fang, A.; Zitnik, M. AutoScientists: Self-Organizing Agent Teams for Long-Running Scientific Experimentation. *arXiv preprint arXiv:2605.28655* **2026**, [arXiv:cs.AI/2605.28655].
332. Ma, Z.; Wang, G.; Xie, X.; Chen, Y.; Du, H.; Li, B.; Sun, Y.; Liu, W.; Chen, K.; Li, Y. TREX: Automating LLM Fine-tuning via Agent-Driven Tree-based Exploration, 2026, [arXiv:cs.AI/2604.14116].
333. Xia, S.; Zhang, Y.; Chen, A.; Wu, S.; Yuan, S.; Xiao, Y. From AI Assistant to AI Scientist: Autonomous Discovery of LLM-RL Algorithms with LLM Agents, 2026, [arXiv:cs.LG/2603.23951].
334. Ma, Y.J.; Liang, W.; Wang, G.; Huang, D.A.; Bastani, O.; Jayaraman, D.; Zhu, Y.; Fan, L.J.; Anandkumar, A. Eureka: Human-Level Reward Design via Coding Large Language Models, 2023, [arXiv:cs.RO/2310.12931].
335. Sun, S.; Liu, R.; Lyu, J.; Yang, J.W.; Zhang, L.; Li, X. A Large Language Model-Driven Reward Design Framework via Dynamic Feedback for Reinforcement Learning. *arXiv preprint arXiv:2410.14660* **2024**, [arXiv:cs.LG/2410.14660].
336. Wu, X.; Zheng, Z.; Xiong, H. LLM-ALSO: LLM-Driven Adaptive Learning-Signal Optimization for Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2605.29293* **2026**, [arXiv:cs.MA/2605.29293].
337. Hazra, R.; Sygkounas, A.; Persson, A.; Loutfi, A.; Martires, P.Z.D. REvolve: Reward Evolution with Large Language Models using Human Feedback. *arXiv preprint arXiv:2406.01309* **2024**, [arXiv:cs.NE/2406.01309].

# 原文第 68 页 | 中文翻译初稿

68 共 70

316.乔，A.； Choe，S.K.；苏布拉马尼亚，S.J.；奈斯万格，W.；何，Q。张，H。甘格，G.R.；邢E.P. Pollux：用于优化深度学习的自适应集群调度。第 15 届 USENIX 操作系统设计与实现研讨会 (OSDI 21) 论文集。 USENIX 协会，2021 年，第 1-18 页。

317.苏布拉马尼亚，S.J.；阿尔芬，D.；林，S。乔，A.；贾，Z。甘格，G.R. Sia：异构感知、Goodput 优化的 ML 集群调度。第 29 届操作系统原理研讨会论文集，2023 年，第 642-657 页。https://doi.org/10.1145/3600006.3613175。

318.卞Z.；李，S。王，W。 You, Y. 用于在 GPU 集群中调度深度学习工作负载的在线进化批量大小编排。高性能计算、网络、存储和分析国际会议论文集，2021 年，第 1-15 页，[2108.03645]。https://doi.org/10.1145/3458817.3480859。

319.戴，Y.；他，K。 Wang, A. DYNAMIX：分布式机器学习系统中基于 RL 的自适应批量大小优化，2025，[arXiv:cs.LG/2510.08522]。

320.张L.；贾，T。翟，Y。方，L.；郑，K。刘，H.；黄X。余，附注； Li, Y. 迈向稳健的法学硕士后期训练：强化微调的自动故障管理。 arXiv 预印本 arXiv:2605.04431 2026 ，[arXiv:cs.LG/2605.04431]。

321.谭，Z.；阿卜杜拉希，M.；石，T。袁，H.；徐，Z。于，C.；李，B。赵 B. EARL：大型语言模型的高效智能体 强化学习（Reinforcement Learning） 系统，2025 年，[arXiv:cs.DC/2510.05943]。

322.彭，Y.；鲍，Y。陈，Y。吴，C.；郭，C. Optimus：深度学习集群的高效动态资源调度器。收录于 2018 年第十三届 EuroSys 会议论文集。https://doi.org/10.1145/3190508.3190517。

323.张X.；赵，H。肖，W。贾X.；徐，F。李，Y。林，W。 Liu, F. Rubick：利用作业可重构性进行深度学习集群调度，2024 年，[arXiv:cs.DC/2408.08586]。

324.方，H.；朱W.；汉，B.；张，A。潘，Z。杨，S。张，S。盖，J.；唐，P。胡，C.；等人。 LLMZero：通过 LLM 智能体发现 RL 训练后的自适应训练策略。 arXiv 预印本 arXiv:2606.18388 2026 ，[arXiv:cs.LG/2606.18388]。

325.陈，G.；石，Y。李，Y。李，B。徐，X。魏，H。尼，S。杨，M。 Ye, J. EvoTrainer：自主智能体运行支撑框架（Harnesses） 的共同发展 LLM 策略和训练 强化学习（Reinforcement Learning），2026 ，[arXiv:cs.AI/2606.03108]。

326.于Z.；尹，P。高，S。他，S。蔡K.；张X.P. AutoTrainess：教授语言模型以自主改进语言模型。 arXiv 预印本 arXiv:2606.31551 2026 ，[arXiv:cs.CL/2606.31551]。

327.李Q.；张，Y。杨X。杨X。王，Z。刘，W.； Bian, J. FT-Dojo：利用语言智能体进行自主 LLM 微调。 arXiv 预印本 arXiv:2603.01712 2026 ，[arXiv:cs.AI/2603.01712]。

328.宁，J.；李，X。曾，J.；康，H.； Xiong, C. 与专业智能体的汽车研究开发出有效且重要的训练方案，2026，[arXiv:cs.MA/2605.05724]。

329.Mroueh，Y.；丰塞卡，C.；贝尔戈德尔，B.； Cox, D. CliffSearch：科学算法发现理论和代码的结构化智能体协同进化，2026，[arXiv:cs.LG/2604.01210]。

330

。陈，S。唐，Z。张，W。杨，F。王，Y。李，T。 Liu, Y. OptiCo：通过协作智能体推理进行自适应分布式训练优化。计算语言学协会第 64 届年会论文集，2026 年。

331.高S.；方，A.； Zitnik, M. AutoScientists：用于长期运行科学实验的自组织智能体团队。 arXiv 预印本 arXiv:2605.28655 2026 ，[arXiv:cs.AI/2605.28655]。

332.马Z.；王，G。谢X。陈，Y。杜，H。李，B。孙，Y。刘，W.；陈，K。 Li, Y. TREX：通过智能体驱动的基于树的探索自动化 LLM 微调，2026，[arXiv:cs.AI/2604.14116]。

333.夏S.；张，Y。陈，A.；吴，S。袁S.；肖，Y.从AI助理到AI科学家：利用LLM智能体自主发现LLM-RL算法，2026年，[arXiv:cs.LG/2603.23951]。

334.马玉杰；梁，W。王，G。黄，D.A.；巴斯塔尼，O.；贾亚拉曼，D.；朱，Y。范L.J.； Anandkumar，A. Eureka：通过编码大型语言模型进行人性化奖励设计，2023 年，[arXiv:cs.RO/2310.12931]。

335.孙S.；刘，R。吕，J。杨，J.W.；张L。 Li, X. 强化学习（Reinforcement Learning） 的通过动态反馈的大型语言模型驱动的奖励设计框架。 arXiv 预印本 arXiv:2410.14660 2024 ，[arXiv:cs.LG/2410.14660]。

336.吴X.；郑Z.； Xiong, H. LLM-ALSO：LLM 驱动的自适应学习信号优化 MultiAgent 强化学习（Reinforcement Learning）。 arXiv 预印本 arXiv:2605.29293 2026，[arXiv:cs.MA/2605.29293]。

337.哈兹拉，R.；西格库纳斯，A.；佩尔森，A.；卢特菲，A.；马蒂雷斯，P.Z.D. REvolve：使用人类反馈通过大型语言模型奖励进化。 arXiv 预印本 arXiv:2406.01309 2024，[arXiv:cs.NE/2406.01309]。

338. Du, S.; Yan, X.; Shi, J.; Cao, Z.; Feng, S.; Liang, Z.; Sun, B.; Peng, T.; Zhou, Y.; Li, X.; et al. MLEvolve: A Self-Evolving Framework for Automated Machine Learning Algorithm Discovery. *arXiv preprint arXiv:2606.06473* **2026**, [arXiv:cs.AI/2606.06473].
339. Wang, H.; Wu, Y.; Chang, D.; Wei, L.; Heldt, L. Self-Evolving Recommendation System: End-To-End Autonomous Model Optimization With LLM Agents, 2026, [arXiv:cs.LG/2602.10226].
340. Li, P.; Tang, H.; Qiao, J.; Zheng, Y.; Hao, J. LaRes: Evolutionary Reinforcement Learning with LLM-based Adaptive Reward Search. In Proceedings of the Advances in Neural Information Processing Systems, 2025.
341. Li, P.; Hao, J.; Tang, H.; Yuan, Y.; Qiao, J.; Dong, Z.; Zheng, Y. R*: Efficient Reward Design via Reward Structure Evolution and Parameter Alignment Optimization with Large Language Models. In Proceedings of the Proceedings of the 42nd International Conference on Machine Learning. PMLR, 2025, Vol. 267, *Proceedings of Machine Learning Research*, pp. 34509–34527.
342. Gao, N.; Zhang, X.; Jiang, X.; You, M.; Zhang, M.; Deng, Y. RF-Agent: Automated Reward Function Design via Language Agent Tree Search. *arXiv preprint arXiv:2602.23876* **2026**, [arXiv:cs.AI/2602.23876].
343. Huang, C.; Chang, Y.; Lin, J.; Liang, J.; Zeng, R.; Li, J. Efficient Language-instructed Skill Acquisition via Reward-Policy Co-Evolution. *arXiv preprint arXiv:2412.13492* **2024**, [arXiv:cs.RO/2412.13492].
344. Liu, Z.; Chai, J.; Zhu, X.; Tang, S.; Ye, R.; Zhang, B.; Bai, L.; Chen, S. ML-Agent: Reinforcing LLM Agents for Autonomous Machine Learning Engineering, 2025, [arXiv:cs.AI/2505.23723].
345. Zhang, Y.; Zhou, K.; Xu, Z.; Ramnath, K.; Zhou, Y.; Woo, S.; Ding, H.; Cheong, L.L. Learning to Ideate for Machine Learning Engineering Agents, 2026, [arXiv:cs.AI/2601.17596].
346. Wu, F.; Zheng, X.; Wang, Z.; ming Dai, Y.; Li, H. RHyVE: Competence-Aware Verification and Phase-Aware Deployment for LLM-Generated Reward Hypotheses, 2026, [arXiv:cs.LG/2604.28056].
347. Chen, M.; Xiao, B.; Liang, D.; Zeng, C.; Wen, Z. Efficient Hyperparameter Optimization for LLM Reinforcement Learning. *arXiv preprint arXiv:2606.03073* **2026**, [arXiv:cs.LG/2606.03073].
348. Lu, C.; Holt, S.; Fanconi, C.; Chan, A.J.; Foerster, J.; van der Schaar, M.; Lange, R.T. Discovering Preference Optimization Algorithms with and for Large Language Models. In Proceedings of the Advances in Neural Information Processing Systems, 2024, [2406.08414].
349. Cheng, S.; Li, T.; Huang, X.; Yin, X.; Zou, D. Differentiable Evolutionary Reinforcement Learning, 2026, [arXiv:cs.AI/2512.13399].
350. Ahmadi, A.; Sharif, S.; Banad, Y. Enhanced LLM Reasoning by Optimizing Reward Functions with Search-Driven Reinforcement Learning, 2026, [arXiv:cs.CL/2605.02073].
351. Han, X.; Yang, Q.; Chen, X.; Chu, X.; Zhu, M. Generating and Evolving Reward Functions for Highway Driving with Large Language Models. *arXiv preprint arXiv:2406.10540* **2024**, [arXiv:cs.AI/2406.10540].
352. Wei, Y.; Shan, X.; Li, J. LERO: LLM-driven Evolutionary framework with Hybrid Rewards and Enhanced Observation for Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2503.21807* **2025**, [arXiv:cs.LG/2503.21807].
353. Jiang, Z.; Schmidt, D.; Srikanth, D.; Xu, D.; Kaplan, I.; Jacenko, D.; Wu, Y. AIDE: AI-Driven Exploration in the Space of Code. *arXiv preprint arXiv:2502.13138* **2025**, [arXiv:cs.AI/2502.13138].
354. Pepe, A.; Lin, C.Y.; Magka, D.; Acun, B.; Wu, Y.N.; Protopopov, A.; Wu, C.J.; Bachrach, Y. Agentic Discovery of Neural Architectures: AIRA-Compose and AIRA-Design, 2026, [arXiv:cs.AI/2605.15871].
355. Jeddi, A.; Le, M.N.; Karaimer, H.C.; Derpanis, K.G.; Taati, B. GEAR: Genetic AutoResearch for Agentic Code Evolution, 2026, [arXiv:cs.AI/2605.13874].
356. Jiang, J.; Zhu, H.; Zhu, Z. SMCEvolve: Principled Scientific Discovery via Sequential Monte Carlo Evolution, 2026, [arXiv:cs.AI/2605.15308].
357. Si, C.; Yang, Z.; Choi, Y.; Candès, E.; Yang, D.; Hashimoto, T. Towards Execution-Grounded Automated AI Research, 2026, [arXiv:cs.AI/2601.14525].
358. Yuan, S.; Chen, Z.; Xi, Z.; Ye, J.; Du, Z.; Chen, J. Agent-R: Training Language Model Agents to Reflect via Iterative Self-Training, 2025, [arXiv:cs.AI/2501.11425].
359. SAE International. Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles, 2021.
360. Chen, M.; Lv, C.; Zhang, G.; Chang, H.; Zhou, S. HarnessForge: Joint Harness and Policy Evolution for Adaptive Agent Systems, 2026, [arXiv:cs.CL/2606.01779].
361. Iacob, A.; Jovanovic, A.; Shen, W.F.; Burkhardt, D.; Kurmanji, M.; Tastan, N.; Sani, L.; Venanzi, N.A.E.; Odonnat, A.; Cao, Z.; et al. The Red Queen Gödel Machine: Co-Evolving Agents and Their Evaluators, 2026, [arXiv:cs.LG/2606.26294].

# 原文第 69 页 | 中文翻译初稿

69 共 70

338.杜S.；严X；石，J.；曹，Z。冯，S。梁，Z。孙，B.；彭，T。周，Y。李，X。等人。 MLEvolve：用于自动机器学习算法发现的自我进化框架。 arXiv 预印本 arXiv:2606.06473 2026 ，[arXiv:cs.AI/2606.06473]。

339.王H.；吴，Y。张，D。魏，L。 Heldt, L. 自我进化推荐系统：利用 LLM 智能体进行端到端自主模型优化，2026 年，[arXiv:cs.LG/2602.10226]。

340.李，P.；唐，H。乔，J。郑，Y。 hao, J. LaRes：基于 LLM 的自适应奖励搜索的进化 强化学习（Reinforcement Learning）。神经信息处理系统进展论文集，2025 年。

341.李，P.；郝，J。唐，H。袁，Y。乔，J。董，Z.；郑 Y. R*：通过大型语言模型的奖励结构演化和参数对齐优化进行高效奖励设计。第 42 届国际机器学习会议论文集。 PMLR，2025 年，卷。 267，机器学习研究论文集，第 34509-34527 页。

342.高，N.；张X；蒋X；你，M。张，M。 Deng, Y. RF-Agent：通过语言智能体树搜索自动奖励函数设计。 arXiv 预印本 arXiv:2602.23876 2026 ，[arXiv:cs.AI/2602.23876]。

343.黄，C.；张，Y。林，J。梁，J。曾，R.； Li, J. 通过奖励策略共同进化实现高效的语言指导技能习得。 arXiv 预印本 arXiv:2412.13492 2024，[arXiv:cs.RO/2412.13492]。

344.刘Z.；柴，J。朱X。唐，S。叶，R。张，B。白，L.； Chen, S. ML-Agent：强化自主机器学习工程的 LLM 智能体，2025 年，[arXiv:cs.AI/2505.23723]。

345.张，Y.；周，K。徐，Z。拉姆纳斯，K.；周，Y。吴，S。丁，H。 Cheong, L.L. 学习构思机器学习工程智能体，2026 年，[arXiv:cs.AI/2601.17596]。

346.吴F.；郑X；王，Z。戴明，Y.； Li, H. RHyVE：LLM 生成的奖励假设的能力感知验证和阶段感知部署，2026 年，[arXiv:cs.LG/2604.28056]。

347.陈，M.；肖，B.；梁，D。曾，C.； Wen, Z. LLM 强化学习（Reinforcement Learning） 的高效超参数优化。 arXiv 预印本 arXiv:2606.03073 2026 ，[arXiv:cs.LG/2606.03073]。

348.Lu，C.；霍尔特，S.；范科尼，C.；陈，A.J.；福斯特，J.；范德沙尔，M.；兰格，R.T.发现大型语言模型的偏好优化算法。神经信息处理系统进展论文集，2024 年，[2406.08414]。

349.程S.；李，T。黄X。尹，X。 Zou, D. 可微进化 强化学习（Reinforcement Learning），2026, [arXiv:cs.AI/2512.13399]。

350.艾哈迈迪，A.；谢里夫，S.； Banad, Y. 通过使用 SearchDriven 强化学习（Reinforcement Learning） 优化奖励函数来增强 LLM 推理，2026，[arXiv:cs.CL/2605.02073]。

351.韩X.；杨，Q。陈X.；楚，X。 Zhu, M. 使用大型语言模型生成和演化高速公路驾驶奖励函数。 arXiv 预印本 arXiv:2406.10540 2024 ，[arXiv:cs.AI/2406.10540]。

352.魏，Y.；单，X。 Li, J. LERO：LLM 驱动的进化框架，具有混合奖励和增强多智能体观察 强化学习（Reinforcement Learning）。 arXiv 预印本 arXiv:2503.21807 2025 ，[arXiv:cs.LG/2503.21807]。

353.蒋Z.；施密特，D.；斯里坎斯，D.；徐，D。卡普兰，I.

;亚琴科，D.； Wu, Y. AIDE：人工智能驱动的代码空间探索。 arXiv 预印本 arXiv:2502.13138 2025，[arXiv:cs.AI/2502.13138]。

354.佩佩，A.；林，C.Y.；马格卡，D.；阿昆，B.；吴，Y.N.；普罗托波波夫，A.；吴，C.J.；Bachrach, Y. 神经架构的智能体发现：AIRA-Compose 和 AIRA-Design，2026，[arXiv:cs.AI/2605.15871]。

355.杰迪，A.； Le，M.N.；卡赖默，H.C.；德帕尼斯，K.G.； Taati, B. GEAR：智能体代码进化的遗传自动研究，2026，[arXiv:cs.AI/2605.13874]。

356.江J.；朱，H.； Zhu, Z. SMCEvolve：通过顺序蒙特卡罗进化进行原则性科学发现，2026，[arXiv:cs.AI/2605.15308]。

357.Si，C.；杨，Z。崔，Y。坎德斯，E.；杨，D。 Hashimoto, T. 迈向以执行为基础的自动化人工智能研究，2026，[arXiv:cs.AI/2601.14525]。

358.袁S.；陈，Z.；习，Z.；叶，J。杜，Z。 Chen, J. Agent-R：通过迭代自我训练训练语言模型智能体进行反思，2025，[arXiv:cs.AI/2501.11425]。

<标记>359。国际汽车工程学会。 2021 年道路机动车辆驾驶自动化系统相关术语的分类和定义。</mark>

360.陈，M.；吕，C.；张，G。张，H.； Zhou, S. HarnessForge：自适应智能体系统的联合 运行支撑框架（Harness） 和策略演进，2026 年，[arXiv:cs.CL/2606.01779]。

361.雅各布，A.；约万诺维奇，A.；沉W.F.；布克哈特，D.；库尔曼吉，M.；塔斯坦，N.；萨尼，L.；韦南齐，N.A.E；奥多纳特，A.；曹，Z。等人。红皇后哥德尔机器：共同进化的智能体及其评估者，2026，[arXiv:cs.LG/2606.26294]。

362. Hu, J.; Shukla, P.; Huang, K. Verifying the Verifiers: Failure Attribution for Agentic Benchmark Diagnostics and Training Data Curation. In Proceedings of the ICLR 2026 Workshop on AI with Recursive Self-Improvement (RSI), 2026.
363. Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Narang, S.; Chowdhery, A.; Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models, 2023, [arXiv:cs.CL/2203.11171].
364. Chen, X.; Aksitov, R.; Alon, U.; Ren, J.; Xiao, K.; Yin, P.; Prakash, S.; Sutton, C.; Wang, X.; Zhou, D. Universal Self-Consistency for Large Language Model Generation, 2023, [arXiv:cs.CL/2311.17311].
365. Wang, Z.; Wang, K.; Wang, Q.; Zhang, P.; Li, L.; Yang, Z.; Jin, X.; Yu, K.; Nguyen, M.N.; Liu, L.; et al. RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning, 2025, [arXiv:cs.LG/2504.20073].
366. Yi, B.; Liu, Q.; Cheng, Y.; Xu, H. Escaping Model Collapse via Synthetic Data Verification: Near-term Improvements and Long-term Convergence, 2026, [arXiv:stat.ML/2510.16657].
367. Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; et al. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *CoRR* **2026**, *abs/2602.08234*, [2602.08234]. https://doi.org/10.48550/ARXIV.2602.08234.
368. Li, Y.; Miao, R.; Qi, Z.; Lan, T. ARISE: Agent Reasoning with Intrinsic Skill Evolution in Hierarchical Reinforcement Learning. *CoRR* **2026**, *abs/2603.16060*, [2603.16060]. https://doi.org/10.48550/ARXIV.2603.16060.
369. Hebbar, P.; Manawat, Y.; Verboomen, S.; Ivanova, A.; Palanimalai, S.; Bhatia, K.; Baskaran, V. SIA: Self Improving AI with Harness & Weight Updates. *CoRR* **2026**, *abs/2605.27276*, [2605.27276]. https://doi.org/10.48550/ARXIV.2605.27276.
370. Li, Y.; Zhang, Y.; Zhang, X.; Liu, X.; Liu, Y. CODESKILL: Learning Self-Evolving Skills for Coding Agents. *CoRR* **2026**, *abs/2605.25430*, [2605.25430]. https://doi.org/10.48550/ARXIV.2605.25430.
371. Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; Anandkumar, A. Voyager: An Open-Ended Embodied Agent with Large Language Models. *CoRR* **2023**, *abs/2305.16291*, [2305.16291]. https://doi.org/10.48550/ARXIV.2305.16291.
372. Xia, C.S.; Wang, Z.; Yang, Y.; Wei, Y.; Zhang, L. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? *CoRR* **2025**, *abs/2511.13646*, [2511.13646]. https://doi.org/10.48550/ARXIV.2511.13646.
373. Lou, X.; Lázaro-Gredilla, M.; Dedieu, A.; Wendelken, C.; Lehrach, W.; Murphy, K.P. AutoHarness: improving LLM agents by automatically synthesizing a code harness. *CoRR* **2026**, *abs/2603.03329*, [2603.03329]. https://doi.org/10.48550/ARXIV.2603.03329.
374. Fang, T.; Zhang, H.; Zhang, Z.; Ma, K.; Yu, W.; Mi, H.; Yu, D. WebEvolver: Enhancing Web Agent Self-Improvement with Coevolving World Model, 2025, [arXiv:cs.CL/2504.21024].
375. Guo, Y.; Lee, T.; Shi, L.X.; Chen, J.; Liang, P.; Finn, C. VLAW: Iterative Co-Improvement of Vision-Language-Action Policy and World Model, 2026, [arXiv:cs.RO/2602.12063].


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.

# 原文第 70 页 | 中文翻译初稿

70 共 70

362.胡，J.；舒克拉，P.； Huang, K. 验证验证者：智能体基准诊断和训练数据管理的故障归因。 ICLR 2026 人工智能递归自我改进 (RSI) 研讨会论文集，2026 年。

363.王X.；魏，J。舒尔曼斯，D.；勒，Q。池，E。纳朗，S.；乔杜里，A.； Zhou, D. 自我一致性改进了语言模型中的思维链推理，2023，[arXiv:cs.CL/2203.11171]。

364.陈X.；阿克西托夫，R.；阿隆，美国；任，J。肖，K。尹，P。普拉卡什，S.；萨顿，C.；王X；Zhou, D. 大型语言模型生成的通用自一致性，2023，[arXiv:cs.CL/2311.17311]。

365.王Z.；王，K。王Q。张，P。李，L。杨，Z。金，X。于，K。阮，M.N.；刘，L。等人。 RAGEN：通过多轮 强化学习（Reinforcement Learning） 了解 LLM 智能体的自我进化，2025，[arXiv:cs.LG/2504.20073]。

366.易，B.；刘，Q。程，Y。 Xu, H. 通过综合数据验证逃避模型崩溃：近期改进和长期收敛，2026，[arXiv:stat.ML/2510.16657]。

367.夏，P.；陈，J。王，H。刘，J。曾，K.；王，Y。汉，S。周，Y。赵X。陈，H。等人。 SkillRL：通过递归技能增强 强化学习（Reinforcement Learning） 进化智能体。 CoRR 2026，abs/2602.08234，[2602.08234]。 https://doi.org/10.48550/ARXIV.2602.08234。

368.李，Y.；苗，R。齐，Z.； Lan, T. ARISE：分层 强化学习（Reinforcement Learning） 中具有内在技能进化的智能体推理。 CoRR 2026，abs/2603.16060，[2603.16060]。 https://doi.org/10.48550/ARXIV.2603.1 6060。

369.赫巴尔，P.；马纳瓦特，Y.；韦尔布门，S.；伊万诺娃，A.；帕拉尼玛莱，S.；巴蒂亚，K.； Baskaran, V. SIA：通过 运行支撑框架（Harness） 和体重更新自我改进人工智能。 CoRR 2026，abs/2605.27276，[2605.27276]。 https://doi.org/10.48550/ARXIV.2605.27276。

370.李，Y.；张，Y。张X；刘X。 Liu, Y. CODESKILL：学习编码智能体的自我进化技能。 CoRR 2026，abs/2605.25430，[2605.25430]。 https://doi.org/10.48550/ARXIV.2605.25430。

371.王，G.；谢，Y。蒋，Y。曼德勒卡，A.；肖，C.；朱，Y。范，L.； Anandkumar，A. Voyager：具有大型语言模型的开放式具身智能体。 CoRR 2023，abs/2305.16291，[2305.16291]。 https://doi.org/10.48550/ARXIV.2305.16291。

372.夏，C.S.；王，Z。杨，Y。魏，Y。张 L. Live-SWE-agent：软件工程智能体能否动态自我进化？ CoRR 2025，abs/2511.13646，[2511.13646]。 https://doi.org/10.48550/ARXIV.2511.13646。

373.卢X.；拉萨罗-格雷迪拉，M.；德迪厄，A.；温德尔肯，C.；莱拉赫，W.；墨菲，K.P. AutoHarness：通过自动合成代码 运行支撑框架（Harness） 来改进 LLM 智能体。 CoRR 2026，abs/2603.03329，[2603.03329]。 https://doi.org/10.48550/ARXIV.2603.03329。

374.方，T.；张，H。张，Z。马，K。于，W.；米，H。 Yu, D. WebEvolver：通过 Coevolving 世界模型（World Model） 增强 Web 智能体 自我改进（Self-Improvement），2025，[arXiv:cs.CL/2504.21024]。

375.郭，Y.；李，T。施L.X.；陈，J。梁，P。 Finn, C. VLAW：视觉-语言行动策略和 世界模型（World Model） 的迭代共同改进，2026，[arXiv:cs.RO/2602.12063]。

免责声明/出版商注：所有出版物中包含的声明、意见和数据仅代表个人作者和贡献者，而非 MDPI 和/或编辑。 MDPI 和/或编辑对内容中提及的任何想法、方法、说明或产品造成的任何人员或财产伤害不承担任何责任。